

Idle Planet

- George Kerr -

Contents

Analysis.....	4
Name: Idle Planet.....	4
Overview	4
Stakeholders:	5
Other similar products:	6
Stakeholder Survey	10
User Requirements + Essential Features:	24
Limitations:	29
Hardware + Software Requirements:	30
Computational methods:	31
Success Crieteria:	33
Design.....	35
Structure Diagram:	36
Stages of development:	37
1. GUI Setup:.....	37
Planet Clicking + Population and Money Generation	41
3. Workers:	44
4. Shop:.....	46
5. Progress Saving System:.....	50
6. Prestige:.....	53
7. Population gambling mechanic.....	56
8. Achievements.....	60
9. Level system:.....	64
10. Effects:.....	67
11. Minigames:	69
12. Multiple Planets	74
13. Custom Planets.....	77
14. Animations.....	80
GUI Design:	81

Post-dev Test Plan:	95
Development	99
Stage 1 – GUI:	99
Features:	99
Development	104
Testing:	147
Stage review:	152
Stage 2 – Planet Clicking + Population and Money Generation	152
Features:	152
Data Dictionary	152
Pseudocode:	153
Development:	155
Testing:	162
Stage review:	164
Stage 3 – Workers:	164
Features:	165
Data Dictionary:	165
Pseudocode:	167
Development	169
Testing	182
Stage review:	183
Stage 4 – Shop:	184
Features:	184
Data Dictionary:	189
Pseudocode:	191
Development	201
Testing	267
Stage review:	272
Stage 5 – Progress Save Feature:	274
Features:	274
Data Dictionary:	274
Pseudocode:	274
Development:	279
Testing:	284
Stage review:	285

Stage 6 – Prestige:	286
Features:	286
Data Dictionary:	286
Psuedocode:	287
Development:	293
Testing:	316
Stage review:	320
Stage 7 – Population Gambling:	321
Features:	321
Data Dictionary:	322
Pseudocode:	324
Development:	332
Testing:	367
Stage review:	374
Stage 8 – Achievements:	375
Features:	375
Data dictionary:	376
HTML and CSS modifications:	379
Pseudocode:	380
Development:	386
Testing	423
Stage review:	427
Stage 9 – Level System:	428
Features:	428
Data dictionary:	428
Pseudocode:	429
Development:	433
Testing:	441
Stage review:	445
Stage 10 – Effects:	446
Features:	446
Data dictionary:	446
Pseudocode:	448
Development:	452
Testing:	468
Stage review:	470

Evaluation	471
Post-development testing:	471
1. Functionality Testing:	471
2. Usability Testing:	475
3. Robustness Testing:	479
Evaluation:	483
Success Criteria Evaluation:	483
Usability Evaluation:	493
Limitations:	494
Maintenance:	495
Conclusion:	496

Analysis

Name: Idle Planet

Overview

The aim of my project is to create an incremental, idle/tycoon style game, where the user is able to develop different planets in a galaxy, clicking them to earn an in-game currency which can be spent to progress them further. The main stat of each planet will be population, increasing as the player progresses and influencing money production. Idle and tycoon games are games where the player progresses through simple, repeated tasks (like clicking something) or automated processes that run without the user's input and can be upgraded to increase their generation of a currency. Despite their simplicity, these types of game have a lot of potential for customisation through various features like upgrades, different worlds/scenes and prestige mechanics. The problem with some idle games is that they all involve the user following the same set of

upgrades, which leads to the same tycoon at the end with little customisation. When I play idle games, I often notice many opportunities there are for more features, and more ways to expand the game, so I am making this game to ensure this won't be a problem. The core functionality of this game will be just like other idle/tycoon games, but with much more customisation, allowing the user to add and customise their own planets, creating a more unique and satisfying experience. This game will have a clean, simple user interface similar to other idle games, but with a space theme to reflect the theme of the game. The planet will be in the centre, allowing the player to click, while upgrades, minigames and other options will be on either side. This draws the main focus to the planet, which is the focus of the game. I will research space and galaxy colour pallets to create a vibrant, themed user interface.

For a successful game, I will have to assess the potential stakeholders of this game, including their age and what type of gamer they are. This will allow me to get an idea of the user requirements for the players of this game, so I can better tailor the experience. I will ask the stakeholders questions about certain game mechanics that I have in mind, and whether they would enjoy playing with them. I will also take inspiration from idle games like cookie clicker, which has a similar concept. I will take some features from it, like certain upgrades and the prestige system, whilst customising the game to a space theme with my own features.

Stakeholders:

The stakeholders for this game are mainly casual gamers, who enjoy relaxed, slow paced games with some challenging features. This is because my game is not very stressful or over-competitive, like first person shooters, so it will attract gamers that enjoy a calm gaming session where they can create something to feel a sense of

achievement. The age range for my stakeholders is between 15 and 20, as the sci-fi, space theme is appealing to a young age group. Furthermore, incremental games like this can be played in short sessions, where players can get on for just 15 minutes and make meaningful progress. This is perfect for students who may have limited time due to homework and tests, as they can use this game to take short, relaxing breaks. I will include my stakeholders in my analysis for the user requirements, by distributing a Microsoft Forms survey that will get their opinion on core features, gameplay style and visual design. This will allow me to tailor the game to the specific needs of the stakeholders, making it more personal and engaging.

Other similar products:

- **Cookie clicker:**

- This game allows the player to click a cookie, which grants them cookies (the in-game currency) which they can use to purchase automated clickers, like grandmas or farms that click the cookie for them, and upgrades that help them progress, like increase cookies per second. As the player progresses, they earn achievements which gains them milk that unlocks unique upgrades. The player can prestige, which resets their progress but gives them a permanent upgrade that helps them earn cookies quicker than before.

Features I will include in my game	Features I won't include in my game
1. The automated clickers/workers that the player can purchase and	1. Sugar lumps – These are the secondary, in-game currency in cookie

upgrade to idly earn cookies. This is a key feature of an idle game, the player needs some form of automated resource generation, which they can purchase and upgrade to make more powerful. I will adapt this to fit the space theme of the game, for example a space station.	clicker, and they are unlocked after the player has baked a certain amount of cookies. They take between 18 and 24 hours to mature, and can be used to make additional purchases that cookies cannot be used for, like levelling up buildings (the automated clickers). I like the concept of an additional currency, however I think these don't give the player enough of a sense of achievement, as they are just collected over time, meaning the player must wait instead of being active and earning them themselves. This makes the game less engaging and active, so I won't implement this type of additional currency in my game (I will still have one, but it will be gained through active tasks).
2. The factory upgrades, like increased total cookie production or decreased costs. This makes progression in the game more interesting, as the player has more ways to increase their progression. I may adapt these to give the player a choice between multiple upgrades, causing them to progress differently based on their individual choices, which further will further add to the uniqueness of each user's tycoon.	2. Heralds – These are people who have donated to the game's Patreon highest tier, which then gives everyone who plays (whether they have donated or not) a boost. For every one person who donates to the Patreon tier, everyone gets a +1% cookies per second increase. This feature is a nice way to recognise the donations of the game's supporters, however I won't be having a Patreon where people donate, so this feature won't be needed in my game.
3. The prestige mechanic, where the user resets their progress to earn a permanent upgrade to make progressing easier. However, the prestige is quite complicated in cookie clicker, with heavenly chips, an upgrade tree, and angel and demon upgrades. I will likely make it a simple progress reset with a permanent currency multiplier.	3. A stock market – This is a feature of the bank building in the game, where the player has options of different items that they can buy and sell to make a profit. I think this feature will be too complicated to implement into my game, and is just not necessary. I don't have much knowledge on how stock markets work, and I cannot assume that the players of my game will either, therefore it may create unnecessary confusion.

- **Idle world:**

- This game starts with the user clicking a small button to earn energy, which then allows them to develop and colonise a planet, using the energy to purchase upgrades like water and land which generate more energy. The player can also purchase a second world (world 2.0) and mars, to colonise. There is an additional currency, diamonds, which is earned through completing missions, which can be spent on upgrades like 2x energy for 4 minutes.

Features I will include in my game	Features I won't include in my game
1. The diamond currency, which is earned by completing various challenges. These diamonds can be spent on upgrades which help the player progress further. I like the idea of completing additionally challenges for an additional currency, as it gives the player more to do, making the game more interesting and exciting. These challenges will be the minigames that I mentioned in the overview, which will grant the user this additional currency, which I will change to make my game unique to Idle world.	1. The random, free rewards in return for watching an advert. Every so often, something like a satellite will pas in front of the planet, and if the player clicks on it, they will get a message offering them a reward (like 2x energy for 2 minutes) if they watch an advert. I like the idea of these free rewards, however having to watch and advert every time can become frustrating, and the player might find them quite annoying. Therefore, I won't be implementing these type of free rewards in my game.
2. The level mechanic, where the player's level increases as they progress in the game. I like this feature as it is an easy way to show that the player is progressing in the game. The player can then receive unique rewards once they level up, giving them a sense of achievement and reward.	2. The upgrades that are purchased by watching adverts. Another way the the game tries to get the player to watch adverts, is through purchasing upgrades. In the upgrades tab, only the first 2 upgrades can be purchased by the in-game currency, whilst the other 10 all require the player to watch an advert to purchase. I think this makes the game boring, as the player is progressing by just watching

	short videos, rather than spending the currency that they worked to get. Therefore, I am not going to implement this in my game.
--	--

- **Adventure capitalist:**

- The player has their own lemonade stand, which they click to earn money. The more they click the stand, the more money it earns them. They can then purchase more stands which earn them even more money. The player can also hire managers, which idly earn money for them. Upgrades are also available which further increase the player's progress, for example 3x profit for a certain stand. Once the player reaches a certain amount of money, they can go to a new world with different upgrades and stands

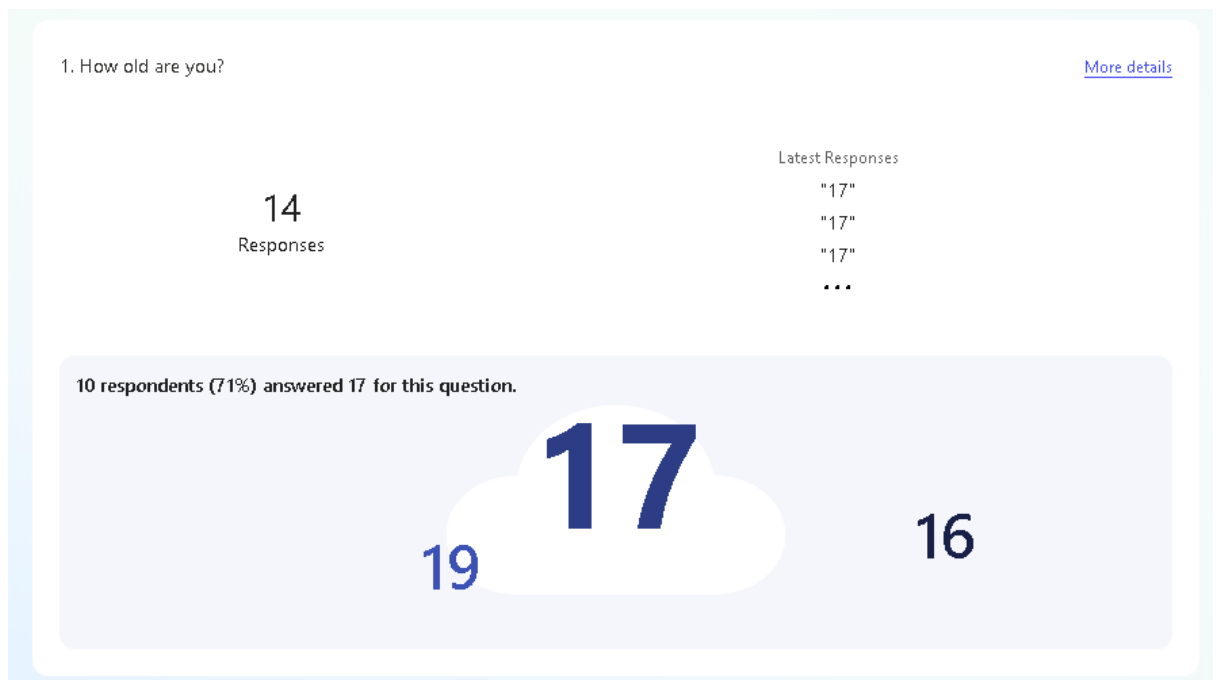
Features I will include in my game	Features I won't include in my game
1. The multi-purchase feature for stands. This allows the player to purchase more than 1 of a stand at a time, if they have enough money. For any stand, they can either purchase 1, 10, 100 or the max possible amount with the money they have at once. This is also a feature in cookie clicker (except without an option to purchase the maximum amount possible), and it is quite a simple feature that makes purchasing much easier when the player starts to earn a large amount of money and wants to spend it all without clicking on a certain building/stand load of times.	1. Angel investors – These are an interesting concept, they are attracted as the player earns money, but they can only earn them when they prestige and restart. However, I don't think these are needed, as it may overcomplicate the prestige process. I will just make some sort of prestige multiplier that increases throughout the player's progression, and the more these are when the player prestiges, the better multiplier they will get. I also think that it's not clear in this game how much money the player needs to start earning these angel investors, so the player will forget about them for a while.

	2. Mega tickets – These are a third currency, on top of the main cash, and gold. It must be exchanged, taking a huge amount of earth dollars just to get 1. These tickets permanently increase investments on any planet by an extremely large amount (777%), however I don't think it is needed as there are already 2 currencies, so it might get confusing. Furthermore, the player's investments could still be increased but just with a purchasable upgrade, rather than a whole extra currency.
--	---

Stakeholder Survey

- **Survey:**
 - To aid my choice of user requirements, I made a survey with questions about desired gameplay features and importance of aspects like the visual design. I distributed this survey to my friends and members of my college. Here were the results:

Question 1:



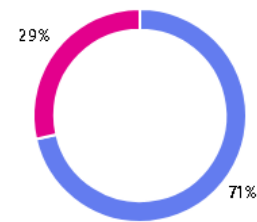
- I made this question to ensure that the answers were coming from users of a specific age range, of 15-20 which is what I decided was the target age range for this game. This confirms that all my responses are from the desired stakeholders, as they were all between 15 and 20. If there were some results from outside the desired age range, and they still showed interest (rated it highly in question

Question 2:

2. Do you like idle/incremental games like Cookie Clicker and Adventure Capitalist

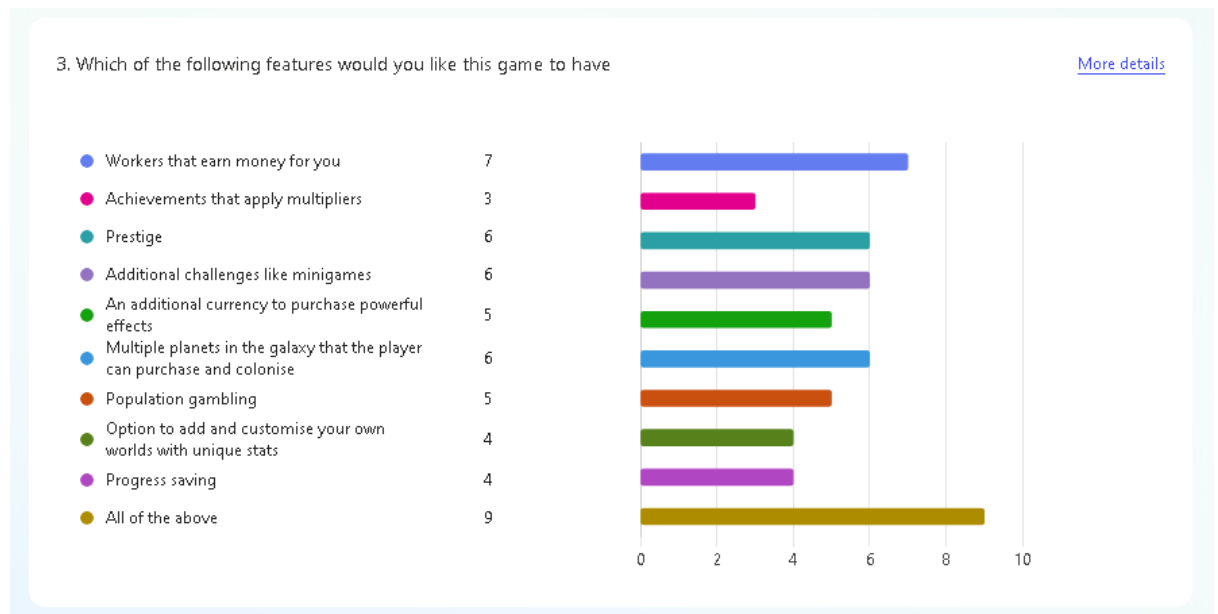
[More details](#)

● Yes 10
● No 4



- This question asked the participants whether they liked idle/incremental games like Cookie Clicker and Adventure Capitalist, which is useful to find out whether the participants enjoy this genre of game. 71% of participants said they liked incremental games, confirming a strong interest, meaning I can focus on the core mechanics of these types of games without needing to make it appealing to a wider audience. However, since there were still some participants (29%) who said they didn't like idle/incremental games, I may have to add some features for a wider audience, that are more engaging rather than simply idle.

Question 3:



- This question was designed to get more of an idea on the specific user requirements. I asked the participants to select all the features they thought the game should have, with an option to select all of them. These are all the core features that I have in mind for the game, however I wanted to find out if any needed to be eliminated due to lack of interest, or if any needed to be prioritised due to high interest. The problem with this question is that there were multiple participants who selected all the features, and also selected the “All of the above”, for example:

[View results](#)

<

Respondent

2

Anonymous

00:51

Time to complete

>

...

1. How old are you? *

17

2. Do you like idle/incremental games like Cookie Clicker and Adventure Capitalist? *

- ☒ Yes
- ☐ No

3. Which of the following features would you like this game to have? *

- ☒ Workers that earn money for you
- ☒ Achievements that apply multipliers
- ☒ Prestige
- ☒ Additional challenges like minigames
- ☒ An additional currency to purchase powerful effects
- ☒ Multiple planets in the galaxy that the player can purchase and colonise
- ☒ Population gambling
- ☒ Option to add and customise your own worlds with unique stats
- ☒ Progress saving
- ☒ All of the above

This creates the problem where some choices appear to be chosen more than they actually have been, if a participant selects it as well as the all of the above option. To solve this problem, I need to clean the data, by removing any individual options that have been selected if the participant has also selected the all of the above option. To do this, I will use the Excel spreadsheet that Microsoft Forms stores all the responses to the survey in. I will use the copy the column for question 3 into a new sheet:

I will then look at each row, and look for rows that contain “All of the above” as well as individual options. Where this is the case, I will remove all of the individual options, leaving the “All of the above” option there. The responses are now clean and look like this:

Which of the following features would you like this game to have
Workers that earn money for you;Additional challenges like minigames ;Progress saving;
All of the above
Workers that earn money for you;Prestige;Multiple planets in the galaxy that the player can purchase and colonise;Population gambling;Progress saving;
All of the above;
Workers that earn money for you;Achievements that apply multipliers;Prestige;Multiple planets in the galaxy that the player can purchase and colonise;Additional challenges like minigames ;An additional currency to purchase powerful effects;
All of the above;
All of the above;
Workers that earn money for you;Achievements that apply multipliers;Prestige;Additional challenges like minigames ;An additional currency to purchase powerful effects;Multiple planets in the galaxy that the player can purchase and colonise;
All of the above;
All of the above;
Option to add and customise your own worlds with unique stats;Workers that earn money for you;Additional challenges like minigames ;An additional currency to purchase powerful effects;Multiple planets in the galaxy that the player can purchase and colonise;
All of the above;
All of the above;
All of the above;

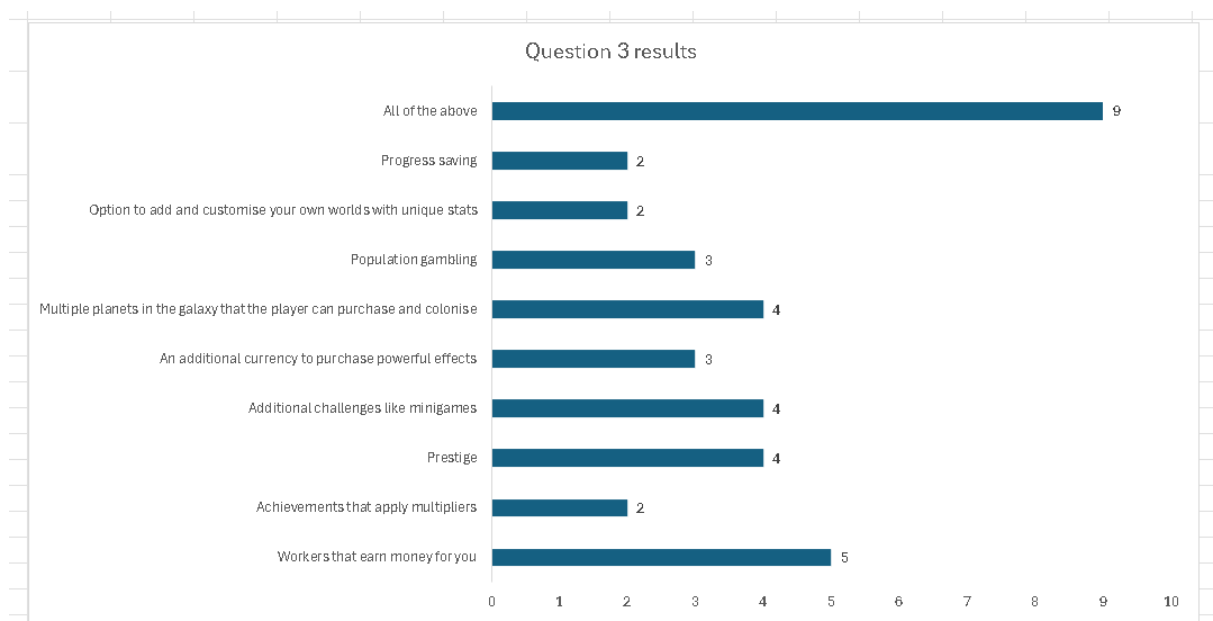
From this, I can calculate how create a table for the correct results for each option:

Option	Count
Workers that earn money for you	5
Achievements that apply multipliers	2
Prestige	4
Additional challenges like minigames	4
An additional currency to purchase powerful effects	3
Multiple planets in the galaxy that the player can purchase and colonise	4
Population gambling	3
Progress saving	2
Option to add and customise your own worlds with unique stats	2
All of the above	9

I can then put this table into excel:

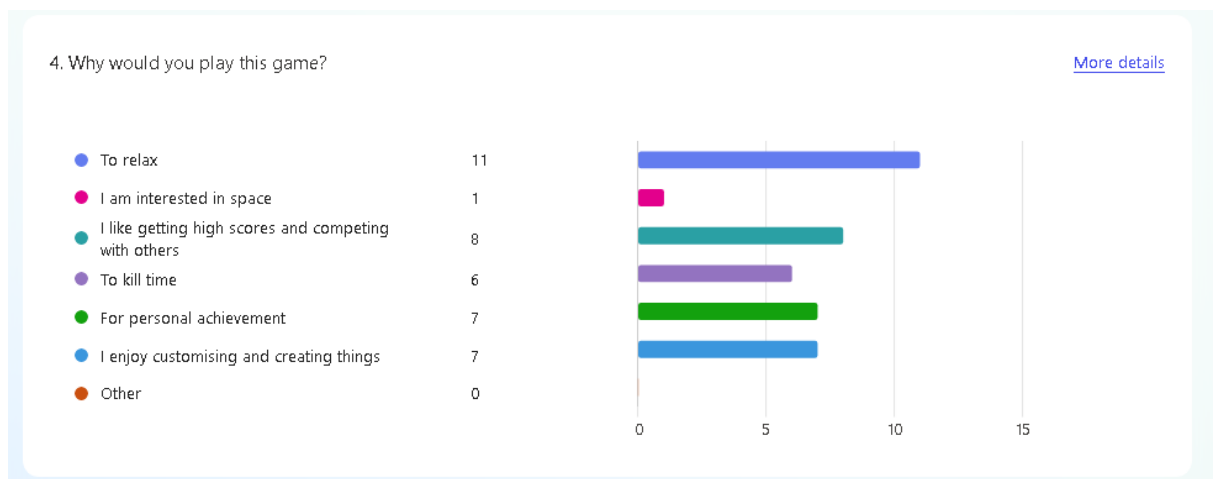
Option	Count
Workers that earn money for you	5
Achievements that apply multipliers	2
Prestige	4
Additional challenges like minigames	4
An additional currency to purchase powerful effects	3
Multiple planets in the galaxy that the player can purchase and colonise	4
Population gambling	3
Option to add and customise your own worlds with unique stats	2
Progress saving	2
All of the above	9

And use a Clusted Bar chart to create a bar chart for the results, similar to the one on Microsoft Forms, but with the accurate, clean results:



This tells me that all of these should be essential features of the game, since all the options were chosen by at least 11 participants, meaning all these features are generally desired.

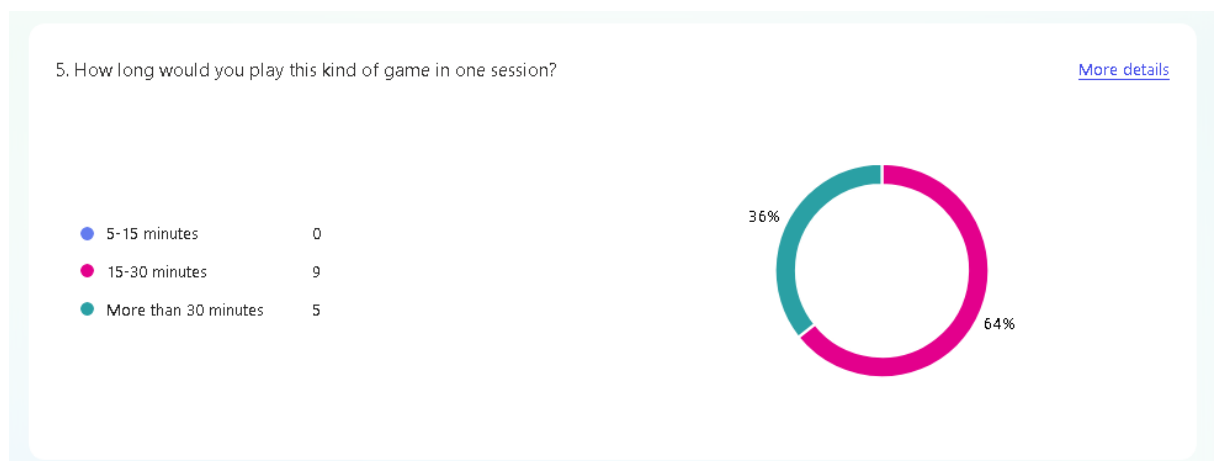
Question 4:



- This question asked the users why they would play my game, allowing me to understand the reasons and motivations for playing it. This then allows me to more accurately tailor the game's features and visual design to its players. The participants were able to select a maximum of 3 choices, and all of them apart from 2 used all 3 of their choices (the other 2 only used 2 of them). The results show that the most common reason that the participants would play this game is to relax and to kill time (as these options combined received 43% of choices), which tells me that I need to create a more calm, cozy visual design, with a focus on relaxing, idle features rather than fast-paced competitive mechanics (I can still include challenging minigames, but potentially make these more of an optional feature that the player doesn't have to do). I also need to ensure there is always something for the player to do, so they don't get bored. The results also show that personal achievement and the love of high scores and competition would play a big part in the motivation to play this game, as both of these options combined received 38% of choices. This tells me that

there also needs to be a focus on the score that the player is achieving and what they have earned as a result of it. I need to make the player feel rewarded by getting a high score, potentially giving them unique rewards for reaching a certain score. Additionally, 18% of participants said that a reason they would play this game is because they enjoy customising and creating things, meaning there must be a focus on the planet they are generating. I must make it clear that they are developing a planet, by visually displaying it's expansion as they play. This also confirms that the feature where the player can add and customise their own planet with unique stats would be a desired feature for many, as it allows for more creation and customisation.

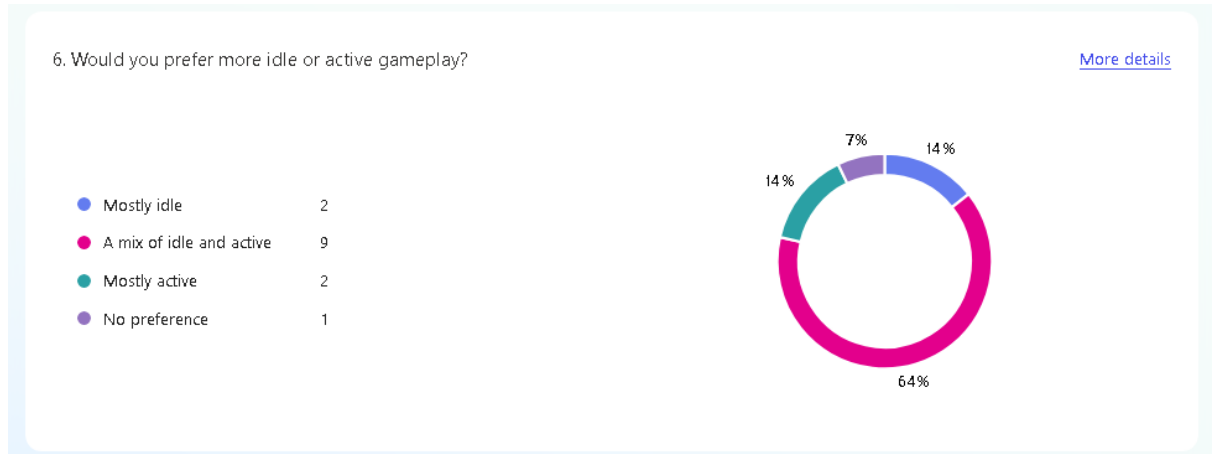
Question 5:



- This question asked the participants how long they would typically play this type of game in a single session, further allowing me to tailor my game's features to its players, as I know how long they need to be playing before they see significant progress. There were no responses of less than 15 minutes, meaning the game doesn't need to be too fast paced and the player shouldn't be seeing much progress before 15 minutes of playing. Most of the participants (64%) said that they would play for between 15 and 30 minutes in a session, and the rest (36%) said they would play for 30+ minutes. This informs me that the game should start to feel rewarding after 15 minutes, where the player can see the progress they have made, ensuring they stay engaged. This

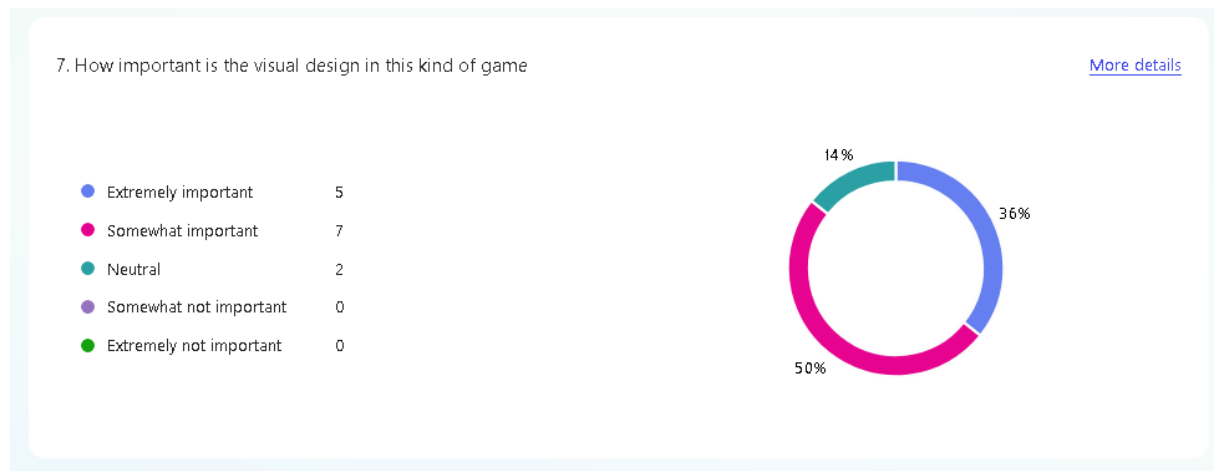
can help me design some of the features, like the speed of the idle resource generation.

Question 6:



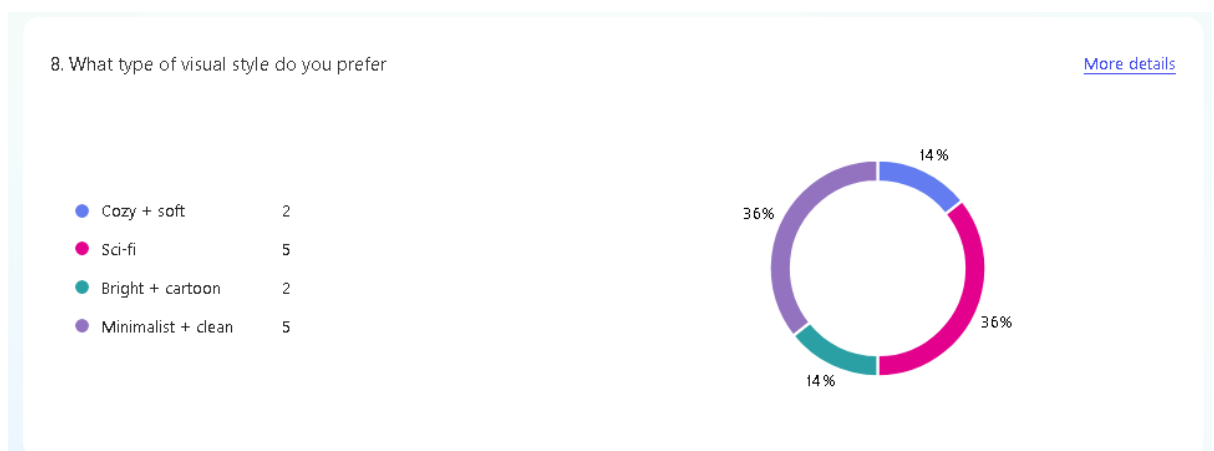
- This question was designed to find out how idle/active the game should be. A more idle game will involve simple tasks that the player completes (like clicking) to then be able to gain a lot of progress without any input. Whereas, a more active game will involve lots more tasks that the user can complete, like challenges and minigames to help them progress further in the game. Since the majority (65%) of participants said they would like a mix of idle and active gameplay, this informs me that the game needs to include lots of interactive features that the user can engage with, whilst also allowing them to gain significant progress without any input.

Question 7:



- This question asked about the importance of the visual design in the type of game that I'm making, allowing me to decide the priority of how the game looks. Since 86% of participants said the visual design is either somewhat important or extremely important, the UI of this game will be a high priority. This question allows me to ensure I focus on an aesthetic, visually appealing design whilst also focussing on the features and game mechanics.

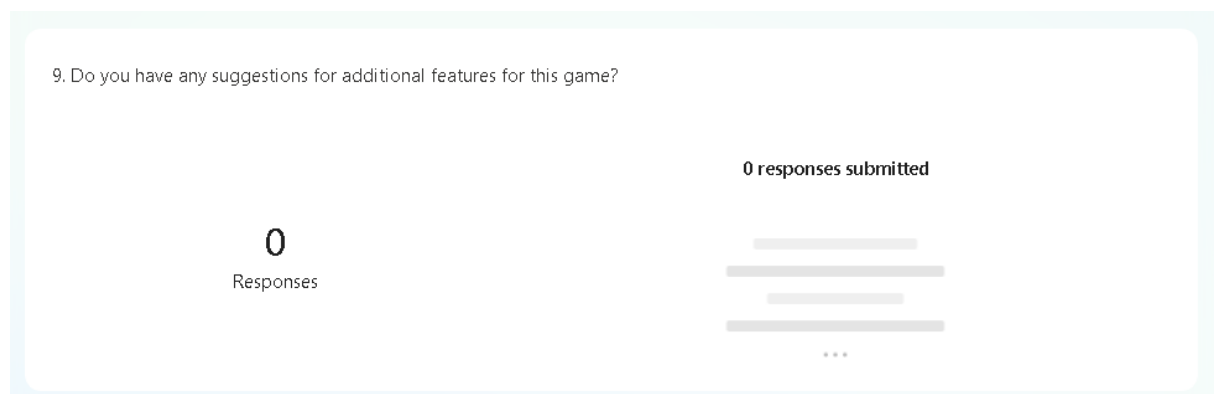
Question 8:



- Following the question that asked the importance of the visual design, this question asked the participants which visual style they prefer.

From the 4 options, there was a split between the most popular. Both Sci-fi and Minimalist + Clean received 36% of choices, informing me that the best style for the game would be a combination of both. The sci-fi style will fit well with the space theme of the game, which I can ensure is still clean, aesthetic and modern.

Question 9:



- This question asked the participants if they had any suggestions for additional features for the game, which was designed to see if there were any features that a lot of people were asking for, that I had missed, however there were no responses to this so I am sure that the game isn't missing any must-have features.

Question 10:



- This question was a quick, rating question where I asked the participants to rate (on a scale of 1-10) the idea of this game. This was just to check that the participants were actually interested in my idea and whether they thought it would make a fun game. Fortunately, I received an average rating of 8.71, and no one rated it below 7, which confirms that this game has potentially to be enjoyable and engaging.

User Requirements + Essential Features:

Requirement	Reason
1. The player should be able to click a planet, to increase it's population which earns them money.	This is the core mechanic of the game, as it's an incremental game so requires a simple action that the user can complete to progress. The population allows them to earn money, which is essential for progress and upgrades.
2. There should be a clean, sci-fi/space-themed GUI with space-themed animations. The GUI should be separated into the main section, with the planet and popuation and money displayed clearly, and then individual sections like challeneges and upgrades.	In the survey, 50% of participants said that the visual design in this kind of game is somewhat important, and 36% said that it is extremely important, therefore the there must be a clean GUI and animations to ensure there is an aesthetic visual design. Furthermore, in response to the question that asked the participants what kind of visual style they prefer, there was quite a strong interest in all the styles. However, the most popular styles were Sci-fi and Minimalist/clean, therefore the visual design needs to have a sci-fi aspect (which fits well with the space-theme), and also be clean and sleek.
3. A shop where the player can purchase upgrades, like cash multipliers and worker upgrades.	This is an essential feature of the game, since it is going to have upgrades that need to be purchaseable. A shop is a perfect way to organise all of the upgrades into one area, so it is clear where they all are and easy to access.
4. A multi-purchase feature, where the player can purchase multiple of one upgrade/automated clicker in a single purchase, if they have enough money.	This is a very useful feature, as it allows the user to purchahse a large number of something in one click, without having to click it loads of times (especially if they have a lot of money, which will take a while). It is also extremely easy to implement, so is an essential

	feature. I am also taking this feature from Adventure Capitalist and Cookie Clicker, as I have benefitted from it in both of them, making purchasing much more convenient.
5. Workers, that the player can purchase, that click the planet for them without their input, increasing the planets population which then increases the players earnings. These workers need to be upgradable, allowing their generation to be increased.	This feature was the most highly requested in the survey (question 3), with every participant saying they would like this feature in the game, thus it should definitely be implemented. Just to add, in the survey this feature was labelled as “Workers that earn money for you”, however the workers will actually be clicking the planet so earning population for the player, however this then increases their earnings so essentially workers do earn money for the player indirectly. This is a minor error in my survey and I don’t think the results would be any different if the feature was labelled “Workers that earn population for you”, since the key concept is the actual workers themselves”. Furthermore, workers are a core mechanic in incremental games, something that idly generates resources and can be upgraded.
6. A prestige mechanic, that allows the player to reset their progress (keeping some progress like achievements), so they can restart the game again but with multipliers that make progression easier than before.	This feature was highly requested, with 93% survey participants choosing it as a feature that should be in the game, meaning it should definitely be implemented. Prestiging is also a core mechanic in many games, including incremental games, as it keeps the game fresh, by allowing the player to restart but with progress multipliers.
7. Multiple planets that the player can purchase and colonise.	This was another popular feature, also with 93% of survey participants selecting it, making it an essential feature. This is also essential to keep

	the game fresh, as it may get boring just colonising a single planet, whereas a galaxy of different planets with different upgrades and effects will make the game much more interesting and engaging.
8. Additional challenges, like space-themed minigames that the player can complete for a reward. These challenges will earn the player the additional currency that I will implement, that will allow them to purchase unique, limited effects/multipliers.	This was highly requested, with 13 survey participants choosing it as a feature that should be in the game. Furthermore, in response to question 6, 64% of participants said they would like a mix of idle and active gameplay in the game, meaning I need to have features like this to ensure the game has active mechanics to allow the player interact with it more rather than just watching idle progress.
9. An additional currency that the user will earn from the challenges/minigames. This currency will allow the player to purchase unique, limited effects/multipliers, for example double money for 2 minutes.	This was a popular feature amongst the participants of the survey, with 12/14 (86% of) survey participants choosing it in question 3, meaning it is in high demand so should definitely be added to the game. Furthermore, the challenges/minigames should have a unique reward to give the player a better sense of achievement, this reward will be this unique, additional currency. It also just makes the game more engaging, with another currency that the player needs to earn.
10. A population gambling feature, where the player can gamble a specific amount of their population, after which they have a chance of either losing it all, or gaining more than they gambled.	This feature was popular amongst the survey participants, as 86% of them selected it as a feature they would like in the game, meaning it should definitely be implemented. Furthermore, it increases player engagement and activity within the game, making it more engaging for them. Again, 64% of participants (in response to question 6) said that they would like

	a mix of idle and active gameplay and 14% said they would like more active gameplay, thus population gambling ensures there is an active side of the game, allowing the player to better influence the game themselves, rather than them just progressing idly.
11. Achievements that the player reaches after time playing, for example “Clicked 500 times”. These achievements will grant xp, which goes towards the player’s level.	This was quite a highly requested feature, with 11/14 (79% of) survey participants saying they would like the game to have it, meaning it is a desired feature amongst the stakeholders. Furthermore, this is a common feature amongst most games, not just incremental games, as it gives the player a sense of achievement, as their progress in the game is being recognised and rewarded. In fact, the (joint) third highest reason for why participants said they would play this game (in response to question 4), with 7 participants choosing it, was for personal achievement. This confirms that the game needs to recognise the progress of players, and reward them with achievements that then give them even more progress, to give them a sense of achievement.
12. An option for the player to add and customise their own planets, with unique stats. This will be available to the player once they have reached and fully colonised all of the game’s planets. The player will be able to give the planet any stats they want, giving them free control of how they progress on it. This will be more of a free roam/sandbox mode that the	This feature was also quite a highly requested one, with 79% of participants choosing it as a feature they would like in the game, so I must make sure it is included. Furthermore, in response to question 4, about the motivations for playing this game, 7 participants said that they would play this game because they enjoy customising and creating things. Therefore, this feature is important to increase the customisability of the

player unlocks at the end of the game	game, giving the player the option to create their own unique world.
13. Progress saving, where the player's progress will be saved in the local storage of their browser, allowing them to return to their save whenever they re-open their browser.	This was another feature that was requested quite highly, by 79% of participants of the survey, so it should definitely be implemented in the game. Furthermore, progress saving is a key mechanic of many games, as it allows the user to come back to the game at a different time and still have their progress, that they worked hard for. Additionally, 64% of survey participants said that they would play this kind of game for 15-30 minutes in one session, therefore many players will have to exit the game after this time, and it is important that the game saves their progress so they haven't wasted their time playing.
14. A level mechanic, where the player earns xp through upgrades, purchases and general progression. This xp will increase their level, and once they reach the next level they will get a reward and a congratulations screen will be displayed to them.	This is another important feature that shows the user their progression in the game, thus giving them a sense of achievement and making them more likely to continue playing to get an even higher level, to earn more rewards. This is a feature that is inspired from Idle World, but with the removal of its manual level-up mechanic and replaced with automatic levelling up.

Limitations:

These are desirable features that I would have wanted to implement, however I won't or am unable to for specific reasons

Limitation	Reason
Complex Animations	Due to the number of features and mechanics in my game, I will be limiting myself to simple animations. This is because I don't have much knowledge on web-based animations and I want to focus on how the UI actually looks and the core features. Focussing too much on unique, detailed animations for all the game's features will take up too much development time, therefore I will only be implementing basic ones. These will still fit the sci-fi, space-theme, however.
Multiple Languages	I am not going to develop any options to change the text language in my game, so it will all be in English. This is because the main language for all games is English, it is the most widely spoken language so I can assume that the majority of my stakeholders will be English speakers. I also don't need to make this game accessible to everyone, but instead I want to focus on making it as good as possible for those who can understand it.
All planets in galaxy	Since there are 8 planets in the galaxy, I am limiting myself to only developing 4, as it would take too long to develop all 8, with unique stats and designs. Instead, I will just implement Earth, Mars, Jupiter and Saturn, so I can focus on making

	high quality planets within the time I have. Furthermore, the player will have the option to add their own planets with custom stats, so if they want to add other planets, like Venus, then they can.
--	--

Hardware + Software Requirements:

Hardware:

- This is a game so any personal computer or laptop will work.
- Mobile phones will be able to access it, however the format may make it unplayable as I won't be developing the game to work for phones.
- The device must have an internet connection, as this is a web-based game.
- The device may need speakers to hear game sound effects and music, however this is not essential.
- The device requires a mouse, for navigating the GUI and clicking buttons to play.
- The device will require a keyboard for certain minigames and text input, like when customising new planets.

- The game is web-based, so won't take up any space in memory. The local storage will take up space in memory, but this will be a very small amount.

Software:

- A browser is required to play this game, as it is a web-based game.
- There are no additional software requirements, it is just the browser, which all computers/laptops come with, pre-installed.

Computational methods:

- Why this is a computable task:

1. The problem can be **decomposed**, which is where it is broken down into smaller, more manageable sub-problems which make development easier to understand, reducing the overall time it takes to solve the problem. The reason this problem can be decomposed, is because there are various sections of the game that can be individually identified, and that all integrate to form the overall solution, for example:
 - Planet development – The program needs to track the users clicks, increase population, and generate a certain amount of money per second
 - Upgrades – The program needs to manage and display upgrade cost and effects, handle the purchasing logic (removing the item price from the player's money) and the increase of price each time the upgrade is purchased

- Minigames – The program needs to handle the minigames, which are a separate game mechanic from the main game, with different rewards based on wins/loses, or how well the player did in a minigame.

I will use decomposition to split up this problem into sub-problems that I can individually design, develop and test. This reduces the complexity of the problem, avoids overlapping of tasks, allows me to track development much more easily, making it easier and quicker to develop and test, giving me more time to fix bugs and improve the game. At this stage, I am only identifying how the gameplay logic will be decomposed, into sections like planet development and upgrade handling. However, in the design section, I will explain how each of these sections will be further decomposed into Javascript subroutines, with flowcharts and pseudocode where appropriate.

2. The problem can be **abstracted**, meaning unnecessary details from it can be removed to make the problem simpler to solve. This is mainly because it is a space-themed game, meaning there are a number of features I could include, like stars, meteors, satellites with unique physics and animations. This would allow for an extremely engaging and immersive experience, however would significantly increase the development time, and I would likely not have enough time to complete the project. However, with abstraction, I can remove these unnecessary elements from the problem, meaning I only need to develop what is needed in the requirements of the solution. For example, when developing the Earth planet, I don't need to design every city and landmark on the Earth, as this is not needed to play the game and would take way too much time. Instead, I can just use/design an image of the Earth, which the player can click, ensuring I have enough time to develop the core features of and the gameplay logic. Abstraction makes the game run smoothly, reducing unnecessary resource usage, and reduces the time it takes to develop the program, due to it's reduced complexity.
3. This problem contains tasks that can run **concurrently**, which is where they overlap. For example, once the player is generating idle income from their planet, they can do other tasks like a minigame, which will overlap

with the idle income from their planet, meaning the player can play minigames while still earning money from their planet. This makes the gameplay feel more smooth and interactive, as the player can play minigames, browse the store, or look at their achievements, whilst earning money in the background, thus improving their overall experience. Additionally, concurrency makes more efficient use of the processor, idle (waiting) time by overlapping processes.

Success Criteria:

1. The player should be able to click a planet, to increase it's population, which automatically generates money

Justification: Core mechanic of an incremental game. Requires simple action that the user can complete to progress. Population should automatically earn the player money, so they can purchase upgrades

2. GUI should be clean and space/schi-fi themed with separate sections for different features, and space-themed animations.

Justification: Fits the space theme of the game, and 86% of survey participants said that the visual design of the game was at least somewhat important

3. The game should include a shop system where the player can purchase upgrades using their money, like worker, population and money upgrades.

Justification: Upgrades are essential for the player to make progress in the game, and a shop is required for them to purchase them.

4. The shop system should include a multi-purchase feature, where the player can buy multiplier upgrades at once (either 1, 10, 100 or the MAX they can buy with their money)

Justification: A multi-purchase feature makes it much easier to purchase upgrades in bulk, removing the need for the player to constantly click upgrades to purchase multiple of them.

5. The player should be able to hire workers, that automatically generate population, with upgrades available that can increase their speed and generation

Justification: Workers are also essential for idle games, since the idle part of the game involves some form of automatic generation. This feature was requested by every survey participant.

6. The player can prestige, where they reset their game progress, to earn permanent multipliers that increase their progress

Justification: Prestiging allows the player to replay the game, so it doesn't get repetitive. This feature was also requested by 93% of survey participants

7. The player should be able to unlock and develop multiple planets, which have different stats

Justification: This feature also adds to the engagement of the game, keeping it fresh and allowing the player to replay the game in a different setting.

8. The game should include simple, space-themed minigames that the player can play to earn an additional currency

Justification: This adds an interactive and engaging feature to the game, requested by 93% of survey participants, so it is not just plain idle gameplay. 64% of survey participants voted for a mix of idle and active gameplay, so this adds that active aspect to the game.

9. There should be an additional currency (like gems), that the player earns through winning minigames, completing achievements and levelling up, which can be spent on temporary effects/multipliers

Justification: An additional currency makes winning minigames feel more rewarding, as the player earns a unique currency. Limited time effects makes the game more engaging and exciting. Requested by 86% of participants.

10. The player can gamble some or all of their population, with a chance of winning and multiplying their bet, or losing it all

Justification: This gives the game another interactive and engaging feature, to give the game more variety. The risk/reward aspect of gambling makes the game more exciting.

11. The player unlocks achievements for completing a certain amount of an action (e.g. clicking the planet), which reward them with xp and the secondary currency

Justification: This gives the player a sense of personal achievement, rewarding their progress and success in the game, a motivation for why survey participants would play the game. Also the feature itself was requested by 79% of participants in the survey.

12. After finishing the main game (unlocking all the default planets), the player should be able to create and customise their own planets, with unique stats that they can choose

Justification: Gives the game a sandbox element, which makes the player feel like they own and can customise their game. 7 survey participants said that a motivation for playing this game would be the enjoyment of customising and creating things.

13. The player's progress should automatically save to local storage and whenever a save button is pressed

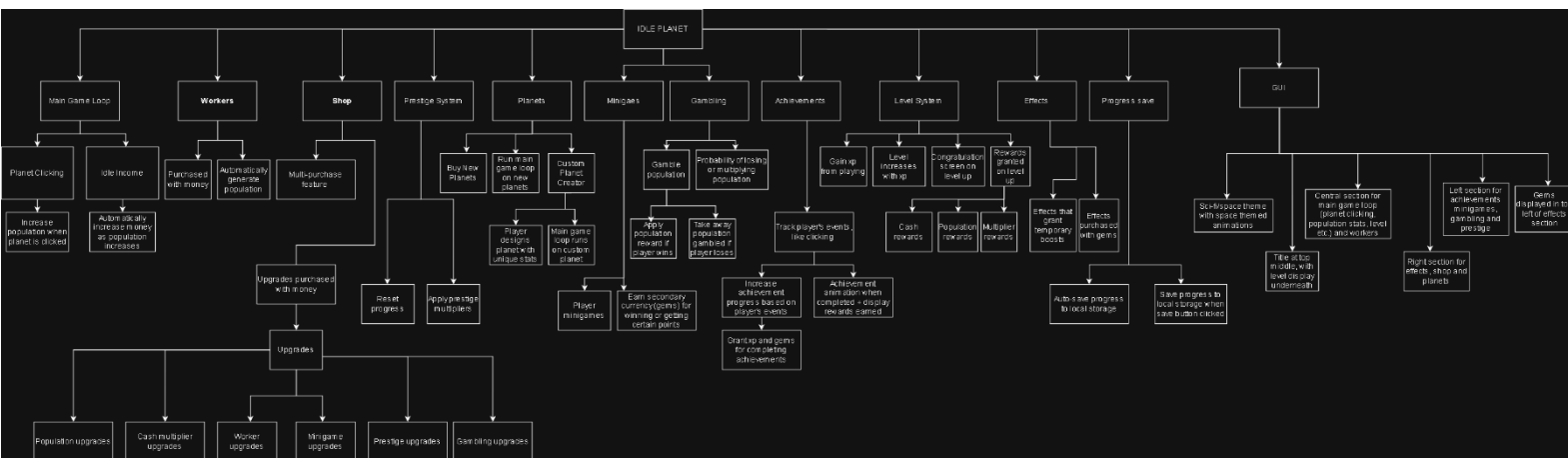
Justification: Requested by 79% of survey participants, this allows players to play the game in short sessions and come back to it whenever they want, with everything saved.

14. The player has a level which increases when they gain xp (through minigames or completing achievements), with a level up congratulations screen and reward granted when they level up

Justification: Another feature that motivates players by giving them a sense of achievement, showing and rewarding their progression.

Design

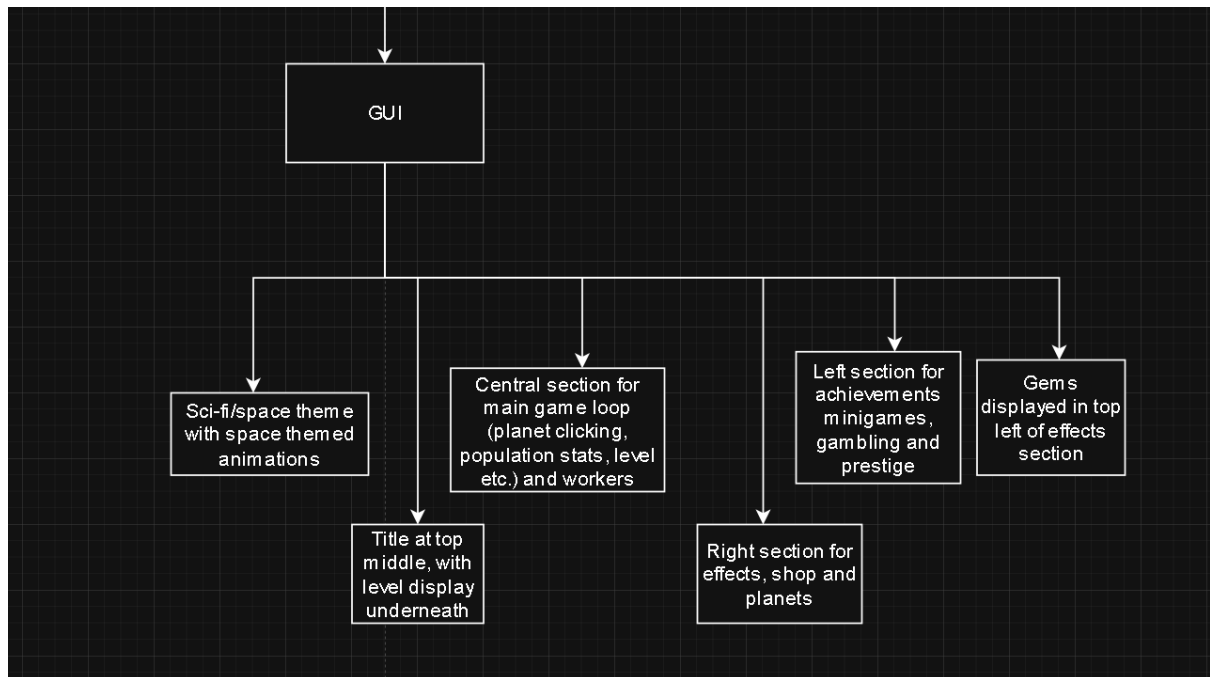
Structure Diagram:



- This is the structure diagram for my project, where I have broken it down into sections to clearly lay out and organise each stage
- Each section is followed by subsections to explain their functionality and implementation
- I will use this structure diagram to design each stage, one by one, rather than approaching the project in one go, as one stage.
- This allows me to decompose the problem, approaching the solution step by step

Stages of development:

1. GUI Setup:



This is the structure diagram section for the GUI, which I have as an individual component as it is separate from the JavaScript code, so can be individually designed with HTML and CSS, before any functionality is added with JavaScript. I have decomposed the GUI into sections for each section of the GUI, plus the space theme and animations, which will all be individually developed within the GUI development stage.

Description:

- In this stage, I need to create the HTML and CSS layout for the game, with the central planet and stats and side panels for other features.

- I need to setup the GUI, with all the sections of the game including their buttons, input forms and text displays
- I need to setup a central section, which displays the planet that the player will be clicking, with text underneath to show population and money
- Under this (still in the central section), I need to create the worker section, where the player can buy new workers, upgrade all workers, and see how quickly workers are clicking the planet
- I need to create the left section of the game, which will be dedicated to more active features of the game:
 - I will have an achievements section in the top left of the page, which will display the achievements to complete. For each achievement, the game will display how much xp the player has got out of how much they need to complete the achievement. I will also make a progress bar for each achievement, that visually displays the progress the player has made for each achievement
 - I will make a minigame section underneath, which will have 3 buttons for each 3 minigames that the player can choose from
 - I will make a prestige section, which will have text to display the current prestige multiplier, and the button that the player can press to prestige
 - I will make a population gambling section, with an input form to allow the player to enter the amount of population they would like to gamble. It will also have a win return display that will display the population that the player will earn if they win. I will also make a display that tells the player they have either won or lost.
- I need to create a the right section of the game, which will be dedicated to purchasable items:
 - I will have an effects section in the top right, which will display all the limited effects that the player can purchase, by clicking, with their gems (the secondary currency), along with the price of each. The player's gems will be displayed in the top left of the effects section.
 - I will also have a shop section, directly to the right of the planet, which will display all the upgrades which the player can purchase, by clicking, along with the price of each and the number of each upgrade the player currently has. In the top right of the shop section, I will make a multi-

purchase button, which allows the player to purchase multiple of the same upgrade at once if they have enough money. The player can click this button, switching it between x1, x10, x100, and MAX (which means the maximum number of upgrades that the player can purchase with the amount of money they have). Under the price of each upgrade, I will make some text that corresponds with the multi-purchase setting selected, and if MAX is selected then this text will display the number of each upgrade that the player can possibly purchase with the money they currently have.

- I will also have a planet section, underneath the shop, which will display the next planets for the player to colonise. I will put a lock symbol next to each planet, apart from the very next one which I will put text next to that displays the level needed to unlock the planet.

Why this stage first:

- Creating the GUI gives the foundation and base layout of the game, allowing me to add and program features into it later. Without creating the GUI first, it would be very hard to test features, as I wouldn't get a visual representation of how they are working.
- It helps show me where all the features will go, preventing me from needing to constantly redesign the GUI later when I add sections

Test plan:

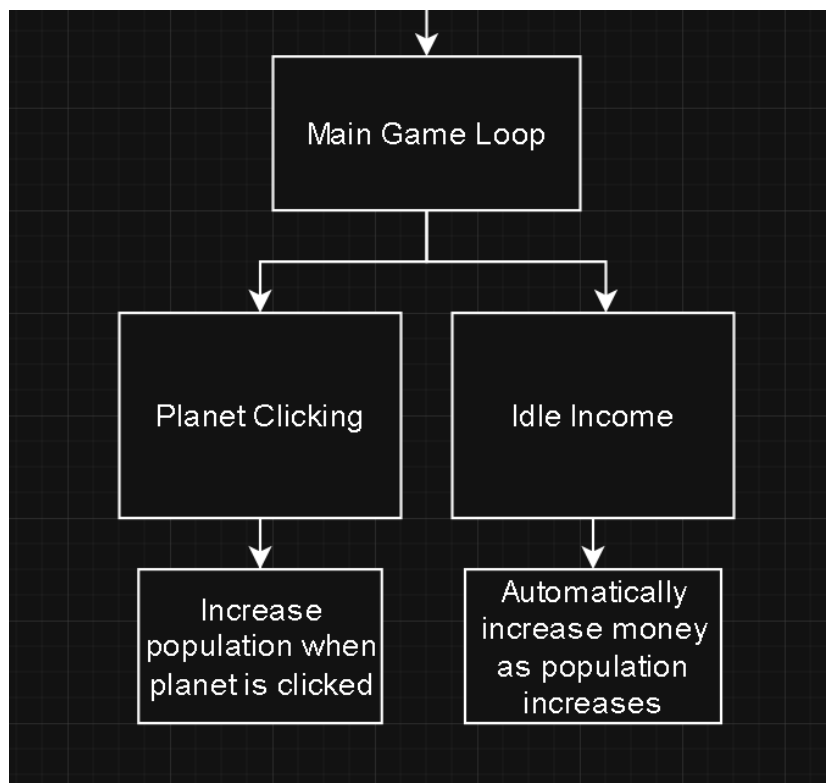
Test no.	Description	Test data	Expected Result	Actual result
1	Is the title section set up correctly	Open the game and look at the top middle of the page	There should be a container which displays the game title and the name of the current planet. The save and load icon should also be	

			in the top left of this container	
2	Is the central section set up correctly.	Open the game and look at the centre of the page	There should be a container, where the planet is displayed at the top, with the money and population display underneath	
3	Is the left section set up correctly	Open the game and look at the left of the page	There should be a container in the top right which displays the achievements. Underneath, there should be another container which has three containers within it (the minigame, prestige and population gambling containers)	
4	Is the right section set up correctly.	Open the game and look at the right of the page	There should be a container in the top right which displays the effects (along with the players gems in the top left of this container). Underneath, there should be a container with two containers within it (the shop container and the planets container)	
5	Is the colour scheme designed correctly		The colour scheme of the game should match that of the GUI colour design, with the correct	

			background, border and text colour.	
--	--	--	-------------------------------------	--

This test data is chosen to check if all the GUI sections are correctly designed and separated, with their correct text displays and buttons. It also checks if the colour scheme designed matches that of the GUI colour design. All together, this will test whether the game meets success criterion 1, that the game has a clean, space-themed design with separate sections for different features

Planet Clicking + Population and Money Generation



This is the structure diagram section for the main game loop, which is the planet clicking, population growth, and automatic money generation. I have the main game loop as an individual component, as it is separate from all the other active and purchase features and is the main focus of the game so should be individually developed. I have decomposed this into planet clicking, and idle income, since the planet clicking leads to idle income, so they should be 2 separate components, with the planet clicking influencing the idle income.

Description:

- Here, I need to implement the planet clicking feature, which allows the player to click the planet which increases the planets population
- Each time the player clicks the planet, the population will increase by 1
- I need make the population automatically increase money
- I will start by making the money increase by 0.01 every second for every 1 population.

Why this stage here:

- This is the core mechanic of the game and without it, nothing else would work as the player needs a way of earning the currency
- Since all the other features depend on this feature, getting this feature done at the start will allow me to test all other features

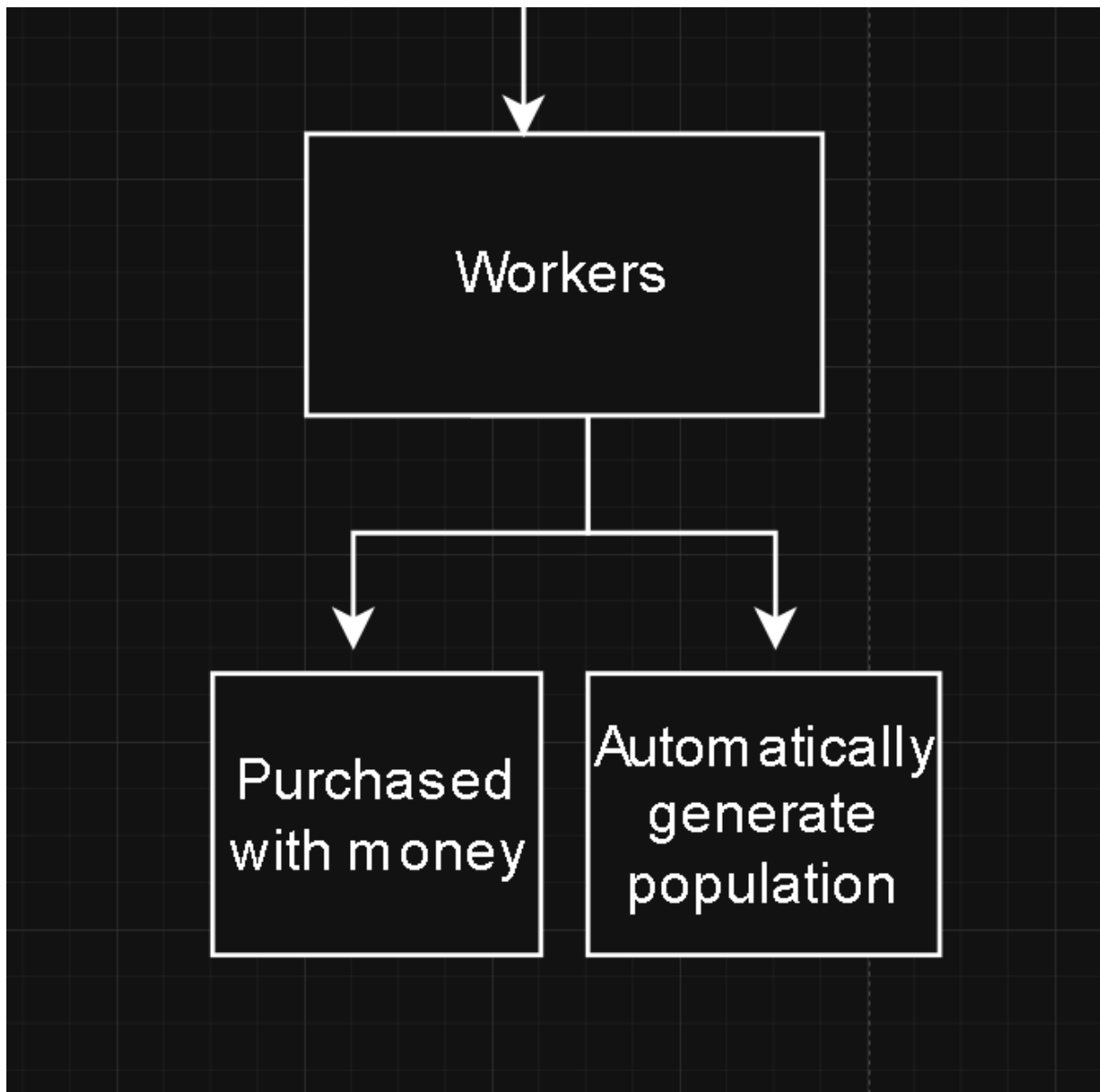
Test plan:

Test no.	Description	Test data	Expected Result	Actual result
1	Does the population increase correctly when you click the planet	Click the planet and see if the population number goes up	The population number increases by 1 (Before any upgrades) every time you click the planet	

2	Does the money increase as the population increases	Click the planet to increase the population, and see if the money starts increasing	The money should start automatically increasing by 0.01 every second once the population increases above 0. The amount that the money increases every second should increase as the population increases.	
3	Does the money per second display increase correctly	Click the planet to increase the population and observe the money/s display underneath the money display.	The money per second should increase by 0.01 per 1 population. At 10 population, for example, it should say 0.1	

I have chosen this test data to check if the the planet clicking is functioning correctly, and that it leads to population growth, which leads to money generation, which are the individual components of this stage according to the structure diagram. This is an extremely important feature as it is the main function of the game, so must be working properly, and it will test whether that the game meets success criterion 1, that the player should be able to click a planet, to increase it's population, which automatically generates money.

3. Workers:



This is the structure diagram section for the workers, which I have as an individual component since it is an individual feature that generates automatic population growth, which no other feature does. Therefore, they should be developed individually. I have broken down the worker component into 2 sections, the purchase handling and the automatic population generation, since these are the 2 main components of the workers. They are purchased with money, and then start generating population.

Description:

- In the worker section inside the main section (underneath the planet) I need to add the option to buy workers that automatically generate population
- I will start by making any workers increase the population by 1 every 2.5 seconds.
- I will make a display in the worker section, to display the total amount of population that all the workers are generating per second.

Why this stage here:

- Once the manual population growth has been developed, the next stage should be the idle income,
- In order for the workers to be able to automatically increase population, I need to first implement the population and money system, which is why this is after stage 2
- Idle growth is also a core feature of this game since it's an idle game, so should be developed early on.

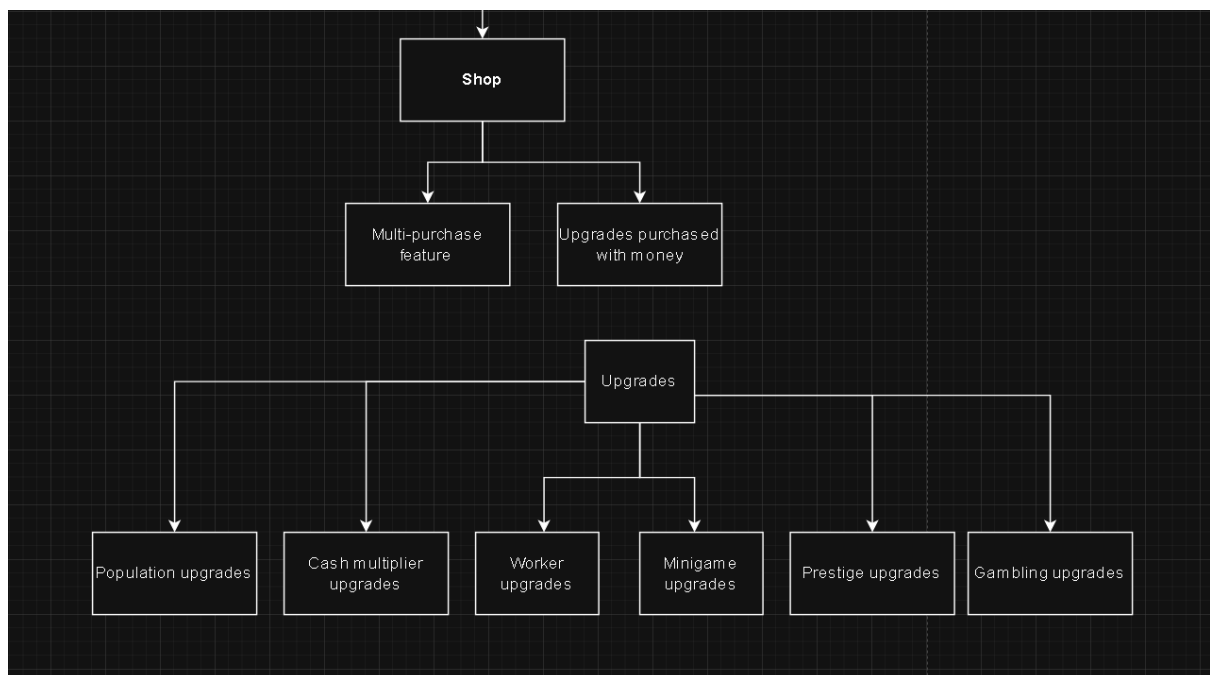
Test plan:

Test no.	Description	Test data	Expected Result	Actual result
1	Is the player prevented from purchasing workers when they don't have enough money	Press the buy worker button without any money	Nothing should happen; the total workers number should not increase at all	

2	Is the player able to purchase a worker	Press the buy worker button with enough money and see if the total workers number increases	The total workers number should increase by 1	
3	Does the worker automatically increase the population	Buy a worker and check whether the population number is increasing automatically	The population number should increase by 1 every 5 seconds (Before any upgrades)	

I have chosen this test data to check that the worker purchase is handled correctly, that the worker's correctly generate population, which are the 2 individual components of the workers according to the structure diagram. This will test whether success criterion 5 is met, which is that the player should be able to hire workers, that automatically generate population, with upgrades available that can increase their speed and generation. The upgrade part of the success criterion will be developed in the shop stage.

4. Shop:



This is the shop section of the structure diagram, which I have as an individual component as it is a single feature, with one purpose which is to contain all the game upgrades for the player to purchase. I have broken this down into the multi-purchase feature, which is an individual component as it will be developed separate from the shop upgrades, since it just influences how much of the upgrades will be purchased and updates their prices. I have also broken the shop down into the main purpose, which is to purchase upgrades with money. I have then broken this down into all the upgrade categories I will have, since I will develop each category of upgrade separately, alongside the associated feature.

Description:

- I need to create the shop, which has all the upgrades available for the player to purchase including their price
- I will have different categories of upgrades:
 - Money and population upgrades
 - Worker upgrades
 - Prestige upgrades
 - Minigame upgrades
 - Gambling upgrades
 - Achievement upgrades
- I need to program the purchasing system and ensure purchased upgrades actually apply effect
- I need to make sure the costs of upgrades and workers increase as the player buys more of them
- I need to develop a multi-purchase feature, which allows the player to purchase multiple of the same upgrade at once. There will be 4 levels:

- **x1** – 1 of the upgrade is purchased at once
 - **x10** – 10 of the upgrade is purchased at once
 - **x100** – 100 of the upgrade is purchased at once
 - **MAX** – The max number of upgrades that the player can purchase with the money they have is purchased
- I need to make the multi-purchase feature take into account the price scaling, where the price of an upgrade increases each time it is purchased. For example, when purchasing 10 of an upgrade, the price of this should be the same as if the player purchased it 10 times individually.

Why this stage here:

- Now that the player can earn money, and there are workers that automatically increase their population, they need a shop to buy these workers and spend their money on.
- The player will not be able to progress much without a shop to buy upgrades from

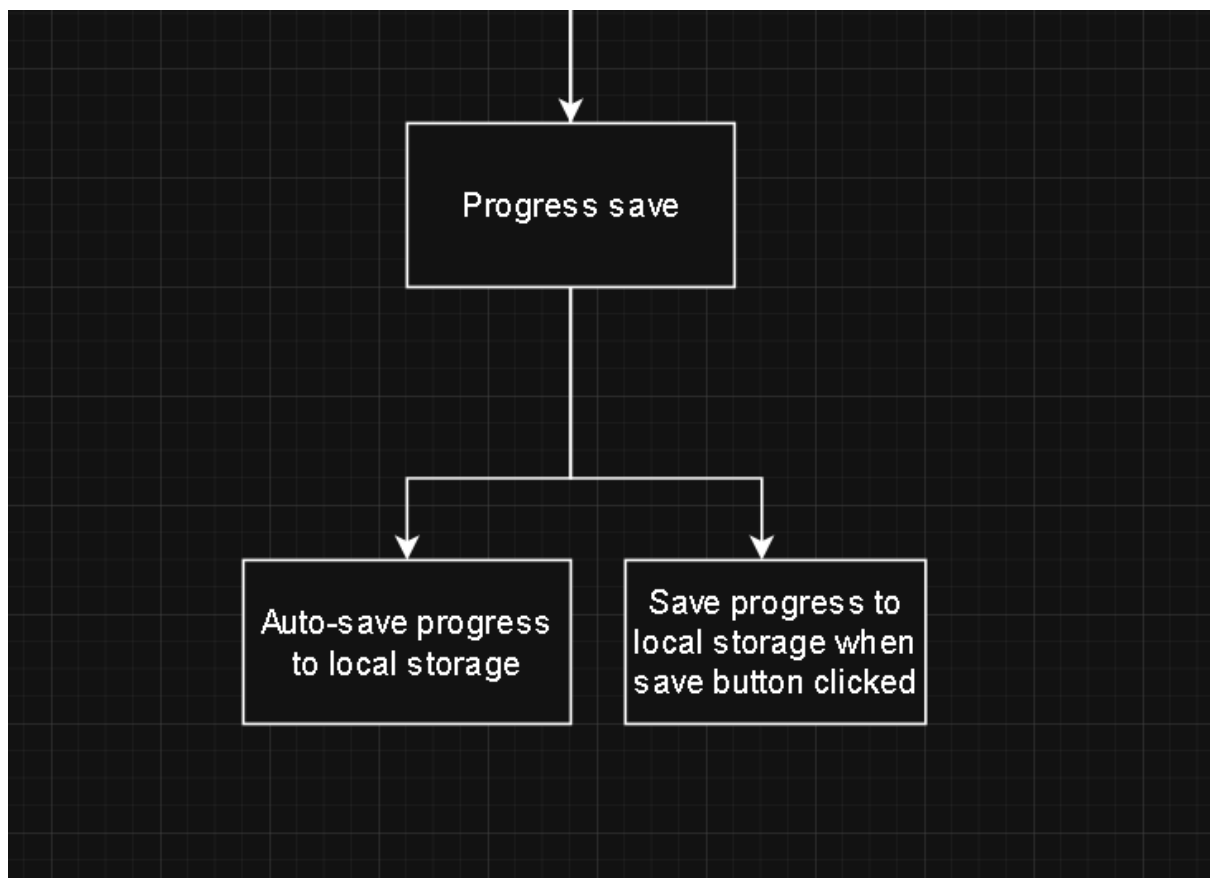
Test plan:

Test no.	Description	Test data	Expected Result	Actual result
1	Is the player prevented from buying upgrades when they don't have enough money	Attempt to buy an upgrade by clicking one in the shop with no money	Nothing should happen, upgrade number for that specific upgrade should not increase	
2	Is the player able to	Click an upgrade in the shop with enough money to	The upgrade number for the upgrade clicked	

	purchase an upgrade	buy it to see if the upgrade number increases	should increase by 1.	
3	Does buying upgrades correctly reduce the player's money	Click an upgrade in the shop with enough money to buy it and observe the money display.	The money should decrease by the price of the upgrade	
4	Do the shop upgrades correctly apply their effects	Buy any upgrade in the shop	The upgrade should do what it says in the name of the upgrade, for example double money should double the money per second	
5	Does the multi-purchase feature correctly calculate the price of purchasing the upgrade 10 times	Click the multi-purchase button to x10 and see if the price of the upgrades increases to the price of purchasing them 10 times	The price of the upgrade should increase to the price of purchasing the upgrade 10 times (with price scaling)	
6	Does the multi-purchase feature correctly calculate the max number of times the upgrade can be purchased, and the price of doing so	Click the multi-purchase button to MAX and see if the price of the upgrades increases to the max amount that can be purchased, and to see if the number on the right of the upgrade is the max number of times it can be purchased	The price of the upgrade should increase to the price of purchasing it the max number of times possible with the current money. The number on the right of the upgrade should be the number of times it can be purchased	

I have chosen this test data to test the upgrade purchase is handled correctly, so that the player can only purchase upgrades when they have enough money, and that the upgrades correctly apply their effects, to ensure the player is getting the boost from their purchase. This will test whether the game meets success criterion 3, which is that game includes a shop system where the player can purchase upgrades using their money, like worker, population and money upgrades. I have also made sure the test data ensures a test to check the multi-purchase feature works by testing that the prices and upgrade numbers are correctly calculated based on the multi-purchase setting, as this is an additional feature of the shop, outlined in success criterion 4.

5. Progress Saving System:



This is the structure diagram section for the progress saving, which I have as an individual component since it has nothing to do with the main game loop, and is not an active or purchase feature. It simply saves the player's

progress, so should be developed individually as it just requires storing all the data in local storage. I have broken down this section into 2 sections, autosave and manual save by save button, since these are the 2 components of the the saving feature and can be developed individually but use the same method of saving the game data.

Description:

- I need to implement the storage of player progress in local storage, I will do this by taking all data that links to the player's progression and things they have unlocked (which will be stored in an object), and store it in local storage, which are:
 - Money
 - Population
 - Workers purchased
 - Upgrades purchased
 - Gems
 - Level
 - Achievements
 - Planets unlocked
- I will make the progress save every 5 seconds, and also have a button that the player can press to save their progress whenever they want

Why this stage here:

- Now that the player can make meaningful progress, they need a way to save it

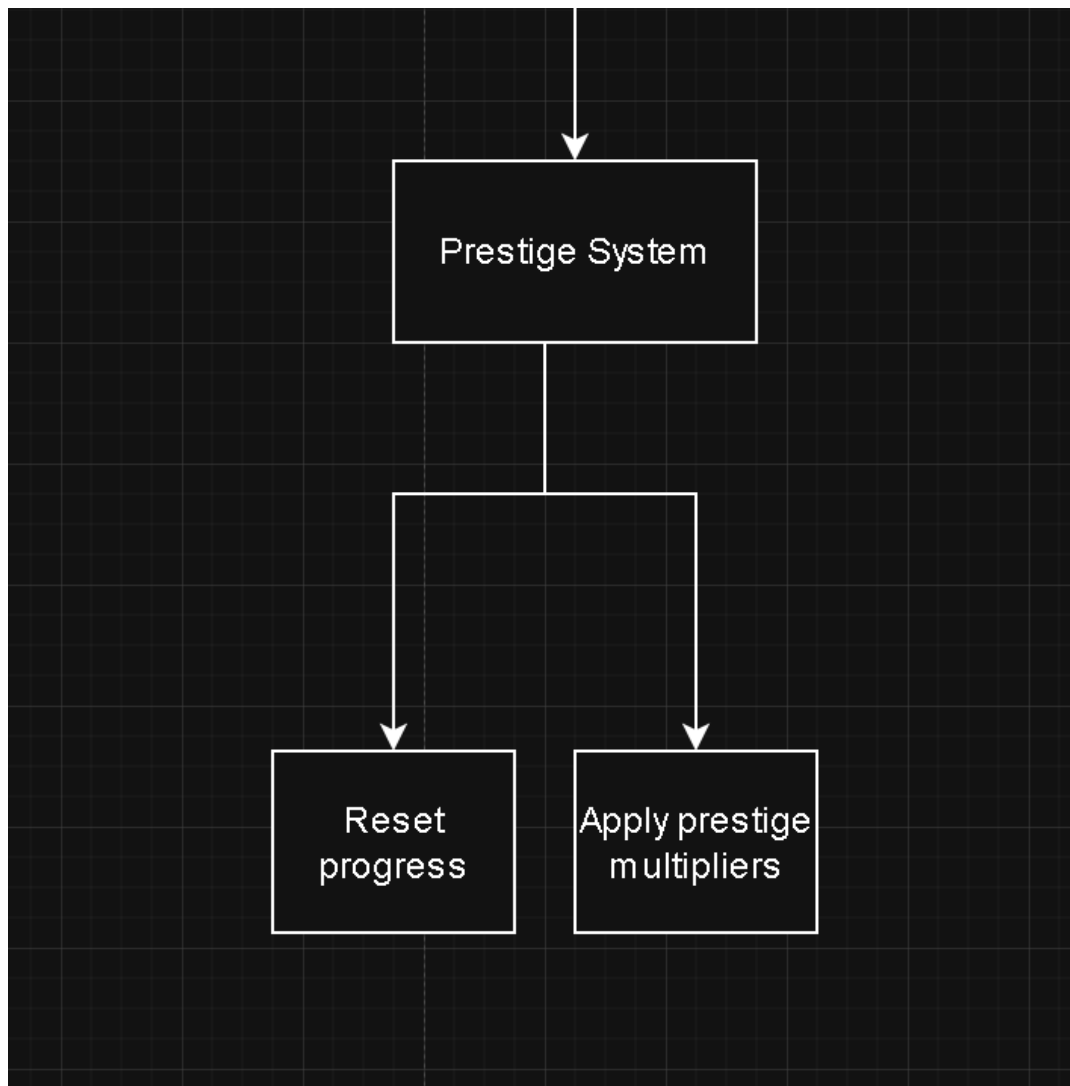
- Also, developing this feature will make testing much easier as I won't have to keep restarting the game to test features that occur later on the game. I can make different saves for different progress levels and test them easily.

Test plan:

Test no.	Description	Test data	Expected Result	Actual result
	Does the player data like money, population and workers save to local storage correctly	After getting some money and at least 1 worker, close the browser, re-open it and go onto the game.	The money, population, and worker count should be the exact same as before the game was closed	
2	Does the save button correctly save game data to local storage	After getting some money and at least 1 worker, press the save button and instantly close the browser	The money, population, and worker count should be the exactly same as when the game was closed	

I have chosen this test data to check that both the core components of the saving feature are functioning. I will test that the game data is correctly saved automatically, and that it saves when the player clicks the save button. This will test whether success criterion 13 is fully met.

6. Prestige:



This is the structure diagram section for the prestige feature, which I have as an individual component as it is an individual game feature with one purpose, that should be developed separately. I have broken it down into the progress reset, and the application of the prestige multiplier from prestiging. This is because that is the 2 functions of the prestige mechanic, the player resets their progress, and then they get a multiplier that boosts their progress. These can, therefore, be developed individually within the prestige stage.

Description:

- I need to develop the option for players to reset their progress, and give them a population multiplier that makes progress easier than before
- The prestige multiplier will depend on the player's population, so the higher their population, the higher the population multiplier will get when they prestige.
- I need to also make prestiging require a certain population, so the player can't prestige as much as they want, whenever they want. The population that will be required to prestige will increase with each prestige.
- I will also make the game keep some of the players progress, so the reset is not too hard. The player will keep their:
 - Gems
 - Achievements
 - Level
 - Prestige upgrades if appropriate

- Why this stage here:

- Now that I have implemented the core mechanics and the saving feature, I need to start adding features that extend the life of the game. Therefore, I will implement prestiging here so I can start to make the game into a long term game.

- Test plan:

Test no.	Description	Test data	Expected Result	Actual result
1	Does the prestige multiplier increase as	Click the planet repeatedly to increase	The population multiplier gradually	

	the population increases	population and check if the population multiplier increases	increases as the population increases	
2	Is the player prevented from prestiging until they reach a certain population	Attempt to prestige with no population to see if it is allowed	Nothing should happen; the player should not be allowed to prestige	
3	Does the game reset the player's progress after prestiging	Prestige with enough money to see if the game resets the player's money, population, workers and upgrades (not including prestige upgrades)	The game should reset the player's money, population, worker count and upgrade count to 0, with all upgrade and worker effects removed.	
4	Does the player keep their achievements, gems, level and prestige upgrades after they prestige	Prestige with enough money to see if the game keeps the player's achievements, gems, level and prestige upgrades	The player's achievements and their completion, gems, level and prestige upgrades should remain unchanged	

I have chosen this test data to check that the core features, according to the structure diagram, of the prestige feature functions correctly, ensuring the game data is fully reset, the population multiplier is correctly calculated and applied, and that the player can only prestige when they meet the population requirements. This will test whether success criterion 6 is met.

7. Population gambling mechanic



This is the population gambling section of the structure diagram, which I have as an individual component as it is also an individual game feature, that should be developed separately as it has one purpose, to allow the player to gamble. I have broken this component down into the actual gamble, which I then break down into population reward or loss based on whether the player wins or loses. This is the main component of the gamble feature, so should be individually developed. I also have a component for the win probability, as a separate component as this determines whether the player wins or loses. Therefore, the gamble component handles what

happens when the player wins or loses, and the win probability component handles the chance of them winning or losing.

Description:

- I need to make the gambling feature, where players can gamble their population with a chance of losing it or multiplying it by a win multiplier
- The odds will start at 50%
- The player should not be allowed to gamble more population than they currently have
- I will make a display that shows the player, before they actually gamble, how much population they will get back if they win.
- I will make a different sound play, depending on whether the player wins or loses.
- I will make some way to tell them whether they have won the gamble or lost it.
- I need to then make the population actually update when the player wins or loses

Why this stage here:

- This is smaller, optional feature that I can add once the base game is working, as it requires the population mechanic to be working.

Test plan:

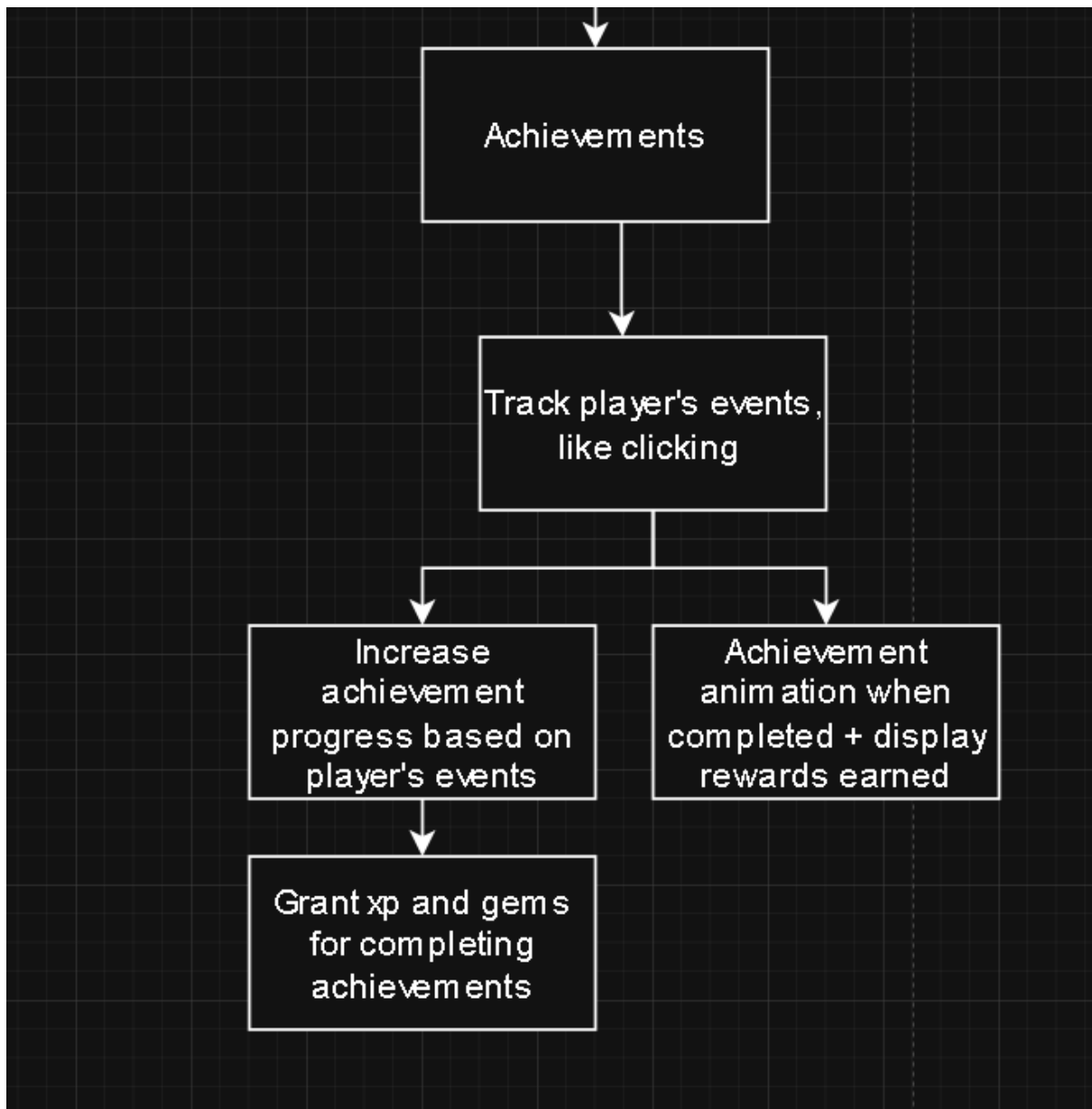
Test no.	Description	Test data	Expected Result	Actual result
1	Does the game	Enter an amount of	The win return display should	

	correctly show how much population the player will win once they enter their gamble	population to gamble in the gambling section and look at the return display	display a number that is the population entered multiplied by the win multiplier, which will be 0.5 (50%) at the start.	
2	Does the player receive the correct amount of population when they win a gamble	Gamble a certain amount of population , remembering the population before the gamble, until a win	The population should increase by the amount displayed by the “win return” display.	
3	Does the player lose the correct amount of population when they lose a gamble	Gamble a certain amount of population, remembering the population before the gamble, until loss.	The population should decrease by the amount that was gambled	
4	Does the player win a random number of gambles	Gamble 10 times and count the number of wins and losses	Around 50% of gambles should be wins, and 50% should be losses. There should be no pattern	
5	Is the player prevented from gambling when they have entered more population than they	With 0 population, enter 1 in the gamble amount form and try to gamble	Nothing should happen. There should be no win or loss message, and the population should not increase or decrease	

	currently have			
--	-------------------	--	--	--

I have chosen this test data to check whether the gamble feature functions properly and the player is granted the correct reward for winning and loses the correct amount for losing. I also make sure to check whether the win probability is being correctly applied, to ensure the player doesn't win or lose all their gambles, as this would not be fair. This will test full functionality of the feature, thus whether success criterion 10 is met.

8. Achievements



This is the structure diagram section for achievements, which I have as an individual component since it is an individual feature, that is not an active or purchase feature, it simply tracks progress and grants rewards, so should be developed individually from the active features that it tracks. I have then broken this component down into player event tracking, as this is the main function of the achievements, to track progress. I then break this down into increasing progress when actions are completed, and then further into granting xp rewards. This is one of the core features of achievements, so should be separately developed within the section. I have also broken

down achievement tracking into the animation and reward display for achievement completion, since this is separate from progress increase and just is just to display the completion to the player, so should be a separate component

Description:

- I need to make achievements that the player reaches once they make certain progress
- I need to make the goal of each achievement get harder each time they complete it
- I will make each achievement progress bar show the player how much progress they have made in it
- I need to make a progress counter that increases as the player makes progress
- I need to implement the rewards for achievements, like multipliers and gems, which will get better as the player completes more achievements
- When the player completes an achievement, I will make a simple animation, where the achievement progress bar will light up and a sound will play.
- I need to restore achievements after prestige, so the player keeps their achievement progress and completion

Why this stage here:

- Now that the player can make progress through all the features I have added, I now need to make achievements so that progress can be tracked
- Once I add achievements, I can test them straight away by using the earlier features I developed to check the progress tracking

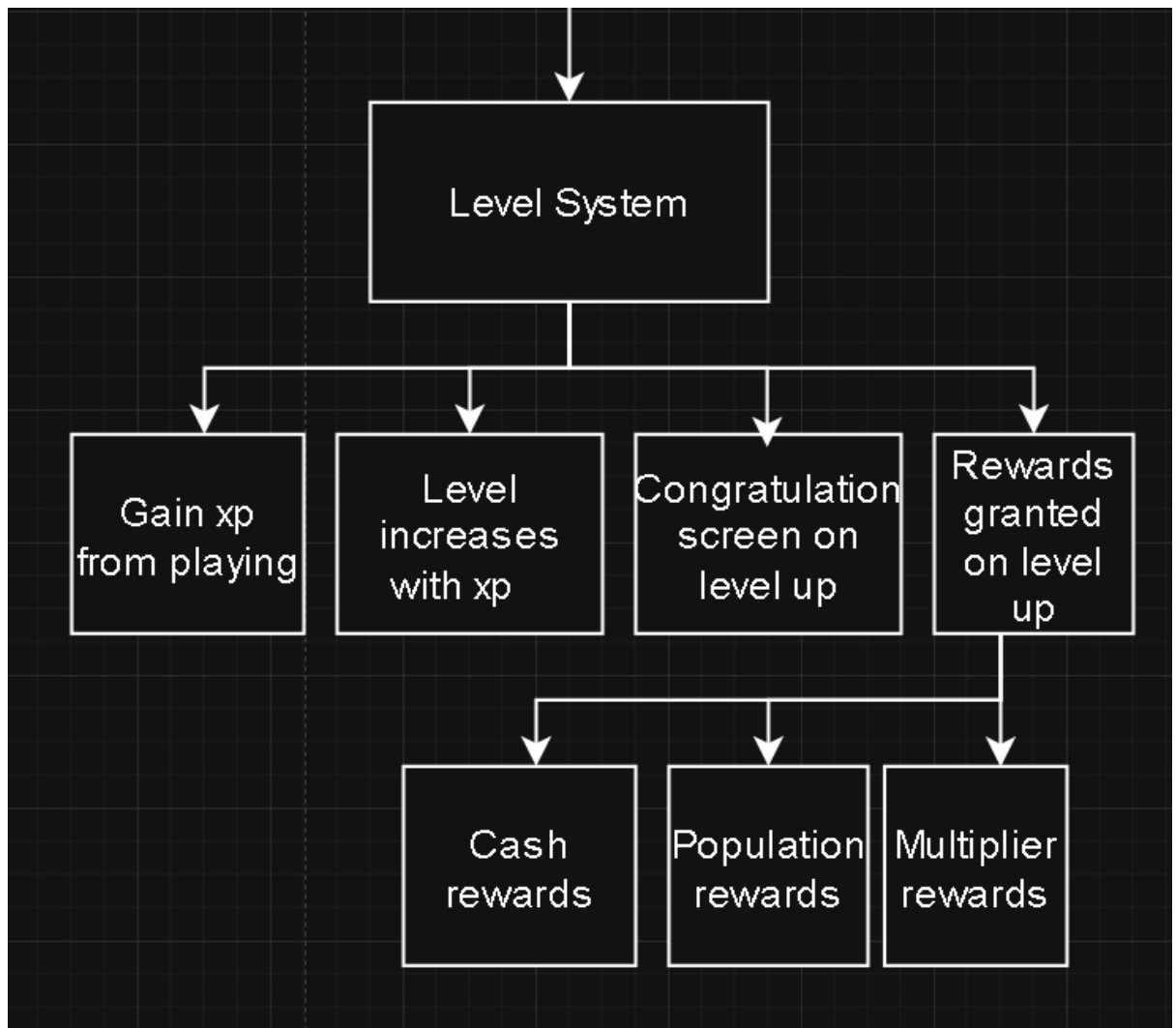
Test plan:

Test no.	Description	Test data	Expected Result	Actual result
1	Do the achievement progress bars and progress counts increase correctly	Complete an achievement action and look for the progress bar and look at the progress count	The progress bar should increase as the action is completed, and the progress count should increase	
2	Does the achievement progress bar and progress counter reset when an achievement is complete	Complete an achievement and look at the progress bar and progress count	The progress bar and progress count should reset to 0	
3	Does the achievement goal scale when the achievement is completed, and is this update reflected in the achievement title	Complete each achievement once and look at the achievement goal value, and the achievement title	The achievement goal should scale and the achievement title should update with the achievement goal.	
4	Do achievements	Complete an achievement	The the xp in the level display and	

	correctly reward the player with gems and xp	and look at the level and gem display	the gems in the gem display should increase	
5	Are achievements and their completion restored after prestige	Prestige with completion of all achievements and observe them	Nothing should happen to the achievements, their progress bars and progress count should remain exactly the same	

I have chosen this test data to test the achievements are fully functioning, to check that they track progress correctly, scale with completion, and correctly grant rewards. This will test the full functionality of the achievements according to the design and whether all of success criterion 11 is fully met

9. Level system:



Description:

- I need to implement a level system, where the player has a level that increases as they earn xp.
- I will make the player will start at level one, needing 1000 xp to get to the next level.
- I need to make the amount of xp that the player needs to get to the next level will increase exponentially, making it harder and harder to level up each time
- I also need to implement a level up congratulations screen, which displays the rewards earned from levelling up.

- The rewards for levelling up will be a certain amount of population, and some gems.
- The rewards for levelling up will increase each time the player levels up.

Why this stage here:

- At this point, I would have created the xp system where the player can earn xp, as well as an achievement system where the player completes achievements to get more xp. Therefore, it makes sense to add the level system at this stage so the xp the player earns will go towards their level
- It will also be easy to implement this once I have the xp set up, as I just need to increase a progress bar based on how much xp there is, and create level up screens that show when the player levels up
- I also want to develop this stage before the minigame feature, so that I have a level system in place for minigame wins to contribute to via xp

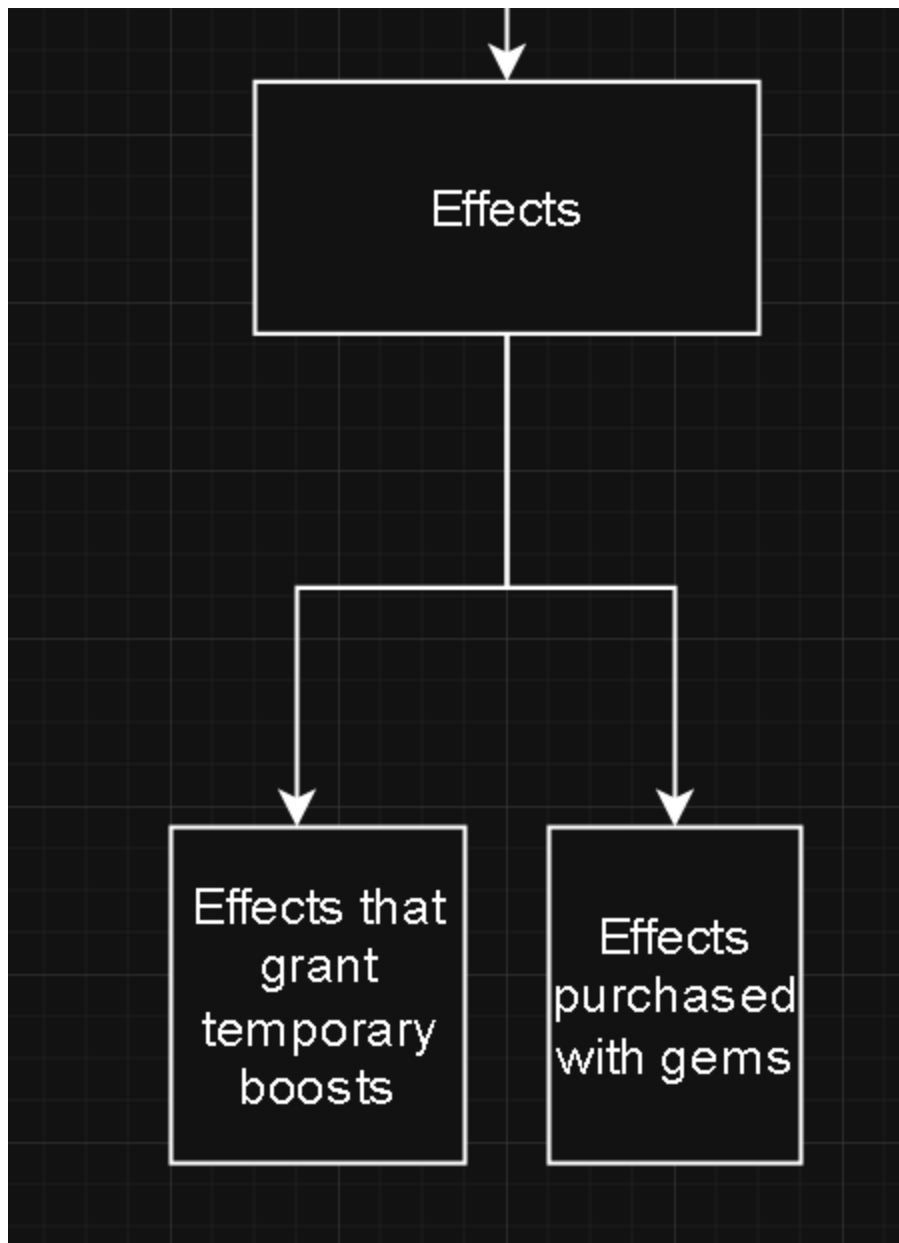
Test plan:

Test no.	Description	Test data	Expected Result	Actual result
1	Do the level progress bar and xp count increase when xp is granted	Complete an achievement and observe the level progress bar and xp count	The level bar should increase slightly, and the xp count should increase to by the achievement reward value	

2	Does the level progress bar and xp count reset correctly after level up	Level up and observe the level progress bar and xp count	The level progress bar and xp count should reset to 0	
3	Does the level goal scale correctly after level up	Level up and observe the level goal value	The level goal value should scale, increasing to a higher number	
4	Does levelling up correctly reward population and gems	Level up and observe the population and gem increase	The population and gems should increase	
5	Does the level up congratulations display correctly	Level up and observe the screen	A level up screen should pop up over the main game, displaying the rewards earned	

I have chosen this test data to check whether the level system is fully functioning, with correct level scaling, the progress being correctly updated with xp, level ups correctly granting rewards, and the level up screen displaying correctly. This will test whether the correct functioning of the level up system according to the design, and that success criterion 14 is fully met.

10. Effects:



This is the structure diagram section for the effects, which I have as an individual component since it is an individual purchase feature, the only feature that contains items purchased with gems. Therefore it should be developed individually. I have broken it down into the 2 main components, which are the actual effects and their temporary boosts, and the purchase handling via gems. This allows me to develop the actual effects and what they do, and then the purchase handling, separately. Therefore, I can develop the effects, and then allow the player to purchase them with gems.

Description:

- I need to add effects that apply temporary boosts when purchased
- I will make effects cost gems
- Effect prices won't scale, they will always stay the same

Why this stage here:

- At this point, I will have implemented gems, and a level and xp system that grant the player gems
- Therefore, I want to make the effects which the player can spend these gems on, otherwise the gems would be pointless.

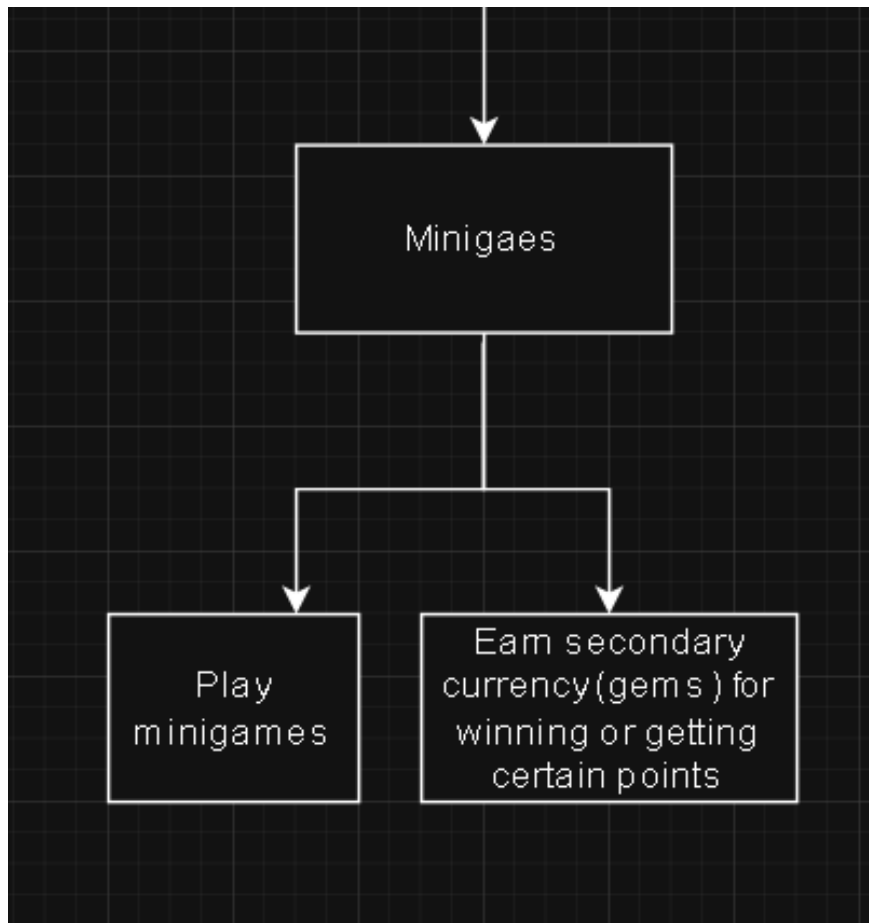
Test plan:

Test no.	Description	Test data	Expected Result	Actual result
1	Do the effects correctly apply their effect	Purchase each effect and see whether they correctly apply their multiplier/boost	Each effect should apply their unique effect after purchase	

I have chosen this test data to ensure the effects are functioning correctly. Once I design the specific effects, I will modify this test table to test for the functionality of each specific effect, but for now I just have a general test to ensure effects function correctly, which tests the final part of success

criterion 9 is fully met, “which (gems) can be spent on temporary effects/multipliers”

11. Minigames:



This is the structure diagram section for the minigames, which I have as an individual component, since they are an individual, active feature, that allow the player to play games, so it is a unique feature and should be developed individually. I have broken it down into the 2 main components, which is playing the minigames, i.e. opening the minigame player and running the game loop, since this is the main component and should be developed individually from the second component of the section, which is the granting of the gem rewards, which should be separate as it comes after the minigames are played.

Description:

- I need to add simple minigames which the player can play
- I need to create a screen for minigames, which will appear over the main game (as an overlay), allowing the player to play the minigame whilst the main game runs in the background
- I need to create a win and loss screen, which will be displayed to the player depending on whether they win or lose the minigame.
- The win screen will also display the gems and xp that the player earned from the minigame.
- The 3 minigames that I will create are:

- **Speed clicker**

- The player has 10 seconds to click a planet as many times as possible
- A timer is displayed to show the time running out
- A counter is displayed to show how many times the player has clicked the planet
- At the end, the player's reward is calculated based on the clicks they got.

- **Memory Game:**

- A grid of 8 cards are displayed, face down and the player flips 2 of them at a time, trying to match pairs
- There is a time limit for the game, and if the player matches all the cards withing this time limit, then they win

- **Planet guesser:**

- The player is shown all 8 planets in the solar system
- A random planet is chosen by the computer, and the player has 2 tries to guess it
- If the player guesses the planet correctly, they win, but if not then they lose

- I need to also implement minigame cooldowns, so the player can't just play minigames whenever they want and get loads of rewards. The cooldown will be a timer, the length of which will depend on the difficulty I make all the minigames:

- Planet guesser (easiest) – 20 minute cooldown
- Speed clicker (medium) – 10 minute cooldown
- Memory game (hardest) – 5 minute cooldown

Why this stage here:

- Minigames are also an optional side feature that the player does not have to play to make progress
- Now that I have the base game sorted, I can add the extra features like minigames that user has the option to play
- The level system and effects will also be in place at this point, the the xp and gem rewards from winning minigames can actually go towards something

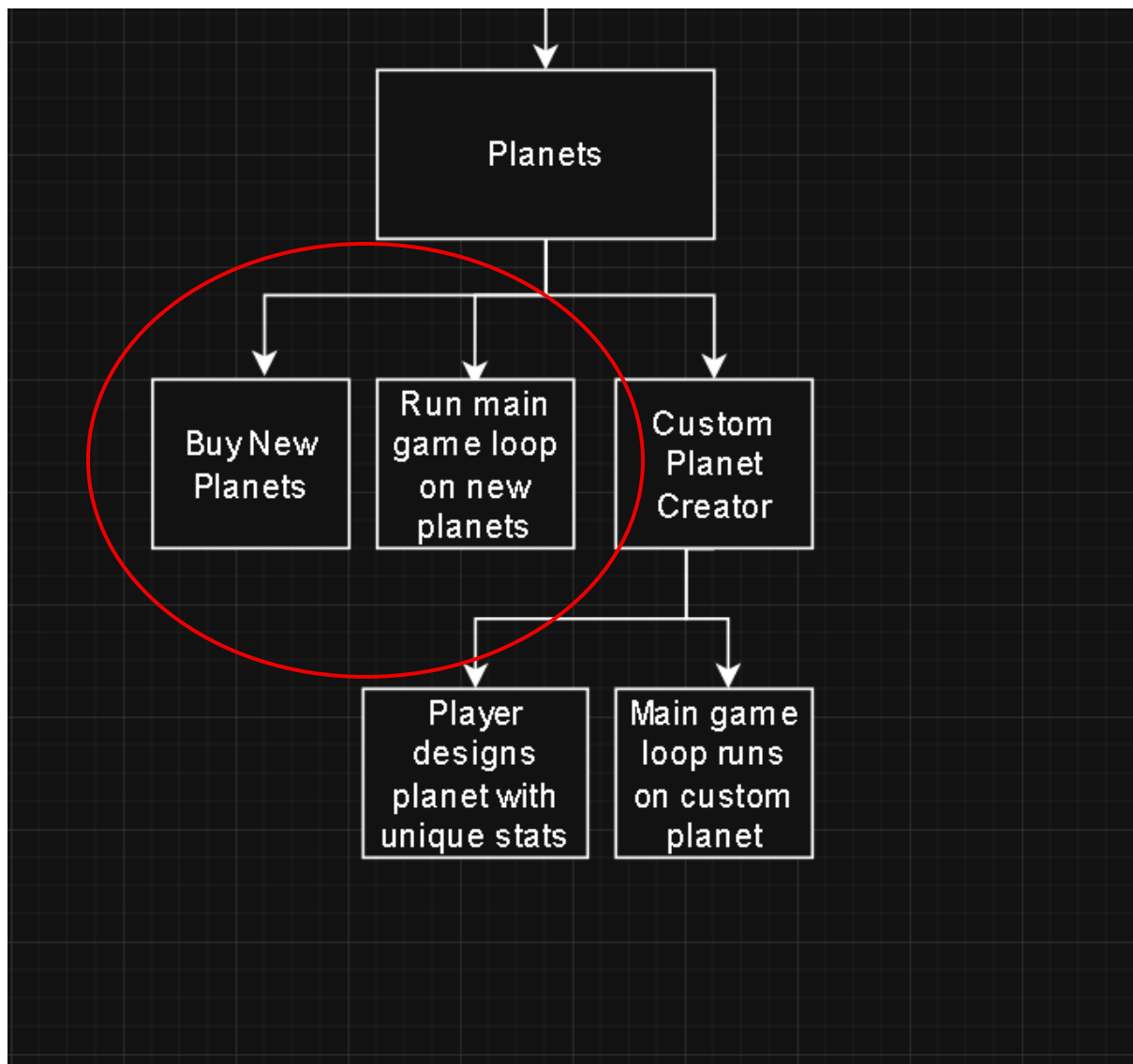
Test plan:

Test no.	Description	Test data	Expected Result	Actual result
1	Does the minigame screen pop up when a minigame is pressed	Press a minigame button	A minigame screen should pop up, overlaying the main game. It should show the minigame name, a container for the actual game, and a container with the game stats	
2	Does the speed clicker minigame minigame function correctly	Press the speed clicker minigame and click the planet button that appears	After 10 seconds, the minigame score screen should pop up, showing gem and xp rewards based on how many times the planet was clicked	
3	Does the memory minigame function correctly	Press the memory game minigame button and press different cards. Flip 2 of the same cards to see if they are removed.	Cards should flip upright when clicked, and the game should only allow 2 cards to be flipped at the same time. Flipping 2 of the same cards should remove them.	
4	Does the player win the memory minigame when they match all cards within the time limit, and lose if they don't	Match all the cards correctly before the timer runs out. Then play again and don't match them all in time.	For the first time, minigame win screen should pop up, showing the rewards. For the second time, the minigame loss screen should pop up.	
5	Does the planet guesser minigame function correctly	Press the planet guess minigame and guess a planet	If the correct planet was guessed in 1 or 2 guesses, the minigame win screen should pop up, but if	

			not then the loss screen should show	
6	Do minigames correctly grant xp and gem rewards	Win a minigame and observe the gem and xp increase	The gems and xp should increase by the reward amounts displayed in the minigame win screen	
7	Are the minigame cooldowns applied correctly	Win the speed clicker minigame and attempt to play it again Win the memory game minigame and attempt to play it again. Win the planet guess minigame and attempt to play it again.	The game should only allow the speed clicker to be played again after 10 minutes, the memory minigame after 5 minutes, and the planet guess after 20 minutes	

I have chosen this test data to check whether the minigames are functioning correctly, including whether the minigame player pops up when a minigame button is pressed, to ensure the player can actually play the minigames, and that each specific minigame works as planned, with cooldowns being correctly applied between games. This will test whether minigames work exactly according to their design, and thus whether success criterion 8 is fully met.

12. Multiple Planets



This is the structure diagram section for the additional planets, which I have an individual component, since it modifies the way the game looks and then runs the main game loop, so should be an individual component that then uses the code from the main game loop to allow the player to replay the game on a different planet. I have circled the components for the additional planets, since I have also broken down this section into the custom planet creator feature, since this is also a planet feature, but will be covered in the next stage.

Description:

- I need to add the ability to purchase new planets, allowing the player to colonise them in the same way as the previous planet
- I need to make planets require a certain level to colonise. For the next planet to colonise, I will show the level required to colonise (which will be 50 for the first planet after Earth), and for planets after that I will show a lock symbol until the next planet is unlocked.
- When a player switches to a new planet, the game will look the exact same, except the planet image will be that of the planet they are on, the name of the planet in the title will update to the current planet name, upgrade prices will increase, but minigames will grant higher rewards and effects will be stronger.

Why this stage here:

- Now that I have built the core mechanics and optional features, I can develop other planets and just replicate these mechanics and features on them, since they will be the same on all planets, but just will slightly different upgrades and progress.

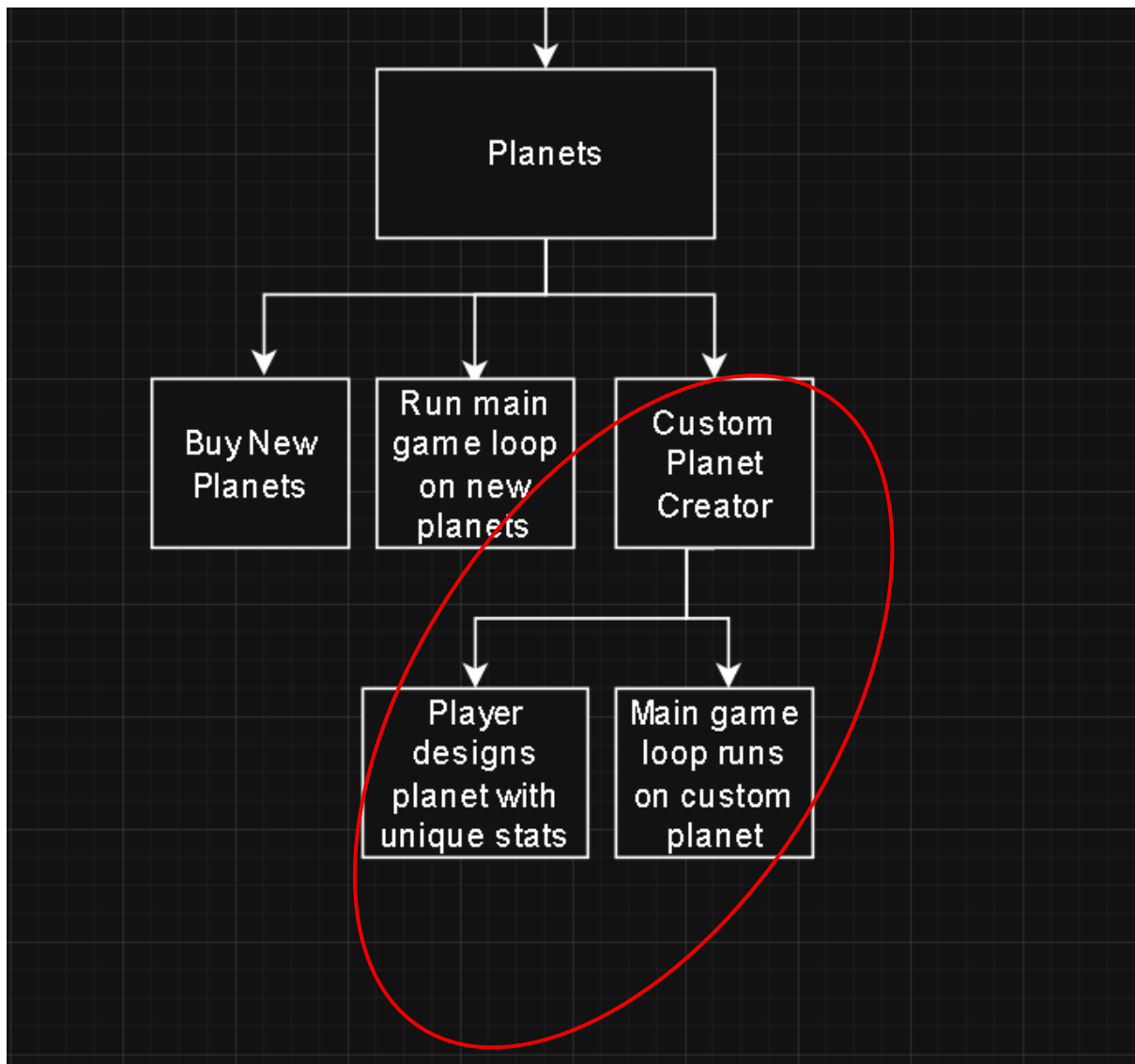
Test plan:

Test no.	Description	Test data	Expected Result	Actual result
1	Is the player prevented from unlocking the next planet if they don't have the required level	Attempt to unlock the next planet with a lower level than the required level, to see if the game allows it	Nothing should happen, the game should not allow the unlocking of the next planet	
2	Does the game correctly switch to	Attempt to unlock the next planet	The game should switch to the next	

	the next planet when the player selects it with a high enough level	with a higher level than the required one, to see if the game switches to it and correctly updates the title	planet. The game should look the same, except the planet in the title should switch to that of the new one.	
3	Does the game correctly increase upgrade prices, minigame rewards, and effect strength/time after switching to a new planet	Take note of the upgrade prices, minigame rewards, and effect strength. Then switch to the next planet and compare them with those of the new planet	The upgrade prices, minigame rewards, and effect strength should have all increased	

I have chosen this test data to check whether the additional planets feature is fully functioning, including the unlock handling to allow unlocking only when the player has the correct level, and that the game works on the new planet, with the main game loop being the same but with modified stats. This will test whether the additional planets work exactly how I designed them, and therefore whether success criterion 7 is fully met.

13. Custom Planets



This is the structure diagram section for the custom planet creator, within the planets section. I have this as an individual component in the planet section, since it is slightly different and allows the player to customise their own planet with unique stats, which will then also run the main game loop on the planet created.

Description:

- I need to develop the planet customisation feature, where the player can add their own planets with unique stats
- I need to make an overlay screen that will appear over the main game, which allows the player to customise their own, unique planet
- The screen will have dropdown menus with different headings, grouping them into the different parts of the planet that they customise. These dropdown menus will be:

- **Planet Basics**
- **Clicking & Workers**
- **Upgrades & Prestige**
- **Gambling & Minigames**
- **Gems**

- Once the player has chosen all the options, they can create their planet and will be taken back to the main screen, where the main game loop will run with their custom settings

- Why this stage here:

- This is an end game feature, which only unlocks once all planets have been colonised, so it should be the final feature
- Without this, the player can still play the majority of the game, just without an extra sandbox feature at the end of the game, meaning it would not be a big problem if I ran out of time for this feature

Test plan:

Test no.	Description	Test data	Expected Result	Actual result
----------	-------------	-----------	-----------------	---------------

1	Does the custom planet creation overlay pop up when the custom planet button is clicked, with the correct drop down menus	Click the custom planet button, then click the different category dropdowns	An overlay should pop up over the game, with different categories for planet customisation: - Planet Basics - Clicking & Workers - Upgrades & Prestige - Gambling & Minigames - Gems Clicking each one should open a dropdown menu with different customisations related to the category	
2	Is the player able to enter custom stats for their custom planet	Click the custom planet button, open the dropdowns and attempt to customise the planet	The dropdown menus should allow the player to enter data, like the name of the planet or population multiplier, and they should be able to select tickboxes like "Infinite money"	
3	Can the player create a custom planet with all their chosen configurations	After customising the planet in the custom planet menu, click the create planet button and play the game	The game should reset, with the custom stats applied	

I have chosen this test data to check whether the custom planet creator is fully working, ensuring that the correct GUI screen pops up, allowing the player to customise their planet and play on it with the normal game loop but with their chosen stats. This will test whether that the custom planets work as I designed them, thus ensuring that success criterion 12 is fully met.

14. Animations

Description:

- I need to add space themed animations and effects, for actions like planet clicking, achievement completion, and gambling.
- There will be button bounce effect for button clicks
- There will be an animation for achievement completion, where the achievement progress bar lights up briefly, with text showing the rewards granted
- There will be an animation for the gambling, where a win/loss message pops up and particles come off it briefly
- There will be an animation for level completion, where the progress bar lights up and particles come off it

Why this stage here:

- This is not an actual functionality of the game, it is just to make it look better so should be left to the end once all the essential features are complete.
- If I ran out of time for this feature, it would not be a big problem since the core game would have been developed, it would just lack some extra aesthetic features that do not add any functionality to it.

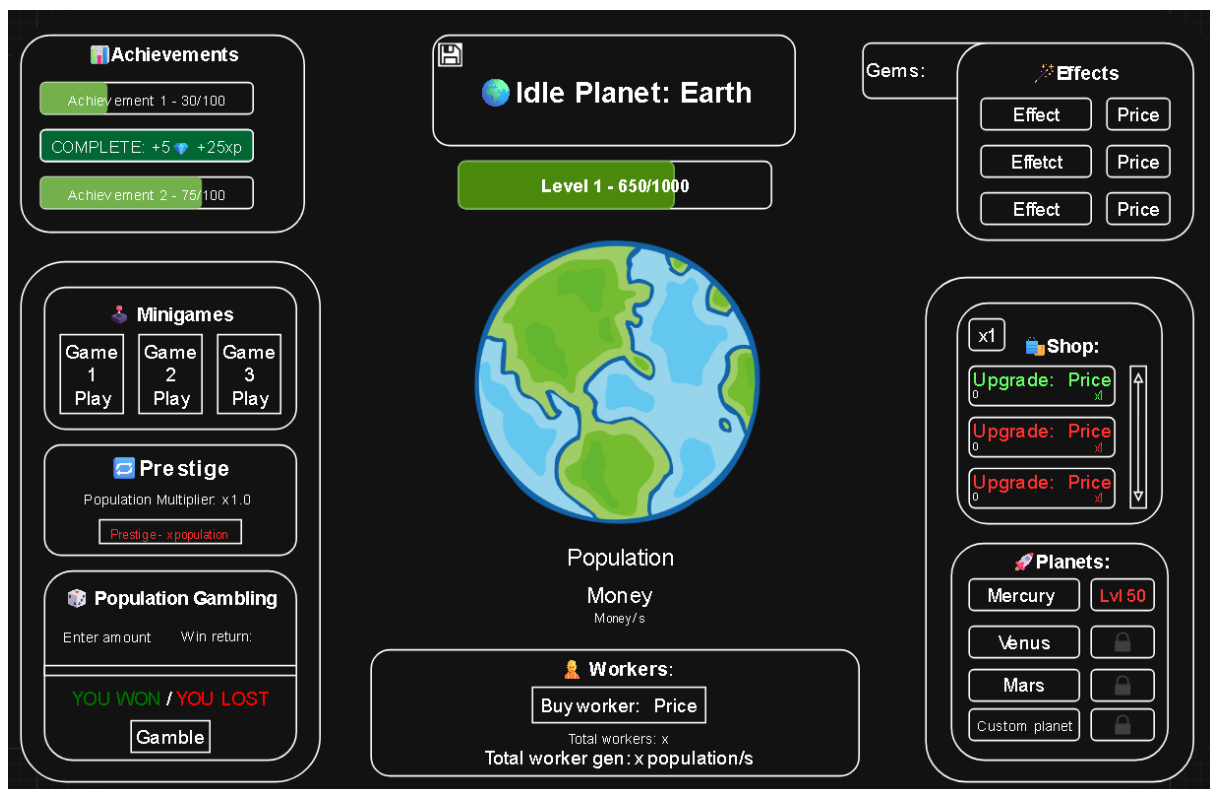
Test plan:

Test no.	Description	Test data	Expected Result	Actual result
1	Are there bounce animations when clicking buttons	Click different buttons and observe them	The buttons should bounce when clicked, where they get smaller and then increase in size	
2	Are there achievement completion animations	Complete an achievement and observe the animation bar	The animation bar should light up, with text popping up over it to show the rewards earned	
3	Are there level up animations	Level up and observe the level bar	The level should bar light up with particles coming of it	
4	Are the animations for winning/losing gambles	Win and lose a gamble and observe the win/loss message	The message should pop up and glow briefly	

I have chosen this test data to test whether the game has a variety of animations, including click, achievement completion, level up, and gamble animations, according to the design. This will test whether the final part of success criterion 2 has been fully met, “space-themed animations”.

GUI Design:

1. Main Game:



Title (Top middle):

This is the title section, displaying the name of the game and the current planet that the player is on. The title is in big text, in the top center, making it clear what game is being played and what planet you are on

Achievements (Top left):

This is the achievements section, which displays the achievements that the user can complete, and how much progress they have made so far in completing them. Since this is not (directly) an active gameplay feature or a purchase feature, I have put it in the top left, as it is not something that the player will be constantly checking and interacting with so can be separate from the gameplay and purchase features. The user can easily check there achievements from time to time by looking to the top left, but it does not get in the way of the main game loop. When

the player completes an achievement, there will be a simple animation on the progress bar, it will light up and display text that says "COMPLETE: ", followed by the reward that the player got from that achievement. The progress bar will then go back to normal, showing the next achievement to complete

Level (Under title):

This is the level section, which displays the player's level and the amount of xp they are into the current level. This is right above the planet, in the top center of the screen, making it easily visible and stand out, which gives the player a sense of achievement as they see their progress increasing at the top of the screen.

Minigames (Under achievements):

This is the minigames section, where the player can select any of the 3 minigames to play, Each button is contained within its own box inside this section, making them easy to differentiate and select the desired one. Game 1, Game 2, and Game 3 are just placeholders and will be replaced with the actual name of the minigame once developed

Prestige (Under minigames):

This is prestige section, where the user can prestige and reset all their progress to earn a population multiplier. The population multiplier that they will receive after their prestige is displayed in this section, and underneath there is a button to prestige. The text of the button is in red to make it stand out, creating a sense of

warning to ensure the player knows that they will lose their progress

Population Gambling (Under prestige):

This is the population gambling section, where the user can enter an amount of their population in the form, and gamble it with a chance of multiplying or losing it. Once the user has entered the amount of population they want to gamble, they can see how much they will get back if they win, allowing them to see if a gamble is worth it to them. To gamble, the player will click the gamble button, and will be presented with either "YOU WON" or "YOU LOST" depending on whether they won or lost. The text colour for the win message is green, as green is associated with winning in this context, and the text colour for the loss message is red as red is associated with losing in this context

Left section as a whole:

The entire left section, apart from achievements which are separated and put in the top left, is dedicated to active gameplay features. There are minigames, which the player actively plays, the prestige feature, which the player actively chooses to use, and the population gambling feature, which is also actively engaged in by the player. Grouping these active features together on the left of the screen, makes them all easy to access and navigate to. When the player wants to use the more active side of the game, they can navigate to the left of the screen and have it all there for them to see and engage with.

Main section (Centre):

This is the main section, where the clickable planet is displayed, which will be a clip art image of either Earth, Mercury, Venus, or Mars depending on what planet the player is on. This section will also contain the planet's population, the player's money, and the money that they are currently earning per second. I have put the population and money right under the planet, which makes it satisfying for the player since they can see how much they're going up as they are clicking. The main section is the biggest section of the game, right in the center, drawing attention to it and making it clear that it is the main part of the game. The planet is the biggest element, as it the core feature of the game loop.

Workers (Bottom centre):

This is the worker section, where the player can purchase workers that automatically increase the population of the planet. Although this is a purchase feature, I have not grouped it with the other purchase features on the right section of the game since workers are a special type of purchase that directly links to the planet, so I have put it under the planet, within the main section, keeping it grouped together with the planet. There is a button to purchase a worker, which displays its current price. Under this, there will be a text display that shows how much population all the workers are generating per second, so the player can decide whether they should buy more or they need to upgrade their workers.

Effects (Top right):

This is the effect section, which displays all the effects (and their price) that the player can purchase with their gems (the secondary currency). Their gems are displayed right next to the available effects, making it easy to see how much gems they have when looking at effects to buy.

Shop (Under effects):

This is the shop section, which displays the upgrades (and their price) that the player can purchase, as well as the number of each upgrade that they have purchased underneath the name in the bottom left. Since there will be lots of upgrade here, There will be a scroll wheel to scroll through them, allowing them all to fit and not be too small or squashed. In the top left, there is the multi-purchase button, allowing the player to purchase multiple of the same upgrade if they have enough money. Clicking the button once will change it to x10, twice to x100 and three times to MAX (Meaning the maximum amount of upgrades that the player can purchase with the amount of money they have, accounting for the price scaling). In the bottom right of each upgrade, there will be a number that shows how much of each upgrade that the the player will be purchasing (which corresponds with the multi-purchase button in the top left of the shop). If this number, and the upgrade name, are in green text then the player has enough money to purchase the upgrade, but if it is in red then they don't. The red and green colours make it obvious to the player whether they can afford an upgrade, due to the connotations of green with money and access and those of red with restriction, meaning they don't constantly have to check their money to see if they can afford

upgrades. If the player selects MAX as the multi-purchase setting, the text in the bottom right of the upgrades will display the number of upgrades that the player has enough money for.

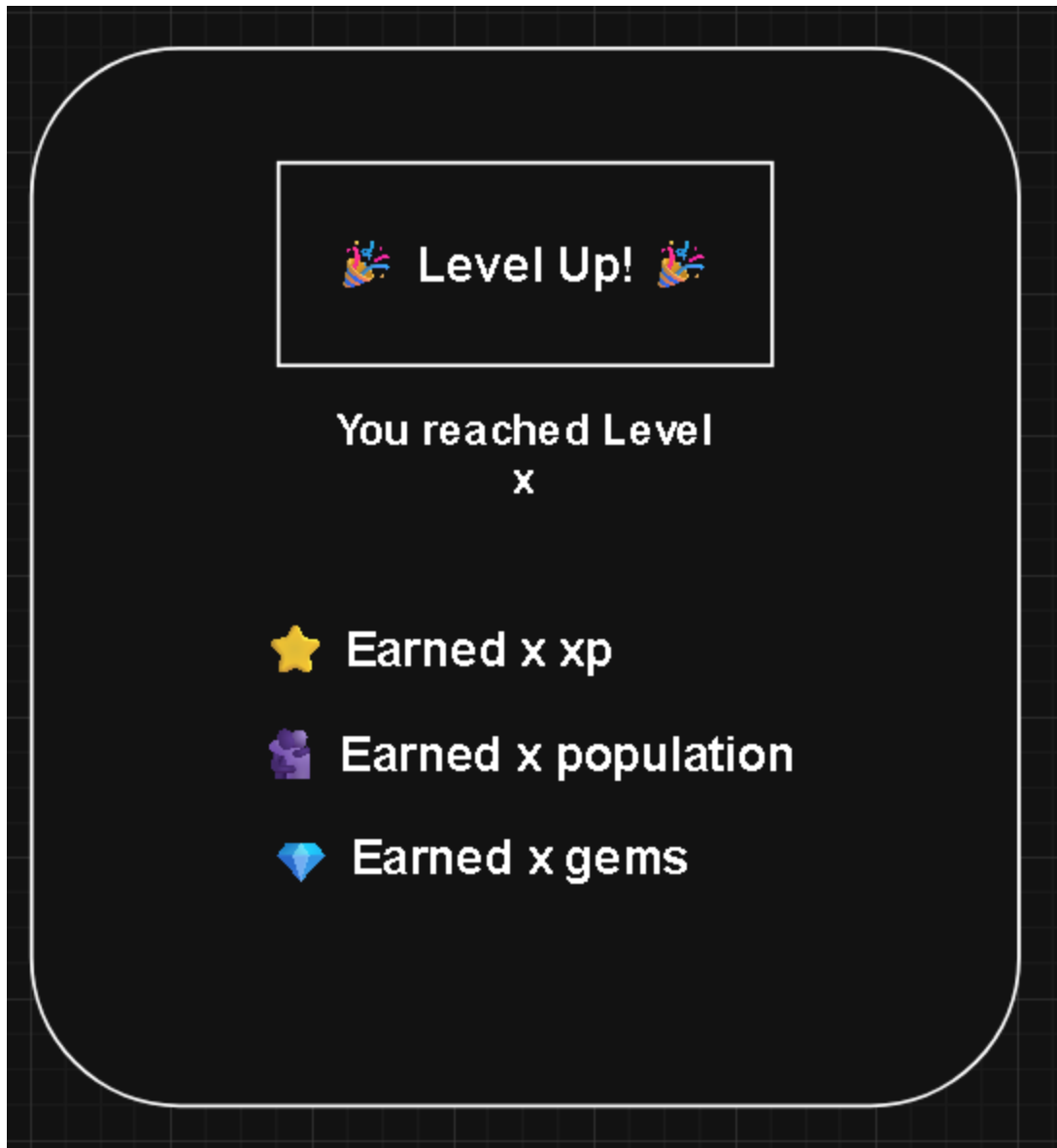
Planets (Under shop):

This is the planets section, where the player can purchase the next planet to colonise, and create their own custom planet if they have colonised all the planets. The level that they need to unlock the planet is displayed next to the upgrade, ensuring they know what they need to achieve to unlock and giving them a goal to hit, and the text colour for this will be red when their level is too low, and green when it is high enough. This makes it clear and obvious to the player whether they can unlock the planet, due to the connotations of green with access and those of red with restriction

Right section as a whole:

The whole right section of the game is dedicated to purchasable items, where the player can buy effects, upgrades and planets. Grouping these features together makes purchases easier, as the player can navigate to the right section of the game to make all their purchases in one go.

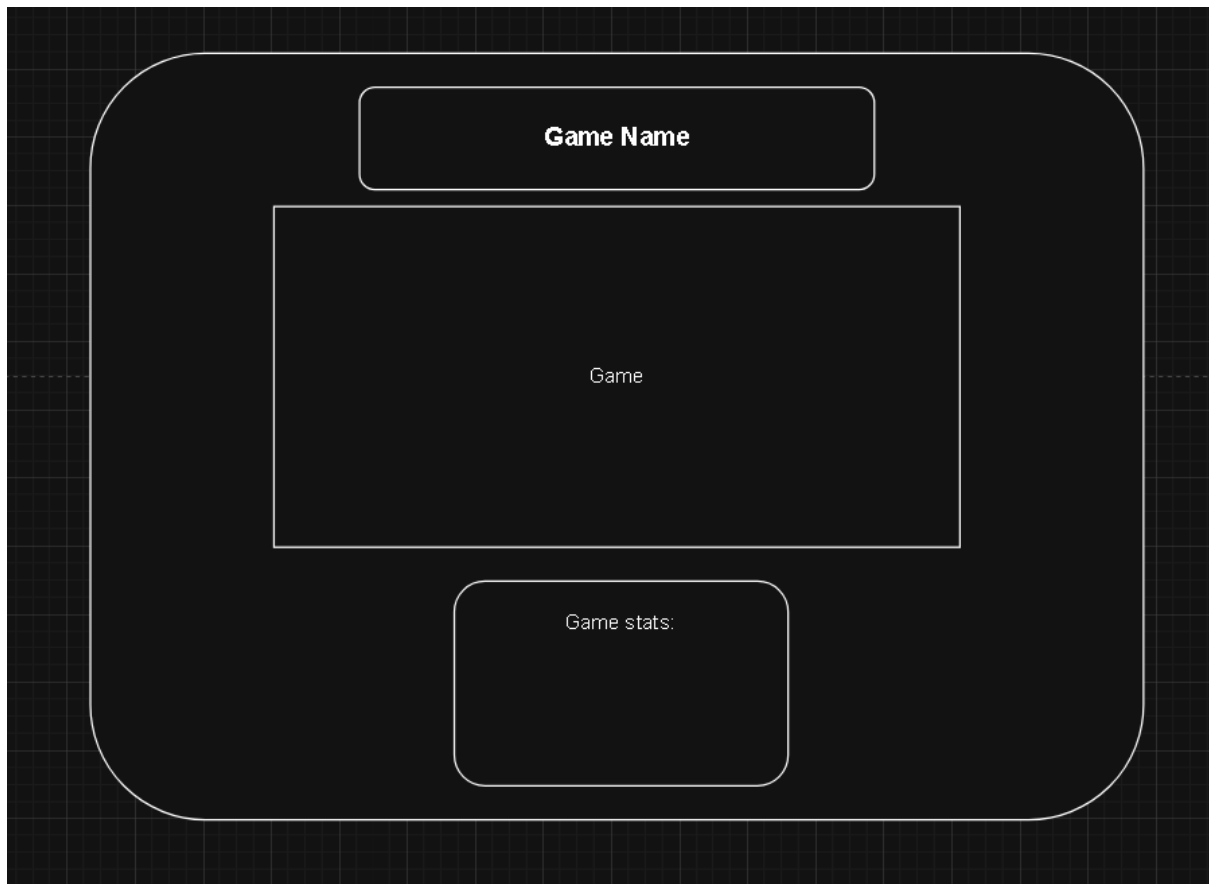
2. Level Up Screen:



This is the level up screen, which appears over the main page when the player levels up. It tells them what level they reached, along with how much xp, population and gems they earned. The emojis make it more exciting and lively, which increases the sense of reward from levelling up. When the player levels up, this screen will appear over the main game, with the game still running in the background.

3. Minigame screens:

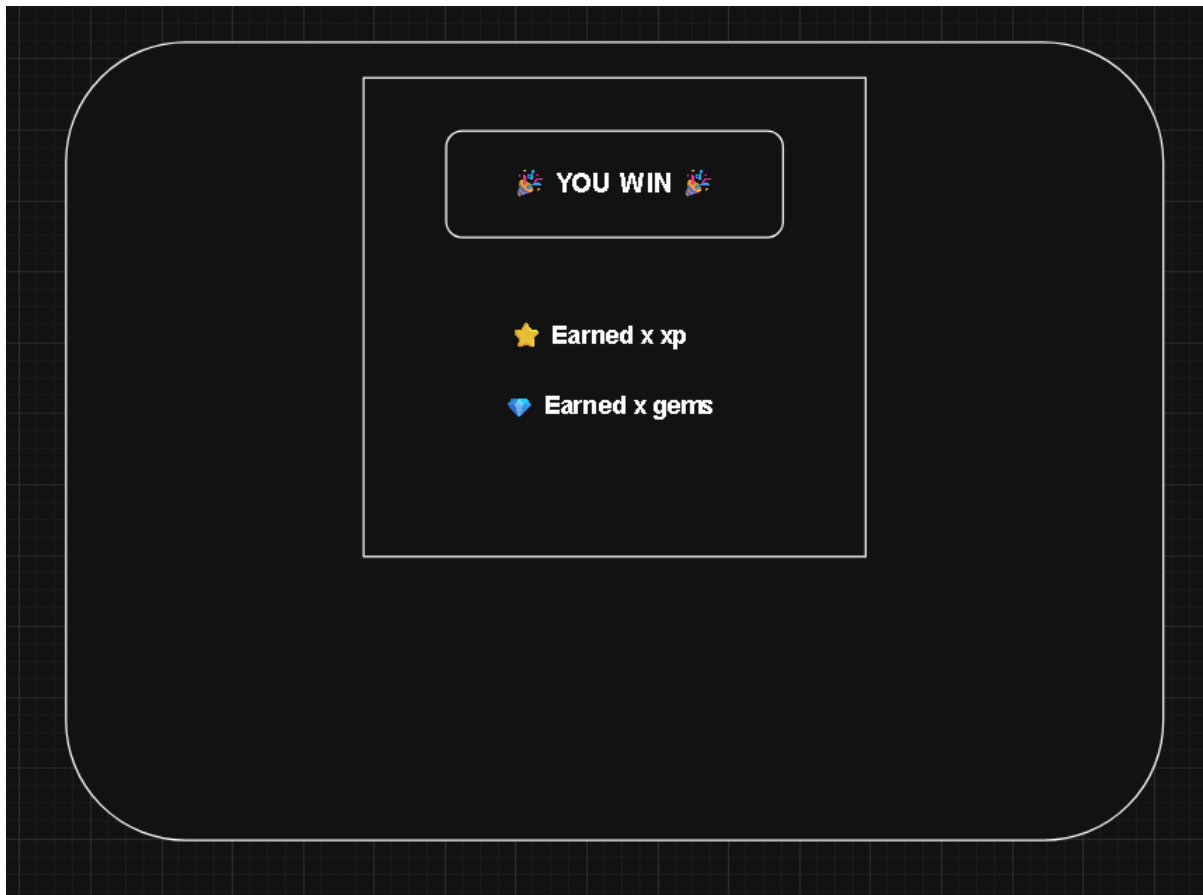
- Minigame screen



This is the minigame screen, where the player will be able to play the minigame they selected. The minigame screen will appear over the main game, with the game running in the background. I have divided this screen into 3 sections. The top section simply displays the name of the current minigame. The middle section displays the actual game being played, and this screen is the biggest out of the 3 as it is the main focus and the player needs to be able to see what they are doing clearly. At the bottom, there is a game stats section, which displays the stats of the game the

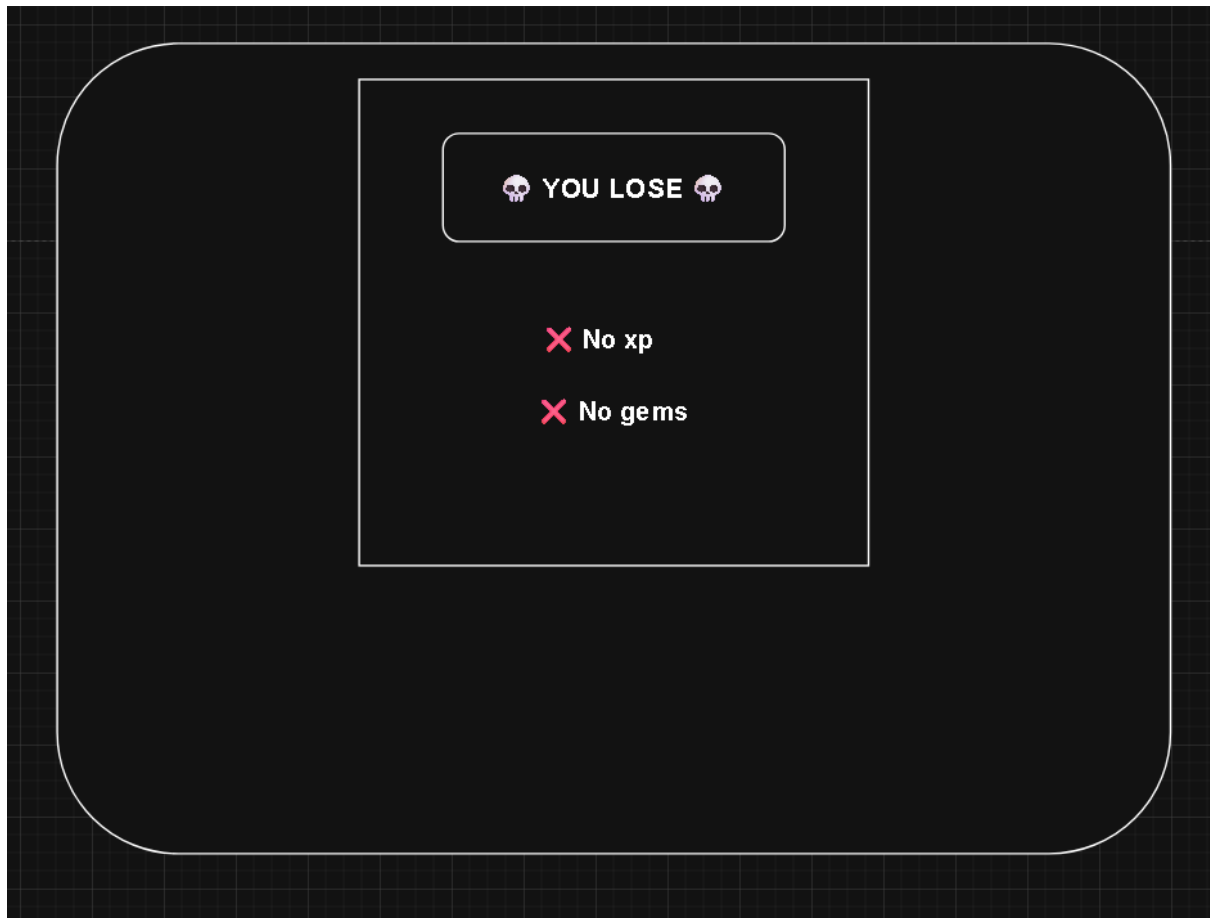
player is playing, allowing the player to easily look at their points whilst still playing, so they know how well they are doing

- Minigame win screen:



This is the minigame win screen, which will display if the player wins the game. It shows them how many gems and xp they earned. The screen has celebration emojis in the title, to make it look fun and rewarding, along with emojis for the rewards to make them feel more special.

- Minigame loss screen:



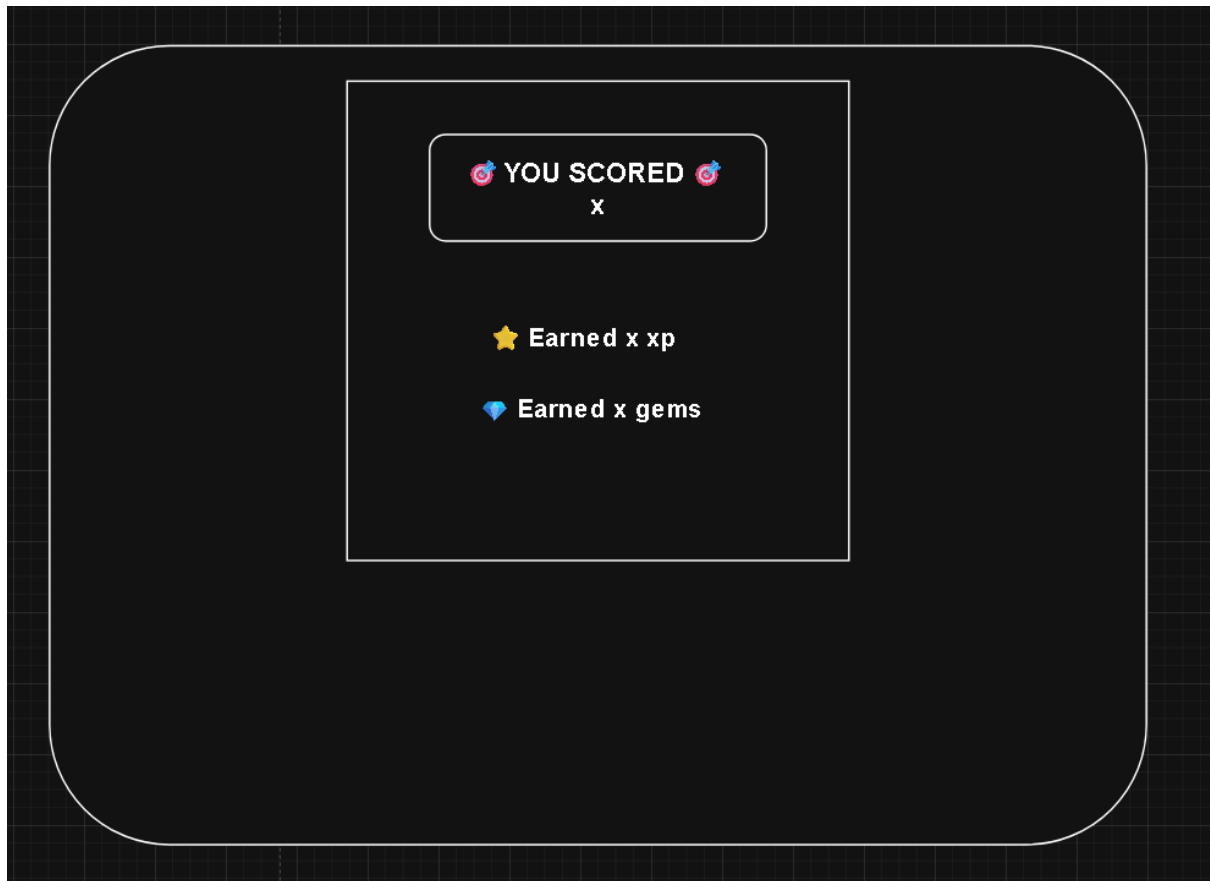
This is the minigame loss screen, which will display if the player loses a game. It shows that they earned no gems or xp. The screen has death symbol emojis for the title and cross symbols for the rewards, which makes it clear that this is a loss and they have not earned any rewards.

Win and loss screens:

These 2 screens are only used for minigames where the player can either win or lose. If they win, they are displayed the win screen and get the set rewards, whereas if they lose they are displayed the loss screen and get no rewards. For minigames that

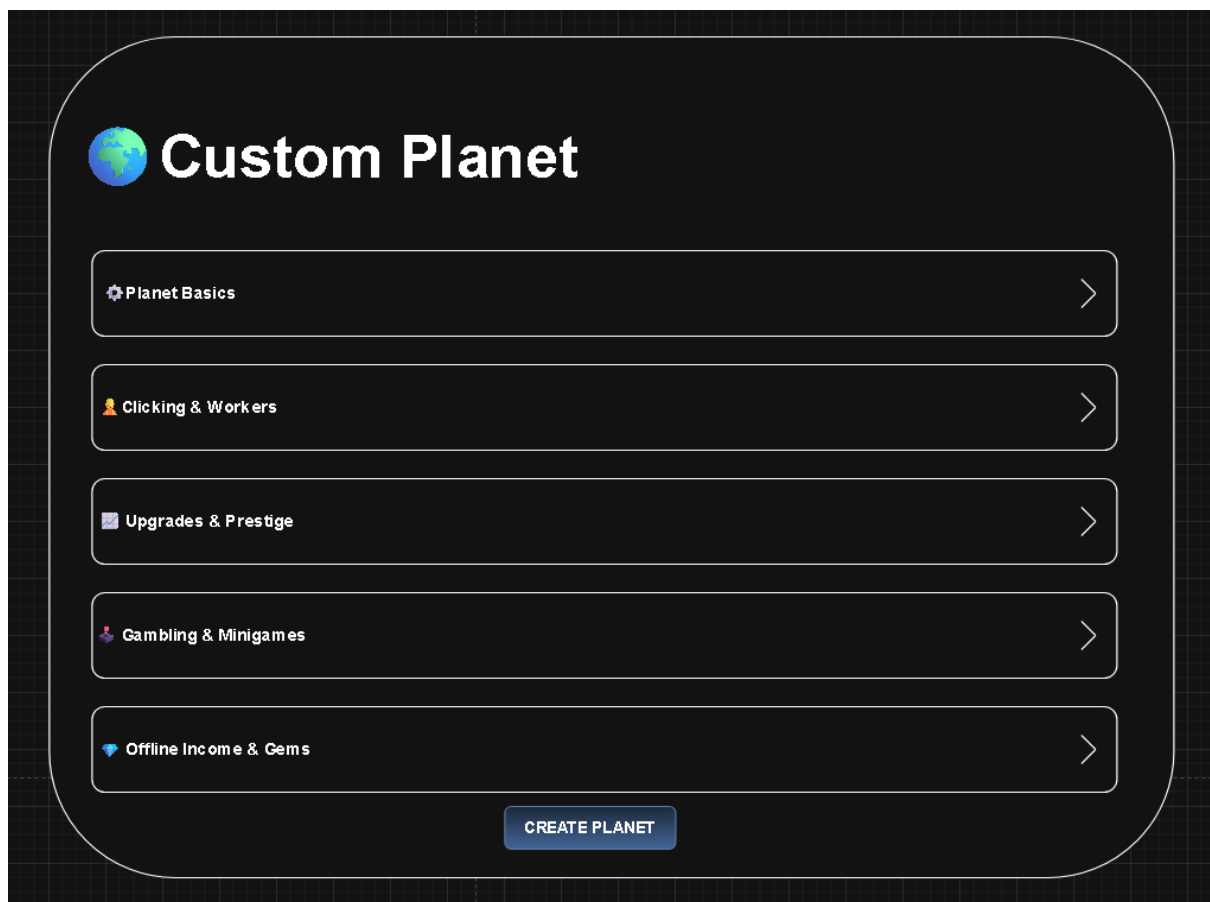
give a reward that is based of how much score the player got, with no way to lose, the following screen will be used:

- Minigame score screen:



This is the minigame score screen, which displays how much the player scored and the rewards they earned from achieving that score. This is used for minigames where the player doesn't either win or lose, but instead gets a score which determines how much reward they get.

4. Custom Planet Screen:



This is the custom planet screen, where the player is given a range of options to customise their own, unique planet. I have split this section into a title, and a series of dropdown boxes with headings. Each box has a title relating to a different part of the planet customisation. When expanded, the dropdown boxes will give the user different options about how to customise that specific part of the planet, allowing them to enter values and tick checkboxes. Once the player has finished configuring the planet, they can press the Create Planet button at the bottom to create their custom planet. Grouping these options into boxes with appropriate headings makes the customisation process much easier for the user, ensuring they don't get lost in all the various

settings they can change. The dropdown boxes allow multiple different options to get compressed into a single box, which can be opened and closed. I have also used emojis for the title and the box titles to make this screen more aesthetically pleasing and fun, which makes the experience more exciting.

Colour design:



I have chosen a space/sci-fi themed colour pallet. The background will be a gradient from a dark purple to a dark shade of pink, creating a cosmic, galaxy style scene. The container borders and title text colours will be blue for the left section, and purple for the right section, combining the colours associated with space and the galaxy. Also, the colour split between these 2 sections further aplifies the split between the active and purchase features. I have kept the achievement and level progress colours the same

(green), since this colour is associated with achievement, progression and reward, which will give the player a stronger sense of achievement as they see the green bars increasing.

Post-dev Test Plan:

After completing the iterative stages of development, I will move on to post-development testing, to ensure that my game meets its success criteria and provides a positive user experience. I will carry out 3 stages of post-dev testing:

Functionality Testing:

- I will play my game as if I am one of the stakeholders, progressing normally and trying out all the features.
- I will test the main planet clicking and automation loop, the worker generation, the shop upgrades, saving, prestige, population gambling, achievements, level system, effects, and planets (including the custom planets).
- This test will be focussed on how the game functions, and whether it does what it is supposed to do
- Of course, I will be performing iterative testing after each stage of development, however this will test the functionality of the game as a whole, with all the components integrated together.
- I will record this test as one video, and annotate it to explain the evidence

Usability Testing:

- I will get a group of stakeholders to test the game and provide feedback, in the form of a survey, on it's usability
- The stakeholders will be of different ages, genders and gaming experience levels to get a range of responses and represent the as many types of users as possible. The stakeholders I select will still be in the age range I mentioned in the analysis (15-20), but I will select a variety of ages within that range.
- After testing the game, I will ask the stakeholders to fill out a survey, with the following questions:

1. How easy was it to understand the main objective of the game? (Very easy/Quite easy/Neutral/Difficult/Very difficult)

- This question finds out whether it is clear to the player how to play the game and what they need to do to progress

2. Were all the buttons and sections easy to locate and identify? (Very Easy/Quite easy/Neutral/Difficult/Very difficult)

- This question is used to find out if GUI is intuitive and easy to navigate

3. Did you find the colour scheme visually appealing? (Very appealing/Appealing/Neutral/Unappealing/Very unaappealing)

- This question finds out whether the colour scheme is well designed and aesthetic

4. Do you think the colour scheme fits the space theme of the game? (Fits very well/Fits well/Neutral/Doesn't fit/Doesn't fit at all)

- This question will find out whether the colour design is well fit to the space/sci-fi theme of the game.

5. Was the population gambling feature easy to use and understand? (Very easy/Easy/Neutral/Hard/Very hard)

- This question is used to find out whether the population gambling feature is intuitive, to ensure it is clear how it works without explanation

6. How would you rate the overall usability of this game out of 10? (1-10)

- This question will give me an overall rating of the game's usability, to determine it's success

7. Do you have any suggestions for improving the layout, colours or accessibility of the game? (Enter text)

- This is an optional question to get stakeholders to provide additional suggestions to improve the usability of the game, which might uncover areas I missed in the design

Robustness Testing:

- I will test my game under extreme or unexpected conditions, and test compatibility across different browsers to ensure it performs correctly in all conditions
- This is my test plan for conducting the robustness testing:

Test No.	Description	Test data	Expected result
1	Can the game handle all invalid data into input forms (The population gambling form)	Attempt to enter/paste characters, symbols and negative numbers into the gambling input form	The game should not allow these inputs. It should not crash or cause any errors

2	Can the game handle rapid clicking of all buttons	Use an autoclicker to click all buttons at 1000 clicks per millisecond	The game should continue to run smoothly without any lag, and should not crash
3	Is the game compatible on multiple browsers	Run the game on Chrome, Firefox, Edge and Safari	The GUI should be consistent across all browsers with no display errors, and the game should run as normal.
4	Does the layout adapt to the screen size of the browser	Resize the browser window by zooming out and test on a smaller screen like a laptop	The GUI should adapt to all sizes, keeping all the elements visible
5	Can the game handle extremely large numbers for population and money	Set the population to an extremely large number and click the planet to start the money update loop	The game should continue to run smoothly with no lag or crashes
6	Can the game handle extremely large upgrade numbers	Purchase each upgrade 100 times, separately	For each upgrade purchase, the game should run smoothly with no lag or crashes

Development

Stage 1 – GUI:

Features:

Title section:

- Container in the top centre of the page
- Large tag in the centre of the container, which reads “🌍 Idle Planet - Earth”.
- Small save icon in the top left of the container

Level display:

- Container underneath the title, shorter and thinner than the title div
- Text in the centre of the container that reads “Level 1 – 0/1000”.

Central section:

- Container in the centre of the page, starting just under the level display.
- Round planet image in the top centre of the container
- Text underneath the planet reading “Population”
- More text underneath that reading “Money”
- Smaller text underneath that reading “Money/s”

Worker section:

- Container at the bottom of the central section

- Text at the top of the container reading “  Workers”
- Button underneath with the text “Buy Worker: Price”
- Small text underneath the button reading “Total workers: x”
- Text underneath that reading “Total worker gen: x population/s”

Left section:

Achievements container:

- Container in the top left of the page
- Text at the top of the container reading “  Achievements”
- 3 containers underneath the text (short and wide), each with text inside reading “Achievement 1 – 0/100”

Container underneath the achievements container, with 3 shorter “sub-containers” inside:

Minigame container:

- 1st sub-container
- Text at the top reading “  Minigames”
- 3 buttons, side by side, with text inside reading “Game 1 Play”, “Game 2 Play” and “Game 3 Play”

Prestige container:

- 2nd sub-container
- Text at the top reading “  Prestige”
- Small text underneath reading “Population Multiplier: x1.0”
- Button underneath with text inside reading “Prestige – x population”

Prestige container:

- 3rd sub-container
- Text at the top reading “ 🎲 Population Gambling”
- Small input form underneath reading “Enter amount”
- Text to the right of the input form reading “Win return: ”
- Solid line underneath that spans across the width of the container
- Larger text underneath reading “YOU WON/YOU LOST”
- Button underneath with text inside reading “Gamble”

Right section:

Effects container

- Container in the top right of the page
- Text at the top of the container reading “ ✨ Effects”
- 3 buttons, one underneath the other, with text inside reading “Effect”
- 3 buttons, adjacent to each of the effect buttons, with text inside reading “Price”
- Container outside of the effects container, in the top left, but still connected to it. Text inside the container reading “Gems: ”

Container underneath the effects container, with 2 shorter “sub-containers” inside

Shop container:

- 1st sub-container:

- Small, square container in the top left of the shop container, with text inside reading “x1”
- Text at the top of the container reading “🛒 Shop”
- 3 buttons, 1 under the other.
- Text inside each button reading “Upgrade: Price”
- Small text in the bottom left of each button reading “0”
- Small text in the bottom right of each button reading “x1”
- Scroll bar on the right on the shop section

Planets container:

- 2nd sub-container:
- Text at the top of the container reading “🚀 Planets:”
- 4 buttons, 1 under the other
- Text in the 4 buttons : “Mercury”, “Venus”, “Mars”, “Custom planet” respectively
- Button adjacent to the first button with text inside reading “Lvl 50”
- Buttons adjacent to the other 3 buttons with a lock image inside

Theme + Styling:

- Text font: Bungee, taken from Google Fonts (Google, n.d.)
- Space themed colour palette according to GUI design:
- South gradient background from # 220B36 to #4F2B35
- # 1C1B29 background for all containers
- 60% opacity for central container
- # 3F9FFF text colour for title
- #7BCFFF title text colour for left containers
- #C77DFF title text colour for right containers
- # 3F0FFF title text colour for worker container
- Red text for Prestige button, Gambling loss message, Worker buy button, Shop upgrades, and Planets the next planet to unlock
- Grey text for planets (including custom planet) after the next one to unlock
- # 3F9FFF border for title section

- # 3F9FFF border for level section
 - # 1D3169 border for central section
 - # 7BCFFF border for achievements container
 - White border for achievement bars
 - # 0B7BDD border minigames, prestige and population gambling containers, including the outer container that they all sit in.
 - # 3F0FFF border for worker section
 - # 9205FF border for effects section
 - White border for each effect and its price
 - # 9205FF border for shop and planets container, including the outer container they sit in
 - #3D2A4F for minigame buttons, gamble button, buy worker button, effect buy buttons, upgrade buy buttons and planet name displays (not the planet unlock button, but the container that displays the name of each planet and the custom planet)
 - #F349CE border for multi-purchase button
 - Purple text for multi-purchase button
 - All other text white
- Text font: Bungee, taken from Google Fonts (Google, n.d.) [Bungee - Google Fonts](#)

Note - In the GUI design, I have an east Gradient from #7BCFFF to #C77DFF for level progress bar, however, I will not implement this in this stage as I am going to add the level progress bar when I actually implement the level system, since it will not go up until the level system is implemented

I also have an east gradient from #7BCFFF to #449BFF for achievement progress bar but I will not implement this until I implement the achievement system, for the same reason

Development

Title + Level Section:

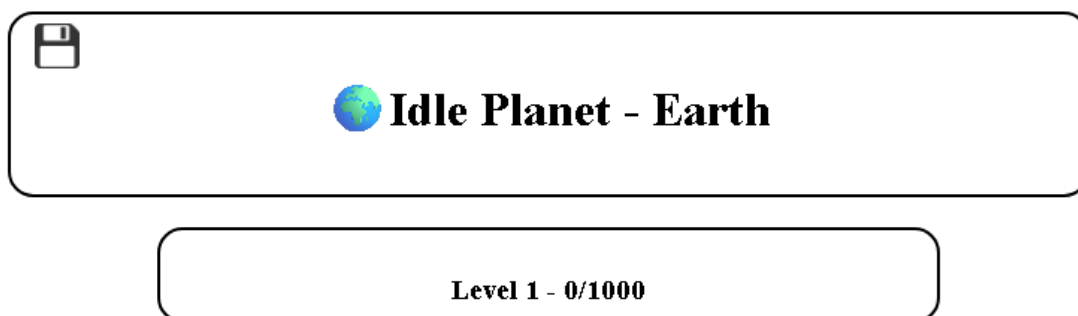
```
<div class="top-center">
  <div class="title">
    <button class="save-button">
      </img>
    </button>
    <h1>🌍 Idle Planet - <span>Earth</span></h1>
  </div>
  <div class="level">
    <div class="progress-bar"></div>
    <h3>Level 1 - 0/1000</h3>
  </div>
</div>
```

I have created a div for the top center of the page, which contains 2 divs, one for the title and one for the level section.

The reason I have put these 2 sections in the same div, is because I am using a grid and these 2 divs are on different rows, meaning there would be a

```
37 .top-center {
38     display: flex;
39     flex-direction: column;
40     align-items: center;
41 }
42
43 .title {
44     grid-row: 1 / span 2;
45     grid-column: 2;
46     height: 100px;
47     width: 700px;
48     display: flex;
49     justify-content: center;
50     align-items: center;
51     position: relative;
52 }
53
54 .save-button {
55     position: absolute;
56     left: 10px;
57     bottom: 80px;
58     background: none;
59     border: none;
60 }
61
62 .save-icon {
63     height: 30px;
64     width: 30px;
65 }
66
67
68
69 .level {
70     grid-row: 2;
71     grid-column: 2;
72     height: 40px;
73     width: 500px;
74     text-align: center;
75 }
76 }
77
```

big gap between them which I don't want. By putting them in the same div, I can set the grid-row of the title div to 1 / span 2, which makes it span over 2 rows, so the 2 divs are essentially in the same row together which will bring the level section up closer to the title section. For the title text, I have used a h1 with a span inside that reads the name of the planet. This is so that I can change the text of just the span when the planet needs to switch. For the save icon, I have used an image from Free Icons Png ([Save Icon PNG Transparent Background, Free Download #5404 - FreelconsPNG](#)). I have put this image inside a button, with the background and border removed so only the icon is visible, but it is still a clickable button. I have set the position of the title div to relative, and the save button to absolute, so I can independantly move the button to the top left of the top centre section without affecting the position of the title. This is what this looks like on the page:



Central + Worker Section:

```
<div class="center">
  <button class="planet-button">
    </img>
  </button>
  <h2>Population: </h2>
  <h2 style="margin-bottom: -10px">Money: </h2>
  <h4>Money/s: </h4>
  <div class="workers">
    <h2> 👤 Workers</h2>
    <div class="worker-elements">
      <button class="worker-button">Buy Worker: <span>Price</span></button>
      <h4>Total Workers: x</h4>
      <h3>Total worker gen: x population/s</h3>
    </div>
  </div>
</div>
```

```
.center {
  grid-row: 2;
  grid-column: 2;
  border: 2px solid black;
  border-radius: 17px;
  padding: 10px;
  margin: 10px;
  text-align: center;
  height: 630px;
  justify-content: flex-start;
  display: flex;
  flex-direction: column;
  align-items: center;
}

.planet-button {
  background: none;
  border: none;
  padding: 0;
  display: flex;
  justify-content: center;
  align-items: center;
}

.planet-icon {
  width: 250px;
  height: auto;
  max-width: 100%;
}

.workers {
  height: 150px;
  width: 600px;
  display: flex;
  text-align: center;
  justify-content: center;
  flex-direction: column;
  margin: 0px;
}

.worker-elements {
  gap: 0px;
}

.worker-button {
  width: 150px;
  height: 30px;
}
```


I have created a div for the central section of the page. I have set the flex-direction of the div to column, so that the elements stack vertically on top of each other, and I have set the justify-content to flex-start so that the elements start at the top of the container (with the planet at the very top). Inside the div I have a button for the clickable planet, with an image inside from Pin Clipart ([Earth Transparent Png Free - Earth Drawing Png Clipart \(#1635836\) - PinClipart](#)) for the Earth. I removed the background and border from the button, so only the planet image is visible. I have also created a div inside of the center div, for the worker section. Inside this, I have the title for the section, and also a div for all the elements inside the worker section, including the worker button , the total workers display and the total worker generation display. The reason I have this nested div, is so that I can arrange the elements to stack on top of each other, and control the sizing and gap between them without affecting the title of the div.

This is what this looks like on the page:



Population:

Money:

Money/s:

 **Workers**

Buy Worker: Price

Total Workers: x

Total worker gen: x population/s

Left Section:

Achievements

```
<div class="achievements">
  <h2> 🏆 Achievements</h2>
  <div class="achievement">
    <h3> Achievement 1 - 0/100</h3>
  </div>
  <div class="achievement">
    <h3> Achievement 2 - 0/100</h3>
  </div>
  <div class="achievement">
    <h3> Achievement 3 - 0/100</h3>
  </div>
</div>
```

```
.achievements {
  grid-row: 1;
  grid-column: 1;
}

.achievement {
  border: 2px solid black;
  border-radius: 10px;
  margin: 10px;
  text-align: center;
  height: 45px;
}
```

I have created a div for the achievements section, with 3 divs inside for each achievements. I've aligned all the elements in the center and given each achievement a border and the correct text. The achievements section is still in the left section, but it is not inside the "left" div

I have created because it is in the top right page, and not embedded inside another container like the next 3 sections. This is what this looks like on the page



Minigames:

```
<div class="left">
  <div class="minigames">
    <h2>🧩 Minigames</h2>
    <div class="minigame-buttons">
      <button class="minigame-button">Minigame 1 Play</button>
      <button class="minigame-button">Minigame 2 Play</button>
      <button class="minigame-button">Minigame 3 Play</button>
    </div>
  </div>
</div>
```

```
.left {  
  grid-row: 2;  
  grid-column: 1;  
  border: 2px solid black;  
  border-radius: 17px;  
  padding: 10px;  
  margin: 10px;  
  text-align: center;  
}
```

```
.minigames {  
  border: 2px solid black;  
  border-radius: 17px;  
  padding: 10px;  
  margin: 10px;  
  text-align: center;  
}  
  
.minigame-buttons {  
  display: flex;  
  justify-content: center;  
  gap: 10px;  
}  
  
.minigame-button {  
  width: 80px;  
  height: 80px;  
  border: 2px solid black;  
}
```

I have created a div for the left section of the game which holds the minigames, prestige, and population gambling sections. Inside this left div, I have created a div for the minigames section, with the title of the section, followed by another div for all the minigame buttons. The reason I have put all the minigame buttons in their own div, is so that I can arrange them side by side, separate from the title. This is what this looks like on the page:



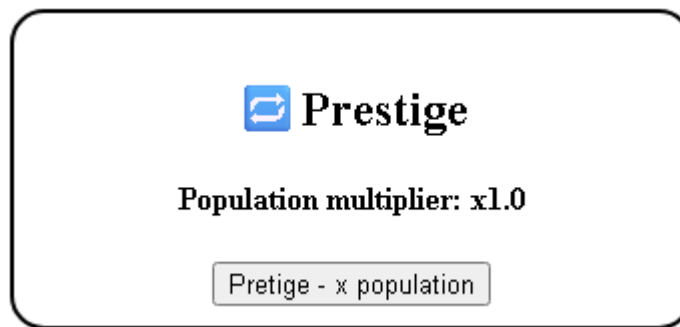
Prestige:

```
<div class="prestige">
  <h2> 🎮 Prestige</h2>
  <h4>Population multiplier: x1.0</h4>
  <button>Pretige - x population</button>
</div>
```

```
.prestige {
  border: 2px solid black;
  border-radius: 17px;
  padding: 10px;
  margin: 10px;
  text-align: center;
}
```

I have created a div for the prestige section (Also inside the left div), with the text for the title and the population multiplier display. I then have a button for the prestige

which contains text showing the required population.
This is what this looks like on the page:



Population Gambling:

```
<div class="population-gambling">
  <h2> 🎲 Population Gambling</h2>
  <div class="gamble-input">
    <input class="gamble-amount" type="text" placeholder="Enter amount"></input>
    <button>Gamble</button>
  </div>
  <h4 style="margin: 5px; text-align: left;">Win return: </h4>
  <div class="separator"></div>
  <h3>YOU WON / YOU LOST</h3>
</div>
</div>
```

```

.population-gambling {
  height: 170px;
  border: 2px solid black;
  border-radius: 17px;
  padding: 10px;
  margin: 10px;
  text-align: center;
}

.gamble-input {
  display: flex;
  height: 20px;
  align-items: center;
  gap: 10px;
}

.separator {
  margin: 10px;
  height: 0.0001px;
  border: solid 2px
}

```

I have created a div for the population gambling section (Also inside the left div), with text for the title at the top. I then have a div for the gamble input, which holds the input form for the gamble and the gamble button. The reason I have these in a separate div is because I can arrange just these two elements side by side. I then have text underneath this div for the Win Return display. This is outside the gamble input div because it is on the line below. After this, I have a separator div, which I have just used to make a line to divide the section according to my GUI design. The separator is just a very short div with a solid border, which makes it look like a line. Finally, I have the win/loss text at the bottom of the section. This will not be visible when I set up the population

gambling, and it will only appear when the player wins or loses (YOU WON will appear if they win and YOU LOST will appear if they lose). This design is slightly different to that of my GUI design, because I thought it made more sense to have the gamble button right next to, on the same line as, the input form, rather than all the way at the bottom of the section. I then thought it would be better to have the win return text display underneath the input form, on its own line, so there is more space to display the number which may be large and take up a lot of horizontal space.

Right Section:

Effects:

```
<div class="effects">
  <div class="gems"><h3>Gems: </h3></div>
<h2> ✂ Effects</h2>
<div class="effects-list">
  <div class="effect-row">
    <div class="effect"><h3>Effect 1</h3></div>
    <div class="effect-price"><span>Price</span></div>
  </div>
  <div class="effect-row">
    <div class="effect"><h3>Effect 2</h3></div>
    <div class="effect-price"><span>Price</span></div>
  </div>
  <div class="effect-row">
    <div class="effect"><h3>Effect 3</h3></div>
    <div class="effect-price"><span>Price</span></div>
  </div>
</div>
</div>
```

```

.effects {
  padding: 5px;
  display: flex;
  flex-direction: column;
  align-items: flex-start;
  position: relative;
}

.effects-list {
  display: flex;
  height: 50px;
  width: 100px;
  align-items: flex-start;
  flex-direction: column;
  gap: 10px;
  margin-top: 10px;
  margin-left: 5px;
}

.effect-row {
  height: 40px;
  width: 350px;
  display: flex;
  align-items: center;
  justify-content: center;
  gap: 10px;
}

.effects h2 {
  margin-top: 5px;
  margin-left: 135px;
  margin-bottom: 45px;
}

.effect-price {
  display: flex;
  height: 40px;
  width: 100px;
  border: 2px solid black;
  border-radius: 10px;
  align-items: center;
  justify-content: center;
}

```

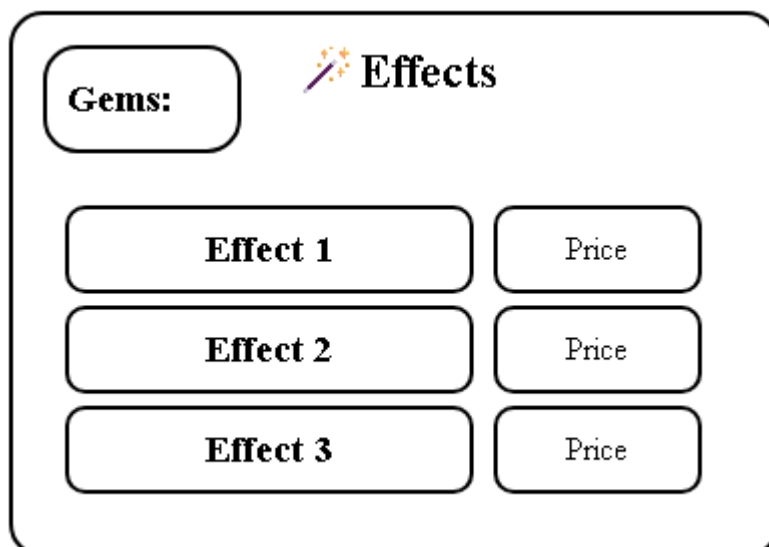
```

.effect {
  border: 2px solid black;
  border-radius: 10px;
  width: 200px;
  height: 40px;
  display: flex;
  align-items: center;
  justify-content: center;
}

.gems {
  border: 2px solid black;
  border-radius: 17px;
  padding: 10px;
  margin: 10px;
  text-align: center;
  width: 75px;
  height: 30px;
  display: flex;
  justify-content: left;
  align-items: center;
  position: absolute;
}

```

I have made a div for the effects section. The effects section is still in the right section of the page, but it's not in the "right" div I have created because it is in the top right of the page and not embedded inside another container like the next 2 sections. Inside the effects div, I have a div for the gems, as this needs to be a container in the top left of the effects section. I then have title text for the section. After this, I have an effects list div, which holds an effects row div for each effect. The effects row div contains a div for the effect, and then a div for the effect price display. The reason I have separate divs for each row of effects, is so that I can group the effect and price together and then arrange them side by side, then stack each one on top of each other. I do this by setting the flex direction to column in the effects list div, so that the effects and their price will be arranged side by side. Therefore, each effect row div will have the effect and then the price next to it, and then the next div will be underneath the previous one. For the gems div, I have set the position to absolute so that it can stay in the top left of the section without affecting the rest of the content. Otherwise, it would push the rest of the elements in the section down. This is what this looks like on the page:



Shop:

```
<div class="right">
  <div class="shop">
    <h2>🛒Shop</h2>
    <div class="shop-list">
      <div class="shop-row">
        <button class="shop-item">Upgrade 1:<span>x1</span></button>
        <span>Price</span>
      </div>
      <span>x0</span>
      <div style="height: 10px"></div>
      <div class="shop-row">
        <button class="shop-item">Upgrade 2:<span>x1</span></button>
        <span>Price</span>
      </div>
      <span>x0</span>
      <div style="height: 10px"></div>
      <div class="shop-row">
        <button class="shop-item">Upgrade 3:<span>x1</span></button>
        <span>Price</span>
      </div>
      <span>x0</span>
    </div>
  </div>
</div>
```

```
.multi-purchase {
  border: 2px solid black;
  border-radius: 17px;
  padding: 10px;
  margin: 0px;
  text-align: center;
  width: 20px;
  height: 20px;
  display: flex;
  justify-content: left;
  align-items: center;
  position: absolute;
  margin-bottom: 215px;
}
```

```
.right {
  grid-row: 2;
  grid-column: 3;
  border: 2px solid black;
  border-radius: 17px;
  padding: 10px;
  margin: 10px;
  text-align: center;
}

.shop {
  border: 2px solid black;
  border-radius: 17px;
  padding: 10px;
  margin: 10px;
  text-align: center;
  justify-content: center;
  display: flex;
  flex-direction: column;
  gap: 5px;
  height: 260px;
}

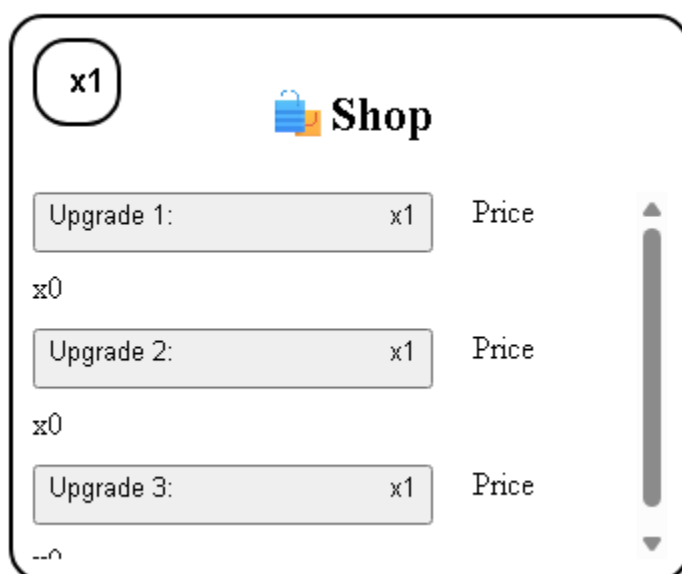
.shop-list {
  gap: 5px;
  max-height: 240px;
  overflow-y: auto;
  text-align: left;
}

.shop-row {
  display: flex;
  flex-direction: row;
  gap: 20px;
  margin-bottom: 8px;
}

.shop-item {
  height: 30px;
  width: 200px;
  display: flex;
  justify-content: space-between;
}
```

I have created a div for the right side of the page, which holds the shop and planets section. Inside this div, I have created a div for the shop, which starts with another div for the multi-purchase button. I have used the same css code for this button as for the gems display in the effects section (except with a margin-bottom of 215px to push it to the top of the section). This css ensures it can be positioned in the top left of the section without affecting the rest of the elements. After this, I have the title text, followed by another div for the shop list. Similar to the effects section, I then have a shop row div for each upgrade, purchase number and its price, allowing me to put the upgrade and its price side by side. Inside the button for each upgrade, I have a span for the purchase number reads x1, which corresponds to the number in the multi-purchase container. I have set the justify content of each shop item to space-between, which pushes that span to the very right hand side of the button. After each shop row div, I have a span that reads x0, which will increase each time the player purchases the upgrade above. I have also set the overflow-y of the shop list to auto, which creates a scrollo bar for the upgrades, so that when I add more the player will be able to scroll through them all, without them taking up loads of space on the page.

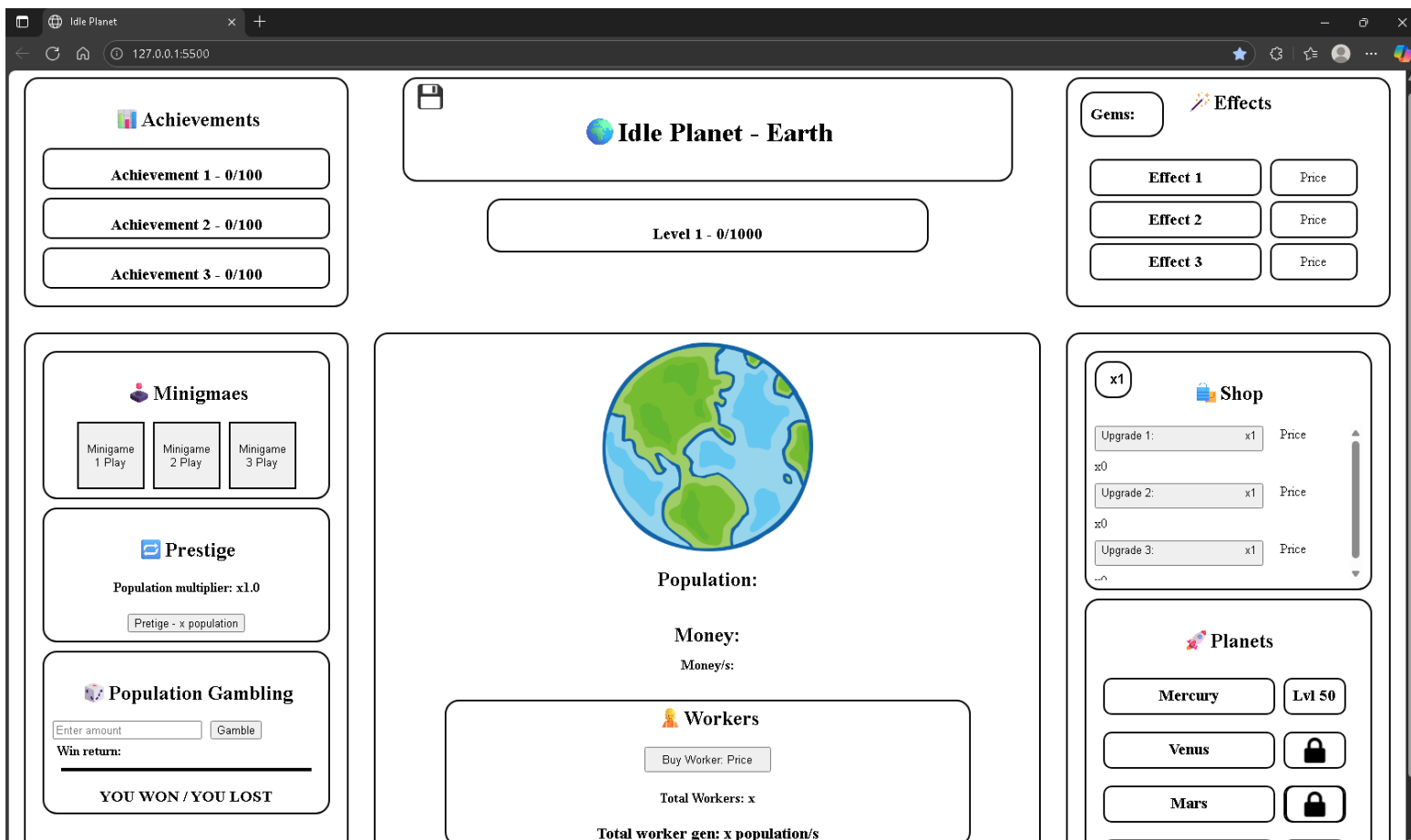
This is what this looks like on the page:



Planets:

```
<div class="planets">
  <h2> 🌌 Planets </h2>
  <div class="planet-list">
    <div class="planet-row">
      <div class="planet">
        <h3>Mercury</h3>
      </div>
      <div class="planet-unlock"><h3>Lvl 50</h3></div>
    </div>
    <div class="planet-row">
      <div class="planet">
        <h3>Venus</h3>
      </div>
      <div class="planet-unlock"></img></div>
    </div>
    <div class="planet-row">
      <div class="planet">
        <h3>Mars</h3>
      </div>
      <div class="planet-unlock"><div class="planet-unlock"></img></div></div>
    </div>
    <div class="planet-row">
      <div class="planet">
        <h3>Custom Planet</h3>
      </div>
      <div class="planet-unlock"><div class="planet-unlock"></img></div></div>
    </div>
  </div>
</div>
```

Entire page:



Cleaning up the CSS:

I have noticed that many my HTML elements have the same 5 lines of CSS code:

```
border: 2px solid black;  
border-radius: 17px;  
padding: 10px;  
margin: 10px;  
text-align: center;
```

This is because for all of these elements, I set a border with the same border radius, add padding and a margin of 10px, and center the text. Because of this, I can set these properties for all the elements at once. The classes of the elements that all have these properties set are:

- left
- right
- achievements
- center
- effects
- gems
- level
- minigames
- planets
- population-gambling
- prestige
- shop

- title
- planet
- workers

Therefore, I will delete the css that individually sets the properties for each of these classes, and do it all at once:

```
.left,  
.right,  
.achievements,  
.center,  
.effects,  
.gems,  
.level,  
.minigames,  
.planets,  
.population-gambling,  
.prestige,  
.shop,  
.title,  
.planet,  
.workers {  
    border: 2px solid black;  
    border-radius: 17px;  
    padding: 10px;  
    margin: 10px;  
    text-align: center;  
}
```

This cleans up the css code a bit, removing duplication of code


```
.planets {
  border: 2px solid □black;
  border-radius: 17px;
  padding: 10px;
  margin: 10px;
  text-align: center;
}

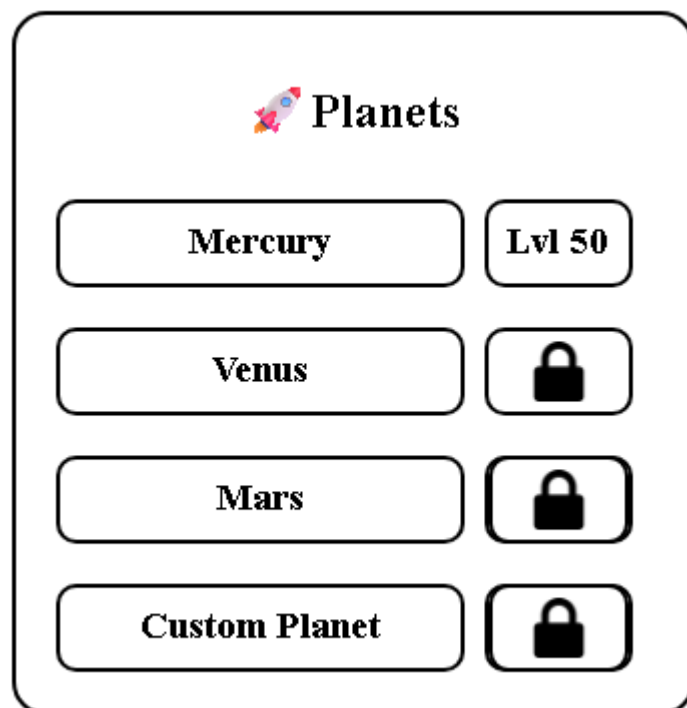
.planet {
  width: 200px;
  height: 40px;
  padding: 0px;
  display: flex;
  text-align: center;
  justify-content: center;
  align-items: center;
  border-radius: 10px;
}

.planet-unlock {
  border: 2px solid □black;
  border-radius: 10px;
  width: 70px;
  height: 40px;
  align-items: center;
  justify-content: center;
  display: flex;
}

.planet-row {
  display: flex;
  align-items: center;
}

.lock-icon {
  background: none;
  border: none;
  padding: 0;
  display: flex;
  justify-content: center;
  align-items: center;
  max-width: 30px;
}
```

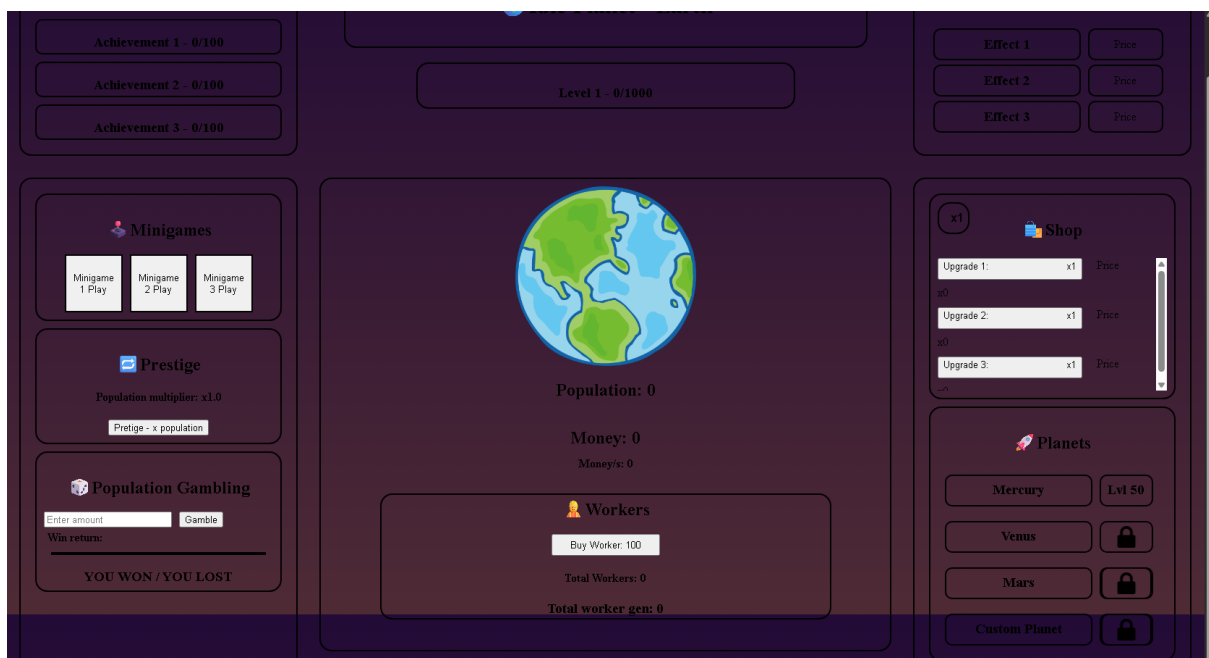
I have created a div for the planets section (also in the right div), which has the title of the section at the top. Like with the previous 2 sections, I have a div for the planets list, and then a planet row div for each row. Inside this, I have a div for the planet, which displays the planet name, and then a div for the planet unlock button. The first planet unlock button reads “Lvl 50”, and then each one after has a lock image inside, which I got from Free Icons Png ([Black Lock Icon PNG Transparent Background, Free Download #29059 - FreelconsPNG](#)). This is what this looks like on the page:



Theme + Styling:

```
body {  
  display: grid;  
  grid-template-columns: 1fr 2fr 1fr;  
  grid-template-rows: auto auto auto;  
  gap: 10px;  
  height: 908px;  
  background: linear-gradient(to bottom, #220B36, #4F2B35);  
}
```

I added a linear gradient to the body, from top to bottom which creates a south gradient. I used the colours from the design. This is what this looks like on the page:

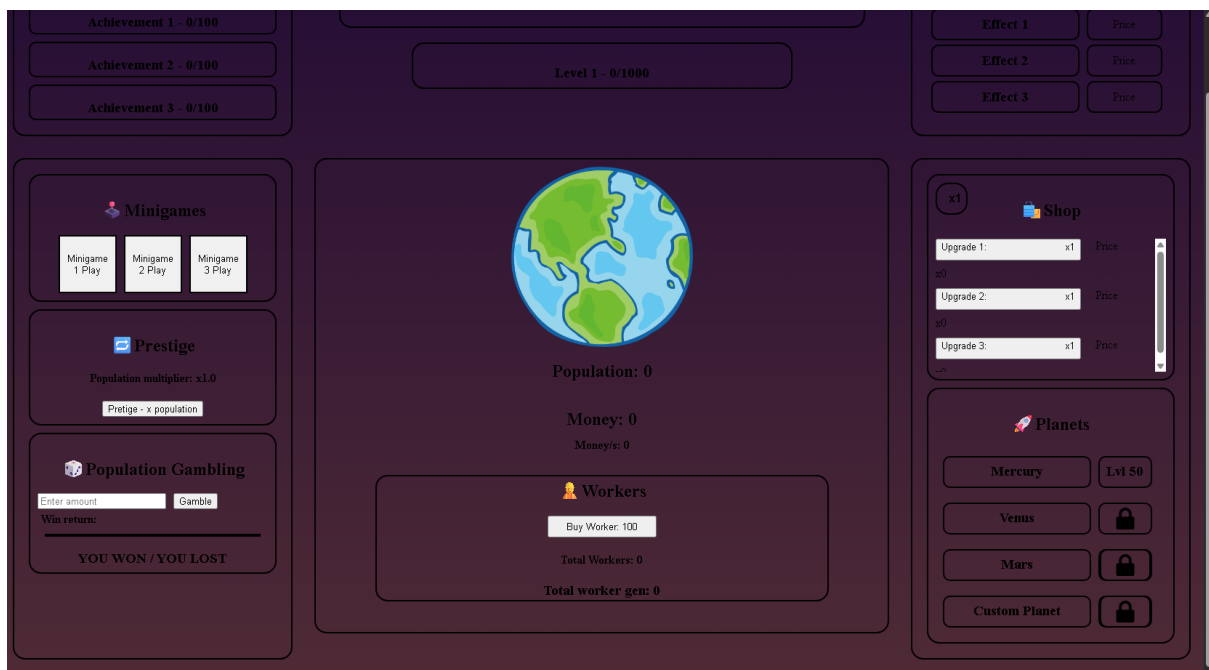


The gradient has been correctly applied, however since my game doesn't fully fit in the visible viewpoint of the page and you need to scroll down to see it all, the gradient resets at the bottom of the

page. To fix this, instead of hard coding the height of the body to 908px, I should just set the height of the body to 100%, so that the gradient will fill the entire body rather than just filling the visible viewpoint and then resetting after that:

```
body {  
  display: grid;  
  grid-template-columns: 1fr 2fr 1fr;  
  grid-template-rows: auto auto auto;  
  gap: 10px;  
  height: 100%;  
  background: linear-gradient(to bottom, #228B36, #4F2B35);  
}
```

This is what this looks like on the page:



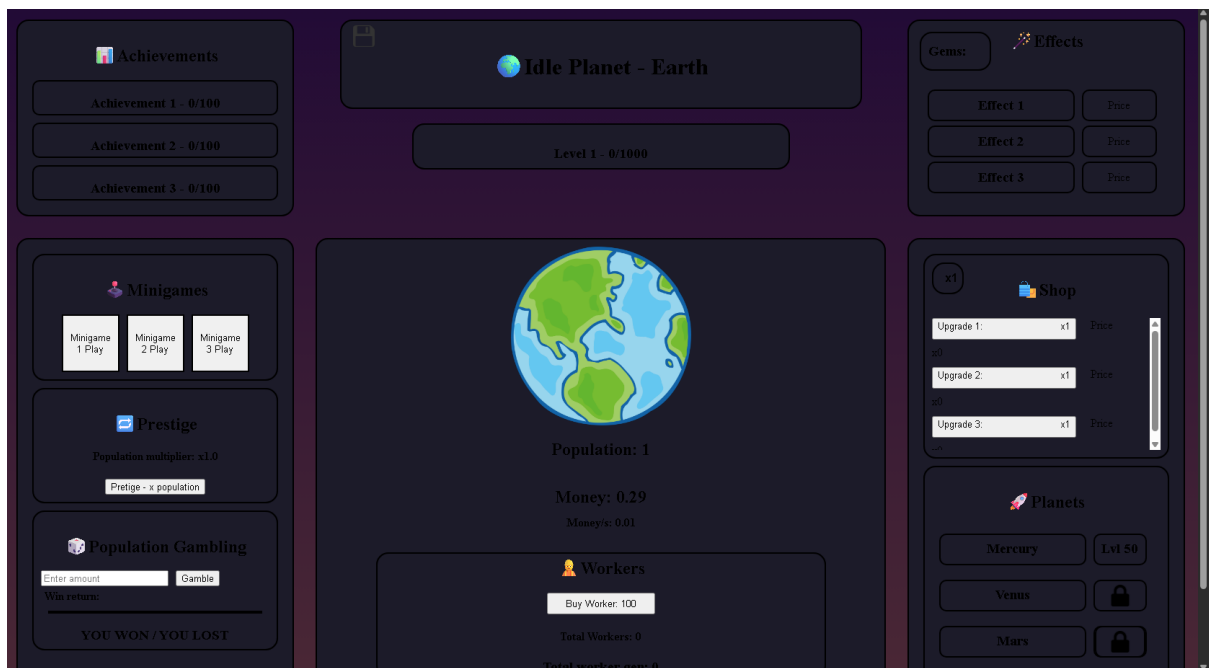
Now, the gradient applies to the bottom of the page (scrolled down all the way), rather than just the visible viewpoint.

```

.left,
.right,
.achievements,
.center,
.effects,
.gems,
.level,
.minigames,
.planets,
.population-gambling,
.prestige,
.shop,
.title,
.planet,
.workers {
  border: 2px solid black;
  border-radius: 17px;
  padding: 10px;
  margin: 10px;
  text-align: center;
  background-color: #1C1B29;
}

```

I then changed the background of all the containers according to the colour from the design, which looks like this on the page:



I then applied all the title text colours according to the design:

```
.left h2 {
  color: #7BCFFF;
}

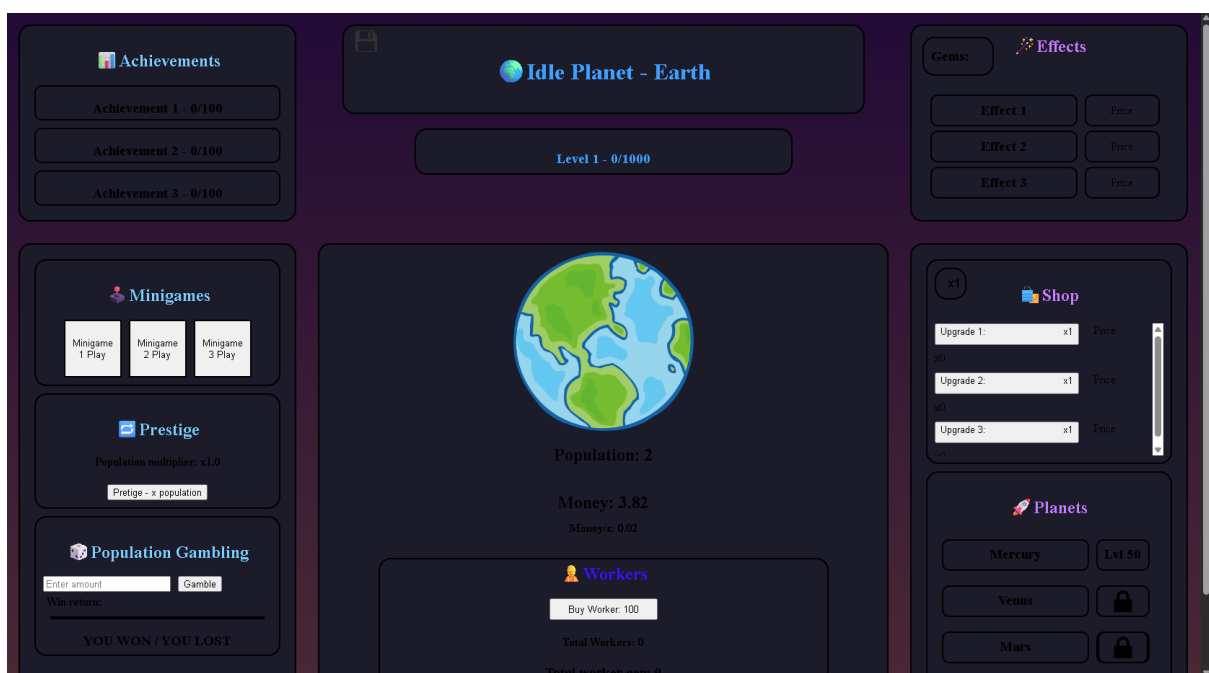
.right h2 {
  color: #C77DFF;
}

.achievements h2 {
  color: #7BCFFF;
}

.effects h2 {
  color: #C77DFF;
}

.workers h2 {
  color: #3F0FFF;
}
```

This is what this looks like on the page:



```
.prestige button,
.gamble-loss,
.shop-item,
.worker-button,
.next-planet,
.planet-unlock {
  color: red;
}

.planet-row {
  color: grey;
}
```

I then changed the text colour of the prestige button, gamble loss message, shop upgrades, worker button, and planets to red. Originally, I had a h3 tag for the gamble outcome messages, reading “YOU WON / YOU LOST”, but at this point I changed it to an h3 tag with 2 spans inside, one that reads “YOU WON” and the other that reads “YOU LOST”, so I could apply different colours to them. I then gave the win message span the class “gamble-win” and the loss message span the class “gamble-loss” (which I apply the red colour to in the screenshot above):

```
<h3><span class="gamble-win">YOU WON</span> <span class="gamble-loss">YOU LOST</span></h3>
```

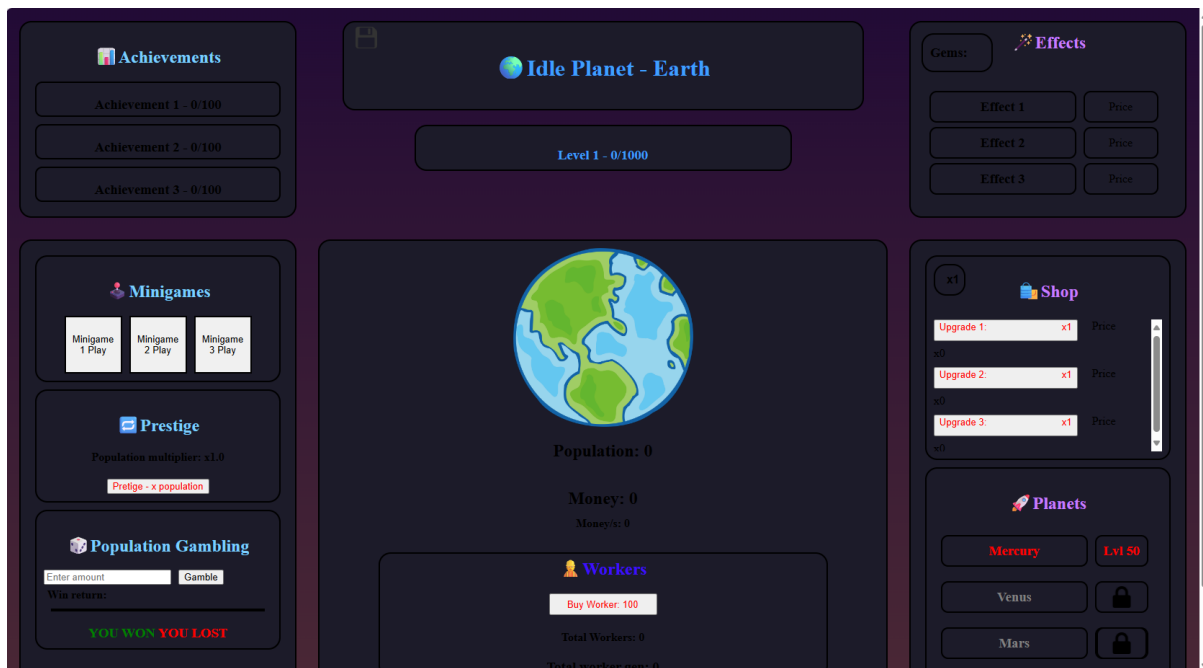
I then made the text of the win message to green using the span id:

Also, in order to make the text of the next planet to unlock red, and the rest of the planets grey, I gave the h3 and unlock text for the Mercury planet the class “next-planet”. I

then applied the red text to this and then the grey text to the rest of the planets using the “planet-row” id. When the player unlocks a planet, I will use the JavaScript to change the give the id “next-planet” to the h3 and unlock text of the next planet to unlock:

```
div class= planet >  
  <h3 class="next-planet">Mercury</h3>  
</div>
```

This is what this looks like on the page:



I am now going to set the correct borders for all the sections. I have already assigned borders to all the containers at this point, but they are just plain black borders and I did this so that I could get an idea of how the GUI is looking before applying the styling. Now I have set the correct colours for all the borders in the css:


```

.title,
.level {
  border-color: #3F9FFF;
}

.center {
  border-color: #1D3169;
}

.achievements {
  border-color: #7BCFFF;
}

.achievement {
  border-color: white;
}

.left,
.minigames,
.prestige,
.population-gambling {
  border-color: #0b7bdd;
}

.workers {
  border-color: #3F0FFF;
}

.effects {
  border-color: #9205FF;
}

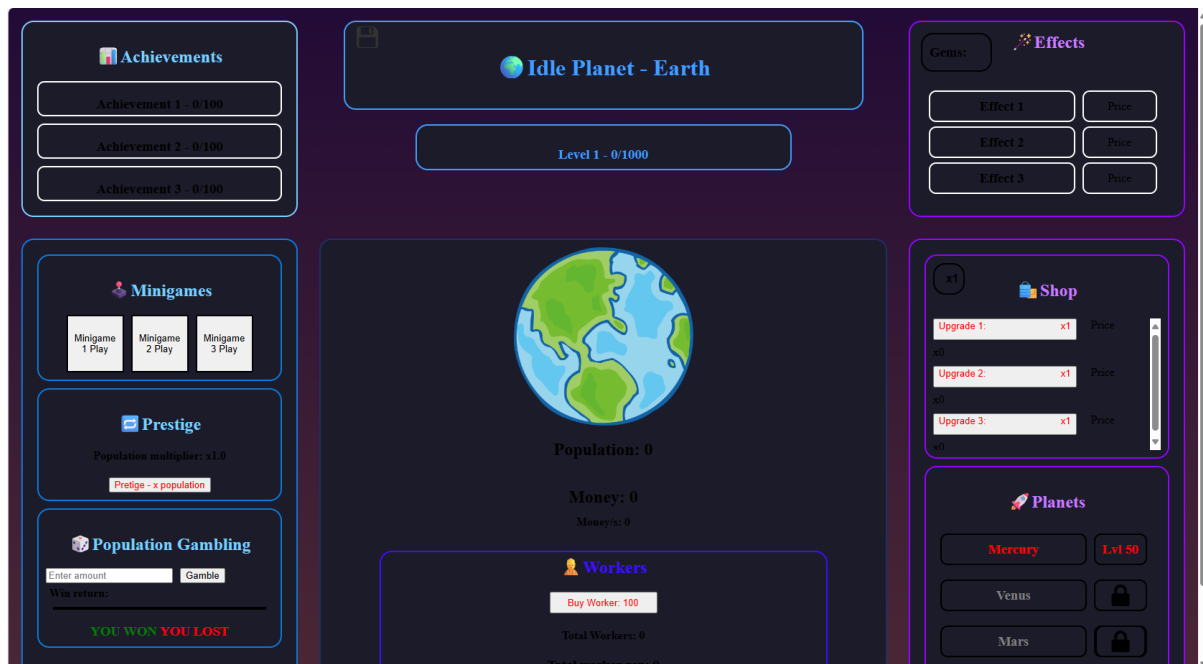
.effect {
  border-color: white;
}

.effect-price {
  border-color: white;
}

.right,
.shop,
.planets {
  border-color: #9205FF;
}

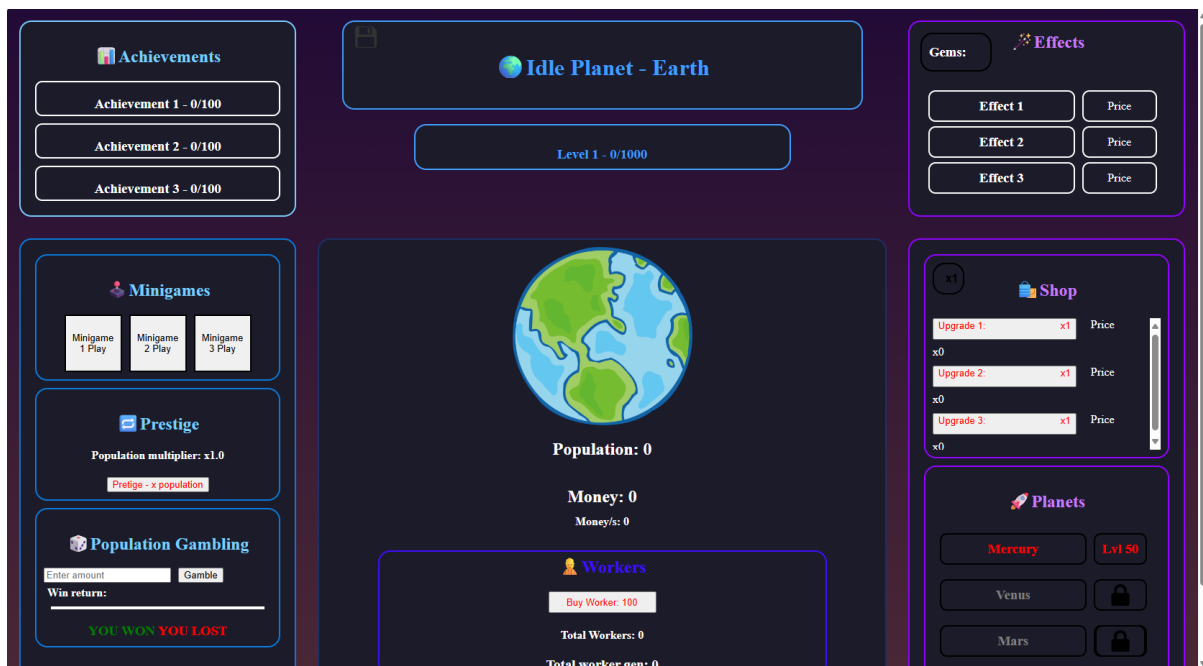
```

This is what this looks like on the page:



I then added the line “color: white;” in the body of the css, to set all text to white. Since this is at the start of the css, every other time the colour of text is set it will overwrite this. This allows me to set the text colour of specific elements, and any elements I don’t set the colour of will be set to the default of white.

This is what this looks like on the page:



I will now apply the button colours according to the design:

```

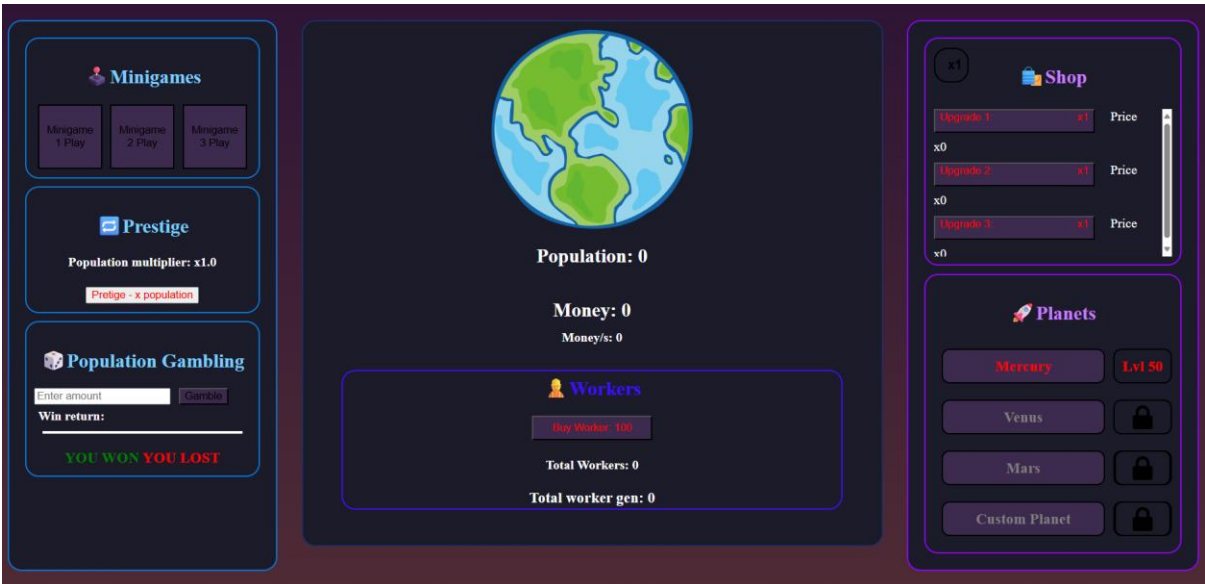
0
1  .minigame-button,
2  .gamble-button,
3  .worker-button,
4  .effect,
5  .shop-item,
6  .planet {
7    background-color: #3D2A4F;
8  }
9

```

This sets the background colour to #3D2A4F for the correct buttons/displays according to the design. I also gave the gamble button the class “gamble-button” to access it in the css as I realised I had not done this:

```
<button class="gamble-button">Gamble</button>
```

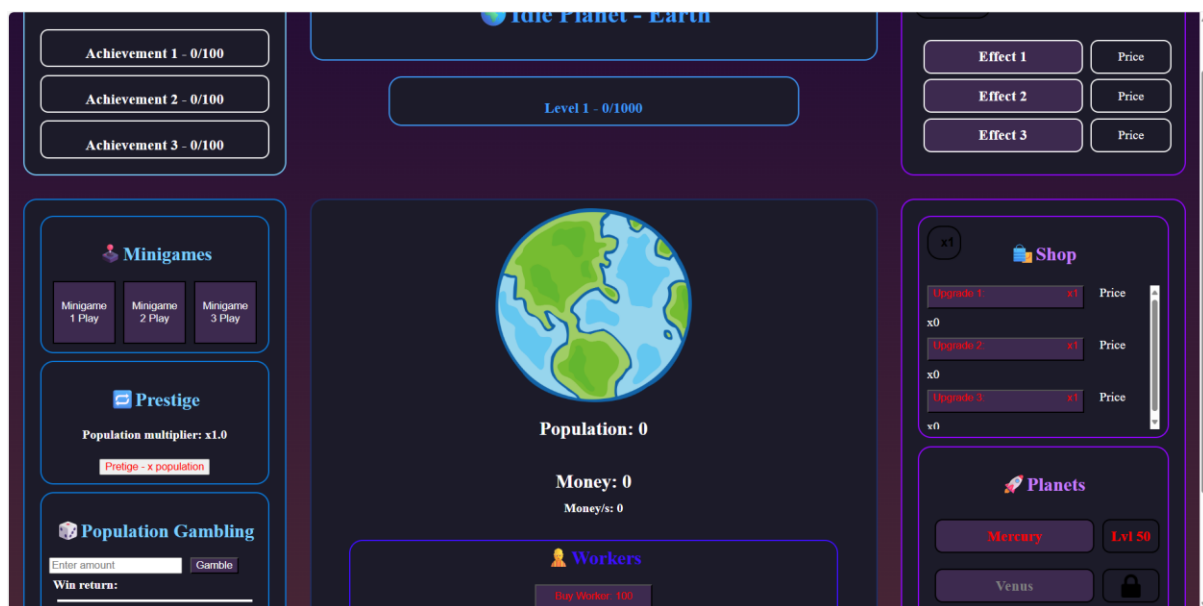
This is what this looks like on the page:



I then realised that the text for the minigames and gamble buttons should be white, so I set them both separately:

```
.minigame-button,  
.gamble-button {  
  color: white;  
}
```

This is what this looks like on the page:



I will now set the correct border and text colour for the multi-purchase button. For the border, I can just set the border of the multi-purchase div. However, currently the text that displays the

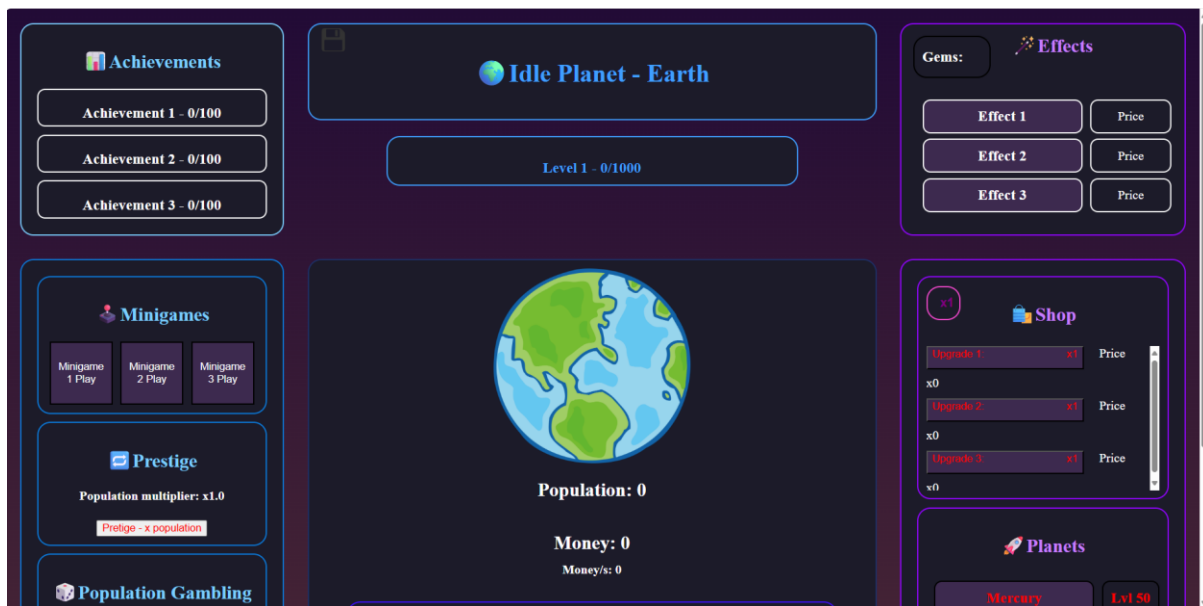
multi-purchase state is inside an h3 tag inside the button tag. Because I will need to access this in the JavaScript later, I put the text inside a span, inside the h3 which is inside the button. I then gave the span the id “multi-purchase-state” so I can access it in the css, and I will give it an id when I need to access it in the JavaScript:

```
<div class="shop">  
  <div class="multi-purchase"><button><h3><span class="multi-purchase-state">x1</span></h3></button></div>  
</div>
```

I then set the border and text colour for the multi-purchase button in the css, using the new class for the text colour:

```
.multi-purchase {  
  border: 2px solid #F349CE;  
}  
  
.multi-purchase-state {  
  color: purple;  
}
```

This is what this looks like on the page:



I will now apply the Bungee font to the text on the page, according to the design. I copied the embed code from google fonts:

Embed code in the <head> of your html

```
<link rel="preconnect" href="https://fonts.googleapis.com">
<link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
<link href="https://fonts.googleapis.com/css2?family=Archivo+Black&family=Bungee&display=swap" rel="stylesheet">
```

Copy code

Bungee: CSS class

```
.bungee-regular {
  font-family: "Bungee", sans-serif;
  font-weight: 400;
  font-style: normal;
}
```

Copy code

I put the HTML code in the head of my HTML under where I link my css stylesheet:

```

<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Idle Planet</title>
  <link rel="stylesheet" href="styles.css">
  <link rel="preconnect" href="https://fonts.googleapis.com">
  <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
  <link href="https://fonts.googleapis.com/css2?family=Archivo+Black&display=swap" rel="stylesheet">
  <link href="https://fonts.googleapis.com/css2?family=Archivo+Black&family=Bungee&display=swap" rel="stylesheet">
</head>

```

I then put the css code near the top of my css, under the body:

```

10
11
12  .bungee-regular {
13    font-family: "Bungee", sans-serif;
14    font-weight: 400;
15    font-style: normal;
16  }
17
18

```

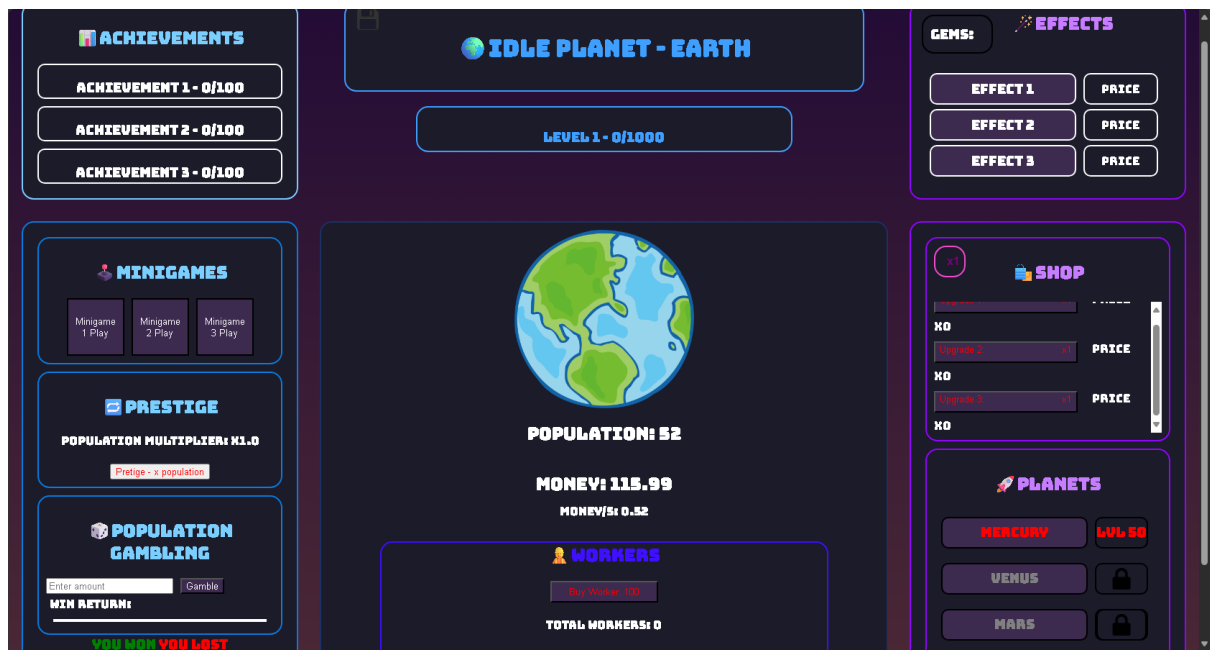
I then set the font family to “Bungee” in the css for the body, which should apply the font to all text on the page:

```

color: white;
font-family: "Bungee", sans-serif;

```

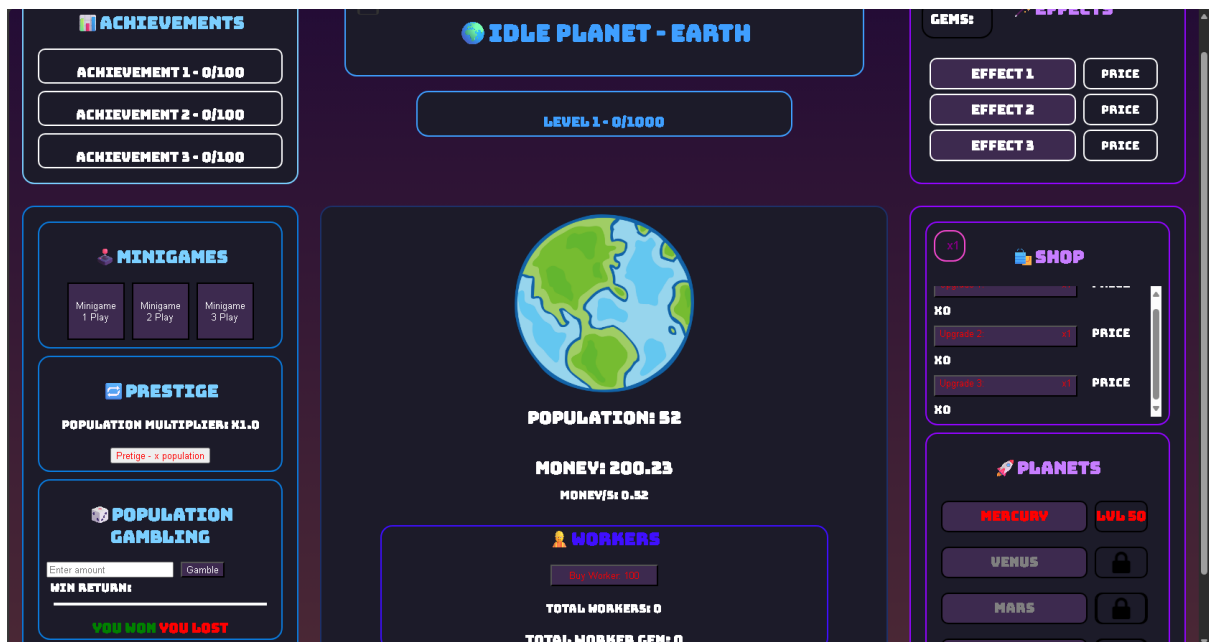
This is what this looks like on the page:



The font has been correctly applied to most the text, however there is a problem with the sizing, as the win/loss message has been moved outside on the population gambling section, due to the size of the font. To fix this, I will increase the height of the population gambling container, so it can fit all the text in it:

```
.population-gambling {
  height: 200px;
}
```

The original height was 170px, so I changed it to 200 and this is what it looks like on the page:

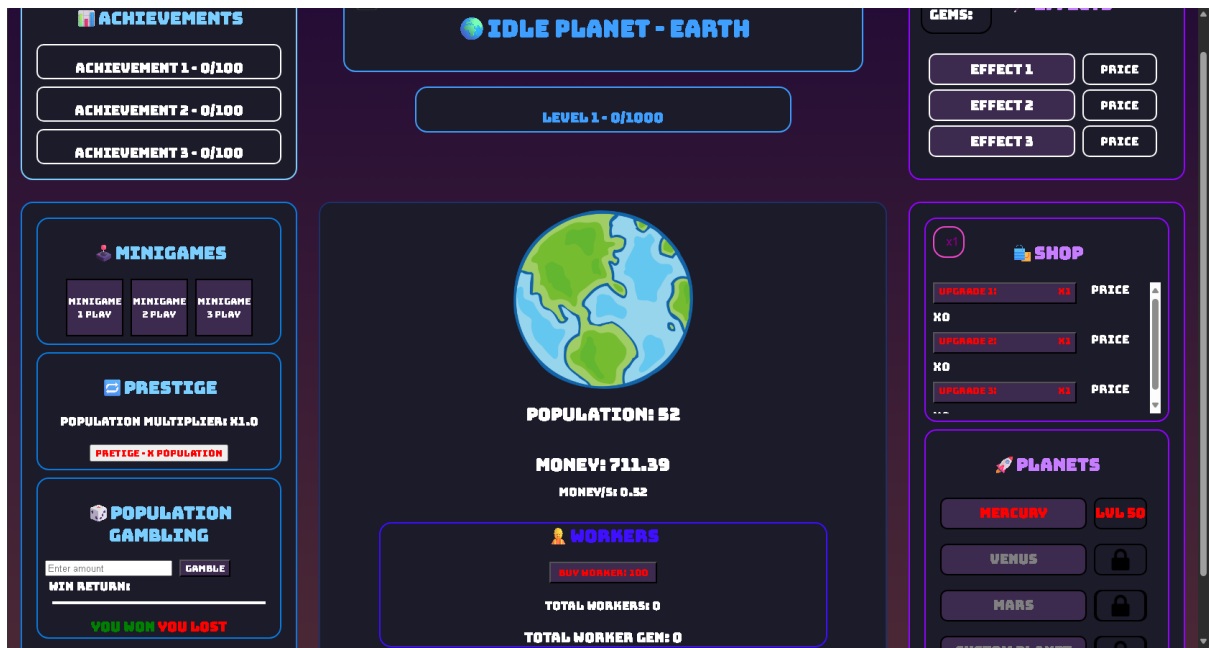


Now, the container is tall enough to fit all of it's text in.

I then realised that the font did not apply to the text for the minigame, prestige, gamble and upgrade buttons so I set it separately in the css:

```
.minigame-button,
.prestige button,
.gamble-button,
.worker-button,
.shop-item {
  font-family: "Bungee";
}
```

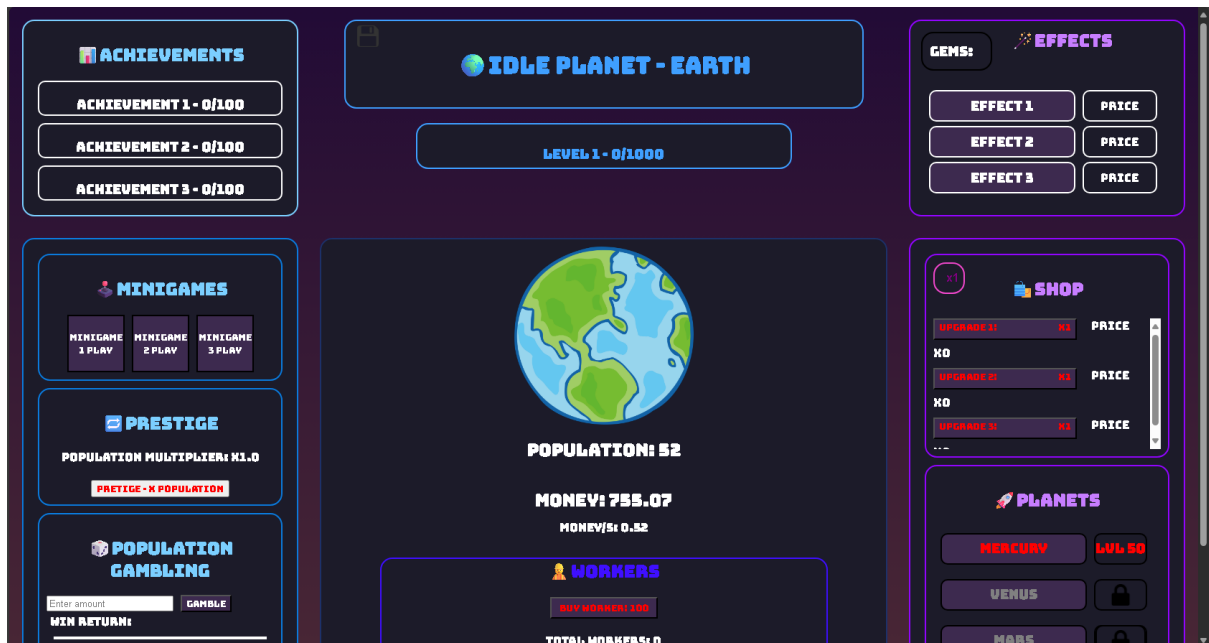
This is what this looks like on the page:



Now, the font is correctly applied to all the text on the page

Final tweaks

Here, I will make some final tweaks in the HTML and CSS, to fix any small imperfections. This is what the current page looks like:



I have noticed that the text for each achievement is slightly off centered (closer to the bottom of each achievement), so I fixed this in the CSS:

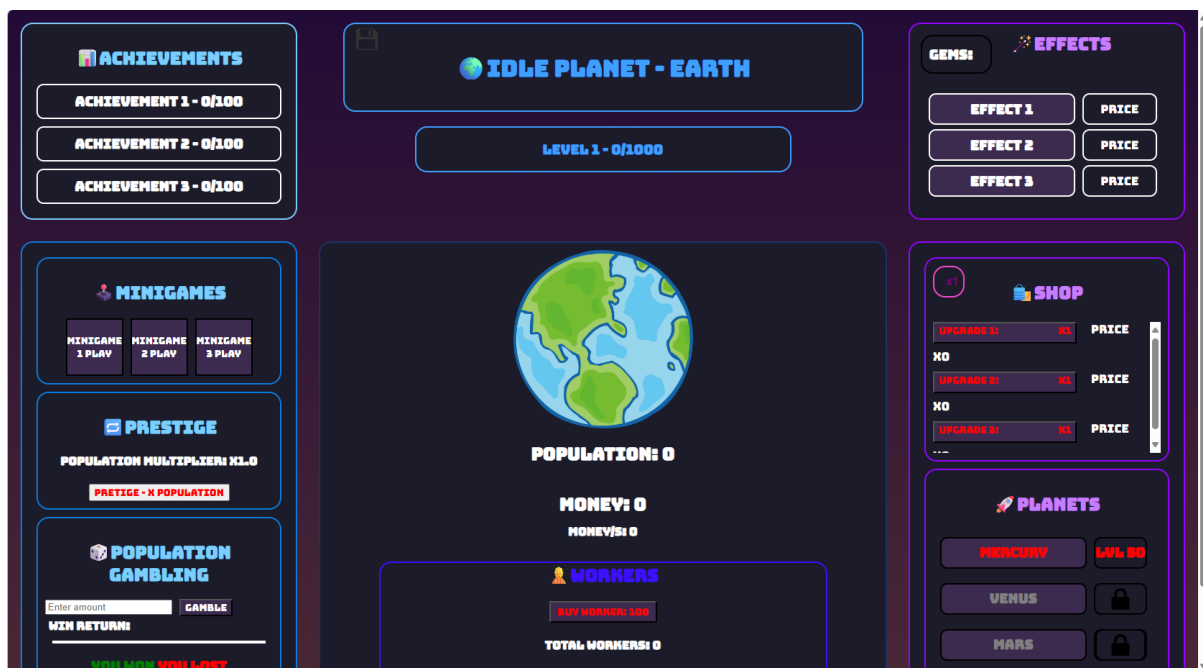
```
.achievement {  
  border: 2px solid black;  
  border-radius: 10px;  
  margin: 10px;  
  text-align: center;  
  height: 45px;  
  display: flex;  
  justify-content: center;  
  align-items: center;  
}
```

I added the last 3 lines to center the text vertically and horizontally, so it is in the middle of each achievement. This just makes the game look neater and more professional.

I also noticed the same thing with the level display, as the text is off centered and closer to the bottom of the container. I did the same thing for this as for the achievement text:

```
.level {
  grid-row: 2;
  grid-column: 2;
  height: 40px;
  width: 500px;
  text-align: center;
  display: flex;
  justify-content: center;
  align-items: center;
}
```

This is what both of these modifications look like on the page:



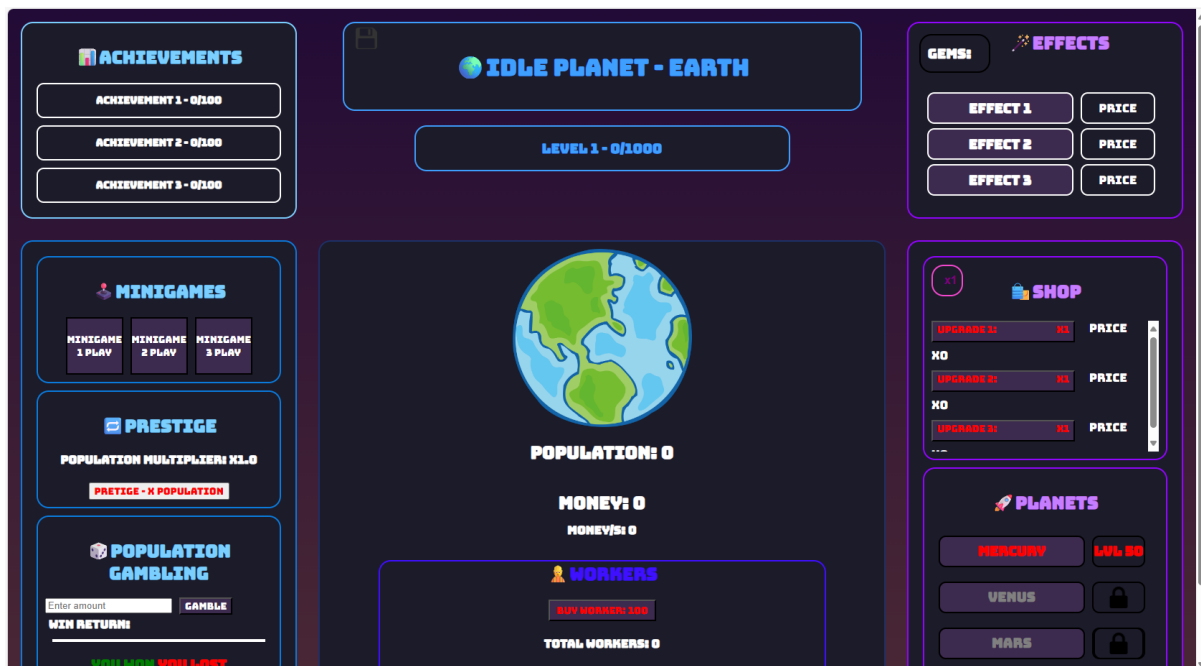
Finally, I think that the text size is slightly too big for the achievements, so I am going to make it smaller in the css:

```

.achievement {
  border: 2px solid black;
  border-radius: 10px;
  margin: 10px;
  text-align: center;
  height: 45px;
  display: flex;
  justify-content: center;
  align-items: center;
  font-size: 12px;
}

```

I changed the font size of each achievement, which is slightly smaller than what it currently is, which look like this:



Testing:

This is my test plan for stage 1:

Test no.	Description	Test type	Test data	Expected Result	Actual result
1	Is the title section set up correctly		Open the game and look at the top middle of the page	There should be a container which displays the game title and the name of the current planet. The save icon should also be in the top left of this container	There is a container that reads “Idle Planet – Earth”, and there is a save icon in the top left. Passed  See test evidence 1
2	Is the level section set up correctly		Open the game and look at the top middle of the page, below the title section	There should be a container that displays the current level and xp (out of the xp needed for the next level)	There is a container underneath the title container that reads “Level 1 – 0/1000”. Passed  See test evidence 2

3	Is the central section set up correctly.		Open the game and look at the centre of the page	There should be a container, where the planet is displayed at the top, with the population, money and money/s text display underneath	There is a container with the planet image at the top, and then a text display for the population, money and money/s. Passed ☒ See test evidence 3
4	Is the worker section set up correctly		Open the game and look at the bottom of the central section	Underneath the population and money display, there should be a container for the workers, with a buy worker button and 2 text displays underneath (One for total workers and one for total worker gen)	At the bottom of the central section, there is a container for the workers, which has a buy worker button, a text display that reads “Total workers: x” and another display that reads “Total worker gen: x population/s”. Passed ☒ See test evidence 4

5	Is the left section set up correctly		Open the game and look at the left of the page	There should be a container in the top right which displays the achievements. Underneath, there should be another container which has three containers within it (the minigame, prestige and population gambling containers)	There is a container in the top left for the achievements. Underneath, there is a container which has 3 containers inside, one for minigames, one for prestige, and one for population gambling. Passed ☒ See test evidence 5
6	Is the right section set up correctly.		Open the game and look at the right of the page	There should be a container in the top right which displays the effects (along with the players gems in the top left of this container). Underneath, there should be a container	There is a container in the top right for the effects, with the player's gems displayed in the top left. Underneath this, there is a container with a container for the shop and the planets

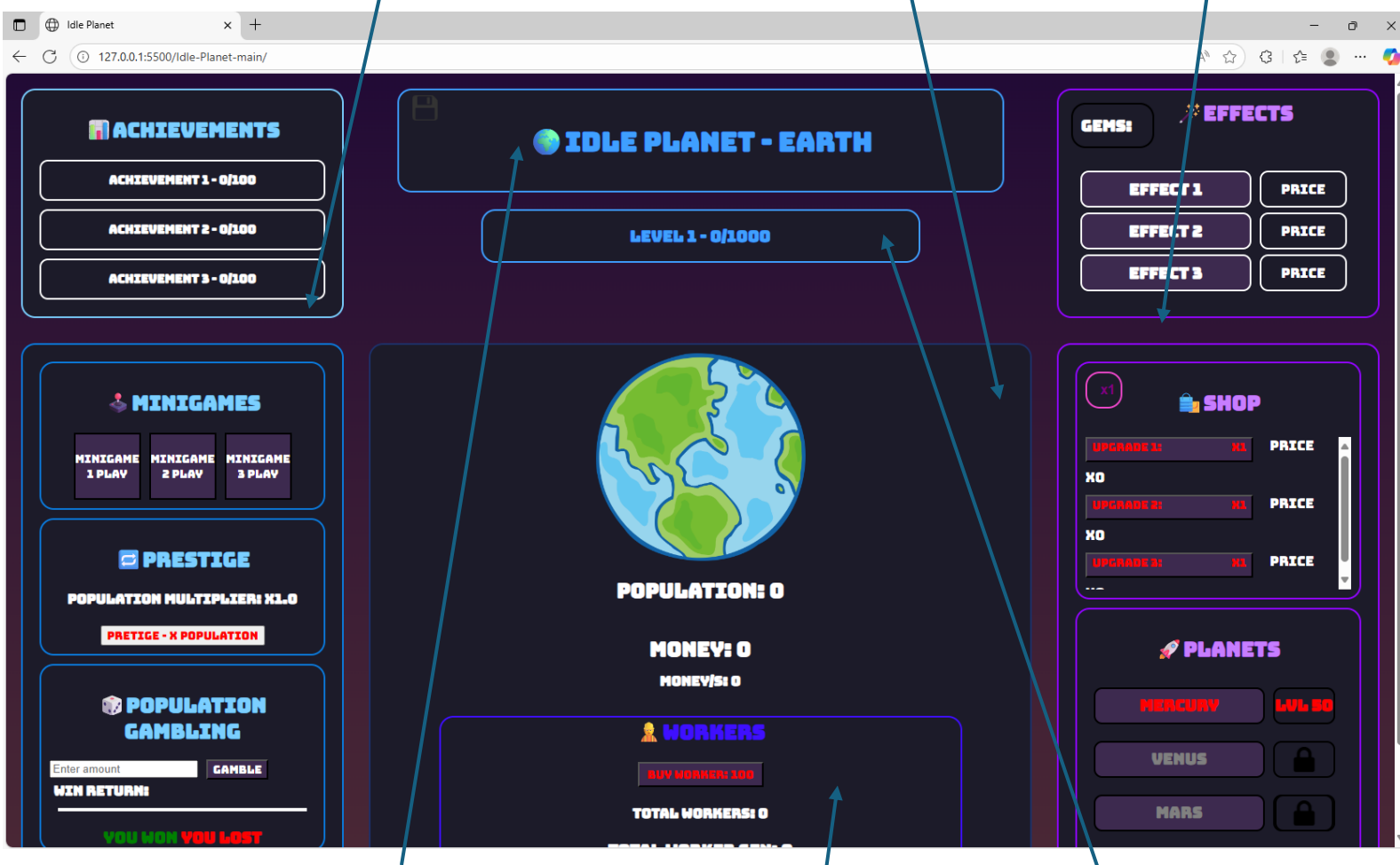
				with two containers within it (the shop container and the planets container)	inside. Passed  See test evidence 6
--	--	--	--	--	--

Test evidence:

Test evidence 5: There is a left section, with an achievements section in the top left, and then a container which has 3 containers within, one for minigames, one for prestige, and one for population gambling

Test evidence 3: In the centre of the page, there is a container with the planet image at the top, with a population, money and money/s display underneath

Test evidence 6: There is a right section, with an effects section in the top right, and then a container underneath with 2 containers within, one for the shop and one for planets



Test evidence 1: At the top middle of the page, there is a title section with the text "Idle Planet - Earth" and a save icon in the top left

Test evidence 4: At the bottom of the central section, there is a worker section with a buy worker button, and a total worker and total worker gen display underneath

Test evidence 2: Underneath the title there is a level section, which reads "Level 1 - 0/1000"

Stage review:

Overall, I am happy with the development of this stage. I correctly designed the GUI according to the GUI design, with all the correct sections, text displays, buttons and input fields. I have placeholder text for all the displays, ready to be updated with their correct values. I also correctly applied the colour scheme according to the GUI design, with all the correct background, border, button and text colours. There were little errors during development, and I managed to complete this stage in quite a short amount of time.

Stage 2 – Planet Clicking + Population and Money Generation

Features:

- Clicking the planet increases population by 1
- Money increases by 0.01 every second for every 1 population
- Money per second display shows the money being generated per second

Data Dictionary

Variable	Data type	Purpose
game_data {}	Dictionary	To store necessary data about the game, like the player's

		money and population.
game_data.population	Key (Integer)	Stores the population of the planet
game_data.money	Key (Float)	Stores the player's money
game_data.money_per_second	Key (Float)	Stores the amount of money that the player's money increases per second.

Pseudocode:

Game data:

```
dict game_data {
    population = 0
    money = 0
    money_per_second = 0
}
```

Planet click:

```
planet-button Eventlistener(click) {
    population += 1
    money_per_second = population * 0.01
```

```

HTML.population.text = population
HTML.money-per-second.text =
money_per_second
}

```

- This is an event listener for the planet button, that will detect any time it is clicked. When it is clicked, it will run the code.
- The code simply increases population by 1 and money per second by $\text{population} * 0.01$. It then updates the text of the HTML elements for the population and money per second display, setting them to the variable for population and money per second (respectively) in the JavaScript.
- **Note:** When I access population and money per second (and any other variables in the game data dictionary in the future), I will need to use `game_data.[variable_name]`, for example `game_data.population`. However, for the purpose of the pseudocode I have left this out as it is generally unnecessary. When it is necessary and important to include, I will.

Money increase:

```

WHILE true
    money += money_per_second
    HTML.money.text = money

```

WAIT 1
ENDWHILE

- This is an infinite loop that runs every second
- It increases money by money per second, and then updates the text of the HTML element for the money display, setting it to the money variable in the JavaScript#

Development:

```
</body>  
<script src="script.js"></script>  
</html>
```

At the bottom of my HTML, I linked my JavaScript file using the script tag

```
JS script.js > ...  
1  const game_data = {  
2    population: 0,  
3    money: 0,  
4    money_per_second: 0  
5  }  
6  
7  
8
```

In the JavaScript, I created a game_data dictionary, which holds population, money and money per second (all of which I set to 0).

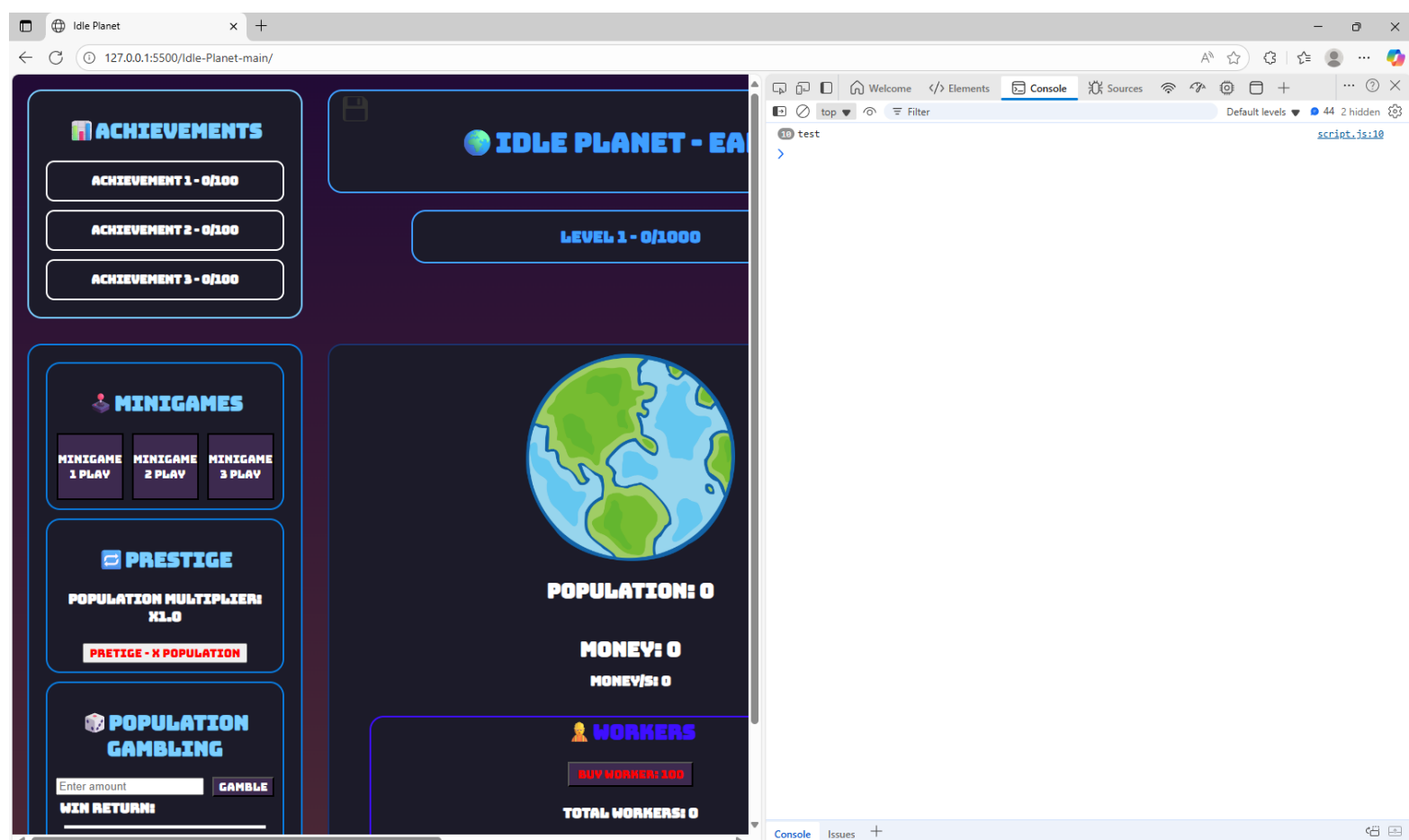
As I develop, I will add more variables to this dictionary for the data the game needs. The reason I have chosen a dictionary data structure is because it groups all the game variables together, so that when I come to implementing the save feature, all the variables are in one place and I can save them all at once. If I were to use individual variables for all the data, it would make the saving process more complicated as I would have to access each variables individually.

```
<div class="center">
  <button class="planet-button" id="planet-button">
    <img class="planet-icon" src="https://www.pinclipart
  </button>
  <h2>Population: </h2>
```

In the HTML, I gave the planet button the id “planet-button”, so I can access it in the JavaScript.

```
document.getElementById("planet-button").addEventListener("click", function (event) {
  console.log("test")
})
```


In the JavaScript, I added an event listener to the planet button, with the event “click” (which is the event when the button is clicked). In the even handler, I added a console.log statement which logs “test” to the console so I can test to see if it is working.



“test” was logged to the console every time I clicked the planet, so the code is working.

```
<h2>Population: <span id="population">0</span> </h2>
<h2 style="margin-bottom: -10px">Money: <span id="money">0</span> </h2>
<h4>Money/s: <span id="money-per-second">0</span></h4>
```

I then added 3 spans inside each text display under the planet with the ids “population”, “money” and “money-per-second” respectively, so I can access each display in the JavaScript, allowing me to update the player’s population, money and money per second on the page. I have put 0 in each span, as the values all start off at 0.

```
// Click event listener for planet button
// Event handler updates population and money_per_second, and updates the population and money per second display in the HTML
document.getElementById("planet-button").addEventListener("click", function (event) {
  game_data.population++;
  game_data.money_per_second = game_data.population * 0.01
  document.getElementById("population").textContent = game_data.population
  document.getElementById("money-per-second").textContent = game_data.money_per_second
})
```

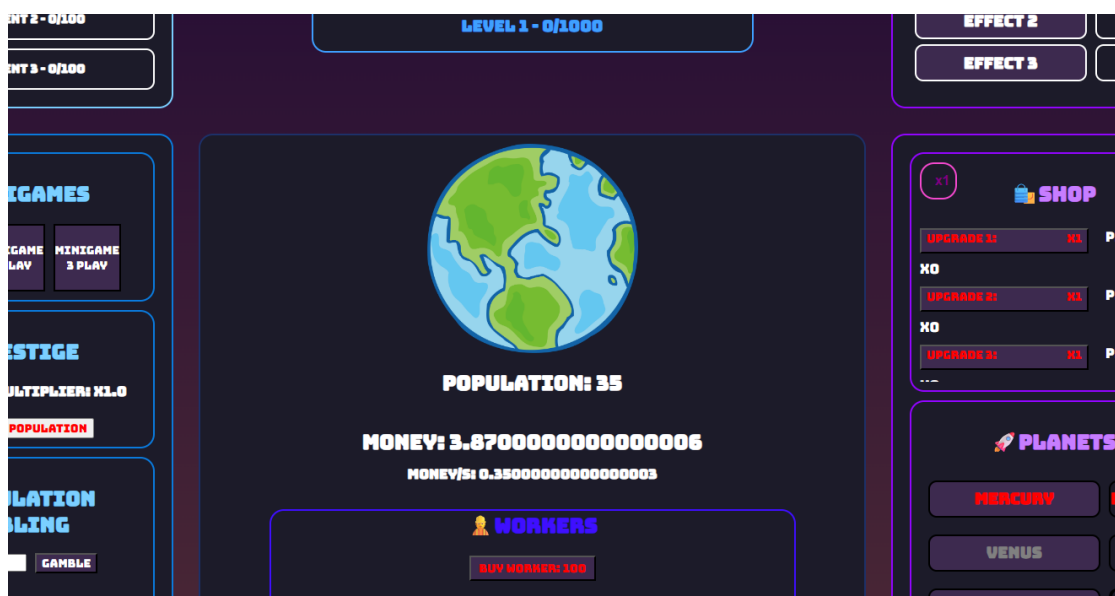
I then added an event listener onto the planet button, using its id, with code that handles when the button is pressed. In the event handler, I increase the population by 1 because clicking the planet increases the population by 1. I also increase the money per second by the population times 0.01, as the player earns 0.01 money per population per second. I then update the population and money per second displays in the HTML using the id of the population and money per second span.

```
// Loop that runs every second, increases money by money_per_second and updates money display in the HTML
setInterval(() => {
  game_data.money += game_data.money_per_second
  document.getElementById("money").textContent = game_data.money
}, 1000)
```

I then created a loop that runs every second (1000 milliseconds) which increase the player’s money by the money per second variable, and then updates the money display in the HTML using

the id of the money span. I could have just increased the money by $\text{population} * 0.01$, and not used a `money_per_second` variable, however I did this because I may want some upgrades to increase the amount of money that the player earns per second, in which case I can just modify the `money_per_second` variable to make the player's money increase by more, making it much easier and less complicated.

I tested this out by running the game and clicking the planet:

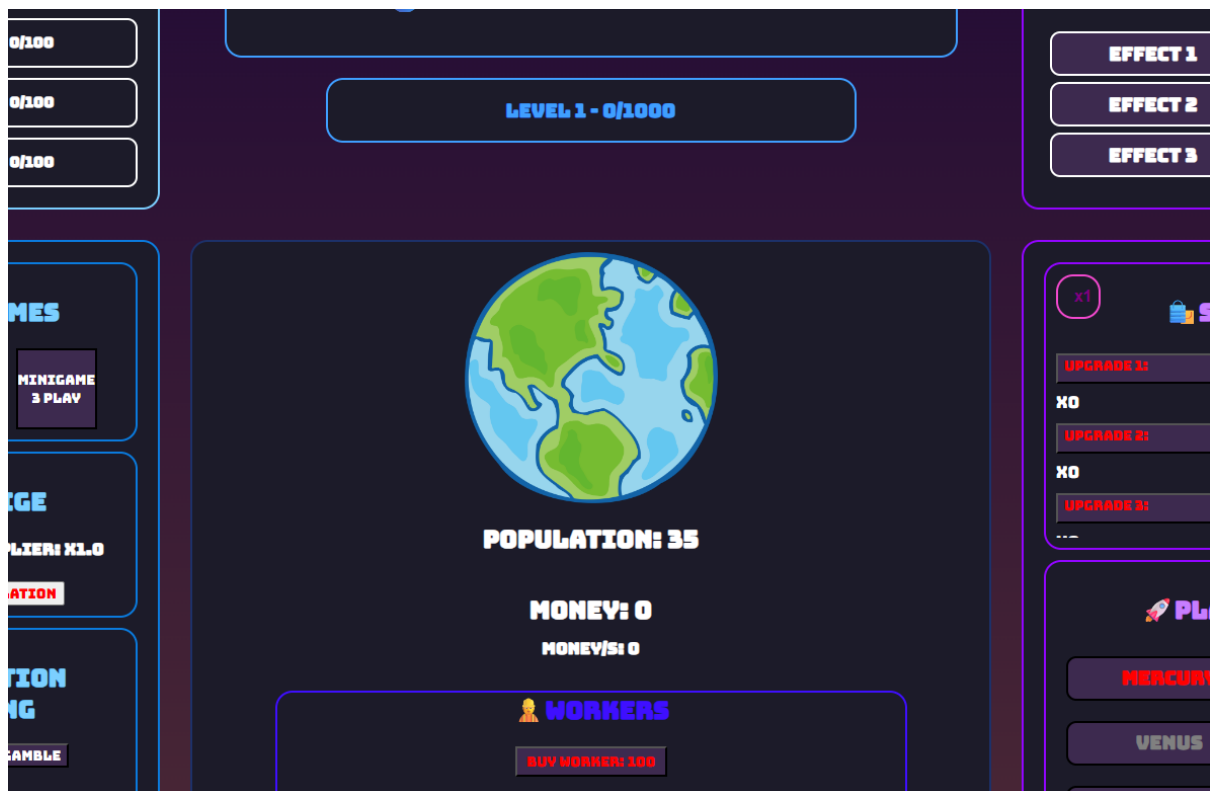


All displays are correctly increasing, when I click the planet the population increases by 1, the money/s then increases by 0.01 and the money then increases by 0.01 every second. However, there is an error with the money and money per second display as there are many 0s added onto the end of the value which should not be the case. I realised that this was because of the way JavaScript handles floats, so I needed to round the money and money/s values to just 2 decimal places:

```
game_data.money = game_data.money + Math.round(game_data.money_per_second, 2)
```

```
game_data.money_per_second = Math.round(game_data.population * 0.01, 2)
```

However, this did not fix the issue and stopped the money and money/s increasing at all:

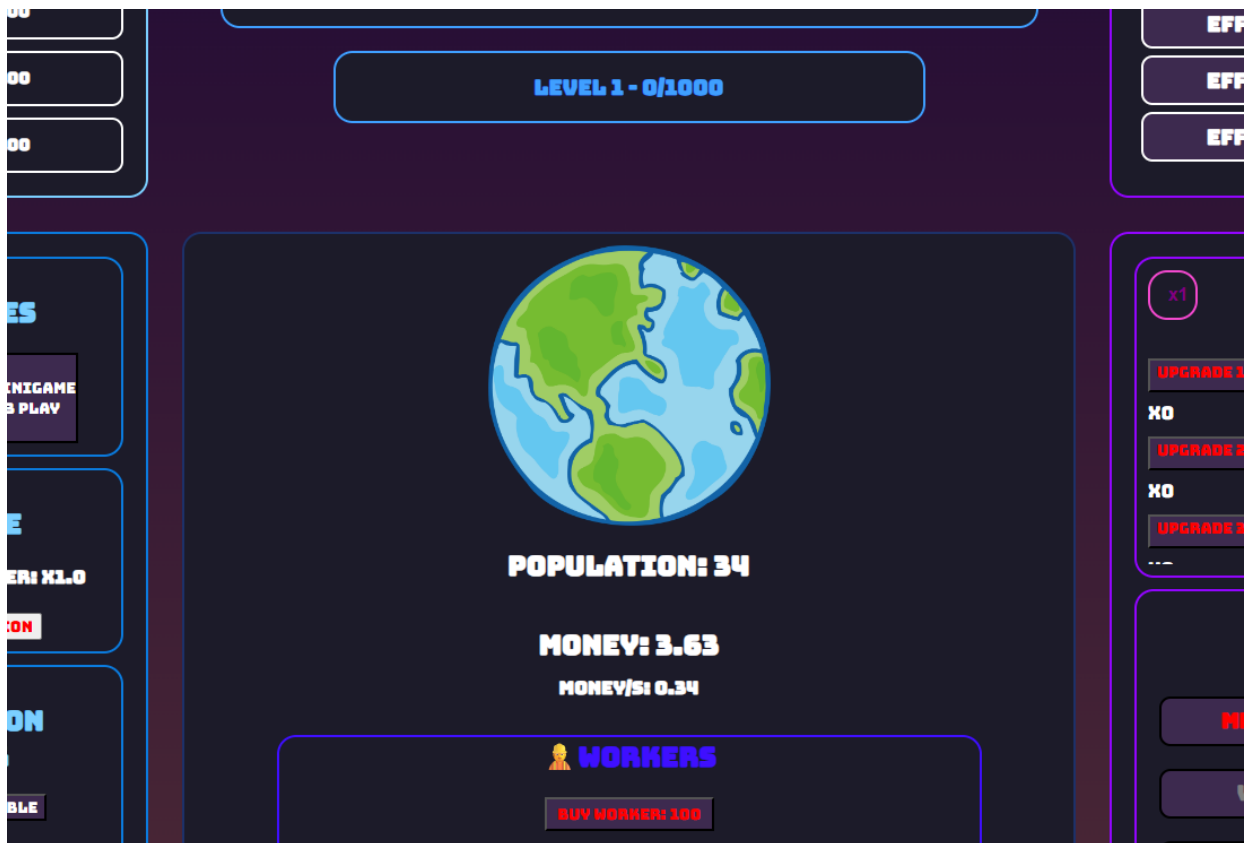


I then realised that this was because when I times the population by 0.01 and then round it to 2 decimal places, this just gets 0. Therefore, instead of rounding I thought of multiplying them by 100 and rounding the result and then dividing them by 100 to get rid of the decimal places. This should work because it will multiply the values, moving the first 2 decimal places (the ones needed) to the right of the decimal point, and then remove the rest of the decimal places by rounding. Then dividing the number by 100 should return it to the correct value with just 2 decimal places

```
game_data.money = Math.round(game_data.money * 100) / 100
```

```
game_data.money_per_second = Math.round(game_data.money_per_second * 100) / 100
```

I tested this by running the code:



This completely fixed the issue, now the money and money/s displays don't display any more than 2 decimal places. I added the comment "//Removes extra, unwanted decimal places" to the end of each of these lines to explain this code as it may not be clear.

Testing:

This is my test plan for stage 1:

Test no.	Description	Test type	Test data	Expected Result	Actual result
1	Does the population increase correctly when you click the planet		Click the planet and see if the population number goes up	The population number increases by 1 (Before any upgrades) every time you click the planet	The population increases by 1 every time the planet is clicked. Passed ☒ SEE STAGE 2 TEST 1.mp4 in evidence folder
2	Does the money increase as the population increases		Click the planet to increase the population, and see if the money	The money should start automatically increasing by 0.01 every second once the population increases above 0. The amount that	The money increases by 0.01 per population (after the population is at least

			starts increasing	the money increases every second should increase as the population increases.	1). At 10 population, the money increases by 0.1, which shows it is correct. Passed ☒ SEE STAGE 2 TEST 2.mp4 in evidence folder
3	Does the money per second display increase correctly		Click the planet to increase the population and observe the money/s display underneath the money display.	The money per second should increase by 0.01 per 1 population. At 10 population, for example, it should say 0.1	The money per second increases by 0.01 each time the population increases by 1. At 10 population, the money per second says 0.1, which shows it is correct. Passed ☒ SEE STAGE 2 TEST 3.mp4 in

					evidence folder
--	--	--	--	--	--------------------

Stage review:

Overall, I am very happy with the development of this stage. I implemented the planet clicking feature exactly how I planned, with the player beign able to click the planet to increase the population. I also correctly implemented the automatic money generation, which increases with population growth. There were very little errors, only a simple formatting/rounding error that I managed to fix quickly, and the implementaiton of the stage did not take long at all.

Stage 3 – Workers:

Features:

- Workers that increase the population without the player doing anything
- Workers will increase the population by 1 every 2.5 seconds
- Workers will start by costing 100, and will increase exponentially, the worker price will increase by a multiplier that increases by 0.05 each time a worker is purchased.
- The total workers display will display the total number of workers the player has
- The total worker gen display will display how much the workers are generating per second. Since the workers start by increasing money every 2.5 seconds, I will have to divide the total amount they are making every 2.5 seconds by 2.5.
- The amount that the workers increase the population, as well as how quickly they do it, will both be upgradable stats later when I implement the shop

Data Dictionary:

Variable	Data type	Purpose
game_data.workers	Key (Integer)	To store the total number of workers the player owns
game_data.worker_price	Key (Float)	To store the current price of a worker
game_data.worker_gen	Key (Float)	To store the value that indicates the population the

		workers generate per action (By action, I mean when the workers make a population increase)
game_data.worker_price_multiplier	Float	To store the multiplier that is used to exponentially scale the price of the worker. This multiplier is increased each time a worker is purchased.
game_data.worker_increase	Float	To store the amount that each worker increases the population by every action
game_data.worker_speed	Float	To store the time interval between each worker increase in seconds

Pseudocode:

Add to game_data:

```
workers = 0
total_worker_gen = 0
worker_speed = 2.5
worker_increase = 1
worker_price = 100
worker_multiplier = 0.1
```

Worker Purchase:

```
worker-button Eventlistener(click) {
  IF money > worker_price THEN
    workers += 1
    money -= worker_price
    worker_price *= 1 +
    worker_price_multiplier
    worker_price_multiplier += 0.05
    total_worker_gen = (workers *
    worker_increase)
```

```

        HTML.workers.text = workers
        HTML.worker-price.text =
        worker_price
        HTML.worker-gen.text =
        worker_gen
    ENDIF
}

```

- I will have a click event listener for the worker button, which will handle the worker purchases
- It will check if the player has enough money to purchase the worker, and if so increase the worker total, take off the worker price from the money, correctly scale the worker price according to the price multiplier, and update the worker gen value according to the current total workers multiplier by the total worker increase
- Finally, it will update the worker text displays in the HTML.

Worker Click:

```

WHILE True
    population += total_worker_gen
    money_per_second = population *
    0.01
    HTML.population.text =
    population
    HTML.money-per-second =
    money_per_second

```

WAIT worker_speed

ENDWHILE

- I will have an infinite loop for the worker action, with the time interval between loops being the worker speed
- The loop will update the population by the total worker gen, then recalculate money per second since it is based off population and the population has just increased
- Finally, it will update the population and money per second text values in the HTML, since their JavaScript values have just been updated

Development

I started the development of this stage by adding the variables needed to the game_data dictionary:

```
const game_data = {  
  // POPULATION AND MONEY  
  population: 0,  
  money: 0,  
  money_per_second: 0,  
  
  // WORKERS  
  workers: 0,  
  worker_speed: 2.5, // The amount of time (in seconds) that each worker takes to increase the population  
  worker_increase: 1, // The amount of population that each worker increases every action  
  total_worker_gen: 0, // The amount of population that all workers combined are generating per second  
  worker_price: 100,  
  worker_price_multiplier: 0.1  
}
```

The reason I have variables for the worker speed and population increase, is because these values will be upgradable through

shop upgrades. Therefore, when the values need to be upgraded I can just update them in the game data dictionary. I added comments to explain the values that may not be clear on their own, and comments to section the dictionary into different parts to make it easier to read as it will likely get long with all the values needed throughout the program.

I then moved onto the worker purchasing:

```
document.getElementById("worker-button").addEventListener("click", function (event) {  
    if (game_data.money > game_data.worker_price) {  
        game_data.workers ++  
        game_data.money -= game_data.worker_price  
    }  
})
```

I created an event listener that listens for a click event on the worker buy button, to check if the player is trying to buy a worker. In the event handler, I used an if statement to check if the player's money is more than the current worker price, to see if they have enough money to buy a worker. If the if statement evaluates to true (the player has enough money), then I increase the workers variables by 1 and reduce the player's money by the current worker price.

I then wrote the code to scale the worker price and increase the price multiplier (This is still inside the if statement inside the event handler):

```
game_data.worker_price *= 1 + game_data.worker_price_multiplier  
game_data.worker_price = Math.round(game_data.worker_price * 100) / 100  
game_data.worker_price_multiplier += 0.05
```

I scale the worker price by multiplying it by $1 + \text{the price multiplier}$. This means that the price of the second worker will become $100 * 1.1$ which is 110. I also remembered from stage 2 that when multiplying float values I had the issue of JavaScript adding lots of 0s onto the end of the number, so I used the same method as last time to remove them (By rounding the result of multiplying the price by 100, and then dividing that result by 100). After this, I add 0.05 to the worker price multiplier. I do this so that each time the price is increased, it is multiplied by an increasing amount, allowing it to scale exponentially. This will make it harder to buy workers as the player progresses into the game (making it more challenging and exciting).

After this, I wrote the code to set the total worker gen value (still inside the if statement inside the event handler):

```
game_data.total_worker_gen = (game_data.workers * game_data.worker_increase)
```

I set the total_worker_gen to the value of the workers multiplied by the worker_increase. This is because multiplying the number of workers by the amount that each worker increases the population by per action will get the amount that all workers are increasing per action. The reason I have used a total worker gen value is, firstly, because in the loop where the workers increase the population, I can just increase it by the total worker gen value. I also may want to make an upgrade that increases total worker generation by a certain amount, meaning I can just modify the

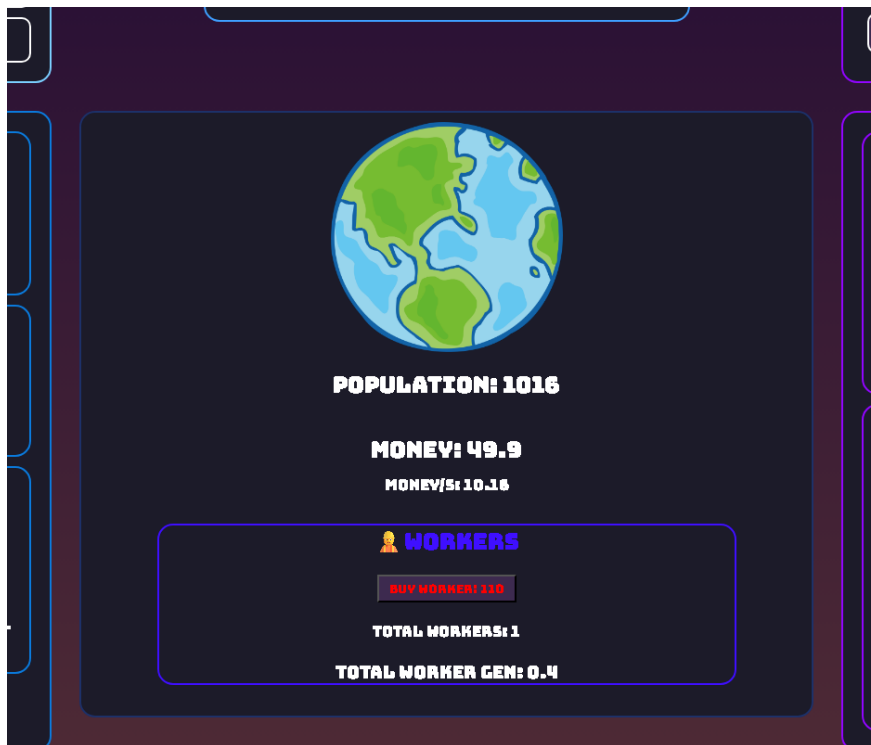
total worker gen to increase the total amount that all workers generate.

Finally, I simply updated the text content of all the html elements whose values will change after a worker is bought

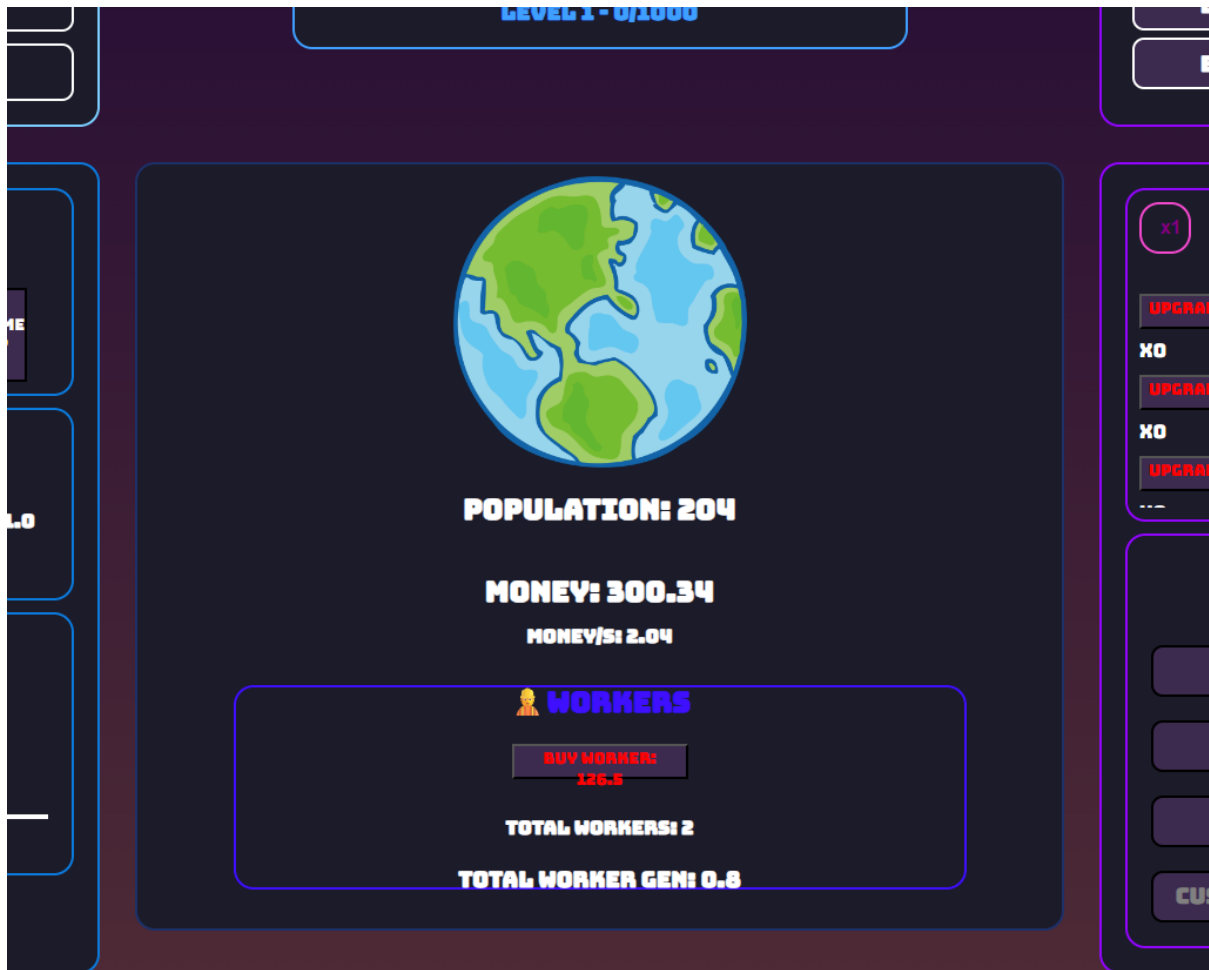
```
document.getElementById("workers").textContent = game_data.workers  
document.getElementById("worker-price").textContent = game_data.worker_price  
document.getElementById("worker-gen").textContent = game_data.total_worker_gen / game_data.worker_speed
```

The reason I have set the text content of the worker gen display to the value of the total_worker_gen divided by the worker_speed is because the total_worker_gen value stores the total population that the workers generate every time interval (which starts at 2.5s), however the total worker gen display on the page is supposed to display the total worker gen per second. Therefore, I divide the total worker gen by the worker speed to get the total worker gen per second, which is what I display on the page.

I then ran the code to see if it worked:



When I pressed the buy worker button, the money decreased by a 100 (Correct as the worker price starts at 100), the total workers display increased to 1 (Correct as 1 worker was purchased), and the total worker gen increased to 0.4 (Correct because 1 population every 2.5 seconds is the same as 0.4 population every second). The only problem I noticed was that when the worker price became a decimal number, it became too long to fit on one line of the button so it went over a line, and overflowed outside of the button:



This means that I need to increase the height of the button to fit all the content. I could just increase the height of the button by 20 pixels or so, however, this would permanently increase the height of the button, even when the worker price can fit on one line. Because I want it to only increase in height when it needs to, I will set the height to auto, meaning it will adjust its height to fit all the content:

```
.worker-button {  
  width: 150px;  
  height: auto;  
}
```

I then ran this to see if it worked:

LEVEL 1 - 0/1000



POPULATION: 100

MONEY: 354.77

MONEY/s: 1



WORKERS

BUY WORKER: 100

TOTAL WORKERS: 0

TOTAL WORKER GEN: 0

x1

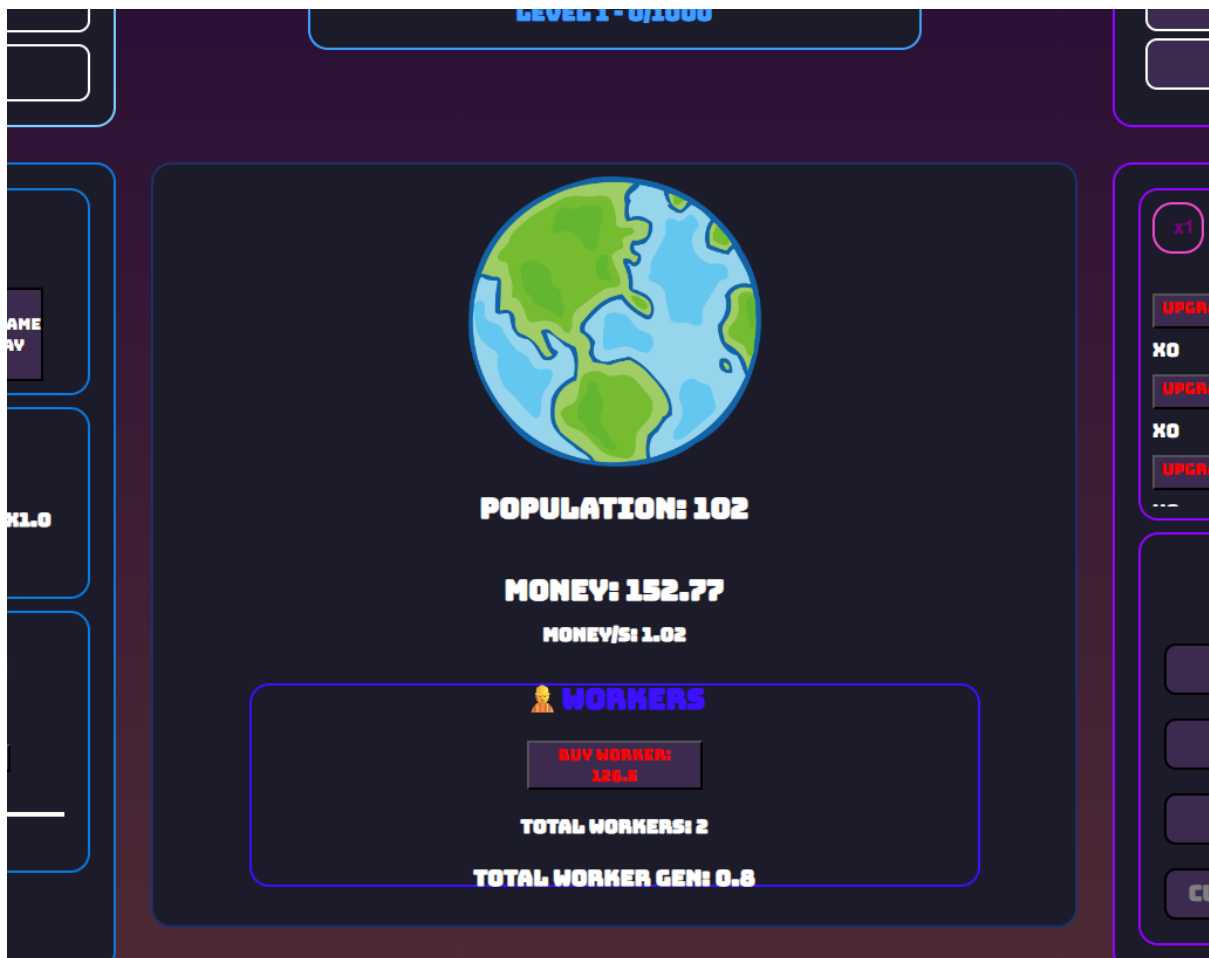
UPGR

X0

UPGR

X0

UPGR

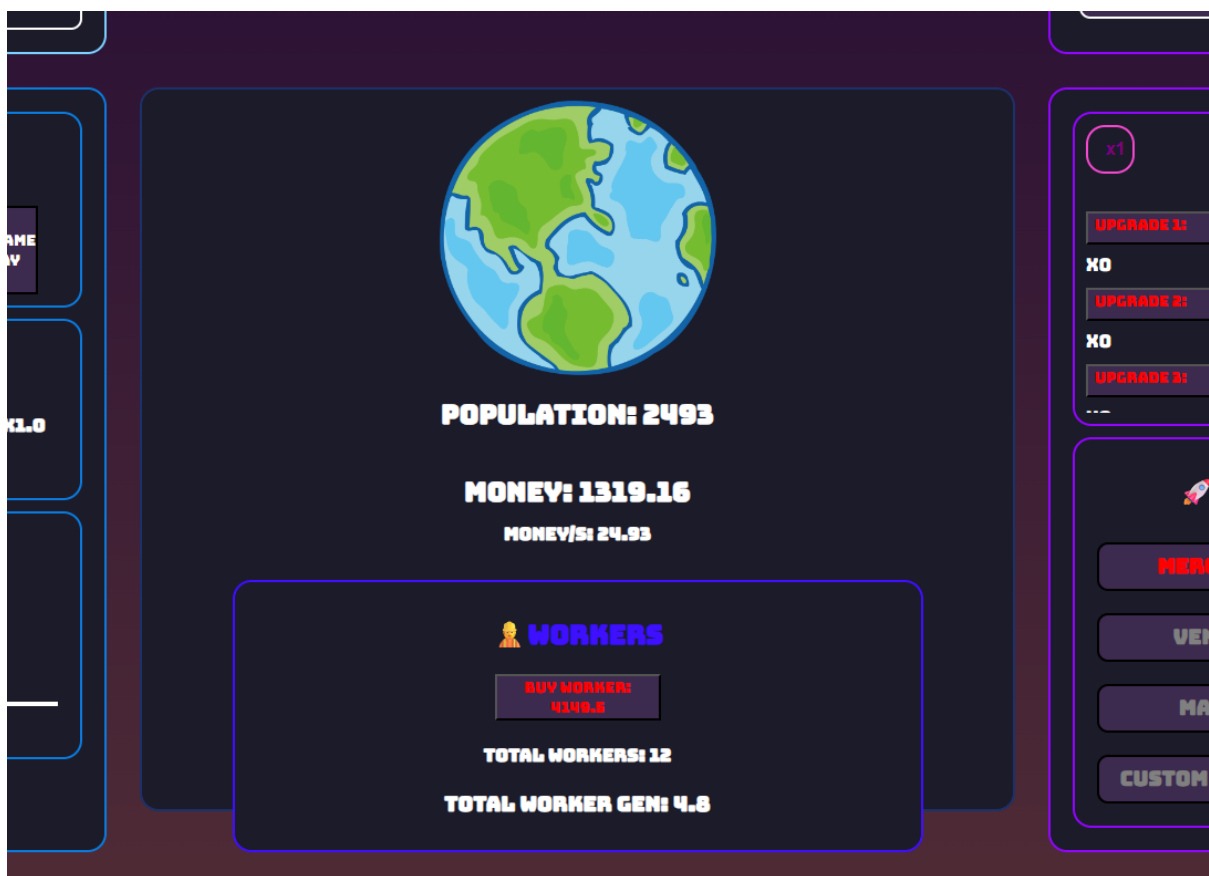


Now, when the worker price is a whole number with no decimal places, the height of the button remains as normal, but when it becomes a number with decimal places and cannot fit on one line, the height of the button increases to fit it all. The only problem with this is that it pushes the rest of the elements up and down, causing the title and total worker display to overflow out of the container.

To fix this, I will also set the height of the worker container to auto, so it will expand to fit all the content when needed:

```
.workers {  
  height: auto;  
  width: 600px;  
  display: flex;  
  text-align: center;  
  justify-content: center;  
  flex-direction: column;  
  margin: 0px;  
}
```

I then ran this to see what it looked like:



This worked but it also caused the worker container to overflow the central container, meaning I have to do the same thing and set the central container height to auto, so it will expand to fit the whole worker container:

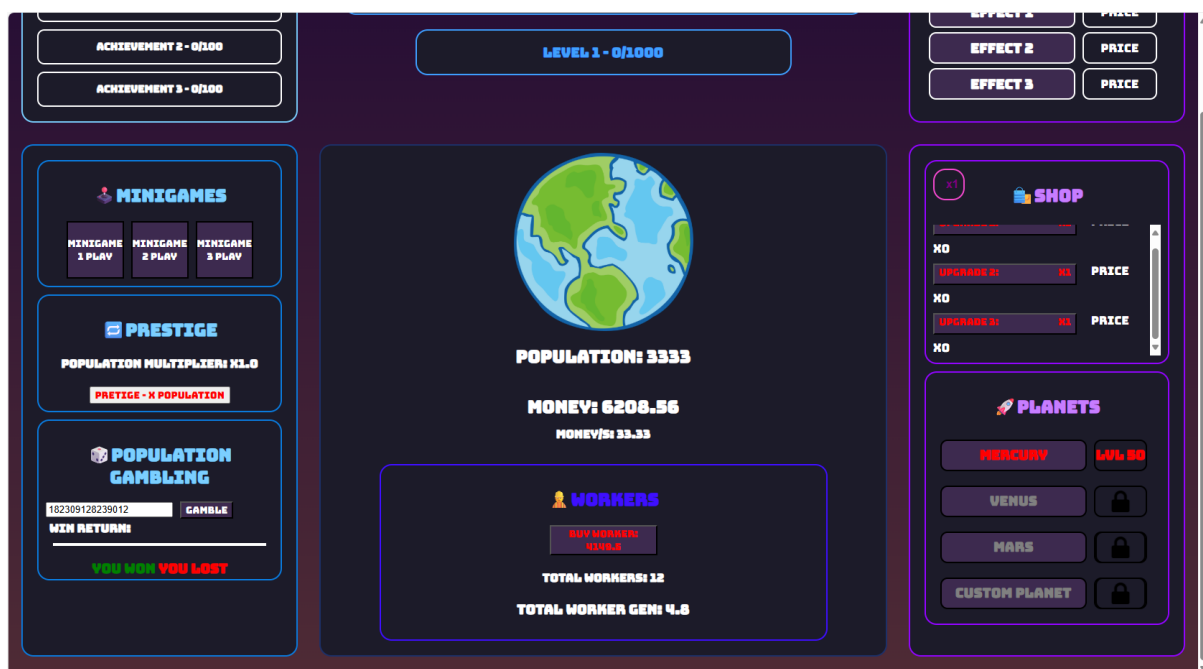
```

.center {
  height: auto;
}

```

I put this separate to the css that styles multiple elements (including the central section) at once, as I don't want to set the height to auto for all those other elements, only for the central section.

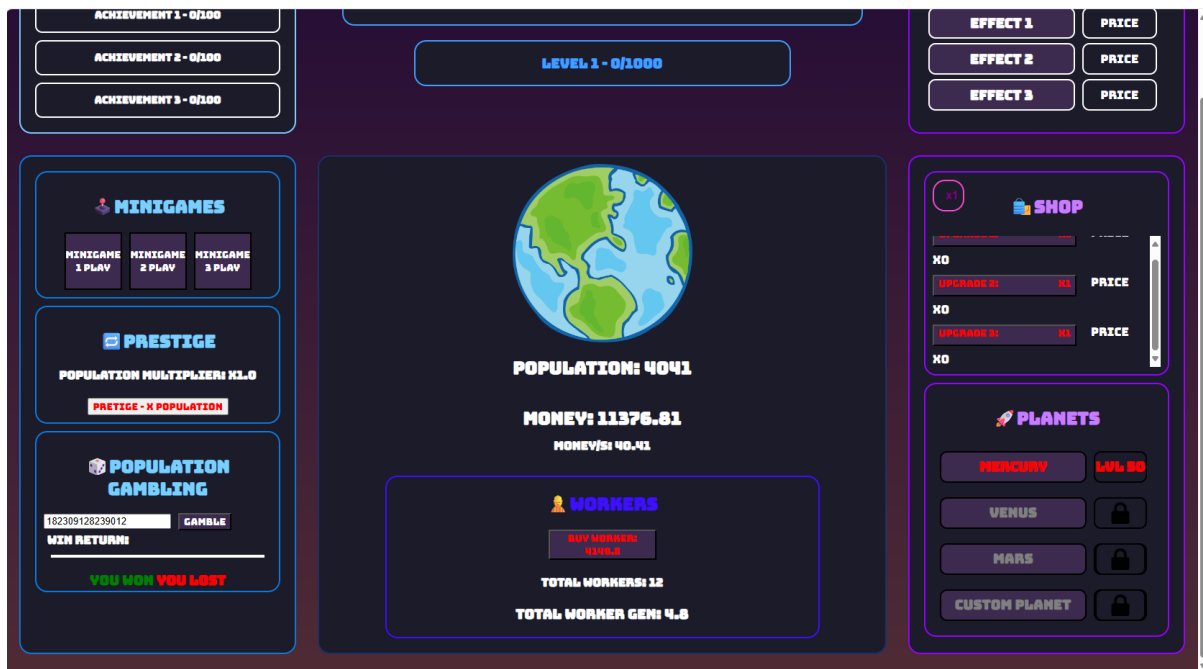
This looks like this on the page:



Now, the central section correctly fits the worker section, with no overflow. However, I noticed the padding in the worker section is too much and creates unnecessary space that makes the container longer. The reason for the padding is that in the css code where I style multiple elements at once (including the worker container), I set the padding to 10px. Therefore, I set it to 0 at the bottom of the page to overwrite this:

```
.workers {
  padding: 0px;
}
```

This is what this looks like on the page:



Now, I am happy with the spacing and sizing of the worker container. It fits the content in the button correctly, and expands to fit everything without overflowing the central container.

I then coded the worker loop, where I will be increasing the player's population by the total worker gen value:

```
setInterval(() => {
  game_data.population += game_data.total_worker_gen
  game_data.population = Math.round(game_data.population * 100) / 100 //Removes extra, unwanted decimal places
}, game_data.worker_speed * 1000)
```

I started by creating an infinite loop. The interval between each loop is the worker_speed value multiplied by 1000. This is because in JavaScript, the time interval between the loops is in

milliseconds, so 1000 for example would be 1 second. Because the worker speed value starts at 2.5 (2.5 seconds) and will decrease with upgrades, I need to multiply it by 1000 to make it 2500 milliseconds which is 2.5 seconds.

Inside the loop, I set the population to the total worker gen value, as this is the value that indicates the total amount of population that is being generated per action by all the workers combined.

I then use the method to remove the extra, unwanted decimal places like I have used before.

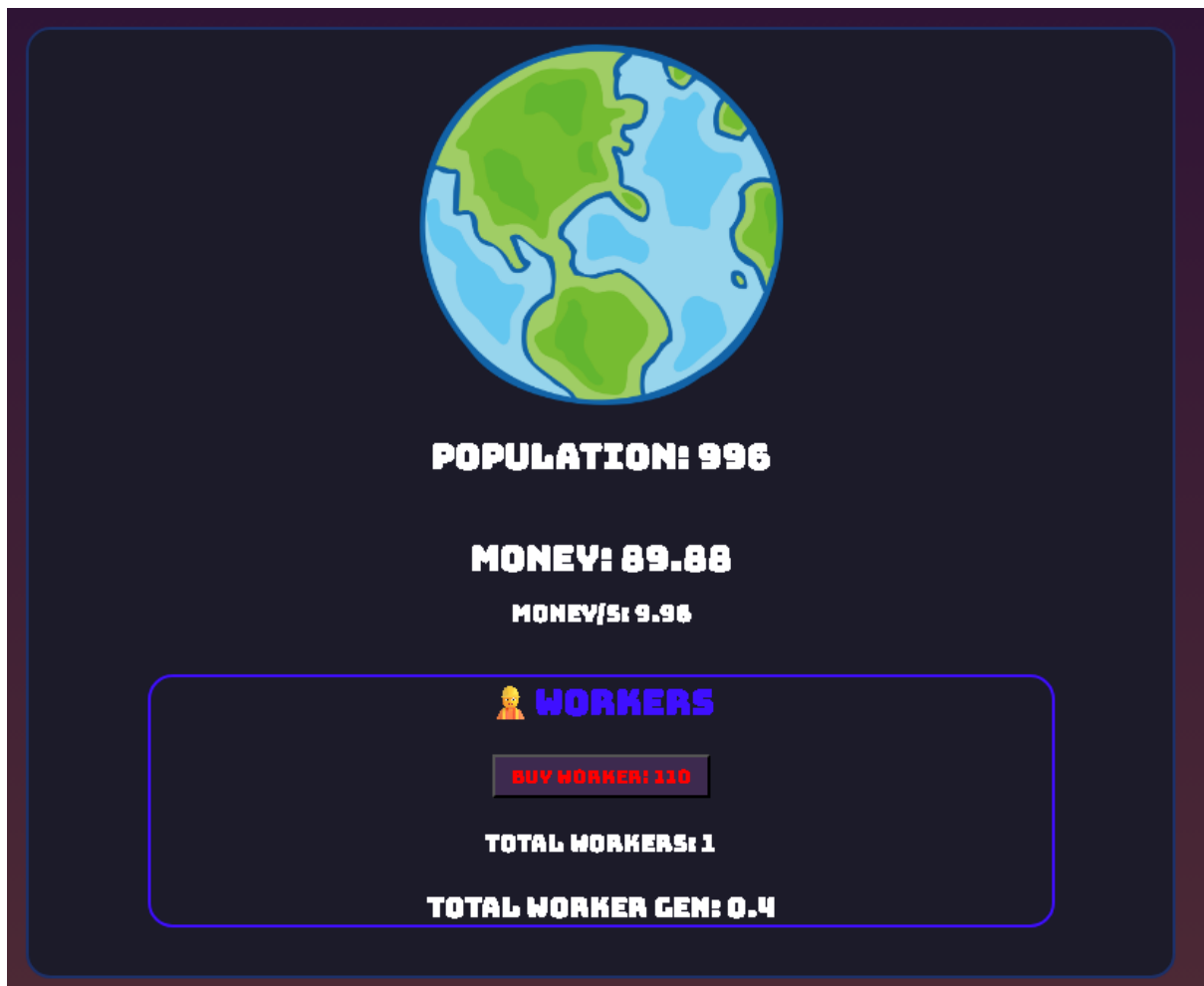
```
game_data.money_per_second = game_data.population * 0.01  
game_data.money_per_second = Math.round(game_data.money_per_second * 100) / 100 //Removes extra, unwanted decimal places
```

I then update the money per second value, since each time the population is increased, the money per second increases. I set the money per second to the population multiplied by 0.01 as money increased by 0.01 for each population. I then remove the extra decimal places from the value like I always do.

Finally, I update the text content of the population and money per second displays as the value of these both change in the loop:

```
document.getElementById("population").textContent = game_data.population  
document.getElementById("money-per-second").textContent = game_data.money_per_second
```

I ran this code to see if it worked:



After purchasing a worker, the population correctly increased, by 1 every 2.5 seconds. The money per second display was also correctly updated each time the worker increased the population, which then correctly updated the amount that the money increased by every second. Therefore, the code is working.

Testing

This is my test plan for this stage:

Test no.	Description	Test type	Test data	Expected Result	Actual result
1	Is the player prevented from purchasing workers when they don't have enough money		Press the buy worker button without any money	Nothing should happen; the total workers number should not increase at all and money should stay the same	Nothing happens, total workers doesn't increase and money stays the same. Passed  SEE STAGE 3 TEST 1.mp4 in evidence folder
2	Is the player able to purchase a worker		Press the buy worker button with enough money and see if the total workers number increases	The total workers number should increase by 1	The total workers display increases by 1. Passed  SEE STAGE 3 TEST 2.mp4 in

					evidence folder
3	Does the worker automatically increase the population		Buy a worker and check whether the population number is increasing automatically	The population number should increase by 1 every 2.5 seconds (Before any upgrades)	The population increases by 1 every 2.5 seconds. Passed ☒ SEE STAGE 3 TEST 3.mp4 in evidence folder

Stage review:

I am also very happy with the development of this stage. I correctly implemented the workers, allowing them to be purchased by the user to automatically generate population. Everything works as expected and there were only a few small layout bugs during development. I also completed this stage in a very short amount of time, which will allow me to spend more time on the more complicated features after.

Stage 4 – Shop:

Features:

- I will create upgrades for all main features of the game
- Like with the workers, I will have a price and price multiplier for each upgrade, that will increase by different amounts when the upgrade is purchased.
- I will choose the amount that the price multiplier increases by depending on how powerful I think the upgrades are. However, I will likely modify the price and price multiplier of the upgrades once I test out the gameplay as they might be too low or high.
- Also, since I have only developed the population, money and workers system at this stage, I will only be able to develop upgrades for these features. For the rest of the upgrades, I will make it so that purchasing one will set a value to true or increase a multiplier, so that when I develop the related feature I can develop the upgrade so it actually has an effect on the feature.
- The colour of the upgrade and upgrade amount text should be green when the player has enough money to purchase it, and red when they don't
- **Population and money upgrades:**
 - **Click Surge** – Clicking grants 2% more population.
Price – 100. Price multiplier increases by 0.01
 - **Income Boost** – Earn 10% more money per population.
Price – 500. Price multiplier increases by 0.02
 - **Industry Boom** – Flat increase to money per second of 20%

Price – 10000. Price multiplier increases by 0.05

- **Worker upgrades:**

- **Worker Training** – Each worker generates 20% more population

Price – 1000. Price multiplier increase by 0.03

- **Worker Tools** – Reduce cost of workers by 10%

Price – 5000. Price multiplier increases by 0.05

- **Worker Efficiency** - Workers generate population 0.1s faster (Until they are generating every 0.5 seconds)

Price – 7500. Price multiplier increases by 0.8

- **Prestige upgrades:**

- **Population Multiplier Boost** – Increase population multiplier by 0.5

Price – 10000. Price multiplier increases by 0.8

- **Quicker Prestige** – Reduce population required to prestige by 5%

Price – 5000. Price multiplier increases by 0.7

- **Permanent click boost** – Each prestige increase click power by 5%

Price – 20000. Price multiplier increases by 0.8

- **Minigame upgrades:**

- **Double minigame rewards** – All minigame rewards are doubled

Price – 30000. Price multiplier increases by 0.125

- **Temporary population surge** – Money per second doubled for 10 seconds after winning a minigame. (Time of the surge will increase by 5s each upgrade)

Price – 20000. Price multiplier increases by 0.125

- **Population gambling upgrades:**

- **Gamble luck** – Gamble winning chance increased by 5%
Price – 50000. Price multiplier increase by 0.125
- **Win streak** – Consecutive gamble wins increases win chance by 5%
Price multiplier 5000. Price multiplier increases by 0.7

- **Achievement upgrades:**

- **Achievement gem boost** - Achievements grant 2% more gems (rounded up)
Price – 2000. Price multiplier increases by 0.03
- **Achievement xp boost** – Achievements grant 10% more xp
Price – 5000. Price multiplier increases by 0.05

- **Multi-purchase feature:**

- The multi-purchase feature will allow the player to purchase multiple of the same upgrade at a time if they have sufficient money
- The player will either be able to purchase 1 upgrade, 10 of an upgrades, 100 upgrades, or the max amount of the upgrade that they can purchase with the money they have
- This feature will take into account the price multiplier for purchasing the upgrade, so if a player wants to purchase 10 of an upgrade for example, then it will be the same total price as if they purchased them individually

- To figure out how this feature would scale the price multiplier to correctly calculate the price of purchasing multiple upgrades in one go, I created a python script to test out values and see if how easily it could be done. The reason I used python for this is because it's quick to develop and I can easily and quickly run it and test out different values to get results in the console:

```
money = 10000
multi_purchase_price = 1000
multiplier = 0.1
multi_purchase_choice = int(input("How many items would you like to purchase\n10\n100\nMAX - Enter 0\n"))
count = 1
prices = [multi_purchase_price]

if multi_purchase_choice == 0:
    while True:
        next_price = round(prices[-1] * (1 + multiplier), 2)

        if sum(prices) + next_price > money:
            break

        multiplier = round(multiplier + 0.05, 2)
        count += 1
        prices.append(next_price)

    multi_purchase_price = sum(prices)

    print("The maximum number of times you can purchase this upgrade with the money you have is " + str(count) + ", which would cost " + str(multi_purchase_price))
else:
    for i in range(multi_purchase_choice-1):
        next_price = round(prices[-1] * (1 + multiplier), 2)
        multiplier = round(multiplier + 0.05, 2)
        count += 1
        prices.append(next_price)

    multi_purchase_price = sum(prices)

    print("To purchase this upgrade " + str(multi_purchase_choice) + " times, it would cost " + str(multi_purchase_price))
```

- This script uses a multi_purchase_price which will be the final price of buying all the upgrades at once
- It also uses a prices array which stores the price of buying the upgrade each time, starting with the base price of the upgrade in it.
- For the MAX purchase option, the script uses an infinite while loop that starts by setting a next_price variable to the product of the previous price multiplied by the multiplier + 1. For the first iteration, this would be $1000 * 1.1$.
- The program then checks if the sum of the prices array + the next price just calculated is more than the player's current money. This checks if buying the upgrade another time would be too much, and if so the program breaks out of the loop, thus stopping before the price would be too much.
- If the sum of the prices array + the next price is not higher than the player's money, then the program increases the price multiplier by adding 0.05 (This number will be different depending on what upgrade it is) to it, then increases the count (which indicates how many of the upgrade the player can purchase), and then adds the next price to the prices array.
- Once the program breaks out of this loop, the multi_purchase_price variable is assigned the value of the sum of all the elements in the prices array, which gives the total price of all the upgrades that the player can purchase with the money they have.

- For the 10 or 100 option, the program uses a for loop, with the number of loops being the value of the multi_purchase_choice (either 10 or 100) - 1. The reason it is -1 is because the multi_purchase_price variable already has the price of the first upgrade, so the loop needs to run one less times.
- The code in the for loop is the same as for the while loop for the max multi-purchase choice, except it does not check if the sum of the prices array plus the next price is bigger than money, because it calculates the price of buying it either 10 or 100 times regardless of whether the player can afford it
- Therefore, at the end of the for loop, the multi_purchase_prices is assigned the sum of the prices array.
- I used this program to see how the prices would scale when buying multiple of the same upgrades in the game, to check whether it will work as intended.
- I started by plugging in the values for the upgrade with the highest price and price multiplier increase, which is gamble luck with a price of 50000 and a price multiplier increase of 0.125:

```
money = 10000
multi_purchase_price = 50000
multiplier = 0.1
multi_purchase_choice = int(input("How many items would you like to purchase\n10\n100\nMAX - Enter 0\n"))
count = 1
prices = [multi_purchase_price]
```

- I then ran the program and entered 100 as the multi_purchase_choice, to check the maximum cost of buying this upgrade:

```
for i in range(multi_purchase_choice-1):
    next_price = round(prices[-1] * (1 + multiplier), 2)
    multiplier = round(multiplier + 0.125, 2)
    count += 1
    prices.append(next_price)

multi_purchase_price = sum(prices)
```

```
How many items would you like to purchase
10
100
MAX - Enter 0
100
To purchase this upgrade 100 times, it would cost 2.8201990147243332e+82
```

- Immediately, I ran into the problem that this number is extremely large and the player would likely never have enough money to purchase it 100 times, meaning it would be useless to them to have the option to buy it 100 times. Even purchasing the cheapest upgrade (Click surge with a price of 100 and price multiplier increase of 0.05) would cost 2.479804038248024e+52 to purchase 100 times, which is also way too high:


```
How many items would you like to purchase
10
100
MAX - Enter 0
100
To purchase this upgrade 100 times, it would cost 2.479804038248024e+52
```

- Therefore, I decided to change this multi-purchase option to 25, instead of 100, as it will make it actually possible to be able to purchase.
- I ran the program again, using the values of the most expensive upgrade, and I entered 25 for the multi-purchase option instead of 100, to check the total price:

```
How many items would you like to purchase
10
100
MAX - Enter 0
25
To purchase this upgrade 25 times, it would cost 59992983634686.76
```

- The total price now (59.9 trillion) is much lower than the previous value I got, and is definitely obtainable in the game, making it much more possible for the player to actually have enough money to make use of this feature.

Data Dictionary:

- Since I have quite a lot of upgrades, I will create a dictionary inside the game_data dictionary, which stores a dictionary for each upgrade, which will store the upgrade's price, level, price multiplier, price multiplier increase, base price, and max increase. Therefore, I will have a dictionary for each upgrade, inside the upgrades dictionary, inside the game data dictionary.

- I will also have an upgrade_elements array, which will store the id of each HTML upgrade element, e.g. "click-surge". The reason is because I will need to manipulate the upgrades a lot in the js, often all at once, so having an array that stores all the id names will allow me to iterate over all the upgrades and do something to them, for example changing the text colour of upgrades depending on their price.

Variable	Data type	Purpose
game_data.upgrades {}	Dictionary	Stores a dictionary of each upgrade. Each upgrade will be a dictionary which stores the price, upgrade level, price multiplier and price multiplier increase of each upgrade.
game_data.upgrades.[upgrade_name] <i>e.g. game_data.upgrades.click_surge {}</i>	Dictionary	Stores the price, upgrade level and price multiplier of the click surge upgrade. This will be the same for every upgrade dictionary
game_data.upgrades.[upgrade_name].price	Key (Float)	Stores the current price of the upgrade
game_data.upgrades.[upgrade_name].level	Key (Integer)	Stores the current upgrade level of the upgrade (increases with every purchase)
game_data.upgrades.[upgrade_name].price_multiplier	Key (Float)	Stores the current price multiplier of the upgrade (the value that is used to scale the price of the upgrade when purchased), which increases by the multiplier increase value each purchase.
game_data.upgrades.[upgrade_name].multiplier_increase	Key (Float)	Stores the price of the current upgrade
game_data.upgrade.[upgrade_name].base_price	Key (Float)	Stores the current price of the upgrade, regardless of the multi-purchase state. This will be used when the multi-purchase state is reset back to 1, the price of the upgrade can be reset to the price before it was increased. It will be increased each time the upgrade is purchased
game_data.upgrades.[upgrade_name].max_purchase	Key (Integer)	Stores the maximum number of times that the player can purchase the upgrade with the money they have. Used when

		the MAX is selected as the multi-purchase setting.
multi_purchase_state	Key (Integer)	This will store the state of the multi purchase feature. It will either be 1, 10, 25 or 0. The number indicates how many of an upgrade will be purchased at once. I will use 0 to indicate the MAX state of the multi purchase feature, where the player will be able to purchase the max number of the upgrade they can with the money they have.
upgrade_elements	Key (Array)	This will store the id of each HTML upgrade element, e.g. “click-surge”

Pseudocode:

Add to Game Data:

```
multi_purchase_state = 1
upgrade_elements = [
'click-surge', 'income-boost', 'industry-boom',
'worker-training', 'worker-tools', 'worker-efficiency',
'population-multiplier-boost', 'quicker-prestige',
'permanent-click-boost',
'double-minigame-reward', 'temporary-population-surge',
'gamble-luck', 'win-streak',
'achievement-gem-boost', 'achievement-xp-boost',
]
```

Upgrade purchase:

```
click-surge Eventlistener(click) {  
    upgrade("click_surge")  
}
```

- I will have an event listener for every upgrade button, which will call the upgrade function and pass in the name of the upgrade, like the example above

```
FUNCTION upgrade(upgrade_name) {  
    IF money > upgrades.upgrade_name.price THEN  
        IF multi_purchase_State == 0 THEN  
            upgrades.upgrade_name.level +=  
            upgrades.upgrade_name.max_purchase  
        ELSE  
            upgrades.upgrade_name.level +=  
            multi_purchase_state  
        ENDIF  
        IF upgrade_name == "industry_boom" THEN  
            money_per_second *= 1.2  
            HTML.money-per-second.text =  
            money_per_second  
        ELSE IF upgrade_name == "worker_efficiency"  
        THEN  
            IF worker_speed > 0.5 THEN  
                worker_speed -= 0.1  
                HTML.worker-gen.text =  
                total_worker_gen /  
                game_data.worker_speed  
            ENDIF  
        ELSE IF upgrade_name == "worker_tools" THEN
```

```

        worker_price *= (1 - (1/10 *
        upgrades.worker_training.level))
        HTML.worker-price.text = worker_price
    ENDIF

    money -= upgrades.upgrade_name.price
    HTML.money.text = money
    HTML.upgrade-level.text =
    upgrades.upgrade_name.level
ENDIF
}

```

- The reason I have an if statement that checks if the upgrade_name is equal to “industry_boom” is because the industry boom upgrade is a flat increase that only applies once (when the upgrade is purchased). Therefore, I can just apply the effect here, so it only runs each time the upgrade is purchased. The code multiplies the money per second by 1.2, which increases it by 20% each time. It then updates the money per second display in the HTML
- The reason I also have an if statement that checks if the upgrade_name is equal to “worker_efficiency” is because this upgrade also just applies its effect once, at purchase. The effect is just reducing the worker speed by 0.1s (if it is above 0.5s, because worker efficiency reduces worker generation time by 0.1s until they are generating every 0.5s), so I have written this code here, so it will run just once, each time the upgrade is purchased. This code also updates the worker gen text display in the HTML, by setting it to total worker gen divided by worker speed to get the worker gen per second.
- The reason I also have an if statement that checks if the upgrade_name is equal to “worker_tools” is because this upgrade also applies its effect straight away, by reducing the price of workers, so I have written this code here so it will run just once each time the upgrade is purchased. The code reduces the price of workers by 10% per upgrade level, each time the worker is purchased. When the upgrade level is 0, the price will be multiplied by $1 - (1/10 * 0)$ which is just 1, so it won't get reduced at all. However, as the upgrade level increase, the price will get divided by 0.9 (price reduced by 10%), then 0.8 (price reduced by 20%) and so on. The code also updates the worker price display in the HTML.
- There is a problem, however, which is that the upgrade_elements array stores the name of the id of each upgrade in the html. The

way the I have formatted the IDs is using a “-“ between each word, e.g. click-surge. However, in the JavaScript I have used snake case, e.g. click_surge. Therefore, when I try to access a value from an upgrade in the upgrades dictionary, using upgrade_elements[N], it won’t work. Therefore, when I come to write the code, I will use the JavaScript function .replaceAll() to replace the “-“ with a “_” when I access the upgrade. For example, I would use click-surge to access the HTML element, then use .replaceAll() to temporarily change it to click_surge to access the upgrade in the upgrades dictionary, so I can set the text of the element.

Price update:

```
WHILE True

    FOR N = 0 TO upgrade_elements.length
        HTML.upgrade_elements[N].text =
            upgrades.upgrade_elements[N].price
    ENDFOR

WAIT 1

ENDWHILE
```

- This is an infinite loop that runs every second, and sets the price of each upgrade to the price of it stored in the javascript
- It has a for loop inside which iterates over each item in the upgrade_elements array, setting the text of it to the value of the upgrades price. This allows the prices of every upgrade to be constantly updated, so they are always accurate.
- The same problem occurs here as with the Upgrade Purchase, as the items in upgrade_elements are in the format x-y-z, but in the upgrades dictionary the names of the upgrades are in the format x_y_z. Therefore, I will use the same solution when I code this as I mentioned in the Upgrade purchase, the .replaceAll() function.

Upgrade Colour:

```
WHILE True
    FOR N = 0 TO upgrade_elements.length
        HTML.upgrade_elements[N].text =
            upgrades.upgrade_elements[N].price
        IF money > upgrades.upgrade_elements[N].price
            THEN
                HTML.upgrade_elements[N].textColour = Green
            ELSE
                HTML.upgrade_elements[N].textColour = Red
            ENDIF
        ENDFOR
    WAIT 1
ENDWHILE
```

- This is the infinite loop I just wrote for the Price update, and inside it I have embedded an if statement.
- The if statement checks if the money is bigger than the price of the current upgrade being looked at. If so, then the colour of the text is set to green, if not then it is set to red.
- The same problem occurs here with accessing the price of upgrade in the upgrades dictionary, as the items in upgrade_elements are in the format x-y-z, but in the upgrades dictionary the names of the upgrades are in the format x_y_z. Therefore, I will use the same solution when I code this as I mentioned in the Upgrade purchase and Price update, the .replaceAll() function.

Code for Click Surge:

```
planet-button Eventlistener(click) {
```

```

    population += 1 *
    (1 + (click_surge.level * 2)/100)
    money_per_second = population * 0.01
    HTML.population.text = population
    HTML.money-per-second.text = money_per_second
}

```



- This is the existing code for the planet click, except I have changed the first line which used to be `population += 1` I have now made it so that when the player clicks the planet, the population increases 2% more with every click surge upgrade:
`population += 1 \* (1 + (click\_surge.level \* 2)/100)`
- When the upgrade level is 0 (no click surge purchased), the population will increase by $1 * (1 + (0 * 2) / 100)$ which is just 1, but when the upgrade level increases (when the click surge upgrade is purchased) then the population will increase by 1.02, then 1.04, then 1.06 and so on.

Code for Income Boost:

```

planet-button Eventlistener(click) {
    population += 1 *
    (1 + (click_surge.level * 2)/100)
    money_per_second = population * (0.01 * (1 +
    (upgrades.income_boost.level * 10) /100))
    HTML.population.text = population
    HTML.money-per-second.text = money_per_second
}

```



- This is the existing code (including the first line that will be changed for the click surge) for the planet click, except I have changed the second line which used to be `money\_per\_second = population \* 0.01`

I have now made it so that when the money per second increases, it increases by 10% with each income boost upgrade:

```
`money_per_second = population * (0.01 * (1 +  
(upgrades.income_boost.level * 10) / 100))`
```

- When the upgrade level is 0, the money per second will increase by $\text{population} * (0.01 * (1 + 0 * 10) / 100)$ which is just $\text{population} * 0.01$, but when the upgrade level increases, the money per second will increase by $\text{population} * 0.011$, then 0.012 , then 0.013 and so on.
- This is the existing code for the planet click (with the updated lines for click surge and income boost), except I have added a line that increases money per second by 20% for each industry boom upgrade
- The increase is calculated in the same way as for the previous 2 upgrades, so that after the first upgrade it will increase by 1.2, then 1.4 for the second upgrade, and so on.

Since I also increase money per second in the worker loop, I will need to update that too, replacing

```
`money_per_second = population * 0.01` with  
`money_per_second = population * (0.01 * (1 +  
(upgrades.income_boost.level * 10) / 100))`
```

Code for Worker Training:

```
WHILE True
    population += total_worker_gen * (1 +
        (upgrades.worker_training.level * 20) / 100)
    money_per_second = population * 0.01
    HTML.population.text = population
    HTML.money-per-second = money_per_second
    WAIT worker_speed
ENDWHILE
```



- This is the existing code for the worker click loop, except I have changed the first line to increase the population by 20% for each worker training upgrade.
- The increase is calculated in the same way as for the previous 3 upgrades, so that after the first upgrade it will increase by 1.2, then 1.4 for the second upgrade, and so on.

Multi-purchase button:

```
multi-purchase-button EventListener(click) {
    IF multi_purchase_state == 1 THEN
        multi_purchase_state = 10
    ELSE IF multi_purchase_state == 10 THEN
        multi_purchase_state = 25
    ELSE IF multi_purchase_state == 25 THEN
        multi_purchase_state = 0
    ELSE
```

```
multi_purchase_state = 1
```

```
update_prices()
```

```
HTML.multi-purchase-state.text = multi_purchase_state
```

```
}
```

- I will have a click event listener for the multi-purchase button, which will check the current multi-purchase state and update it to the next one.
- If the multi-purchase state is 0 (MAX), it will set it back to 1, which is the first one.
- This allows the player to cycle between multi-purchase settings
- After the multi-purchase state is updated, the code will call the `update_prices()` function which will update the prices according to the current multi-purchase state
- Finally, the multi-purchase button text will be updated to reflect the updated setting.

```
FUNCTION update_prices {
```

```
    FOR dictionary in game_data.upgrades
```

```
        Arr prices = []
```

```
        prices.add(dictionary.price)
```

```
        count = 1
```

```
        next_price = 0
```

```
        IF multi_purchase_state == 0 THEN
```

```
            WHILE True
```

```
                next_price = prices[count -1] *  
                (1+dictionary.price_multiplier)
```

```
                IF sum(prices) + next_price > money THEN
```

```
                    BREAK
```

```
                ENDIF
```

```
                prices.add(next_price)
```

```

        dictionary.price_multiplier +=
        dictionary.price_multiplier_increase
        count += 1
    ENDWHILE
    dictionary.price = sum(prices)
    dictionary.max_purchase = count
    HTML.price.text = dictionary.price

ELSE

    FOR N = 0 TO multi_purchase_state
        next_price = prices[count -1] *
        (1+dictionary.price_multiplier)
        prices.add(next_price)
        dictionary.price_multiplier +=
        dictionary.price_multiplier_increase
        count += 1
    ENDFOR
    dictionary.price = sum(prices)
    HTML.price.text = dictionary.price
ENDIF
}

```

- I will have a function that will update the prices of the upgrades depending on the current multi-purchase state, and the player's current money.
- I created this pseudocode from the python script I designed in the **Features** section of this stage, where I explain how it works.

Development

To start the development of this section I will add all the shop upgrades, by duplicating the shop-row div for each upgrade, and changing the button text to the name of the upgrade. Before I duplicate each div, I will add IDs to all the elements I will need to change in the javascript, so that they will be in each div when I duplicate them:

```
<div class="shop-row">
  <button class="shop-item" id="click-surge">Click Surge<span id="click-surge-amount">x1</span></button>
  <span id="click-surge-price">Price</span>
</div>
<span id="click-surge-level">x0</span></h3>
<div style="height: 10px"></div>
```

This span will display the number of the upgrade that will be purchased when the button is clicked, reflecting

This span will display the upgrade level of the upgrade (the number of times it has been purchased)

This span will display the price of the upgrade

I then duplicated the shop-row div for each upgrade I will make, changing the text in the button to the name of the upgrade:

```
<div class="shop">
  <div class="multi-purchase"><button id="multi-purchase-button"><h3><span class="multi-purchase-state" id="multi-purchase-state">x1</span></h3></button></div>
  <h2>🏪 Shop</h2>
  <div class="shop-list">
    <div class="shop-row">
      <button class="shop-item" id="click-surge">Click Surge<span id="click-surge-amount">x1</span></button>
      <span id="click-surge-price">Price</span>
    </div>
    <span id="click-surge-level">x8</span></h3>
    <div style="height: 10px"></div>

    <div class="shop-row">
      <button class="shop-item" id="income-boost">Income Boost<span id="income-boost-amount">x1</span></button>
      <span id="income-boost-price">Price</span>
    </div>
    <span id="income-boost-level">x8</span>
    <div style="height: 10px"></div>
    <div class="shop-row">
      <button class="shop-item" id="industry-boom">Industry Boom<span id="industry-boom-amount">x1</span></button>
      <span id="industry-boom-price">Price</span>
    </div>
    <span id="industry-boom-level">x8</span>
    <div style="height: 10px"></div>
    <div class="shop-row">
      <button class="shop-item" id="worker-training">Worker Training<span id="worker-training-amount">x1</span></button>
      <span id="worker-training-price">Price</span>
    </div>
    <span id="worker-training-level">x8</span>
    <div style="height: 10px"></div>
    <div class="shop-row">
      <button class="shop-item" id="worker-tools">Worker Tools<span id="worker-tools-amount">x1</span></button>
      <span id="worker-tools-price">Price</span>
    </div>
    <span id="worker-tools-level">x8</span>
    <div style="height: 10px"></div>
    <div class="shop-row">
      <button class="shop-item" id="worker-efficiency">Worker Efficiency<span id="worker-efficiency-amount">x1</span></button>
      <span id="worker-efficiency-price">Price</span>
    </div>
    <span id="worker-efficiency-level">x8</span>
    <div style="height: 10px"></div>
  </div>
</div>
```

```

<div class="shop-row">
  <button class="shop-item" id="population-multiplier-boost">Population Multiplier Boost<span id="population-multiplier-boost-amount">x1</span></button>
  <span id="population-multiplier-boost-price">Price</span>
</div>
<span id="population-multiplier-boost-level">x8</span>
<div style="height: 10px"></div>
<div class="shop-row">
  <button class="shop-item" id="quicker-prestige">Quicker Prestige<span id="quicker-prestige-amount">x1</span></button>
  <span id="quicker-prestige-price">Price</span>
</div>
<span id="quicker-prestige-level">x8</span>
<div style="height: 10px"></div>
<div class="shop-row">
  <button class="shop-item" id="permanent-click-boost">Permanent Click Boost<span id="permanent-click-boost-amount">x1</span></button>
  <span id="permanent-click-boost-price">Price</span>
</div>
<span id="permanent-click-boost-level">x8</span>
<div style="height: 10px"></div>
<div class="shop-row">
  <button class="shop-item" id="double-minigame-reward">Double minigame reward<span id="double-minigame-reward-amount">x1</span></button>
  <span id="double-minigame-reward-price">Price</span>
</div>
<span id="double-minigame-reward-level">x8</span>
<div style="height: 10px"></div>
<div class="shop-row">
  <button class="shop-item" id="temporary-population-surge">Temporary population surge<span id="temporary-population-surge-amount">x1</span></button>
  <span id="temporary-population-surge-price">Price</span>
</div>
<span id="temporary-population-surge-level">x8</span>
<div style="height: 10px"></div>
<div class="shop-row">
  <button class="shop-item" id="gamble-luck">Gamble luck<span id="gamble-luck-amount">x1</span></button>
  <span id="gamble-luck-price">Price</span>
</div>
<span id="gamble-luck-level">x8</span>
<div style="height: 10px"></div>
<div class="shop-row">
  <button class="shop-item" id="win-streak">Win streak<span id="win-streak-amount">x1</span></button>
  <span id="win-streak-price">Price</span>
</div>
<span id="win-streak-level">x8</span>
<div style="height: 10px"></div>

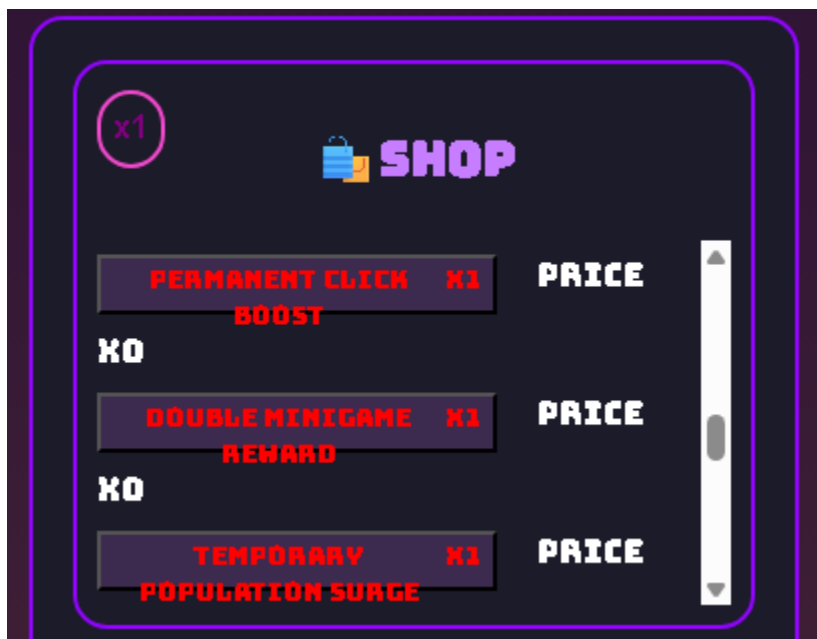
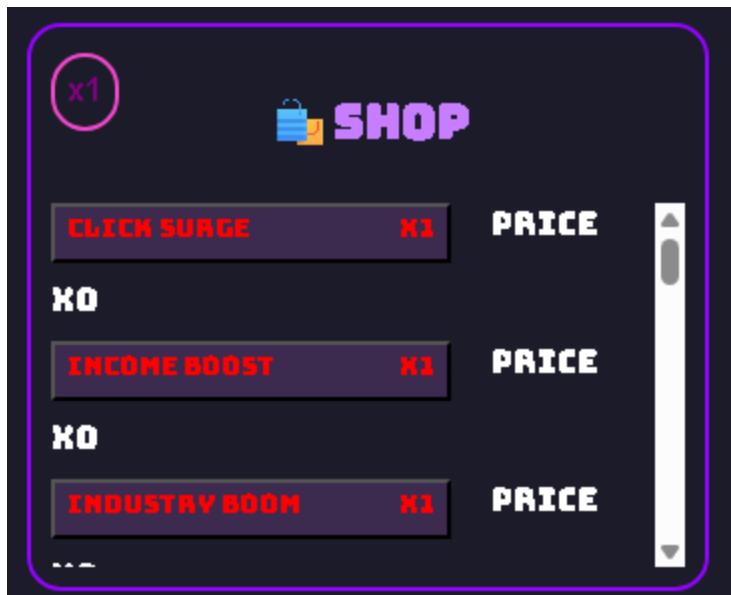
```

```

<div class="shop-row">
  <button class="shop-item" id="achievement-gem-boost">Achievement gem boost<span id="achievement-gem-boost-amount">x1</span></button>
  <span id="achievement-gem-boost-price">Price</span>
</div>
<span id="achievement-gem-boost-level">x8</span>
<div style="height: 10px"></div>
<div class="shop-row">
  <button class="shop-item" id="achievement-xp-boost">Achievement xp boost<span id="achievement-xp-boost-amount">x1</span></button>
  <span id="achievement-xp-boost-price">Price</span>
</div>
<span id="achievement-xp-boost-level">x8</span>

```

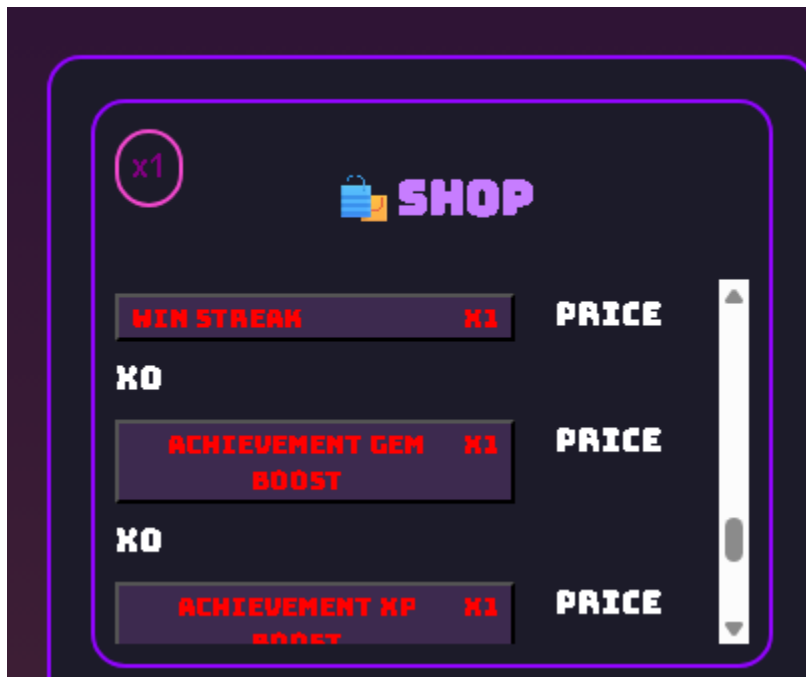
I then ran this to check if all the upgrades had been correctly applied:



All the upgrades had been correctly added to the shop, however some of the text for the upgrades with longer names overflowed out of the button, so I changed the height of the buttons to auto in the css, so they would size to fit the content within them:


```
.shop-item {  
  height: auto; ←  
  width: 200px;  
  display: flex;  
  justify-content: space-between;  
}
```

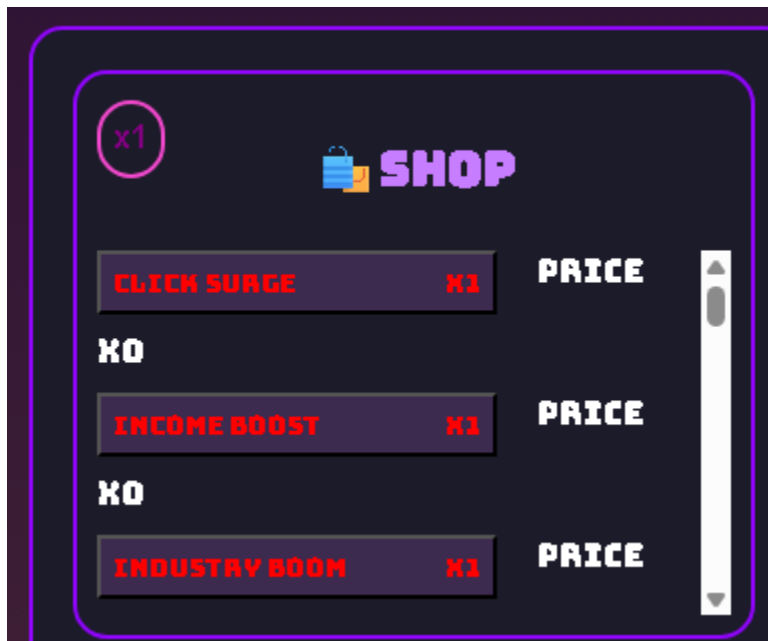
I then ran this to see if it fixed the problem:



This has fixed the problem, as all text now fits inside the buttons without overflowing. The only problem is that for the upgrade names that do fit on one line, the buttons are shorter than I would like. This is because, before I had the height set to 30px, but now since it is auto the height adjusts to fit the height of the text which is slightly less than 30px. Since I would like the buttons to be a bit taller for the upgrade names that fit on one line, I will add some top and bottom padding to the buttons to increase it:

```
.shop-item {  
  height: auto;  
  width: 200px;  
  padding-top: 5px;  
  padding-bottom: 5px;  
  display: flex;  
  justify-content: space-between;  
}
```

This should increase the space between the upgrade text and the bottom and top of the button. I ran this to see if it worked:



Now, the buttons are slightly taller, at my desired height.

I then added the multi_purchase_state and upgrade_elements to the game_data dictionary:

```
// SHOP
multi_purchase_state: 1,

upgrade_elements : [
    'click-surge', 'income-boost', 'industry-boom',
    'worker-training', 'worker-tools', 'worker-efficiency',
    'population-multiplier-boost', 'quicker-prestige', 'permanent-click-boost',
    'double-minigame-reward', 'temporary-population-surge',
    'gamble-luck', 'win-streak',
    'achievement-gem-boost', 'achievement-xp-boost',
],
```

I then added the upgrades dictionary to game_data, with a dictionary inside for each upgrade, and manually set the values for each upgrade:

```

upgrades: {
  click_surge: {
    price: 100,
    level: 0,
    price_multiplier: 0.1,
    multiplier_increase: 0.01,
    base_price: 100,
    max_purchase: 0,
  },
  income_boost: {
    price: 500,
    level: 0,
    price_multiplier: 0.1,
    multiplier_increase: 0.02,
    base_price: 500,
    max_purchase: 0,
  },
  industry_boom: {
    price: 10000,
    level: 0,
    price_multiplier: 0.1,
    multiplier_increase: 0.05,
    base_price: 10000,
    max_purchase: 0,
  },
  worker_training: {
    price: 1000,
    level: 0,
    price_multiplier: 0.1,
    multiplier_increase: 0.03,
    base_price: 1000,
    max_purchase: 0,
  },
  worker_tools: {
    price: 5000,
    level: 0,
    price_multiplier: 0.1,
    multiplier_increase: 0.05,
    base_price: 5000,
    max_purchase: 0,
  },
},

```

```

worker_efficiency: {
  price: 7500,
  level: 0,
  price_multiplier: 0.1,
  multiplier_increase: 0.08,
  base_price: 7500,
  max_purchase: 0,
},
population_multiplier_boost: {
  price: 10000,
  level: 0,
  price_multiplier: 0.1,
  multiplier_increase: 0.08,
  base_price: 10000,
  max_purchase: 0,
},
quicker_prestige: {
  price: 5000,
  level: 0,
  price_multiplier: 0.1,
  multiplier_increase: 0.07,
  base_price: 5000,
  max_purchase: 0,
},
permanent_click_boost: {
  price: 20000,
  level: 0,
  price_multiplier: 0.1,
  multiplier_increase: 0.08,
  base_price: 20000,
  max_purchase: 0,
},
double_minigame_reward: {
  price: 30000,
  level: 0,
  price_multiplier: 0.1,
  multiplier_increase: 0.125,
  base_price: 30000,
  max_purchase: 0,
},

```

```

temporary_population_surge: {
  price: 20000,
  level: 0,
  price_multiplier: 0.1,
  multiplier_increase: 0.125,
  base_price: 20000,
  max_purchase: 0,
},
gamble_luck: {
  price: 50000,
  level: 0,
  price_multiplier: 0.1,
  multiplier_increase: 0.125,
  base_price: 1000,
  max_purchase: 0,
},
win_streak: {
  price: 5000,
  level: 0,
  price_multiplier: 0.1,
  multiplier_increase: 0.07,
  base_price: 5000,
  max_purchase: 0,
},
achievement_gem_boost: {
  price: 2000,
  level: 0,
  price_multiplier: 0.1,
  multiplier_increase: 0.03,
  base_price: 2000,
  max_purchase: 0,
},

```

```

achievement_xp_boost: {
  price: 5000,
  level: 0,
  price_multiplier: 0.1,
  multiplier_increase: 0.05,
  base_price: 5000,
  max_purchase: 0,
},

```

After this, I developed the code to handle the upgrade purchase:

```

for(let i = 0; i < game_data.upgrade_elements.length; i++) {
  document.getElementById(game_data.upgrade_elements[i]).addEventListener("click", function (event) {
    upgrade(upgrade_elements[i])
  })
}

```

This iterates over every element in the `upgrade_elements` array, which are all the ids for each upgrade button in the HTML. It then adds an event listener to each button, calling the function `upgrade()` in the event handler, passing in the current item in `upgrade_elements` (the button id). The reason I have used a for loop to add all the event listeners, is because it significantly reduces the number of lines of code. Without using iteration, I would have to manually code an event listener for all 15 upgrade buttons, which would waste time and cause unnecessary duplication of code. Using a for loop allows me to add an event listener to every upgrade button in 4 lines of code. Since I pass in the upgrade button id to the `upgrade()` function, this means the event handler for each event listener will call the same function with a different value passed in, allowing something different to happen for each event listener.

I then coded the `upgrade()` function:

```
function upgrade(upgrade_element) {  
  let upgrade_name = upgrade_element.replaceAll("-", "_")  
}
```

I started creating a variable called `upgrade_name`, and setting it to the value of the `upgrade_element` (the element id passed in, in the event handler for the event listener for each upgrade button), with all “-” replaced with “_”. This would allow me to have a variable that I can use to access the current upgrade and its necessary values (price, max increase etc.) in the upgrade dictionary.

```
function upgrade(upgrade_element) {  
  let upgrade_name = upgrade_element.replaceAll("-", "_")  
  
  if(game_data.money >= game_data.upgrades[upgrade_name].price) {  
    if(game_data.multi_purchase_state == 0) {  
      game_data.upgrades[upgrade_name].level += game_data.upgrades[upgrade_name].max_purchase  
    } else {  
      game_data.upgrades[upgrade_name].level += game_data.multi_purchase_state  
    }  
  }  
}
```

I then wrote an if statement that checks if money is bigger than or equal to the upgrade price. I had to use bracket notation to access the upgrade within the upgrades dictionary, as I was using a variable (upgrade_name) to access a key within the dictionary. Inside the if statement, I wrote another if statement that checks if the multi_purchase_state is equal to 0. If so then the level of the upgrade is increased by the max_purchase_value of the upgrade. This is because when the multi_purchase_state is 0, it means MAX, and the max_purchase value will store the number of the upgrade that the player can purchase with the money they have (once I have coded the multi-purchase feature). If the multi-purchase state is not 0, then I simply increase the level of the upgrade by the multi-purchase state value, either 1, 10, or 25.

I then added the extra checks in the function (still inside the outer if statement), for the upgrades that apply a single, instant effect:

```
if(upgrade_name == "industry_boom") {  
    game_data.money_per_second *= 1.2  
    document.getElementById("money-per-second").textContent = game_data.money_per_second  
} else if (upgrade_name == "worker_efficiency") {  
    game_data.worker_speed -= 0.1  
    document.getElementById("worker-gen").textContent = game_data.total_worker_gen / game_data.worker_speed  
} else if (upgrade_name == "worker_tools") {  
    game_data.worker_price *= (1 - ((1 / 10) * game_data.upgrades.worker_tools.level))  
    document.getElementById("worker-price").textContent = game_data.worker_price  
}
```

I used an if statement to check whether the name of the upgrade was one of three different upgrades. If the upgrade name is equal to "industry_boom", I apply the industry boom effect by increasing money per second by 20% (and then I update the display money per second display in the HTML); If the upgrade name is equal to "worker_efficiency" and worker speed is greater than 0.5, I apply the worker speed effect by reducing the worker speed by 0.1 (and then I update the worker gen display in the HTML) by setting it to the total worker gen divided by the worker speed). If the upgrade name is equal to "worker_tools", I apply the worker tools effect by reducing worker price by 10% (and I update the worker price display in the HTML). The reason I have these checks is because for these 3 upgrades,

the effect is instant and it only applies once, at purchase. If I just increased the level of these upgrades, I would have to have an infinite loop that constantly checks if the level of the upgrade increases, and then runs the line of code for the effect just once. This would be inefficient and would waste processing power, so I just do a check in the upgrade function to see if the upgrade has an effect that can be applied right there, just once.

Finally, still inside the outer if statement, I added the code to reduce the player's money by the price of the upgrade and to update the money display in the HTML:

```
game_data.money -= game_data.upgrades[upgrade_name].price
document.getElementById("money").textContent = game_data.money
}
```

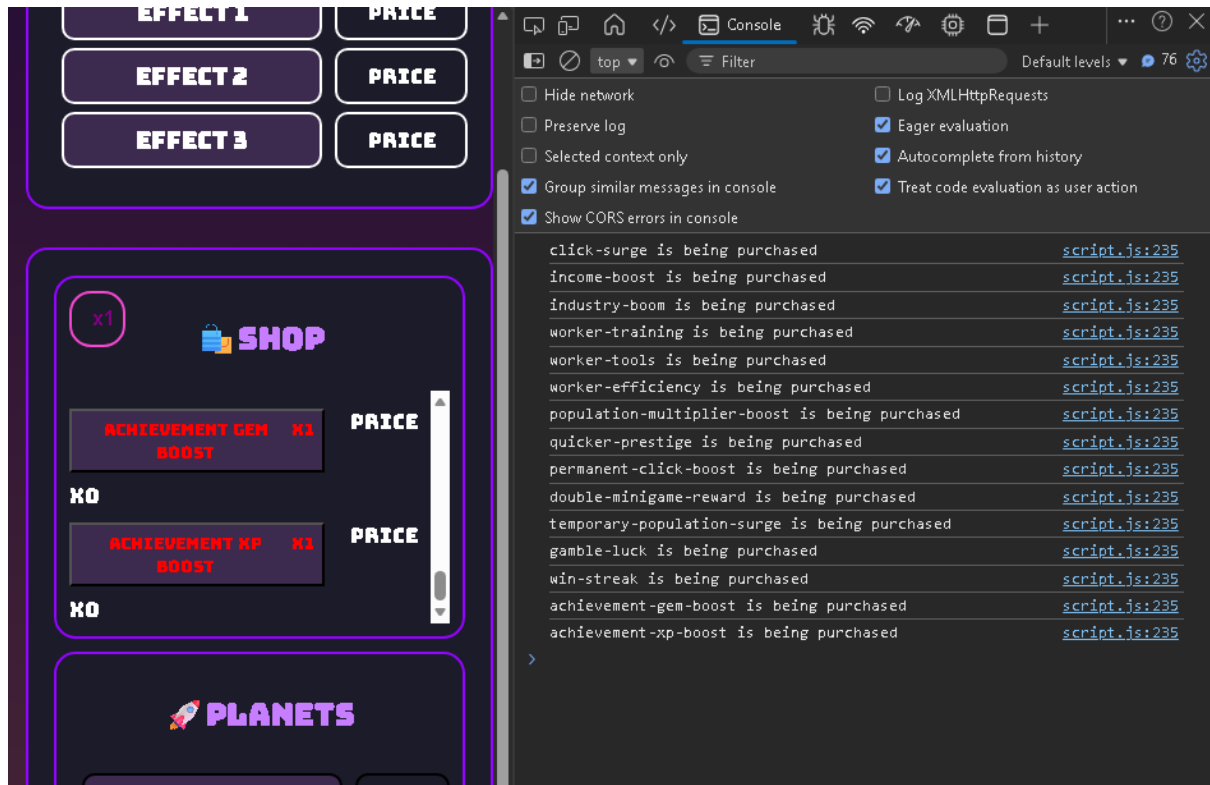
As I will be running lots of tests with this upgrades throughout this stage, I will set the money in game data to 1 million so I always have enough to purchase and test upgrades:

```
// FOR GETTING THE MONEY
population: 0,
money: 1000000,
money_per_second: 0,
```

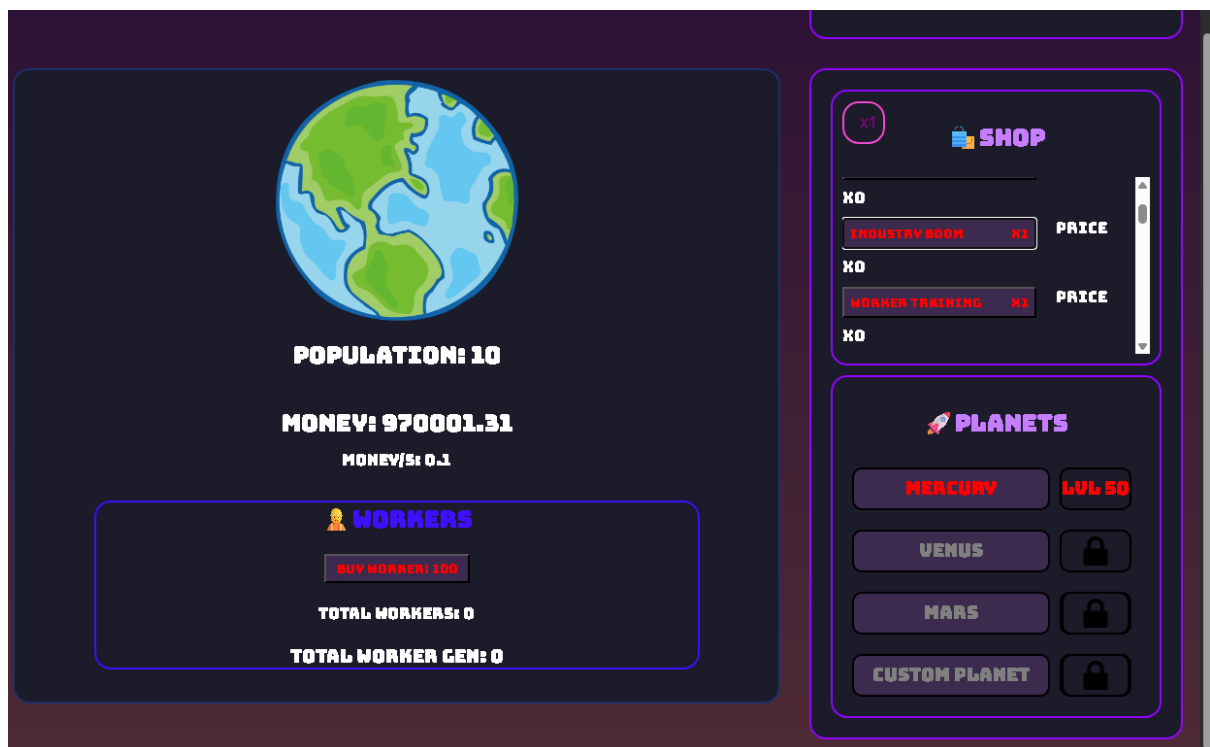
Before I ran the code, I added a line in the upgrade function to check if the correct upgrades were being passed in:

```
function upgrade(upgrade_element) {
  console.log(upgrade_element + " is being purchased")
  let upgrade_name = upgrade_element.innerHTML(" " + " ")
```


This will print a line to the console, with the name of the upgrade being purchased:



As I clicked each upgrade, the name of the upgrade (the id of its button) was output to the console, so I know the first part of the system was working. I then wanted to check if the instant-effect upgrades were working. I started with industry boom:



Purchasing this upgrade should increase the money per second straight away, which it did, however I noticed after a second or two it returned to what it was previously. I realised that this was because I calculate money per second by multiplying the population by 0.01. Therefore, when I increase it by 20%, it goes up but since the population stays the same, when it is calculated in the money loop it returns to its previous value. To fix this, I could change the way in which I calculate money per second, however since it is already working I don't want to change anything that could cause errors. Therefore, after each time the money per second is calculated in the planet button click event handler, I will apply the industry boom effect again so money per second returns to the correct value. I will use a loop to apply the effect, so it will run the same number of times as the level of the industry boom upgrade. Firstly, however, I will write the code for click surge as this is just changing the way population is calculated:

```
document.getElementById("planet-button").addEventListener("click", function (event) {
  game_data.population += 1 * (1 + (game_data.upgrades.click_surge.level * 2)/100)
  game_data.money_per_second = game_data.population * 0.01
  game_data.money_per_second = Math.round(game_data.money_per_second * 100) / 100 //Removes extra
  document.getElementById("population").textContent = game_data.population
})
```

I have set the population to increase by 1 multiplied by the click surge multiplier, to apply the 20% increase to click power for each upgrade level. This means that each time the click surge upgrade is purchased, the amount that the population increases by is multiplied by 1, then 1.02, then 1.04 and so on.

I will then write the code for income boost, as this is also just changing the way money per second is calculated:

```
game_data.population += 1 * (1 + (game_data.upgrades.click_surge.level * 2)/100)
game_data.population = Math.round(game_data.population * 100) / 100
game_data.money_per_second = game_data.population * (0.01 * (1 + (game_data.upgrades.income_boost.level * 10) / 100))
game_data.money_per_second = Math.round(game_data.money_per_second * 100) / 100
```

I have set the money per second to the population multiplied by the income boost multiplier to apply the 10% increase to money per second per upgrade. When the upgrade level is 0, money per second will increase by population multiplied by 0.01, like normal. But as the upgrade level increases, it will be set to population multiplied by 0.011, then 0.12, then 0.13, meaning the number that population is multiplied by to calculate money per second increases by 10%. Now, I can add the for loop to apply the industry boom effect, to ensure money per second returns to its correct value after the upgrade:

```
for(let i = 0; i < game_data.upgrades.industry_boom.level; i++) {
  game_data.money_per_second *= 1.2
  game_data.money_per_second = Math.round(game_data.money_per_second * 100) / 100
}
```

This multiplies money per second by 1.2 for each industry boom level, which correctly applies the flat 20% increase.

Before I ran the code, I realised that I also calculate money per second in the worker loop, by simply setting it to population multiplied by 0.01. Instead, I will just increase it by the total worker gen multiplied by 0.01 so that it just increases money per second rather than fully recalculating it and cancelling the effect from industry boom:

```
setInterval(() => {  
  game_data.population += game_data.total_worker_gen  
  game_data.population = Math.round(game_data.population * 100) / 100 //Removes extra, unwanted decimal places  
  game_data.money_per_second += game_data.total_worker_gen * 0.01  
  game_data.money_per_second = Math.round(game_data.money_per_second * 100) / 100 //Removes extra, unwanted decimal places  
  document.getElementById("population").textContent = game_data.population  
}, 1000)
```

After this, though, I just realised that I need to include the income boost effect here as well, since the upgrade increases money per second by 10% per population. Therefore, total worker gen should be multiplied by the income boost multiplier, not just 0.01. So, I copied the code for the income boost multiplier from the planet button click even handler and replaced 0.01 with it:

```
game_data.population = Math.round(game_data.population * 100) / 100 //Removes extra, unwanted decimal places  
game_data.money_per_second += game_data.total_worker_gen * (0.01 * (1 + (game_data.upgrades.income_boost.level * 10) / 100))  
game_data.money_per_second = Math.round(game_data.money_per_second * 100) / 100 //Removes extra, unwanted decimal places  
document.getElementById("population").textContent = game_data.population
```

I ran this code to see if it worked:



Now, the money per second was being correctly increased when I purchased industry boom, and not getting reset. However, there were extra digits being added onto money per second, and I realised that this was because I multiply it by 1.2 when industry boom is purchased, but don't use the method to remove the extra 0s, so I added that in:

```
if(upgrade_name == "industry_boom") {  
    game_data.money_per_second *= 1.2  
    game_data.money_per_second = Math.round(game_data.money_per_second * 100) / 100 //Removes extra, unwanted decimal places  
    document.getElementById("money-per-second").textContent = game_data.money_per_second
```

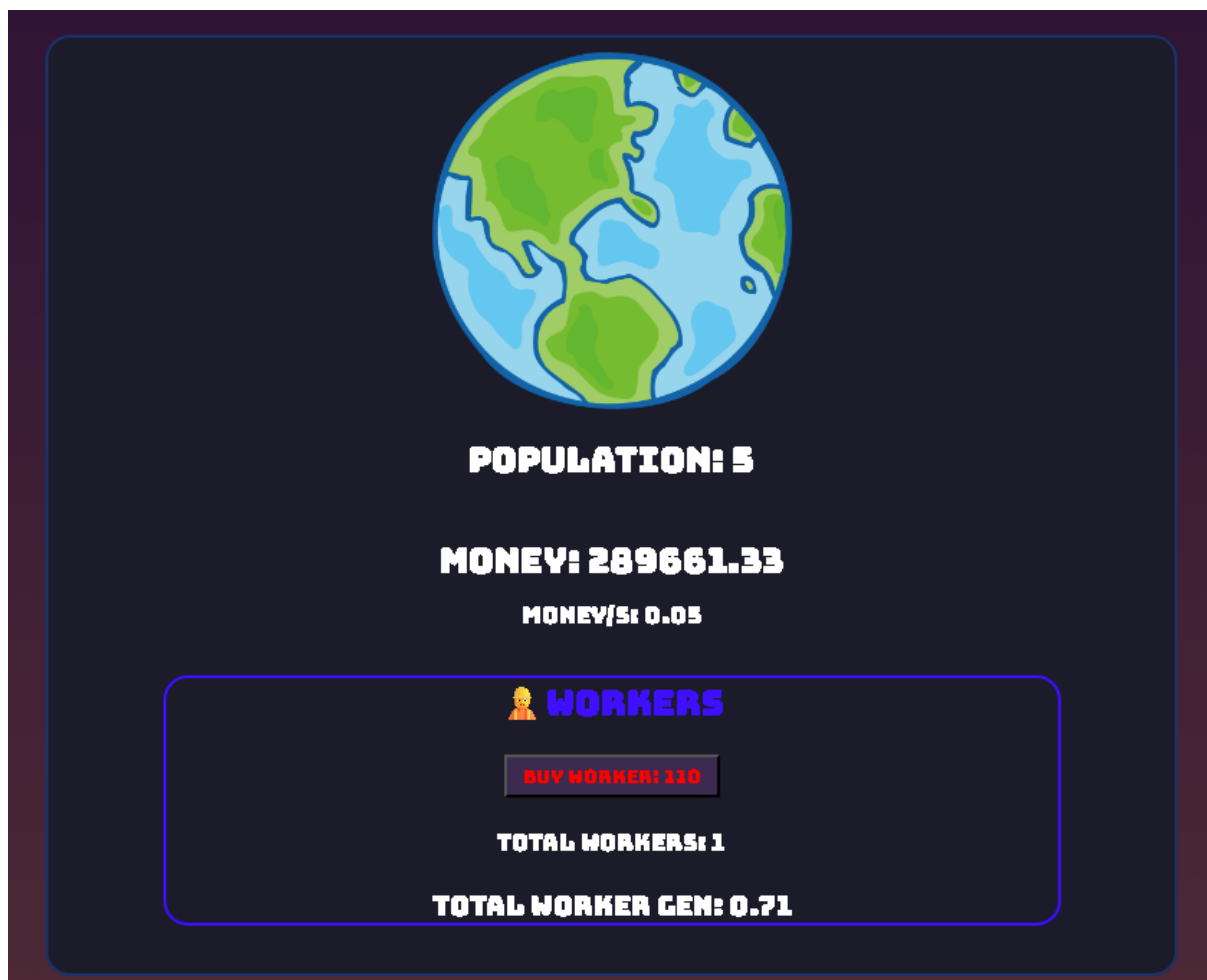
Now, I will check the worker efficiency upgrade. Before I do though, I realised that since I am updating the worker speed (reducing it by 1s), I will need to also update the worker gen, setting it to workers multiplied by worker increase. I will then need to remove extra 0s and update the HTML display:

```
} else if (upgrade_name == "worker_efficiency" && game_data.worker_speed > 0.5) {  
    game_data.worker_speed -= 0.1  
    game_data.total_worker_gen = (game_data.workers * game_data.worker_increase)  
    game_data.total_worker_gen = Math.round(game_data.total_worker_gen * 100) / 100  
    document.getElementById("worker-gen").textContent = Math.round((game_data.total_worker_gen / game_data.worker_speed) * 100) / 100
```

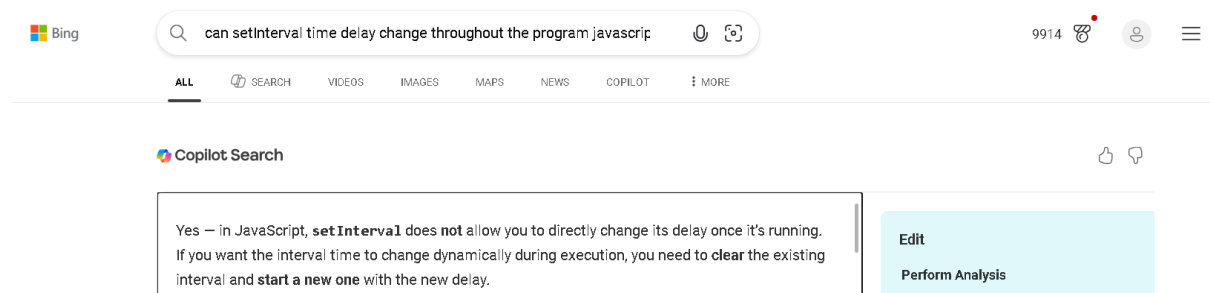
The reason I also did the rounding method when setting the text of the worker gen display in the HTML, is because I am setting the tex to the result of a divison calculation, so there will be extra digits added on, so I use my method to remove extra digits there as well. Since I also set the value of total worker gen in the worker purchase event handler, I did this there as well:

```
game_data.total_worker_gen = (game_data.workers * game_data.worker_increase)
game_data.total_worker_gen = Math.round(game_data.total_worker_gen * 100) / 100
document.getElementById("workers").textContent = game_data.workers
document.getElementById("worker-price").textContent = game_data.worker_price
document.getElementById("worker-gen").textContent = Math.round((game_data.total_worker_gen / game_data.worker_speed) * 100) / 100
```

I ran this to check if the upgrade works, by purchasing a worker and then a few of the worker efficiency upgrades:



After purchasing the upgrade a few times, the worker gen did increase, however the worker seemed to still be increasing the population at the same rate as before. I had a feeling that this was due to the setInterval loop. For the delay of the loop, I set it to worker_speed multiplied by 1000. I then reduce worker_speed when the worker_efficiency upgrade is purchased, so I assumed that this would change the delay of the loop, however I think this problem might be because the delay of a setInterval loop is checked once at the start and cannot change. I looked this up to see if it was true:



The copilot search result said that “setInterval does not allow you to directly change its delay once it’s running”, therefore I knew this was the problem. I looked on a GeeksForGeeks website about this, (GeeksForGeeks, GeeksForGeeks, n.d.), which talked about how to change the time interval of the setInterval method at runtime. It said that you can use a method called setTimeout, instead of setInterval, which allows you to change the interval whenever. The requirements (according to the website) are:

```
let interval
```

```
let timer = 1000
```

```
function loop ()
```

```
    clearInterval(interval)
```

```
    interval = setInterval(loop, timer)
```

```
setTimeout(loop, timer)
```

This creates an infinite loop that runs every second, and the time can be changed by changing the timer, but for me instead of a timer I will use

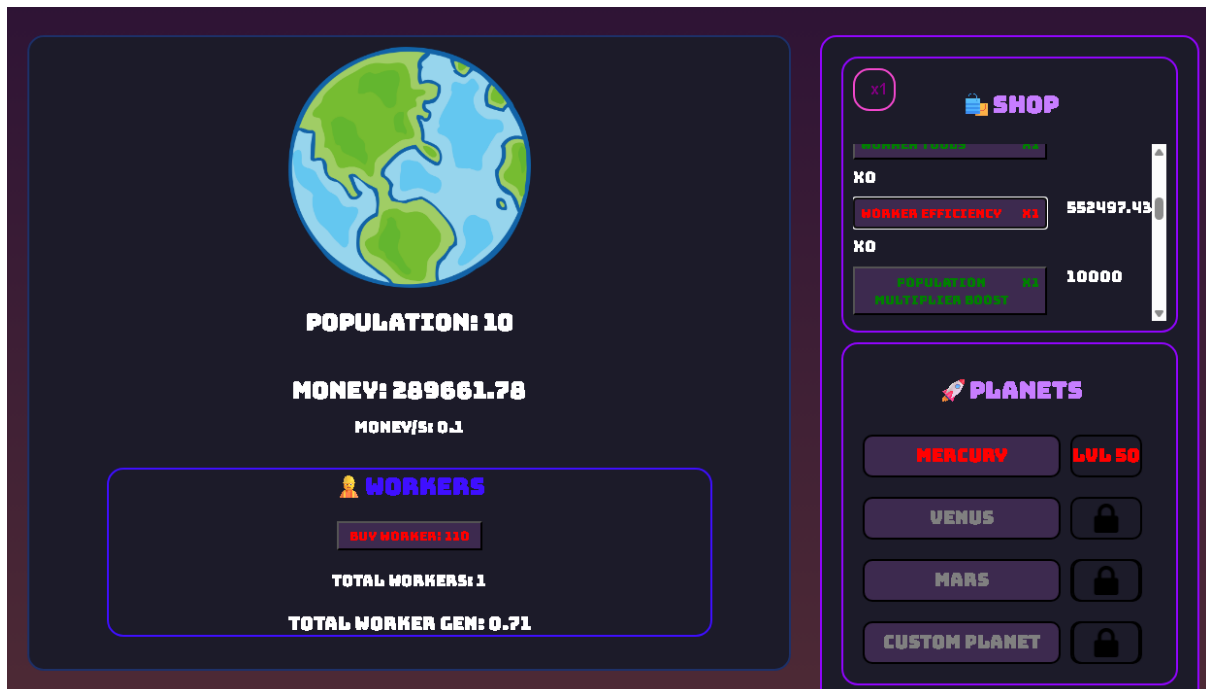
worker_speed multiplied by 1000. Therefore, I will just put my worker loop into a function and then set the interval to the worker speed multiplied by 1000 to get it in seconds:

```
let interval
// Function that handles worker generation
function worker_click() {
  clearInterval(interval)
  // Increases population by the worker gen value
  game_data.population += game_data.total_worker_gen
  game_data.population = Math.round(game_data.population * 100) / 100 //Removes extra, unwanted decimal places
  game_data.money_per_second += game_data.total_worker_gen * (0.01 * (1 + (game_data.upgrades.income_boost.level * 10) / 100)) // Recalculates money per second
  game_data.money_per_second = Math.round(game_data.money_per_second * 100) / 100 //Removes extra, unwanted decimal places
  document.getElementById("population").textContent = game_data.population
  document.getElementById("money-per-second").textContent = game_data.money_per_second
  interval = setInterval(worker_click, game_data.worker_speed * 1000)
}

// Runs worker click infinitely, sets interval to worker speed
setTimeout(worker_click, game_data.worker_speed * 1000)
```

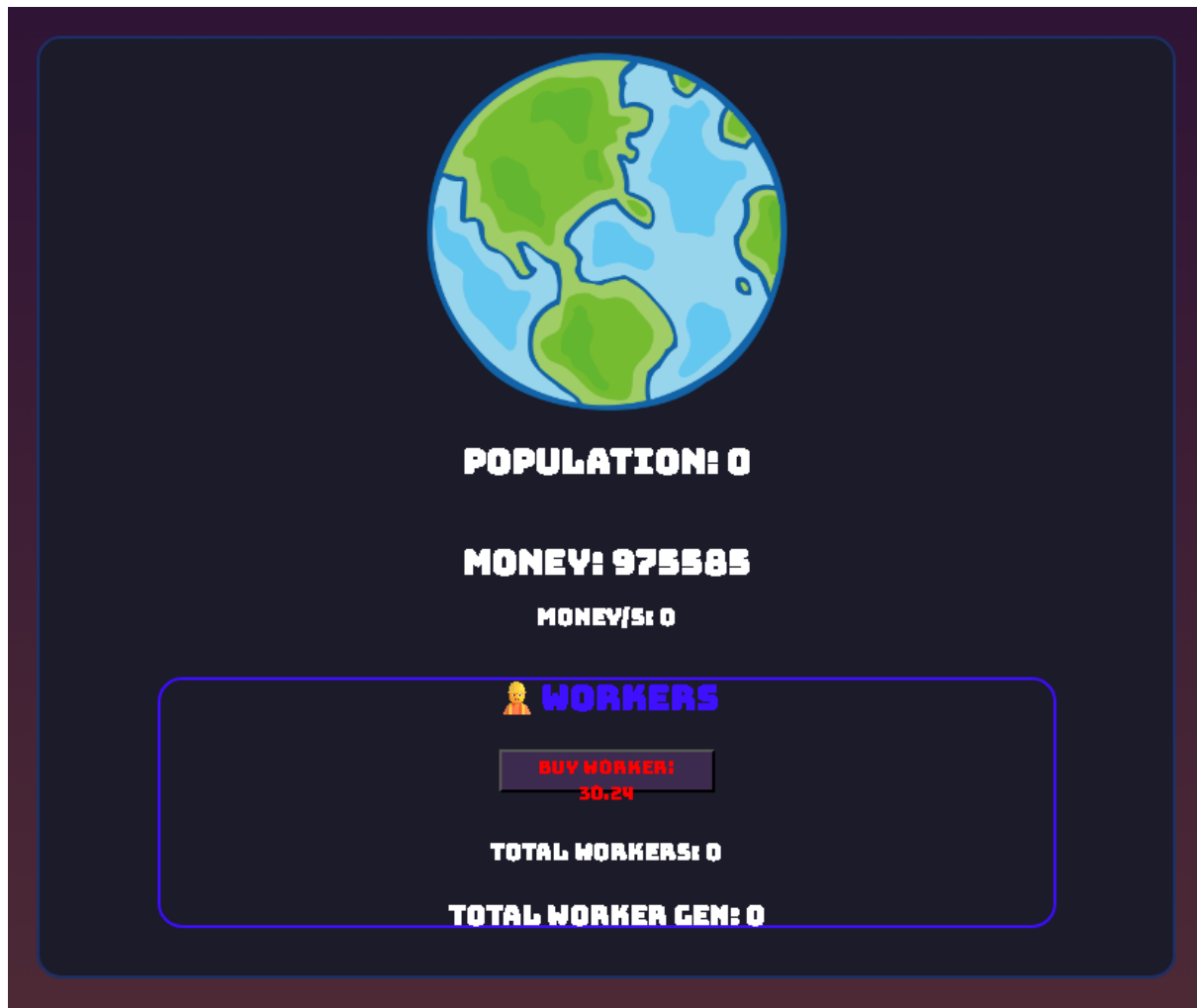
I also added comments explaining the function and loop

I then ran this to see if it worked:



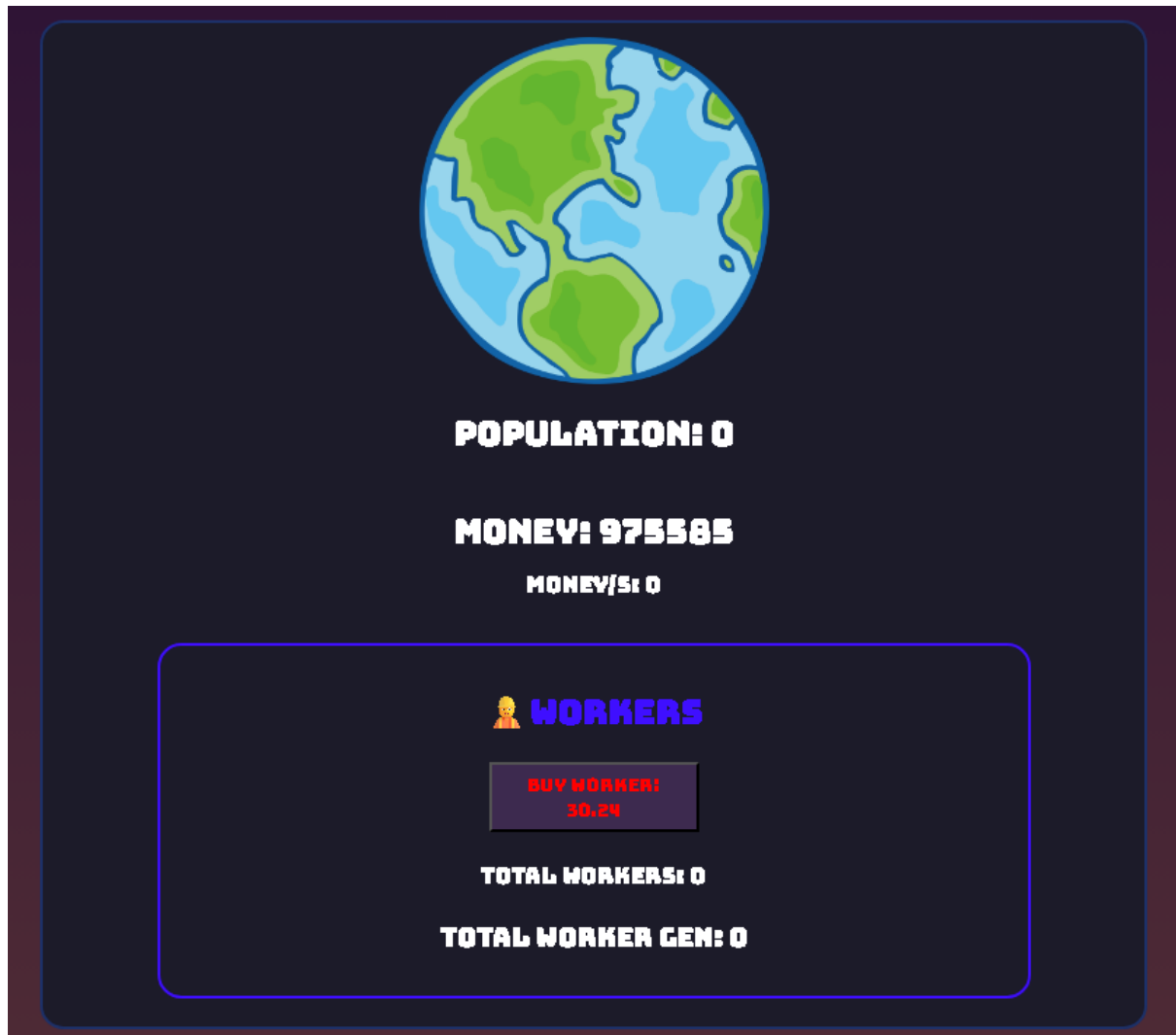
Now, after purchasing the worker efficiency upgrade a few times, the worker began increasing the population much faster, showing it is now working.

Now I need to check the worker tools upgrade, by purchasing it and seeing if the price of the workers decreases:



Each time I purchased the upgrade, the price decreased which shows it is working. However, earlier I changed the css for the worker button, worker container and central container, setting the height of all of them to auto so they would fit all the content when the text in the worker button became too long to fit on one line. This code must not have saved however, coz the height is clearly not set to auto due to the clear overflow. So, I added this code back and added some padding to the worker button, so that when it

was only one line, there would be enough of a gap between the text and the border of the button so it didn't look too thin:



Also, I realised that this upgrade could cause a problem. If the player has enough to purchase it a certain amount of times, they can just keep reducing the price of the workers until it's 0. Therefore, I decided to make it reduce the price until it was less than 50% of the original price, and then it cannot be purchased any more. I did this by first adding a variable to `game_data`, called `worker_base_price`. This is going to store the current price of workers before worker tools affects it, and will stay the same even when `worker_tools` is purchased. It will only be changed when the upgrade is actually purchased. I can then use this to check if the current worker price is half of its original price.

```
// WORKERS
workers: 0,
worker_speed: 2.5, // The amount of time (in seconds) that each worker takes to increase the population
worker_increase: 1, // The amount of population that each worker increases every action
total_worker_gen: 0, // The amount of population that all workers combined are generating per second
worker_price: 100,
worker_price_multiplier: 0.1,
worker_base_price: 100,
```

I then write an if statement to check if the worker base price divided by 2 is bigger than the current worker price, and only if so do I allow the upgrade:

```
} else if (upgrade_name == "worker_tools") {
  if (game_data.worker_base_price / 2 > game_data.worker_price) {
    game_data.worker_price *= (1 - ((1 / 10) * game_data.upgrades.worker_tools.level))
    document.getElementById("worker-price").textContent = game_data.worker_price
  }
}
```

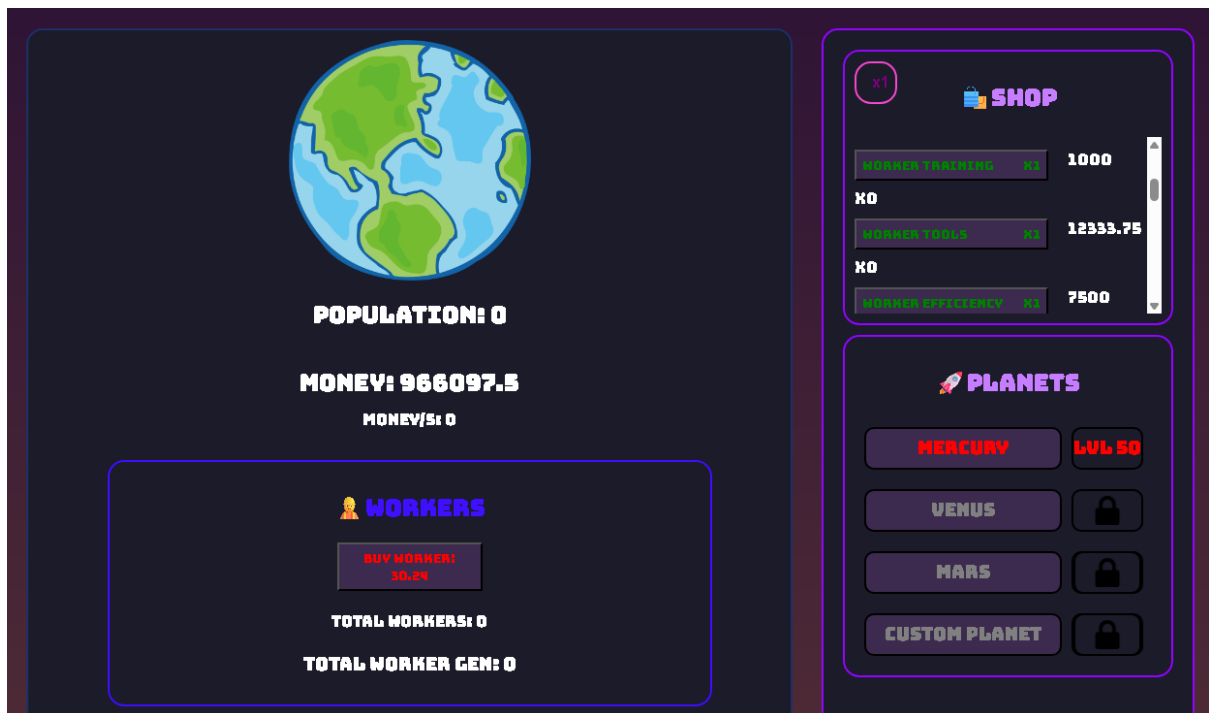
I then set the worker_base_price to the worker_price in the event handler to the worker purchase, so that it is updated each time the worker is purchased:

```
document.getElementById("worker-button").addEventListener("click", function (event) {
  if (game_data.money > game_data.worker_price) {
    game_data.workers ++
    game_data.money -= game_data.worker_price

    game_data.worker_price *= 1 + game_data.worker_price_multiplier
    game_data.worker_price = Math.round(game_data.worker_price * 100) / 100
    game_data.worker_price_multiplier += 0.05
    game_data.worker_base_price = game_data.worker_price
    game_data.total_worker_gen = (game_data.workers * game_data.worker_increase)
    game_data.total_worker_gen = Math.round(game_data.total_worker_gen * 100) / 100

    document.getElementById("workers").textContent = game_data.workers
    document.getElementById("worker-price").textContent = game_data.worker_price
    document.getElementById("worker-gen").textContent = Math.round((game_data.total_worker_gen / game_data.worker_speed) * 100) / 100
  }
})
```

I tested this to see if it worked:



The worker price stopped reducing after it got below 50% of 100, however each time I pressed the upgrade button, the money kept reducing. I realised that this was because I reduce the player's money by the upgrade price at the button of the upgrade() function, outside of the if statement that checks if the worker_tools upgrade is allowed. Therefore I decided to add the code that reduces the money (and updates price, price multiplier, and price display in HTML) inside this if statement, and then only run the one near the bottom of the function if the upgrade name doesn't equal worker_tools. This means that when the upgrade is worker tools, the price will only be reduced if the current worker price is higher than 50% of its original price:

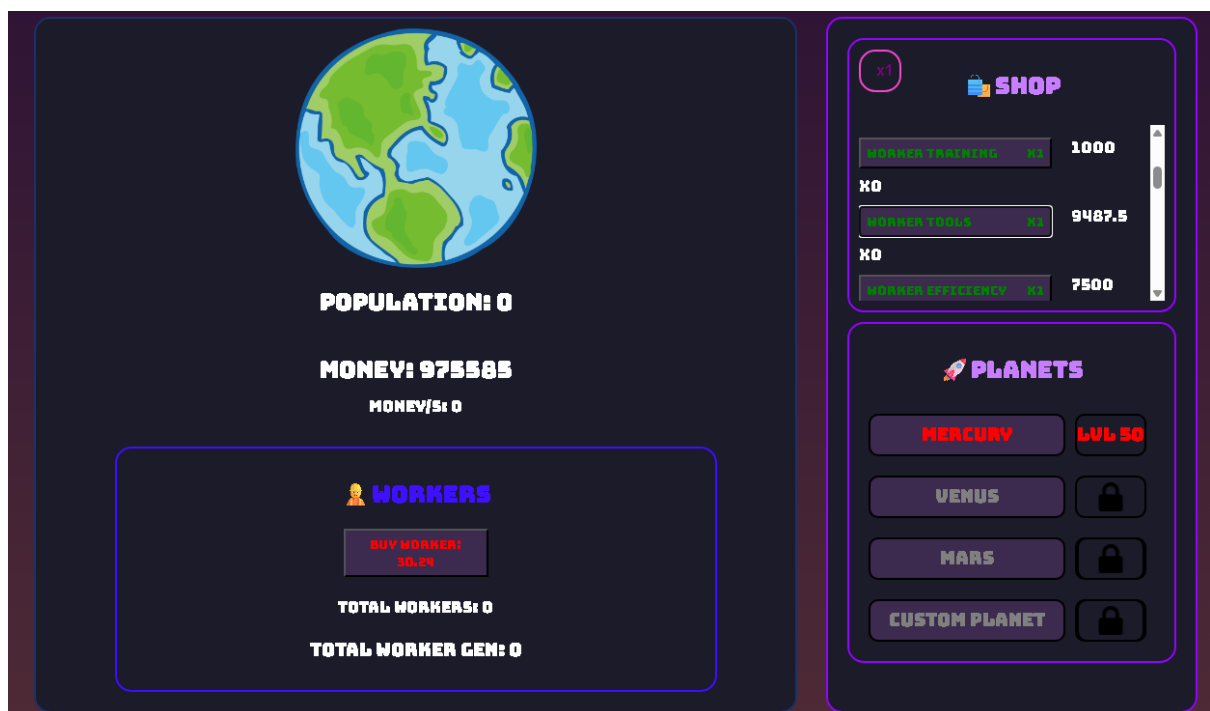
```

} else if(upgrade_name == "worker_tools") {
    if(game_data.worker_base_price / 2 < game_data.worker_price) {
        game_data.worker_price *= (1 - ((1 / 10) * game_data.upgrades.worker_tools.level))
        document.getElementById("worker_price").textContent = game_data.worker_price
        game_data.money -= game_data.upgrades[upgrade_name].price
        document.getElementById("money").textContent = game_data.money
        game_data.upgrades[upgrade_name].price *= 1 + game_data.upgrades[upgrade_name].price_multiplier
        game_data.upgrades[upgrade_name].price_multiplier += game_data.upgrades[upgrade_name].multiplier_increase
        game_data.upgrades[upgrade_name].price = Math.round(game_data.upgrades[upgrade_name].price * 100) / 100 //Removes extra, unwanted decimal places
    }
}

if(upgrade_name != "worker_tools") {
    game_data.money -= game_data.upgrades[upgrade_name].price
    document.getElementById("money").textContent = formatNum(game_data.money)
    game_data.upgrades[upgrade_name].price *= 1 + game_data.upgrades[upgrade_name].price_multiplier
    game_data.upgrades[upgrade_name].price_multiplier += game_data.upgrades[upgrade_name].multiplier_increase
    game_data.upgrades[upgrade_name].price = Math.round(game_data.upgrades[upgrade_name].price * 100) / 100 //Removes extra, unwanted decimal places
}
}

```

I tested this to see if it worked:



Now, once the worker price gets below 50% of its original, it becomes impossible to purchase workers, so the worker tools upgrade is working and more appropriate. However, I realised that since the base price gets set to the current worker price when a worker is purchased, the player can then purchase the upgrade again and reduce the price to 50% of that, and then buy a worker and keep going. They can then reduce the price to such a low number that they could get unlimited workers. Because the fix for this seems too complicated, especially to try and integrate it with the code I have. I decided to remove this upgrade. I believe workers are already powerful enough and do not need the price reduction, especially since their price multiplier increase is only 0.1. Therefore, I removed the if branch for worker tools in the upgrade function, I removed the dictionary for worker tools, I removed the “worker-tools” item from the upgrade_elements array, and I removed the upgrade’s div in the HTML of the shop

However, I realised that worker efficiency would have a similar problem that worker tools had, since even if the worker speed is at 0.5, the code at

the bottom of the upgrade function will always run so the player will still be charged for the upgrade and the upgrade's price will increase. Therefore, I decided to initialise a boolean flag variable that will become false when the worker speed reaches 0.5. Then, I will add a condition to the if statement (that checks if the money is greater than the upgrade being purchased) that checks if the boolean flag is true.

```
var worker_efficiency_allowed = true
function upgrade(upgrade_element) {
  let upgrade_name = upgrade_element.replaceAll("-", "_")
  if(game_data.money >= game_data.upgrades[upgrade_name].price && worker_efficiency_allowed == true) {
    if(game_data.multi_purchase_state == 0) {
      game_data.upgrades[upgrade_name].level += game_data.upgrades[upgrade_name].max_purchase
    } else {
```

```
    if(game_data.worker_speed <= 0.5) {
      worker_efficiency_allowed = false
    }
  }
```

I now need to code the price update, where I run an infinite loop to update the price displays for all the upgrades:

```
setInterval(() => {
  for(let i = 0; i < game_data.upgrade_elements.length; i++) {
    let upgrade_name = game_data.upgrade_elements[i].replaceAll("-", "_")
  }
}, 1000)
```

I started with an infinite loop that runs every second (1000 milliseconds), with a for loop inside that iterates over each item in upgrade elements (the array that stores the id name of each upgrade button. Since I need to access the price of the upgrades in game data, I made a variable in the same way as in the upgrade() function, by using .replaceAll to swap all occurrences of - with _ for when I access the upgrades in game_data.

I then wrote the code to update the display in the HTML:

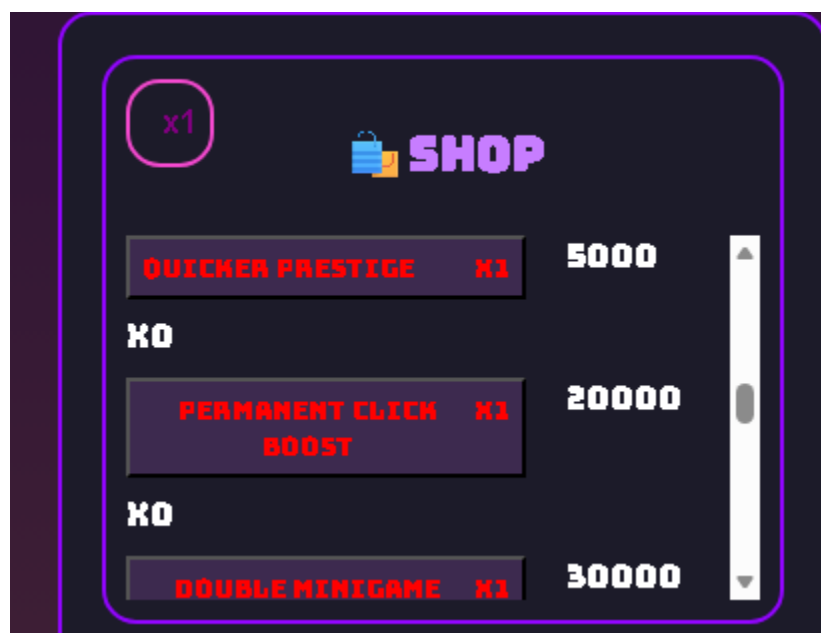
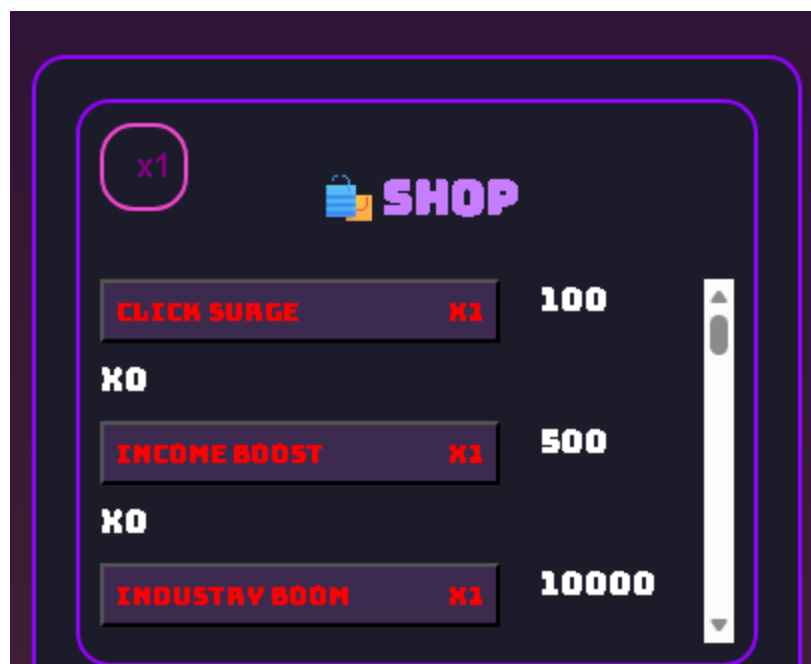
```
setInterval(() => {
  for(let i = 0; i < game_data.upgrade_elements.length; i++) {
    let upgrade_name = game_data.upgrade_elements[i].replaceAll("-", "_")
    document.getElementById(game_data.upgrade_elements[i]).textContent = game_data.upgrades[upgrade_name].price
  }
}, 1000)
```

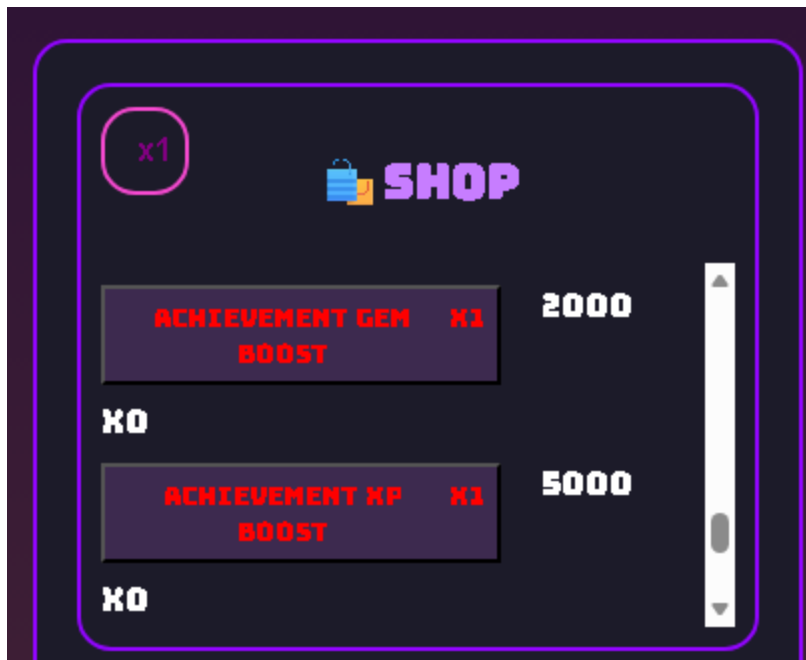
This was according to my pseudocode, however I realised that this would change the text in the upgrade button, rather than the price display, because the id of the price display is [upgrade_name]-price. Therefore, I will need to add -price onto the end of each item in upgrade_elements, when I want to access the price display. I will use string interpolation to do this:

```
setInterval(() => {
  for(let i = 0; i < game_data.upgrade_elements.length; i++) {
    let upgrade_name = game_data.upgrade_elements[i].replaceAll("-", "_")
    document.getElementById(`${game_data.upgrade_elements[i]}-price`).textContent = game_data.upgrades[upgrade_name].price
  }
}, 1000)
```

Now, the id I access to change the text content, is the each upgrade element item, followed by -price. For example, the first item in the array is click-surge, which would become click-surge-price, which is the id for the price display element for click surge in the HTML. The reason I have used a for loop to iterate over each item in the upgrade_elements array is so that I can change the price of each upgrade's price display with one loop. Without using iteration, I would have to manually set the price of each html element, which would mean one line for each element so 15 lines, which would waste time and effort.

I then ran this code to check if it worked:





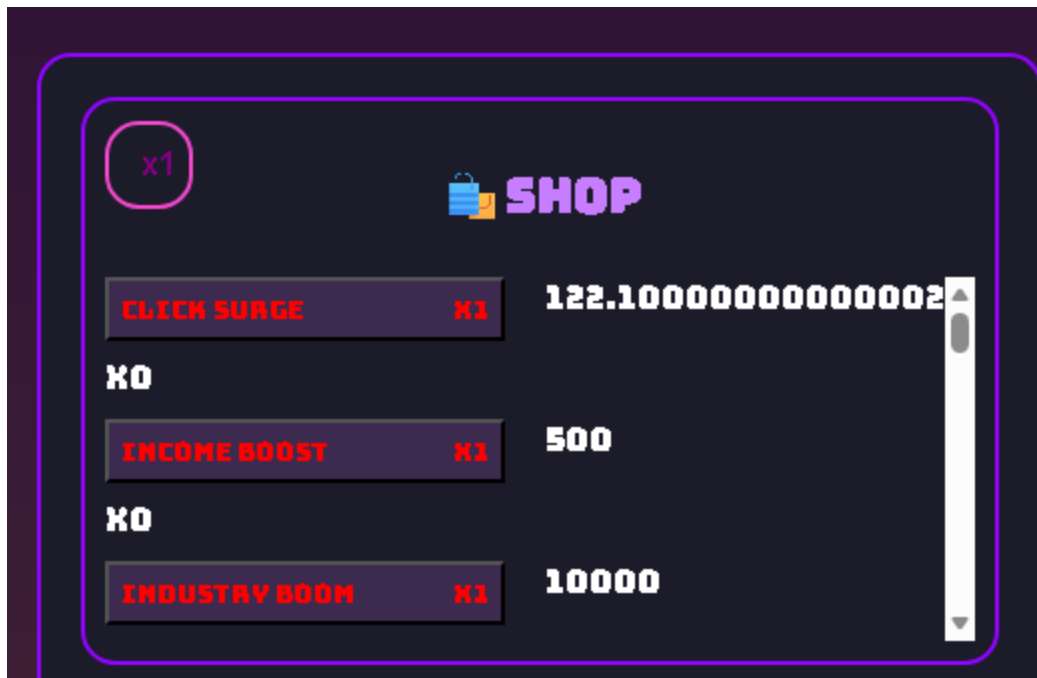
Now all the prices are correctly updated, automatically.

However, I tried pressing upgrades and the prices did not update after purchase, which I realised is because I am not actually increasing the price of the upgrade in the upgrade function, so I need to write the code to do this (code is at the bottom of the upgrade() function):

```
document.getElementById('money').textContent = game_data.money
game_data.upgrades[upgrade_name].price *= 1 + game_data.upgrades[upgrade_name].price_multiplier
game_data.upgrades[upgrade_name].price_multiplier += game_data.upgrades[upgrade_name].multiplier_increase
game_data.upgrades[upgrade_name].price = Math.round(game_data.upgrades[upgrade_name].price * 100) / 100 //Removes extra, unwanted decimal
```

This increases the price of the upgrade by the price multiplier + 1 (e.g. 1.02) and then increases the price multiplier by the multiplier increase value associated with the upgrade.

I checked to see if this worked by running the code and then purchasing a click surge upgrade:



The price correctly increased, but again, I had the problem of the extra 0s which I resolved with the method I use:

```
game_data.upgrades[upgrade_name].price = Math.round(game_data.upgrades[upgrade_name].price * 100) / 100
```

After this I coded the colour change for the upgrade text, where upgrades turn green when the player has enough money to purchase them, and red when they don't:

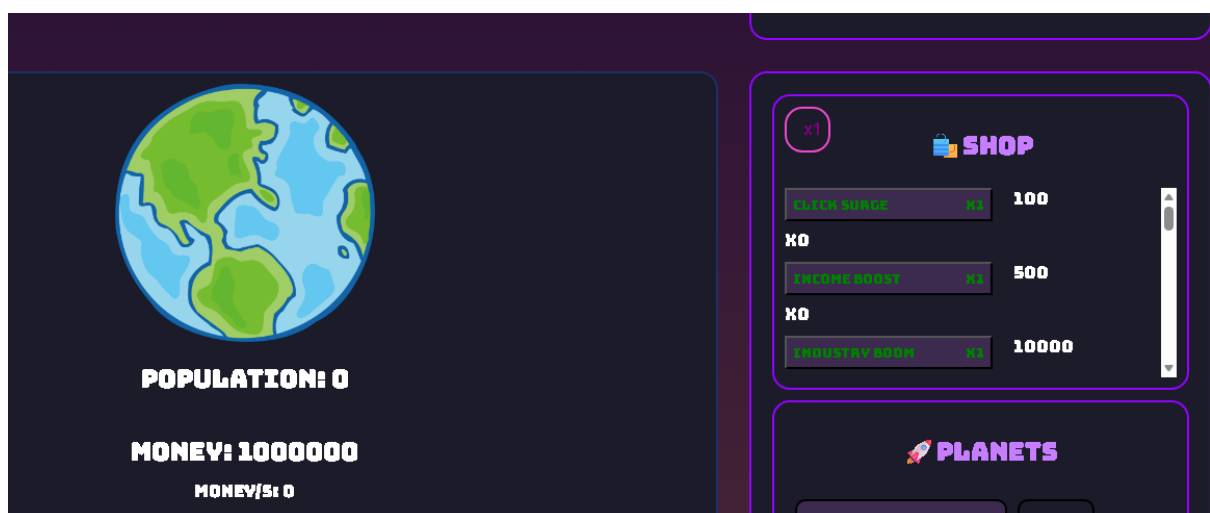
```
setInterval(() => {
  for(let i = 0; i < game_data.upgrade_elements.length; i++) {
    let upgrade_name = game_data.upgrade_elements[i].replaceAll("-", "_")
  }
}, 1000)
```

As with the previous loop, I made an infinite loop that runs every second, with a for loop inside that iterates over each item in the upgrade_elements array, to set the text colour of each. I then also made an upgrade_name variable, which is set to the current item in upgrade_elements with every

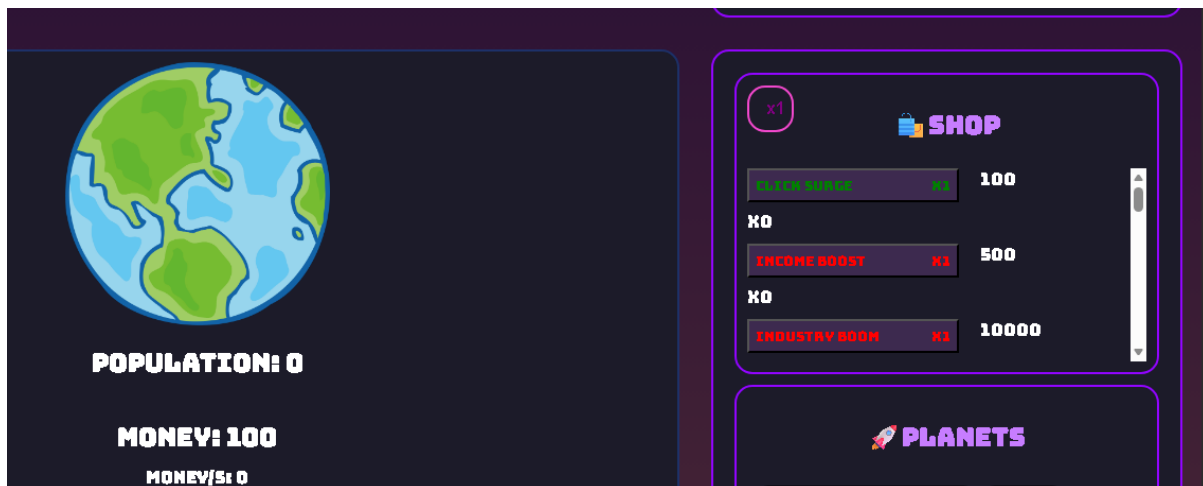
“-” replaced with a “_” so the upgrade price can be accessed in game_data. I then wrote the code to change the colour of the button text depending on whether it is purchasable:

```
setInterval(() => {  
  for(let i = 0; i < game_data.upgrade_elements.length; i++) {  
    let upgrade_name = game_data.upgrade_elements[i].replaceAll("-", "_")  
    if(game_data.money >= game_data.upgrades[upgrade_name].price) {  
      document.getElementById(game_data.upgrade_elements[i]).style.color = "green"  
    } else {  
      document.getElementById(game_data.upgrade_elements[i]).style.color = "red"  
    }  
  }  
}, 1000)
```

I used an if statement to check if the player's current money is bigger than (or equal to the price of the upgrade. If it is, I set the colour of the upgrade button text to green, using the item in upgrade_elements for the id and style.colour to change the colour. If the player's money is less than the current price, I set the colour of the text to red. I ran this to check if it worked:



Since I had 1000000 money, all the upgrade text was set to green. I then set the money to only 100:

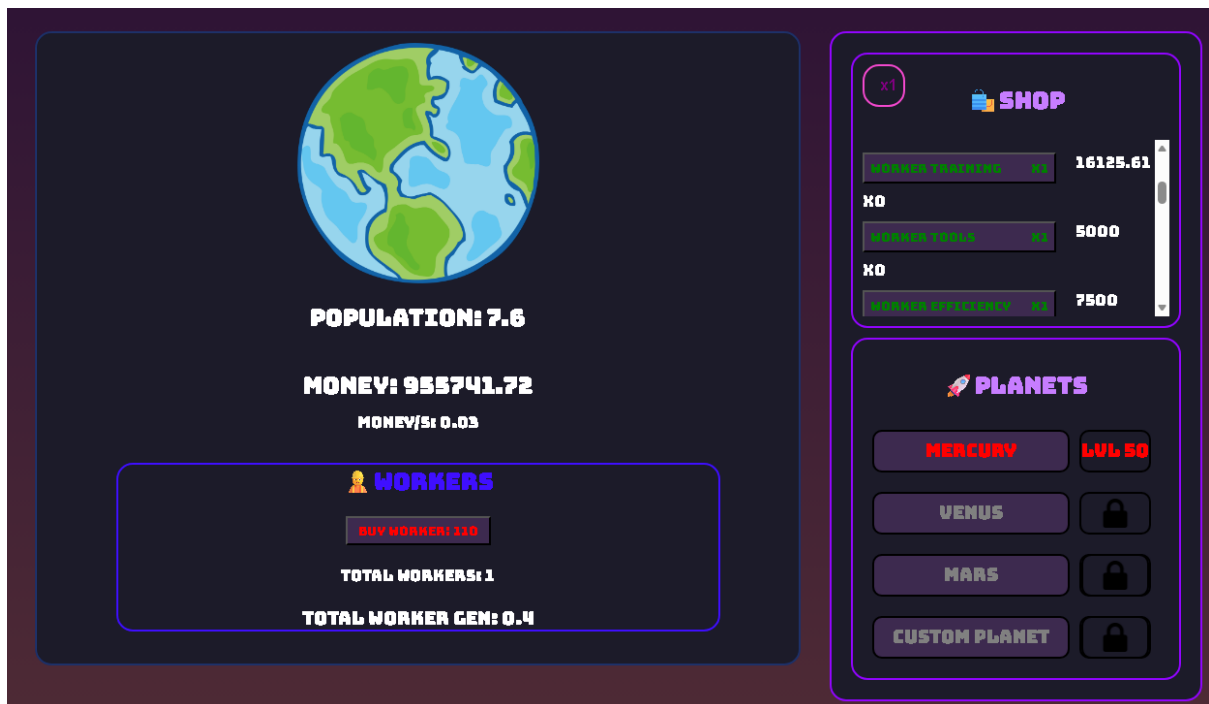


The rest of the upgrades then became red, apart from click surge which costs 100 so can be purchased by the player. This shows that it is working.

I then developed the worker training upgrade effect. For this, I just needed to change the way that the population was increased in the worker loop, by making it go up by the total worker gen multiplied $(1 + (\text{upgrades.worker_training.level} * 20) / 100)$. This will increase the amount that the workers increase the population by, by 20% each upgrade:

```
let interval
// Function that handles worker generation
function worker_click() {
  clearInterval(interval)
  // Increases population by the worker gen value, with boost from worker training upgrade
  game_data.population += game_data.total_worker_gen * (1 + (game_data.upgrades.worker_training.level * 20) / 100)
```

I then tested this by buying a worker and then purchasing the worker training upgrade a few times:

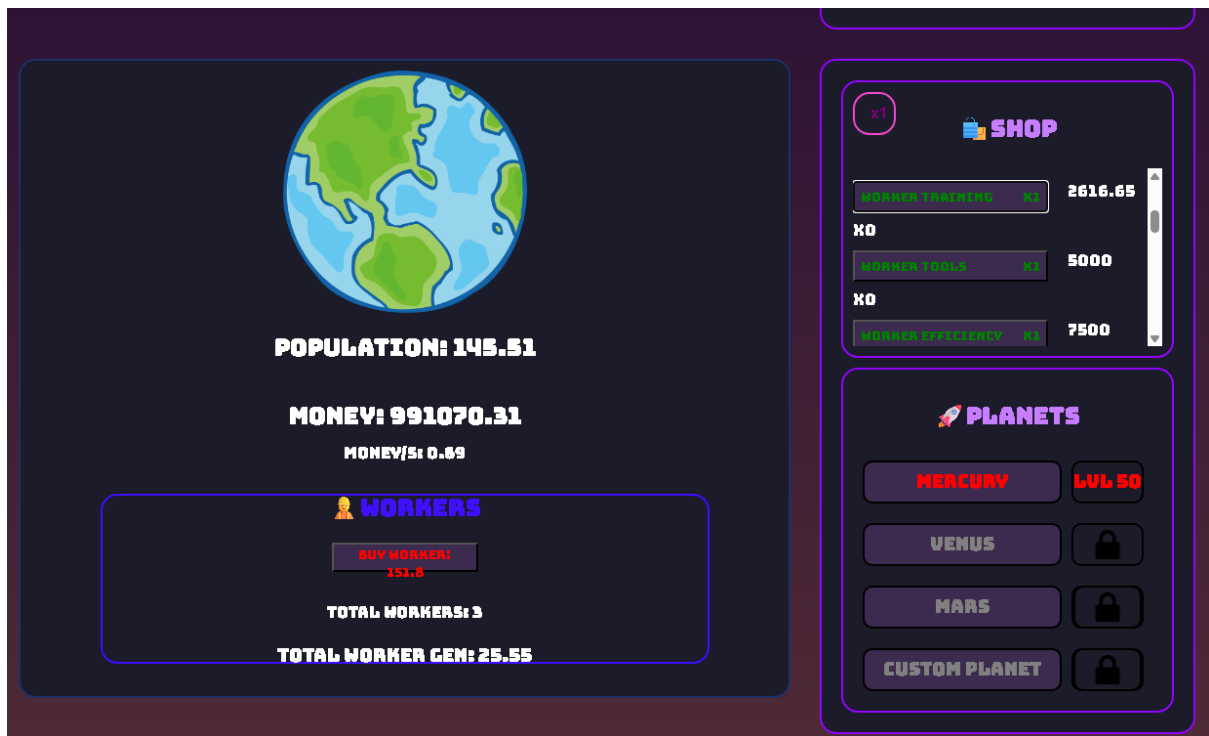


The worker began increasing population more than usual, which shows the upgrade is working, however the only thing is that I forgot to update the total worker gen display to the increased value with the upgrade. Since this is a single, instant effect that would only occur when the upgrade is purchased, I added another else branch into the upgrade function, that checks if the name of the upgrade is equal to worker_training. Since the total worker gen is calculated by multiplying the number of workers by the worker_increase value (which is 1), I will increase the worker_increase value by 20%, and then recalculate the total worker gen:

```
else if (upgrade_name == "worker_training") {
  game_data.worker_increase *= (1 + (game_data.upgrades.worker_training.level * 20) / 100)
  game_data.total_worker_gen = (game_data.workers * game_data.worker_increase)
  game_data.total_worker_gen = Math.round(game_data.total_worker_gen * 100) / 100
  document.getElementById("worker-gen").textContent = Math.round((game_data.total_worker_gen / game_data.worker_speed) * 100) / 100
}
```

I also updated the worker gen display in the HTML and removed the extra digits.

I ran this to see if it worked:



Now, the total worker gen correctly increases as I purchase worker training upgrades, with all no extra digits present.

I also realised that I have not written the code to update the upgrade level display in the HTML for each upgrade, which is the number underneath the upgrade button that shows how many of that upgrade the player has purchased. I forgot this in the pseudocode. For this, I need an infinite loop that iterates over the `upgrade_elements` array, and sets the value of the upgrade-level id (e.g. `click-surge-level`). Because I already have the exact same loop for the upgrade price, I will just use the same loop, and add in the line to change the upgrade level:

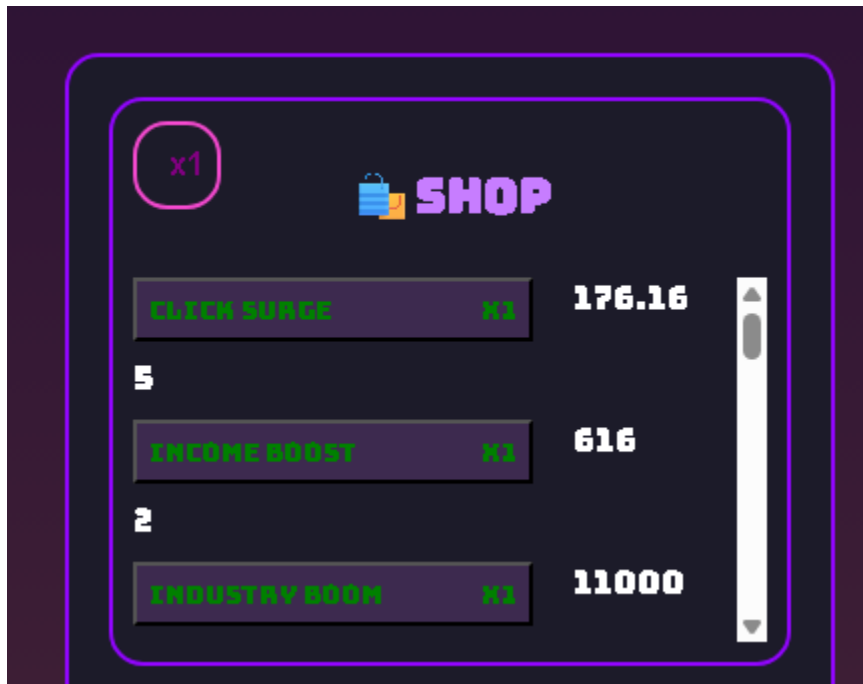
Here, I do the same thing with setting the price display of each upgrade, I

```
setInterval(() => {
  for(let i = 0; i < game_data.upgrade_elements.length; i++) {
    let upgrade_name = game_data.upgrade_elements[i].replaceAll("-", "_")
    document.getElementById(`${game_data.upgrade_elements[i]}-price`).textContent = game_data.upgrades[upgrade_name].price
    document.getElementById(`${game_data.upgrade_elements[i]}-level`).textContent = game_data.upgrades[upgrade_name].level
  }
}, 1000)
```

use string interpolation to add `-level` onto the end of each item in `upgrade_elements`, so I can access the upgrade level display of each in the

HTML, and set it to the upgrades price (using `upgrade_name` to access the specific upgrade in the upgrades dictionary).

I then ran the code to see if this worked:



As I bought upgrades, their upgrade level display increased, showing that the code is working.

I will now code the `multi_purchase` feature, starting with the event listener for the multi-purchase button, with the event handler that changes the multi-purchase-state depending on what it was when the button was clicked:

```
document.getElementById("multi-purchase-button").addEventListener("click", function (event) {
    if(game_data.multi_purchase_state == 1) {
        game_data.multi_purchase_state = 10
    } else if (game_data.multi_purchase_state == 10) {
        game_data.multi_purchase_state = 25
    } else if (game_data.multi_purchase_state == 25) {
        game_data.multi_purchase_state = 0
    } else {
        game_data.multi_purchase_state = 1
    }
})
```

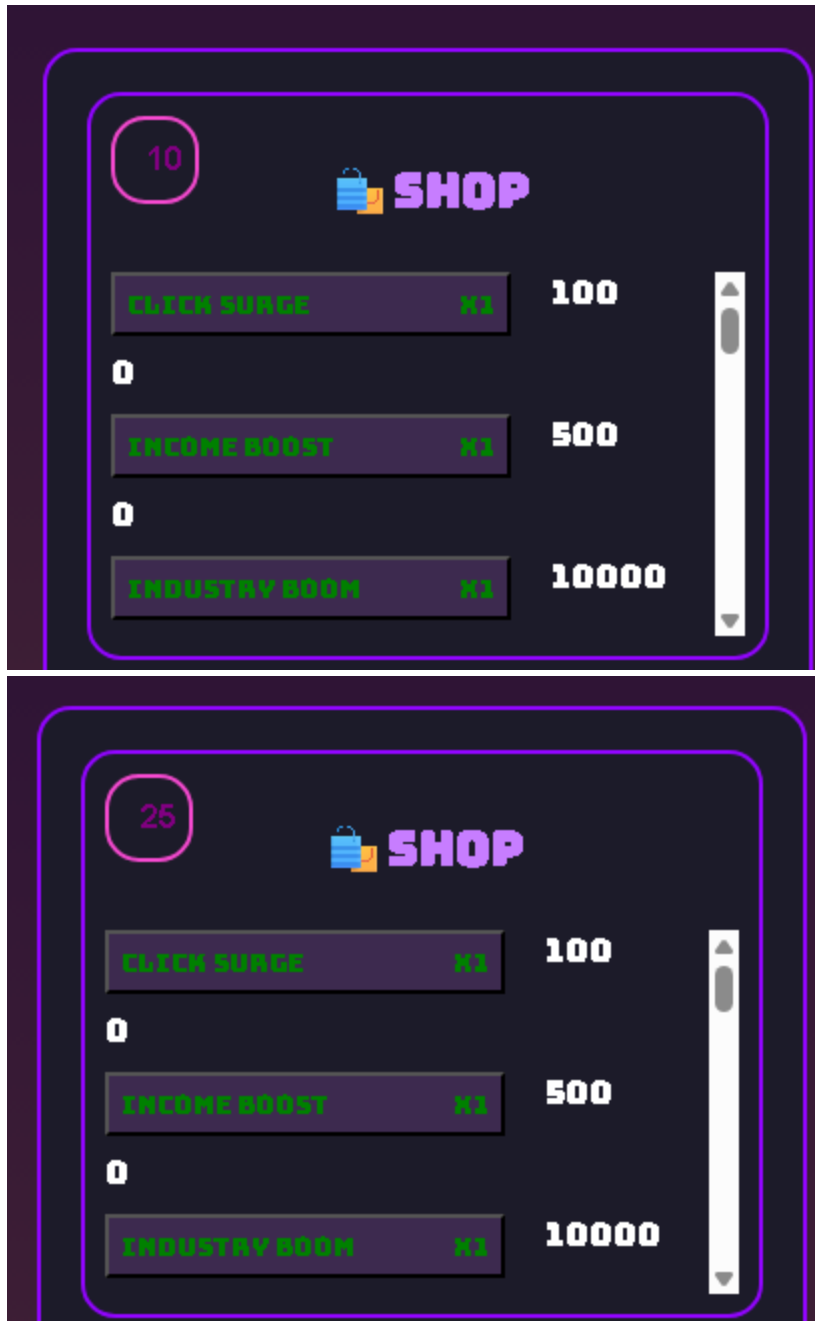
I added an event listener to the multi-purchase button, by accessing its HTML id. In the event handler, I used an if statement to check the current multi-purchase state and update it accordingly. Essentially, the multi-purchase state can be either 1, 10, 25 or 0, this code will set it to the one after. For example, if it is 10 then it will be set to 25. If it is 0, it will be put back to the first one which is 1. This allows the player to cycle through the different multi-purchase options. I could have used an array with all the possible multi-purchase options, and then a pointer to the current one, however this would increase memory usage as I would have to have an extra array as well as a pointer, so I decided just to use an if statement to update the multi-purchase state, meaning I just need one variable. I did, however, realise it would be neater and take less lines of code to use a switch statement for this, so I converted it:

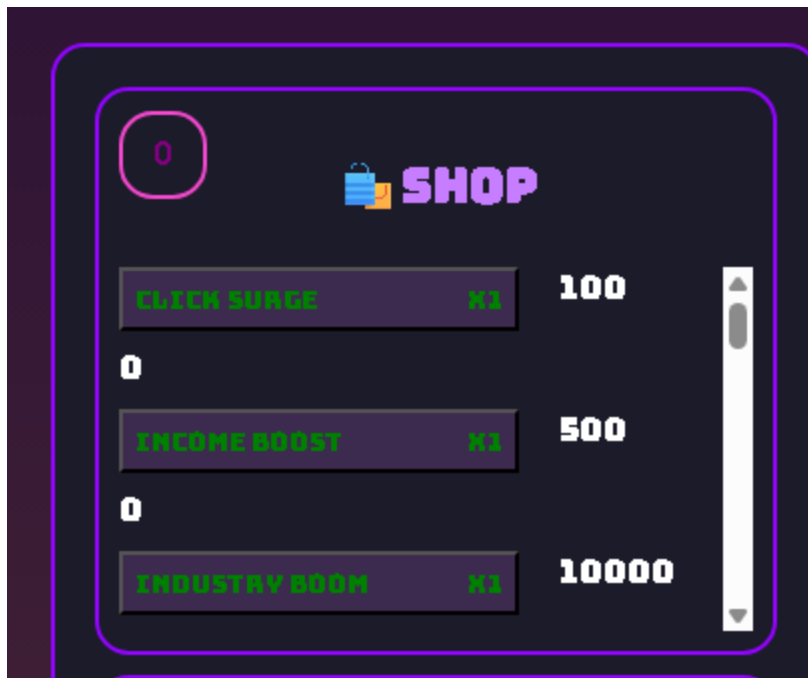
```
// Updates multi_purchase_state to its next value each time player clicks the button
document.getElementById("multi-purchase-button").addEventListener("click", function (event) {
    // Checks current state and switches to the one after
    switch(game_data.multi_purchase_state) {
        case 1: game_data.multi_purchase_state = 10; break
        case 10: game_data.multi_purchase_state = 25; break
        case 25: game_data.multi_purchase_state = 0; break
        default: game_data.multi_purchase_state = 1 // If at the max state, reset back to x1
    }
})
```

I then added the line to update the multi-purchase button text to the multi-purchase state:


```
// Updates multi-purchase state button to the updated value
document.getElementById("multi-purchase-state").textContent = game_data.multi_purchase_state
```

I then ran the code to see if this worked:





The text in the button was correctly updating, however I noticed 2 problems. Firstly, when the multi-purchase state is 1, the button reads “x1”, which is what I want. However, at every other state there is no x before the value, as I forgot to code it. Therefore, I will use string interpolation when I set the text content of the multi-purchase-state, to add an x before the multi purchase state value:

```
// Updates multi-purchase state button to the updated value
document.getElementById("multi-purchase-state").textContent = `x${game_data.multi_purchase_state}`
```

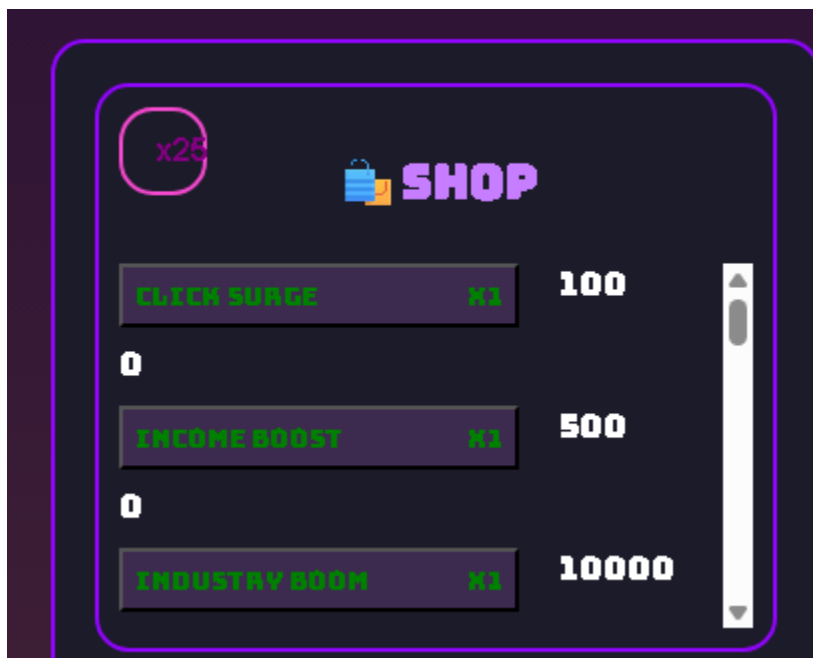
The second problem is that I forgot to make the button say “MAX” when the multi-purchase state is 0, as I am using 0 in the JavaScript to represent MAX. The reason I am using 0 is because the multi_purchase_state value is an integer, so I chose an integer value that would work in the code and that I could use to represent the MAX multi-purchase state. Therefore, I need to use an if statement to check if the multi-purchase state is 0, and if so set the text content to “MAX”, but if not set it to the value of multi_purchase_state (with an x before):

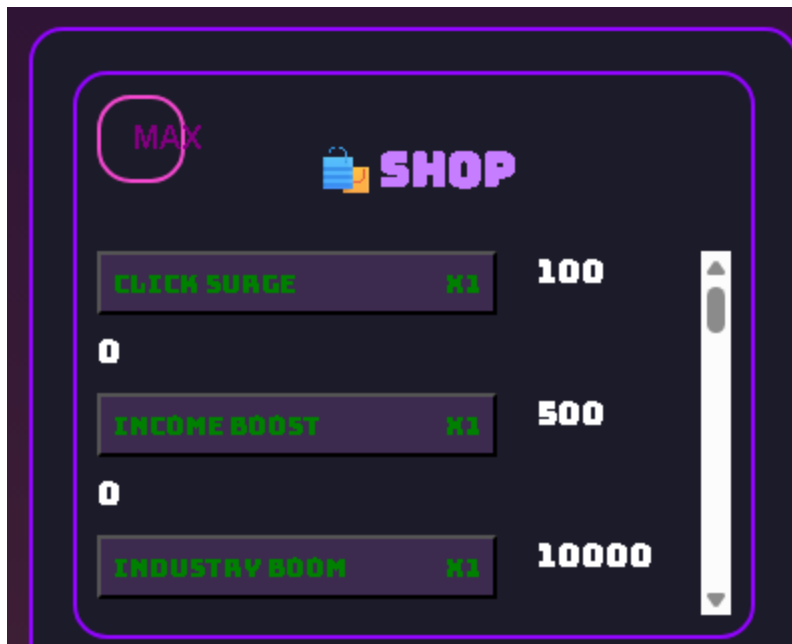
```

// If multi-purchase state is at MAX setting, update button text to "MAX"
if (game_data.multi_purchase_state == 0) {
  document.getElementById("multi-purchase-state").textContent = "MAX"
}
// Otherwise, set it to the multi_purchase_state value
else {
  document.getElementById("multi-purchase-state").textContent = `x${game_data.multi_purchase_state}`
}
})

```

I ran this to see if it worked:



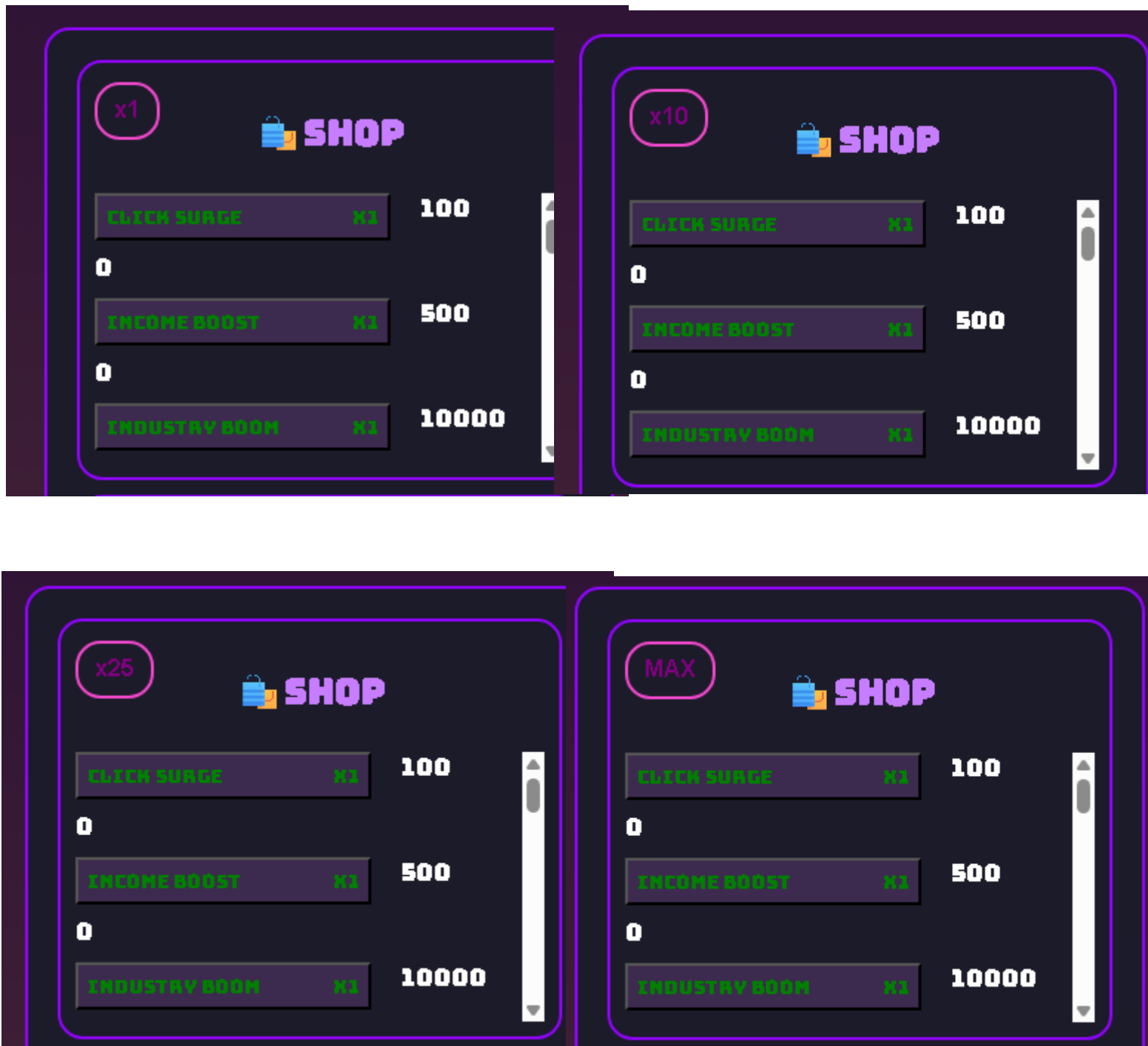


Now, the x is added before the numbers and the multi-purchase button reads MAX when the multi_purchase_state variable is 0. However, this introduced another problem, the text was overflowing the button.

Therefore, I will set the width of the button to auto, so it would re-size to fit the size of the text inside it, meaning it will be the perfect width each time it is pressed. I also decided to reduce the padding from 10 to 5px, as I thought it was too much, and increase the height from 20 to 25 as I thought it was too short:

```
.multi-purchase {  
  border: 2px solid black;  
  border-radius: 17px;  
  padding: 5px;  
  margin: 0px;  
  text-align: center;  
  width: auto;  
  height: 25px;  
  display: flex;  
  justify-content: left;  
  align-items: center;  
  position: absolute;  
  margin-bottom: 215px;  
}
```

I ran this to check it worked:



Now, the size of the width of the button updates as it is pressed, to fit the contents.

I will now code the next part of the multi-purchase feature, the function that calculates and updates the price of each upgrade according to the multi-purchase state.

```
function update_prices() {
  for(const key in game_data.upgrades) {
    let current_upgrade = game_data.upgrades[key]
    var prices = []
    prices.push(current_upgrade.price)
    var count = 1
    var next_price = 0
    var prices_sum = 0

    if(game_data.multi_purchase_state == 0) {

    } else {

    }
  }
}
```

I started by defining a function with the identifier `update_prices`, with a `for` loop inside that will iterate over each key in the `upgrades` dictionary. Since each key is a dictionary for an upgrade, this iterate over each upgrade dictionary in the `upgrades` dictionary. I then declared a variable called `current_upgrade`, and set it to the current upgrade dictionary being iterated over. The reason I did this was so that I would not have to type `game_data.upgrades[key]` each time I accessed the current upgrade, instead all I will need to type is `current_upgrade`. I then declared the variables that will be used in the price calculations. The `prices` array will be used to store the price of buying each upgrade, and I start by adding the current price of the upgrade to it, as this is the starting price. The `count` will be used to access the previous item in the `prices` array (I will index it using `count-1`), and will also be set to the max purchase of the upgrade at the end

(if the multi-purchase state is 0), as it will count the number of times the upgrade can be purchased. The `next_price` variable will be used to store the price of buying the upgrade again after the previous time, and the `prices_sum` variable will just be used to total all the values in the `prices` array. I then wrote an if statement to check if the multi-purchase state is 0, since the code will be slightly different if the multi-purchase state is 0 since the code needs to calculate the max number of upgrades the player can purchase.

I then began writing the code for the MAX multi-purchase state:

```
if(game_data.multi_purchase_state == 0) {  
    while(true) {  
        next_price = prices[count - 1] * (1 + current_upgrade.price_multiplier)  
    }  
}
```

According to the python script I made and my pseudocode, I made an infinite loop, which starts by assigning the `next_price` variable the value of the previous element (`count - 1`) multiplied by `1 +` the price multiplier. This calculates what the cost would be of buying upgrade, taking into account the price scaling. Since I then need to sum the items in the `prices` array, to see if adding the `next_price` would be too much, I will need to use a for loop to add up the items since JavaScript doesn't have a `sum()` function, making sure I reset the `prices_sum` to 0 before it, otherwise it will keep the value from the previous sum calculation:

```
prices_sum = 0  
for(let i = 0; i < prices.length; i++) {  
    prices_sum += prices[i]  
}
```

Here, I use a for loop which iterates over each item in the prices array, adding each one to the prices_sum variable. This sums all the items in the prices array. I then need to check if the sum + the next price variable is more than the players money, so I will use an if statement to check this:

```
if(prices_sum + next_price > game_data.money) {  
    break  
}
```

If the prices sum plus the next price is more than the player's money, the algorithm will break out of the loop. The reason I have done this is because I want to check if buying the upgrade again would be too much for the player to afford, without adding the next price to the array. This ensures that the max purchase value will never go above the players money, since I check before I add the next price whether it would be too much, and I break out of the loop. If buying the upgrade again would not be too much money, then the algorithm continues and I will add the next price to the prices array:

```
prices.push(next_price)  
current_upgrade.price_multiplier += current_upgrade.multiplier_increase  
count ++
```

Now, the next price is added onto the array, and I also increase the price multiplier by the multiplier increase value of the upgrade, and increase count which I will use at the end for the max purchase value.

I will now set the price of the upgrade to the prices sum value, since it stores the cost of buying the upgrade until the price is too much, and I also was going to set the text content of the price display in the HTML for the upgrade, but I realised that I already have an infinite loop that runs every second that does this. Therefore, it would not be absolutely necessary, however since it runs every second there will sometimes be a slight delay in the price update when the player clicks the multi-purchase button.

Therefore, I decided to make the price (and level) update loop a function which I can call here, and also that runs every second:

```
function update_price_and_level() {
  for(let i = 0; i<game_data.upgrade_elements.length; i++) {
    let upgrade_name = game_data.upgrade_elements[i].replaceAll("-", "_")
    document.getElementById(`${game_data.upgrade_elements[i]}-price`).textContent = game_data.upgrades[upgrade_name].price
    document.getElementById(`${game_data.upgrade_elements[i]}-level`).textContent = game_data.upgrades[upgrade_name].level
  }
}

setInterval(() => {
  update_price_and_level()
}, 1000)
```

I then called this function in the update_prices function, after setting the price of the upgrade to the prices sum, after the while loop:

```
function update_prices() {
  for(const key in game_data.upgrades) {
    let current_upgrade = game_data.upgrades[key]
    var prices = []
    prices.push(current_upgrade.price)
    var count = 1
    var next_price = 0
    var prices_sum = 0
    if(game_data.multi_purchase_state == 0) {
      while(true) {
        next_price = prices[count - 1] * (1 + current_upgrade.price_multiplier)

        prices_sum = 0
        for(let i = 0; i<prices.length; i++) {
          prices_sum += prices[i]
        }

        if(prices_sum + next_price > game_data.money) {
          break
        }

        prices.push(next_price)
        current_upgrade.price_multiplier += current_upgrade.multiplier_increase
        count ++
      }

      current_upgrade.price = prices_sum
      update_price_and_level()
```

This means that, once the correct max upgrade price has been calculated, the price text will be instantly updated.

I also remembered that I have a max_purchase value for each upgrade, which stores the number of times that the upgrade can be purchased with the amount of money they have, so I added a line in to set that to the count variable, since this is the amount of times the while loop ran:

```
current_upgrade.price = prices_sum
current_upgrade.max_purchase = count
update_price_and_level()
```



I also remembered that I need to update the purchase amount display for each upgrade. This is the number that indicates how much of the upgrade the player will be purchasing. However, since we will need to access the upgrade_elements array to get each element in the HTML to change the text content. For this, I will need a value to index the array. Normally, I do this in a for loop so I have the loop counter, but the loop counter for this for loop is the key of each dictionary in game_data.upgrades, which is not a number. Therefore, I will initialise a variable called upgrade_number, which will start at 0 and increase for each loop in the for loop. I will then use it to index the upgrade_elements array to get set the text content of the purchase amount display:

```
function update_prices() {
  let upgrade_number = 0
```

```
document.getElementById(`${game_data.upgrade_elements[upgrade_number]}-amount`).textContent = `x${current_upgrade.max_purchase}`
upgrade_number ++
```

I have also used string interpolation to add -amount onto each item in the upgrade_elements array, since this is the format of the id of each purchase amount display in the HTML (e.g. click-surge-amount). I then set the text content to the max_purchase value of the upgrade.

I will now write the code for when the multi-purchase state is not 0 (MAX), for which I just need a for loop to calculate the price of purchasing the upgrade either 1, 10 or 25 times:

```
else {  
    for (let i = 0; i < game_data.multi_purchase_state - 1; i++) {  
        next_price = prices[count - 1] * (1 + current_upgrade.price_multiplier)  
        prices.push(next_price)  
        current_upgrade.price_multiplier += current_upgrade.multiplier_increase  
        count ++  
    }  
}
```

Here, I have made a for loop with the number of loops being the multi-purchase state. Each time the loop runs, I set the value of next_price to the previous price (prices[count-1]) multiplier by the price multiplier plus 1. This calculates the price of buying the upgrade again after the last one, I then add next_price to the prices array, as this stores the price of buying each upgrade. I then increment the price multiplier like normal, and increase the count so that in the next iteration, the next price in the prices array will be used to set next_price. After this, I created a for loop to sum the prices array, like I did before, and then set the price of the upgrade to the result:

```
prices_sum = 0  
for (let i = 0; i < prices.length; i++) {  
    prices_sum += prices[i]  
}
```

I also incremented the upgrade number variable like I did before:

```
document.getElementById(`${game_data.upgrade_elements[upgrade_number]}-amount`).textContent = `x${game_data.multi_purchase_state}`  
upgrade_number ++  
}
```

Finally, I called the `update_price_and_level` function as the price is updated too so I need to update the HTML elements for the price. I do this in the `if` statement before this, and I will need to do it here as well, but since it happens in both, I can take it out of the `if` statement and just have it at the end of the `for` loop, so it happens no matter what. This is because the prices are always going to be updated in this function, so I can call the function every time it runs:

```
} else {  
  for (let i = 0; i < game_data.multi_purchase_state - 1; i++) {  
    next_price = prices[count - 1] * (1 + current_upgrade.price_multiplier)  
    prices.push(next_price)  
    current_upgrade.price_multiplier += current_upgrade.multiplier_increase  
    count ++  
  }  
  
  prices_sum = 0  
  for (let i = 0; i < prices.length; i++) {  
    prices_sum += prices[i]  
  }  
  
  current_upgrade.price = prices_sum  
  
  document.getElementById(`${game_data.upgrade_elements[upgrade_number]}-amount`).textContent = `x${game_data.multi_purchase_state}`  
  upgrade_number ++  
}  
  
update_price_and_level() ←
```

I also added comments explaining this function, as it was quite complicated:

```
// Function that updates prices when multi-purchase button is clicked
function update_prices() {
    let upgrade_number = 0
    // Iterates over shop upgrade objects in game_data
    for(const key in game_data.upgrades) {
        let current_upgrade = game_data.upgrades[key]
        // Resets upgrade price and price multiplier to base values
        current_upgrade.price = current_upgrade.base_price
        current_upgrade.price_multiplier = current_upgrade.base_multiplier
        var prices = []
        prices.push(current_upgrade.price)
        var count = 1
        var next_price = 0
        var prices_sum = 0
        //MAX multi-purchase setting
        if(game_data.multi_purchase_state == 0) {
            while(true) {
                // Calculates the next price of the upgrade, by multiplying previous price by current price multiplier
                next_price = prices[count -1] * (1 + current_upgrade.price_multiplier)

                prices_sum = 0
                // Calculates the sum of the upgrade prices array
                for(let i = 0; i<prices.length; i++) {
                    prices_sum += prices[i]
                }

                // Checks if the next price of the upgrade makes the total too much to afford
                if(prices_sum + next_price > game_data.money) {
                    // Exits loop if so
                    break
                }

                // Adds next price of upgrade to array and increments price multiplier and count
                prices.push(next_price)
                current_upgrade.price_multiplier += current_upgrade.multiplier_increase
                count ++
            }
        }
    }
}
```

```

    // Sets upgrade price to the sum of the total cost of buying all the upgrade the player can afford
    current_upgrade.price = prices_sum
    // Sets the max purchase number to the loop count, to give the number of times the player can purchase the upgrade
    current_upgrade.max_purchase = count

    // Updates the upgrade purchase number in HTML
    document.getElementById(`${game_data.upgrade_elements[upgrade_number]}-amount`).textContent = `x${current_upgrade.max_purchase}`
    upgrade_number ++

    // x1, x10, x100 Multi-purchase settings:
    } else {

        // Runs either 1, 10, or 100 times
        for (let i = 0; i<game_data.multi_purchase_state-1; i++) {
            // Calculates next price of upgrade and adds to array
            next_price = prices[count -1] * (1 + current_upgrade.price_multiplier)
            prices.push(next_price)
            // Increments price multiplier and count
            current_upgrade.price_multiplier += current_upgrade.multiplier_increase
            count ++
        }

        prices_sum = 0
        for(let i = 0; i<prices.length; i++) {
            prices_sum += prices[i]
        }

        // Sets upgrade price to the sum of the total cost of buying all the upgrade the player can afford
        current_upgrade.price = prices_sum

        // Updates the upgrade purchase number in HTML
        document.getElementById(`${game_data.upgrade_elements[upgrade_number]}-amount`).textContent = `x${game_data.multi_purchase_state}`
        upgrade_number ++
    }

    // Updates upgrade price and level displays
    update_price_and_level()
}
}
```

Now I have completed this function, I need to make sure I call it in the even handler for the multi-purchase button, so that the prices are updated each time it is clicked:

```
// Updates multi_purchase_state to its next value each time player clicks the button
document.getElementById("multi-purchase-button").addEventListener("click", function (event) {

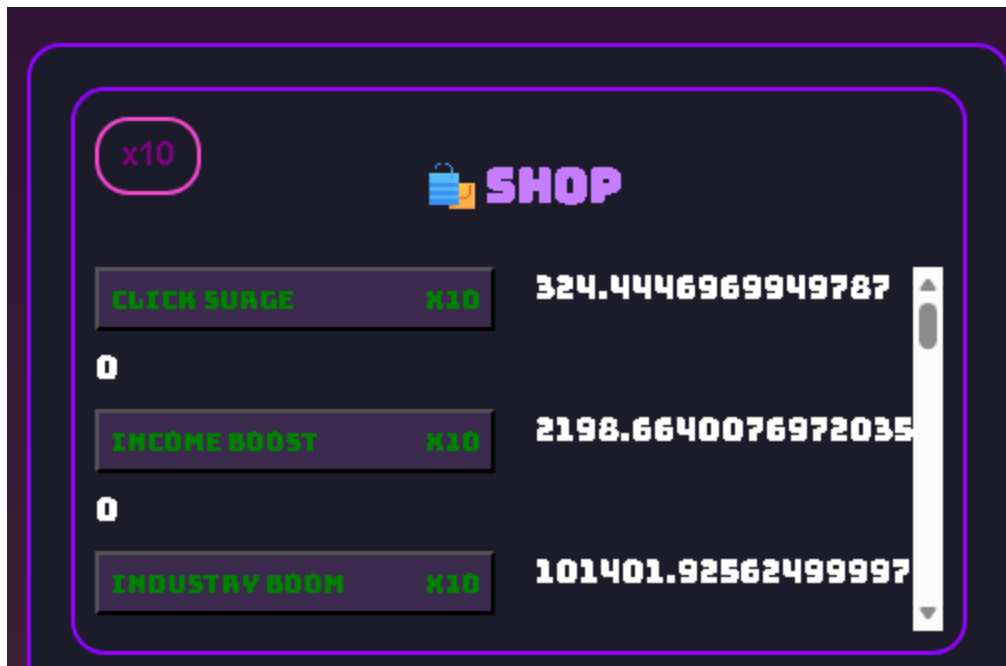
    // Checks current state and switches to the one after
    switch(game_data.multi_purchase_state) {
        case 1: game_data.multi_purchase_state = 10; break
        case 10: game_data.multi_purchase_state = 25; break
        case 25: game_data.multi_purchase_state = 0; break
        default: game_data.multi_purchase_state = 1 // If at the max state, reset back to x1
    }

    // If multi-purchase state is at MAX setting, update button text to "MAX"
    if (game_data.multi_purchase_state == 0) {
        document.getElementById("multi-purchase-state").textContent = "MAX"
    }
    // Otherwise, set it to the multi_purchase_state value
    else {
        document.getElementById("multi-purchase-state").textContent = `x${game_data.multi_purchase_state}`
    }

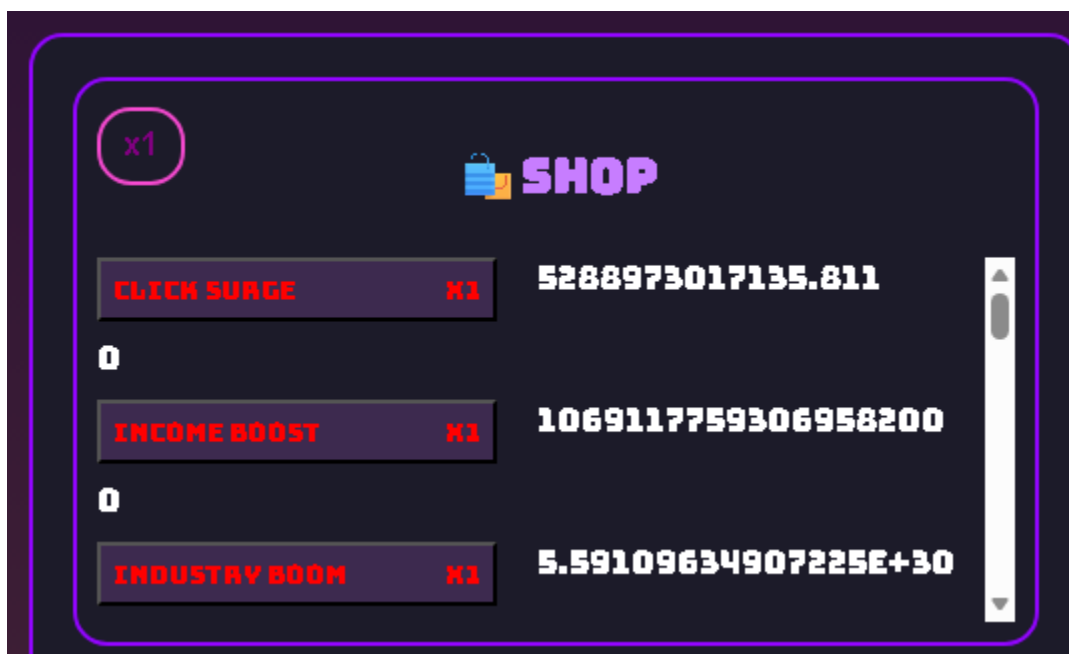
    // Updates prices according to multi_purchase_state
    update_prices()

})
```

I ran this to see if it worked:



After clicking the first time, everything worked (apart from the extra digits added on which I will fix after), except as I kept clicking, the prices kept increasing even when I got back to x1:



This was because I forgot to reset the price, using the base_price value for each upgrade. At the start of each loop, I needed to set the price of the current upgrade to it's base price, so each time the price is reset to its original, and then it is recalculated according to the multi-purchase state. I

also realised that, since the price multiplier is modified as well, I will need to reset this at the start of the loop as well. I don't have a `base_price_multiplier` value so I can't reset it to this, however since all the multipliers start at 0.1, I can just set it to this:

```
// Function that updates prices when multi-purchase button is clicked
function update_prices() {

    // Iterates over shop upgrade objects in game_data
    for(const key in game_data.upgrades) {
        let current_upgrade = game_data.upgrades[key]
        // Resets upgrade price and price multiplier to base values
        current_upgrade.price = current_upgrade.base_price ←
        current_upgrade.price_multiplier = 0.1 ←
    }
}
```

I will then also need to set the base price to the actual price of the upgrade when it is purchased, because the base price stores the current price, regardless of the multi-purchase state, but when the upgrade is purchased it needs to increase. So I will put it in upgrade function:

```
function upgrade(upgrade_element) {
    let upgrade_name = upgrade_element.replaceAll("-", "_")

    if(game_data.money >= game_data.upgrades[upgrade_name].price) {
        if(game_data.multi_purchase_state == 0) {
            game_data.upgrades[upgrade_name].level += game_data.upgrades[upgrade_name].max_purchase
        } else {
            game_data.upgrades[upgrade_name].level += game_data.multi_purchase_state
        }

        if(upgrade_name == "industry_boom") {
            game_data.money_per_second *= 1.2
            game_data.money_per_second = Math.round(game_data.money_per_second * 100) / 100 //Removes extra, unwanted decimal places
            document.getElementById("money-per-second").textContent = formatNum(game_data.money_per_second)
        } else if (upgrade_name == "worker_efficiency" && game_data.worker_speed > 0.5) {
            game_data.worker_speed -= 0.1
            game_data.total_worker_gen = (game_data.workers * game_data.worker_increase)
            game_data.total_worker_gen = Math.round(game_data.total_worker_gen * 100) / 100
            document.getElementById("worker-gen").textContent = Math.round((game_data.total_worker_gen / game_data.worker_speed) * 100) / 100
        } else if (upgrade_name == "worker_training") {
            game_data.worker_increase *= (1 + (game_data.upgrades.worker_training.level * 20) / 100)
            game_data.total_worker_gen = (game_data.workers * game_data.worker_increase)
            game_data.total_worker_gen = Math.round(game_data.total_worker_gen * 100) / 100
            document.getElementById("worker-gen").textContent = formatNum(Math.round((game_data.total_worker_gen / game_data.worker_speed) * 100) / 100)
        }

        game_data.money -= game_data.upgrades[upgrade_name].price
        document.getElementById("money").textContent = formatNum(game_data.money)
        game_data.upgrades[upgrade_name].price *= 1 + game_data.upgrades[upgrade_name].price_multiplier
        game_data.upgrades[upgrade_name].price_multiplier += game_data.upgrades[upgrade_name].multiplier_increase
        game_data.upgrades[upgrade_name].price = Math.round(game_data.upgrades[upgrade_name].price * 100) / 100 //Removes extra, unwanted decimal places
        game_data.upgrades[upgrade_name].base_price = game_data.upgrades[upgrade_name].price ←
    }
}
```


After this I then realised that I will need to call the `update_prices` function to update the prices and how much the player can purchase after the upgrade is purchased, so I did that underneath:

```
game_data.upgrades[upgrade_name].base_price = game_data.upgrades[upgrade_name].price  
update_prices() ←
```

Looking at this function as a whole, I have realised that for the upgrades where I apply the effect straight away in the function, they will only be applied once regardless of the multi-purchase state. Therefore, I will need to use a loop to apply them. I have decided to create a variable called `upgrade_number`, which will control the number of times the effects are applied. I will set this number based on the multi-purchase state. If it is 0, I will set it to the `max_purchase` of the upgrade since this is the number of times the upgrade will be applied. If not, I will set it to the multi-purchase state. Since I already have an if statement at the start that checks whether the multi-purchase state is 0 or not, I can set the value of this variable there:

```
let upgrade_number = 0  
if(game_data.money >= game_data.upgrades[upgrade_name].price) {  
  if(game_data.multi_purchase_state == 0) {  
    game_data.upgrades[upgrade_name].level += game_data.upgrades[upgrade_name].max_purchase  
    upgrade_number = game_data.upgrades[upgrade_name].max_purchase  
  } else {  
    game_data.upgrades[upgrade_name].level += game_data.multi_purchase_state  
    upgrade_number = game_data.multi_purchase_state  
  }  
}
```

Now, the upgrade number will always be the number of times that the upgrade will be purchased. I can then use this to control a for loop to apply the effects of the 3 upgrades that I apply in this function:

```

for(let i = 0; i < upgrade_number; i++) {
  if(upgrade_name == "industry_boom") {
    game_data.money_per_second *= 1.2
    game_data.money_per_second = Math.round(game_data.money_per_second * 100) / 100 //Removes extra, unwanted decimal places
    document.getElementById("money-per-second").textContent = game_data.money_per_second
  } else if (upgrade_name == "worker_efficiency") {
    game_data.worker_speed -= 0.1
    game_data.total_worker_gen = (game_data.workers * game_data.worker_increase)
    game_data.total_worker_gen = Math.round(game_data.total_worker_gen * 100) / 100
    document.getElementById("worker-gen").textContent = Math.round((game_data.total_worker_gen / game_data.worker_speed) * 100) / 100
  } else if (upgrade_name == "worker_training") {
    game_data.worker_increase *= (1 + (game_data.upgrades.worker_training.level * 20) / 100)
    game_data.total_worker_gen = (game_data.workers * game_data.worker_increase)
    game_data.total_worker_gen = Math.round(game_data.total_worker_gen * 100) / 100
    document.getElementById("worker-gen").textContent = Math.round((game_data.total_worker_gen / game_data.worker_speed) * 100) / 100
  }
}

```

At this point, I was pretty sure that the upgrade() function was fully working and did not need any more tweaks (Apart from when I add more upgrades later), so I added comments explaining it all. The reason I did not add comments as I was coding it, was because I knew I would have to change and tweak it a lot to get it to work right, and I did. I made too many changes to this function, so I would have to constantly change my comments to reflect the new functionality of the upgrade:

```

var worker_efficiency_allowed = true
// Function that handles purchasing upgrades
function upgrade(upgrade_element) {
  // Converts upgrade name from the case used in HTML to snake case so the upgrade can be accessed in the upgrades dictionary
  let upgrade_name = upgrade_element.replaceAll("-", "_")
  // Stores the number of times to upgrade
  let upgrade_number = 0

  // Checks if player can afford upgrade and if worker efficiency upgrade is allowed (cannot be purchased if capped)
  if(game_data.money >= game_data.upgrades[upgrade_name].price && worker_efficiency_allowed == true) {
    if(game_data.multi_purchase_state == 0) {
      // For MAX multi-purchase setting, upgrade level is increased by to the max purchase value
      // This is the value that stores the number of times the upgrade can be purchased with the money the player has
      game_data.upgrades[upgrade_name].level += game_data.upgrades[upgrade_name].max_purchase
      upgrade_number = game_data.upgrades[upgrade_name].max_purchase
    } else {
      // For the x1, x10 and x25 settings, upgrade level is increased by multi_purchase_state which is the number of times the upgrade will be purchased
      game_data.upgrades[upgrade_name].level += game_data.multi_purchase_state
      upgrade_number = game_data.multi_purchase_state
    }
  }

  // Applies upgrade effects depending on type of upgrade
  // Upgrade number controls how many times they are upgraded
  for(let i = 0; i < upgrade_number; i++) {
    if(upgrade_name == "industry_boom") {
      game_data.money_per_second *= 1.2 // 20% increase to money per second
      game_data.money_per_second = Math.round(game_data.money_per_second * 100) / 100 //Removes extra, unwanted decimal places
      document.getElementById("money-per-second").textContent = formatNum(game_data.money_per_second)
    } else if (upgrade_name == "worker_efficiency" && game_data.worker_speed > 0.5) {
      game_data.worker_speed -= 0.1 // Reduces worker generation time by 0.12
      game_data.total_worker_gen = (game_data.workers * game_data.worker_increase) // Recalculates worker generation
      game_data.total_worker_gen = Math.round(game_data.total_worker_gen * 100) / 100 //Removes extra, unwanted decimal places
      document.getElementById("worker-gen").textContent = Math.round((game_data.total_worker_gen / game_data.worker_speed) * 100) / 100
    } else if (upgrade_name == "worker_training") {
      game_data.worker_increase *= (1 + (game_data.upgrades.worker_training.level * 20) / 100) // 20% increase to worker generation
      game_data.total_worker_gen = (game_data.workers * game_data.worker_increase) // Recalculates worker generation
      game_data.total_worker_gen = Math.round(game_data.total_worker_gen * 100) / 100 //Removes extra, unwanted decimal places
      document.getElementById("worker-gen").textContent = formatNum(Math.round((game_data.total_worker_gen / game_data.worker_speed) * 100) / 100)
    }
  }
}

```

```

    }
    // Caps worker speed at 0.5s, prevents further upgrades once reached
    if(game_data.worker_speed <= 0.5) {
        worker_efficiency_allowed = false
    }

    // Caps win chance at 75%, prevents further upgrades once reached
    if(game_data.win_chance + 0.05 > 0.75) {
        gambleLuck_allowed = false
    }

    // Takes upgrade price of money
    game_data.money -= game_data.upgrades[upgrade_name].price
    document.getElementById("money").textContent = formatNum(game_data.money)
    // Scales upgrade price and increases price multiplier for next upgrade
    game_data.upgrades[upgrade_name].price *= 1 + game_data.upgrades[upgrade_name].price_multiplier
    game_data.upgrades[upgrade_name].price_multiplier += game_data.upgrades[upgrade_name].multiplier_increase
    game_data.upgrades[upgrade_name].price = Math.round(game_data.upgrades[upgrade_name].price * 100) / 100 //Removes extra, unwanted decimal places

    // Brings base price of upgrade up to current price
    game_data.upgrades[upgrade_name].base_price = game_data.upgrades[upgrade_name].price
    update_prices()
}
}

```

I also then fixed the problem of the extra digits added on the end of the prices, by using my method at the bottom of the update_prices function:

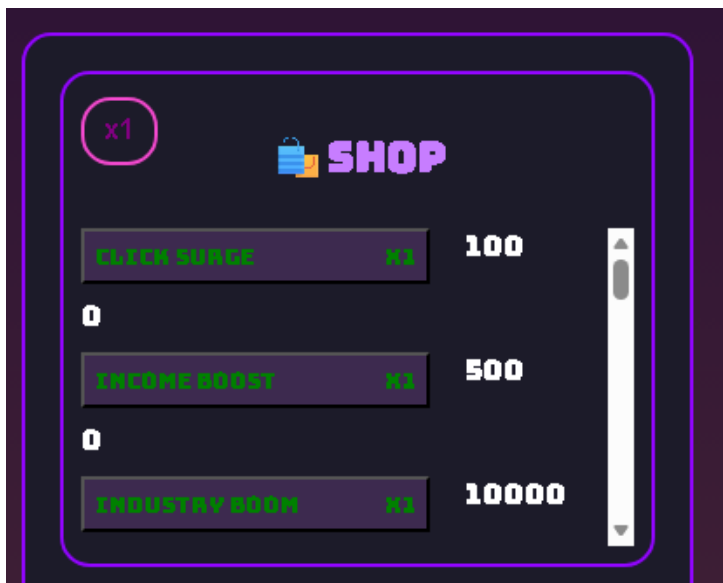
```

    current_upgrade.price = Math.round(current_upgrade.price * 100) / 100 ←
    // Updates upgrade price and level displays
    update_price_and_level()
}
}

```

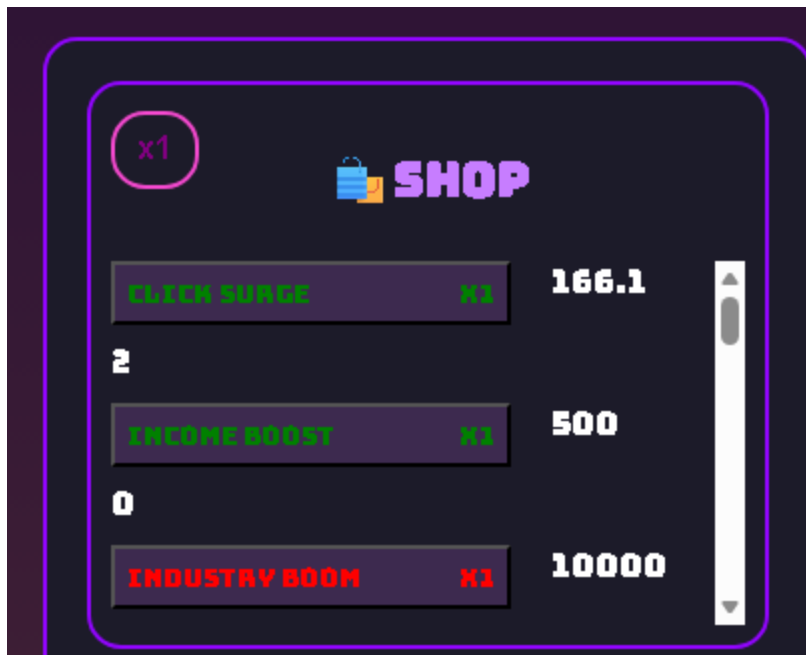
I put this before I call the update_price_and_level so that it would correct the value and then update the HTML.

I ran this again to see if it worked:



Now, the prices correctly increase and reset when the multi-purchase state goes back to x1 and there are also no extra digits added onto the end of them. However, I realised that after purchasing an upgrade and pressing the multi-purchase button a few times, buying the upgrade again increased it more than it should:





Here, the second purchase of click surge should always increase the price to 122.1 (110×1.11), however here it is 166.1. This is because the price multipliers are increased when calculating the multi-purchase prices, and they are not reset at all. Therefore, as I have a base_price for each upgrade, I will also need a base_multiplier for each, so I can reset the price_multiplier to the base_multiplier at the start of the update_prices for loop (like I set the price to the base price). I added a base multiplier to all the upgrade dictionaries and set it to 0.1, as this is the starting multiplier for all upgrades:

```

upgrades: {
  click_surge: {
    price: 100,
    level: 0,
    price_multiplier: 0.1,
    base_multiplier: 0.1,
    multiplier_increase: 0.01,
    base_price: 100,
    max_purchase: 0,
  },
  income_boost: {
    price: 500,
    level: 0,
    price_multiplier: 0.1,
    base_multiplier: 0.1,
    multiplier_increase: 0.02,
    base_price: 500,
    max_purchase: 0,
  },
  industry_boom: {
    price: 10000,
    level: 0,
    price_multiplier: 0.1,
    base_multiplier: 0.1,
    multiplier_increase: 0.05,
    base_price: 10000,
    max_purchase: 0,
  },
},

```

(I did this for every upgrade, but I am only showing 3 as proof)

Now, when I set the upgrade price to the base price at the start of the `upgrade_prices()` for loop, I can also set the upgrade multiplier to the base multiplier:

```

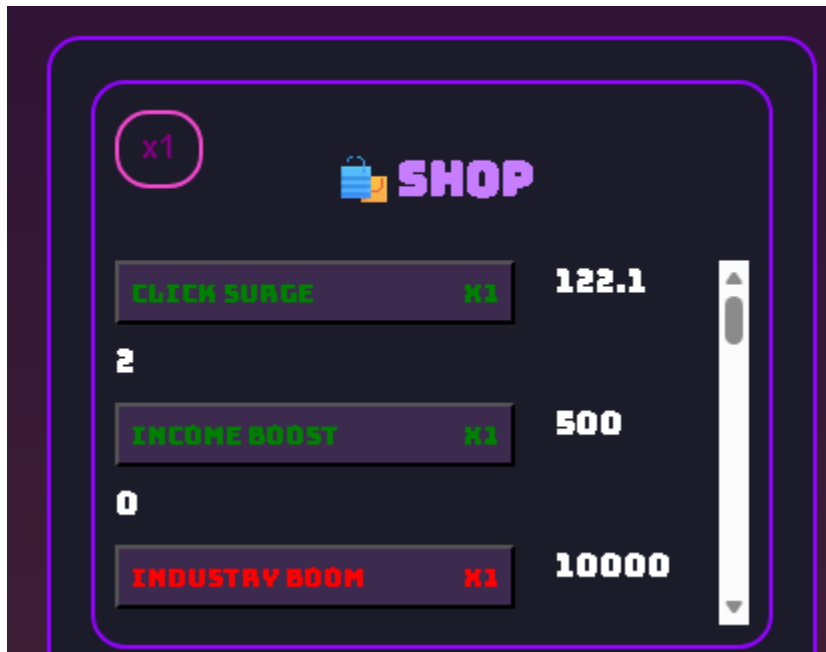
for(const key in game_data.upgrades) {
  let current_upgrade = game_data.upgrades[key]
  // Resets upgrade price and price multiplier to base values
  current_upgrade.price = current_upgrade.base_price
  current_upgrade.price_multiplier = current_upgrade.base_multiplier
  upgrade_prices[key] = 1
}

```

I will then also need to set the base multiplier of the upgrade to the price multiplier in the `upgrade()` function, like I do with the price and base price:

```
game_data.upgrades[upgrade_name].base_price = game_data.upgrades[upgrade_name].price
game_data.upgrades[upgrade_name].base_multiplier = game_data.upgrades[upgrade_name].price_multiplier
update_prices()
```

I ran this to see if it fixed the problem:



Now, after clicking the multi-purchase button a few times, and then purchasing click surge, the price is correct as the multiplier is being reset to its base multiplier each time the price is calculated in the `update_prices()` function.

As a final tweak, I applied the upgrade text colour change to the worker button, so that the text will turn green when the player can afford workers as well. This will make it clear to the player when they can purchase a worker, so they don't have to look at the worker price and then their money, which might be confusing when their money gets into the high numbers:

```

setInterval(() => {
  for(let i = 0; i < game_data.upgrade_elements.length; i++) {
    let upgrade_name = game_data.upgrade_elements[i].replaceAll("-", "_")
    if(game_data.money >= game_data.upgrades[upgrade_name].price) {
      document.getElementById(game_data.upgrade_elements[i]).style.color = "green"
    } else {
      document.getElementById(game_data.upgrade_elements[i]).style.color = "red"
    }
  }

  if(game_data.money >= game_data.worker_price) {
    document.getElementById("worker-button").style.color = "green"
  } else {
    document.getElementById("worker-button").style.color = "red"
  }
}, 1000)

```

I added this in the infinite loop that updates the text colour of the upgrade buttons, so that the worker button text would be updated infinitely as well. I tested this to see if it worked:



POPULATION: 0

MONEY: 5000

MONEY/S: 0

 **WORKERS**

BUY WORKER: 100

TOTAL WORKERS: 0

TOTAL WORKER GEN: 0



POPULATION: 11

MONEY: 14.27

MONEY/S: 0.11

 **WORKERS**

BUY WORKER:
2514.85

TOTAL WORKERS: 11

TOTAL WORKER GEN: 4.4

Now, when money is higher than worker price, the text for the worker button is green, and when money is lower than worker price, the text is red.

The only problem I have noticed now is when the numbers get really big, especially when the x25 multi-purchase setting is selected, they get too long to fit in the shop so they extend the page to the right. I could solve this by using abbreviations for the prices, for example 1000000000 would be 1B, so I added a function for this:

```
function formatPrice(price) {  
  if (num < 1000) {  
    return price  
  }  
  const unit_abbreviations = ["K", "M", "B", "T", "Q", "Qi", "Sx", "Oc", "No", "Dc"]  
  let index = 0
```

I started by creating a function called formatPrice, which has the parameter price. This will be the price passed in to return the formatted price. I used camel case for the function identifier, to make it clear it is not a game feature function, but instead a special function just for formatting.

I then check if the price is less than 1000, and return the price if so. This is because if the number is less than 1000, it doesn't need any abbreviations as it's too small. For example, 900 doesn't need to be formatted as it is very easy to tell that it says 900. I then create an array of the abbreviations I will need, which abbreviate the numbers kilo (thousand), million, billion, trillion, quadrillion, quintillion, sextillion, octillion, nonillion, decillion. I then initialise an index variable, which will be used to index the abbreviations array.

```
function formatPrice(price) {
  if (num < 1000) {
    return price
  }
  const unit_abbreviations = ["K", "M", "B", "T", "Q", "Qi", "Sx", "Oc", "No", "Dc"]
  let index = 0

  while (price >= 1000 && index < unit_abbreviations.length) {
    price /= 1000
    index ++
  }
}
```

I then added a while loop, which runs while the number is bigger than 1000 and the index is smaller than the length of the array-1. This is because I will be repeatedly dividing the number by 1000 each loop, because the difference between each abbreviation is a factor of 1000, and then also increasing the index so the next abbreviation will be used each time the number is reduced. Therefore, when the number gets smaller than 1000, or the index goes past the end of the list, the loop will stop.

```
function formatPrice(price) {
  if (num < 1000) {
    return price
  }
  const unit_abbreviations = ["K", "M", "B", "T", "Q", "Qi", "Sx", "Oc", "No", "Dc"]
  let index = 0

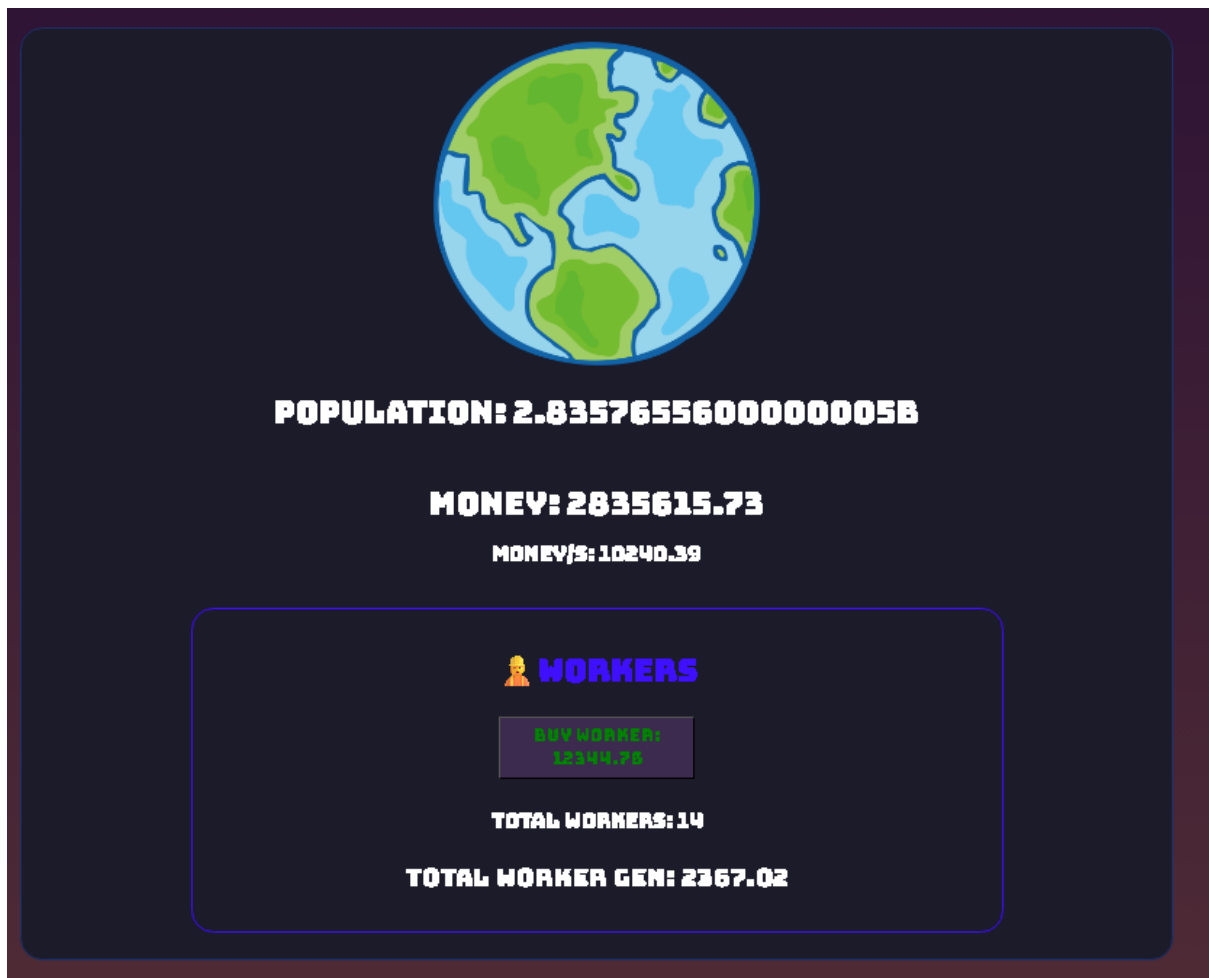
  while (price >= 1000 && index < unit_abbreviations.length) {
    price /= 1000
    index ++
  }
  return price + unit_abbreviations[index] ←
}
```

I then return the price with the unit added on, by indexing unit_abbreviations with the index variable

I tested this in the worker loop, where the population text is updated. Where I set the text content to game_data.population, I changed it to formatPrice(game_data.population), to test the formatting function:

```
document.getElementById("population").textContent = formatPrice(game_data.population)
document.getElementById("money_per_second").textContent = game_data.money_per_second
```

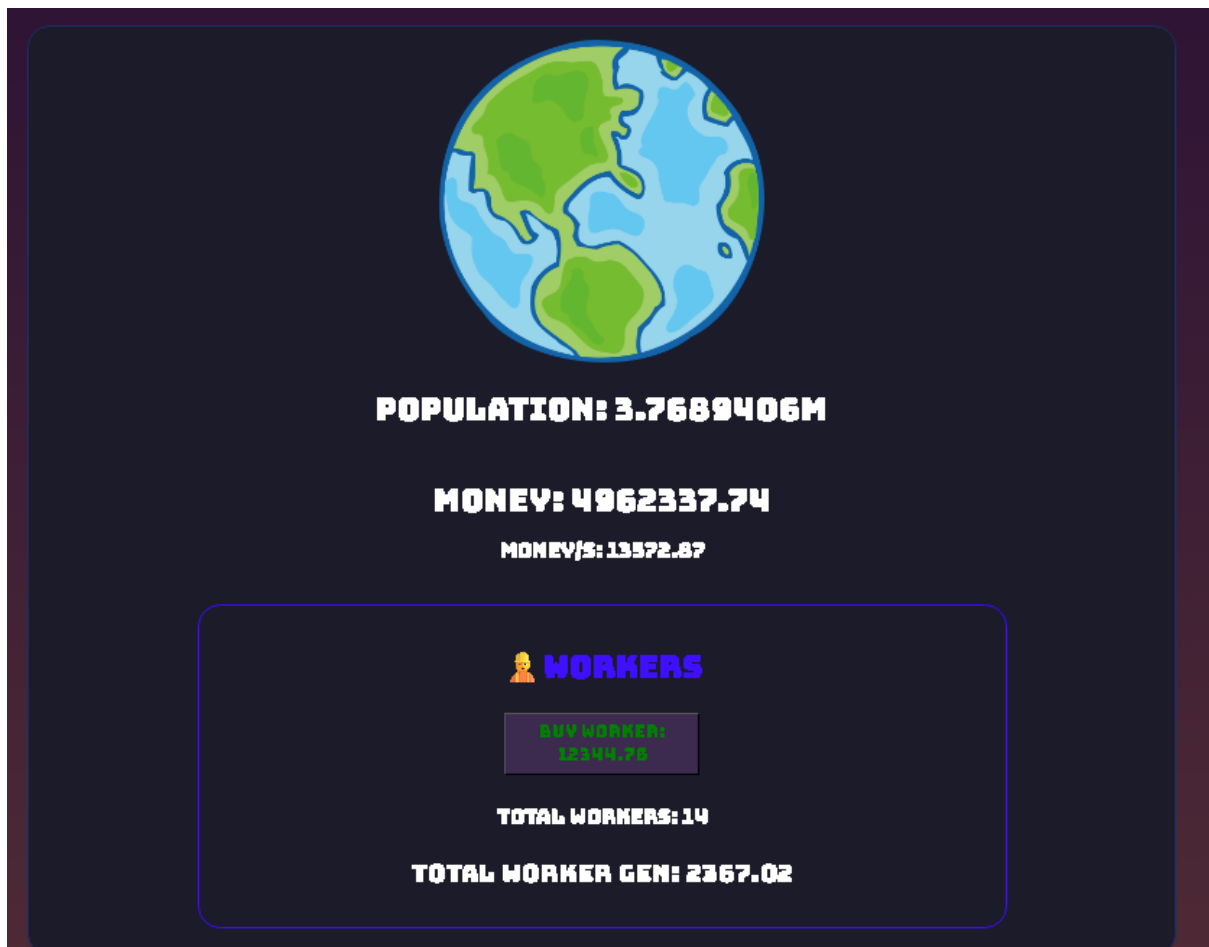
I then ran the code



I noticed 2 problems. The number needs to be rounded, as there are way too many extra digits. Also, the population before this was actually 2.8 million, not billion. I assumed this was because I was indexing to high in the array, so I changed the final return statement in the formatprice function to `index unit_abbreviations[index-1]`:

```
    return price + unit_abbreviations[index-1]
}
```

This fixed the second problem:

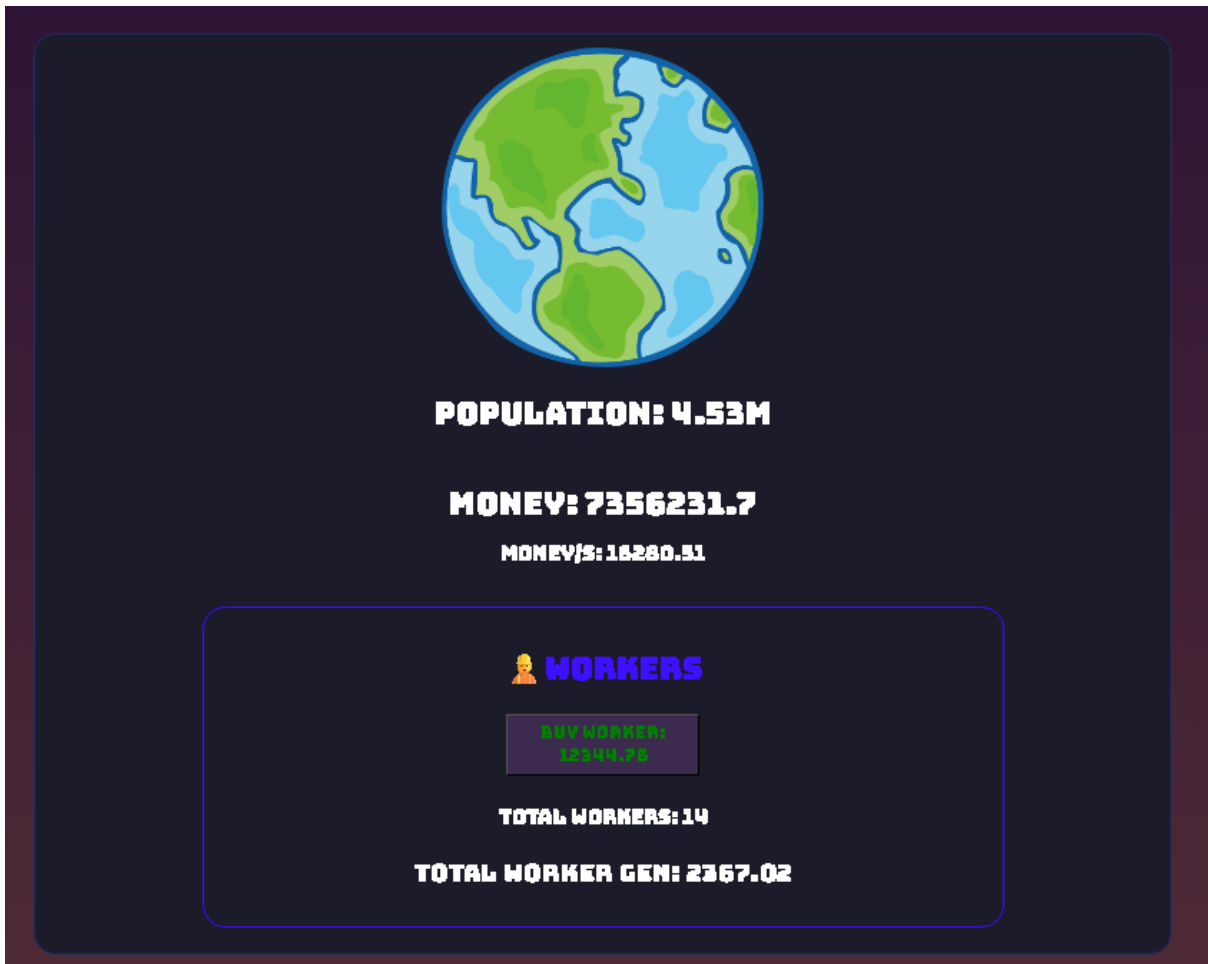


Now, to fix the problem of the extra digits, I used the `.toFixed(2)` JavaScript function, to make sure there were only 2 extra digits after the decimal point:

```
return price.toFixed(2) + unit_abbreviations[index-1]
}
```

I also realised that I am using the identifier `price`, when it is not always a price that is being formatted (e.g. population in this case), therefore I changed it to a more general identifier, “`num`”, including the function name (`formatPrice` → `formatNum`), and commented it all to explain it:

I ran this to see if the first problem was fixed:



Now, there are only 2 decimal places. I also remembered to parse the value of the number passed into the function to a float, since this will remove 0s in numbers like 12.00K:

```
// Returns num followed by the appropriate abbreviation
return parseFloat(num.toFixed(2)) + unit_abbreviations[index-1]
}
```

Finally, everytime I set the textContent of population, money, shop upgrade prices, workers, worker price and worker gen, I will pass them into the formatNum function.

```
if
document.getElementById("population").textContent = formatNum(game_data.population)
document.getElementById("money-per-second").textContent = game_data.money_per_second
})
```

```
document.getElementById("workers").textContent = formatNum(game_data.workers)
document.getElementById("worker-price").textContent = formatNum(game_data.worker_price)
document.getElementById("worker-gen").textContent = formatNum(Math.round((game_data.total_worker_gen / game_data.worker_speed) * 100) / 100)
```

```
game_data.money = Math.round(game_data.money * 100) / 100 //Removes extra, un
document.getElementById("money").textContent = formatNum(game_data.money
}, 1000)
```

```
document.getElementById("population").textContent = formatNum(game_data.population)
document.getElementById("money-per-second").textContent = formatNum(game_data.money_per_second)
```

```
game_data.money -= game_data.upgrades[upgrade_name].price
document.getElementById("money").textContent = formatNum(game_data.money)
```

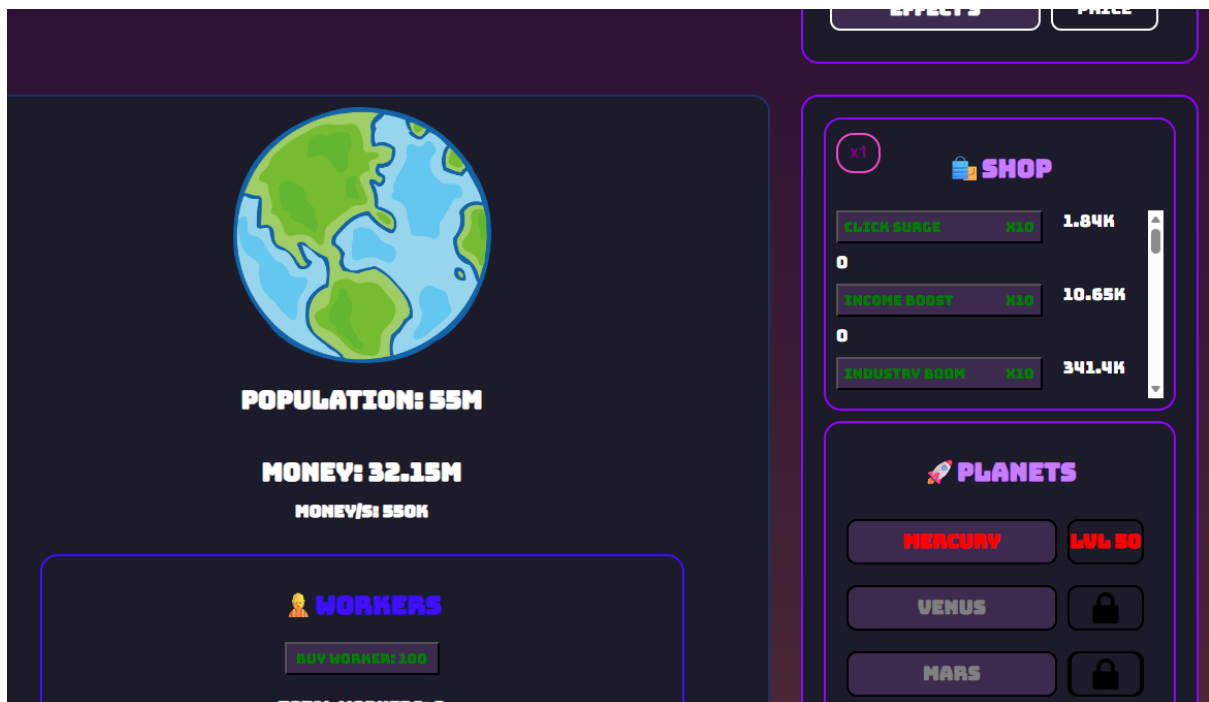
```
document.getElementById(`${game_data.upgrade_elements[1]}-level`).textContent = game_data.upgrades[upgrade_name].level
document.getElementById(`${game_data.upgrade_elements[1]}-price`).textContent = formatNum(game_data.upgrades[upgrade_name].price)
```

```
document.getElementById("population").textContent = formatNum(game_data.population)
document.getElementById("money").textContent = formatNum(game_data.money)
document.getElementById("money-per-second").textContent = formatNum(game_data.money_per_second)
document.getElementById("workers").textContent = formatNum(game_data.workers)
document.getElementById("worker-price").textContent = formatNum(game_data.worker_price)
document.getElementById("worker-gen").textContent = formatNum(Math.round((game_data.total_worker_gen / game_data.worker_speed) * 100) / 100)
}
```

```
game_data.money_per_second = Math.round(game_data.money_per_second * 100) / 100 //Removes extra, unwanted
document.getElementById("money-per-second").textContent = formatNum(game_data.money_per_second)
} else if (upgrade_name === "worker_efficiency" && game_data.worker_speed > 0.5) {
```

```
document.getElementById("population").textContent = formatNum(game_data.population)
document.getElementById("money-per-second").textContent = formatNum(game_data.money_per_second)
interval = setInterval(worker_click, game_data.worker_speed * 1000)
```

I tested this to see if it worked:



Now, all numbers were being correctly abbreviated.

Testing

This is my test plan for this stage, with some modifications:

- My test plan in the design planned to test to see if “the shop upgrades correctly apply their effects”. At that point, I had not designed any specific shop upgrades, but for this test plan, I have a test for each shop upgrade that I can test at this stage in the game (I will test other upgrades in later stages, when their associated game mechanic has been developed)

- **NOTE:** I will give myself money in some tests, by setting the money value in the game_data dictionary to a number like 1000000, so I can purchase upgrades for the purpose of the tests.

Test no.	Description	Test data	Expected Result	Actual result
1	Is the player prevented from buying upgrades when they don't have enough money	Attempt to buy an upgrade by clicking one in the shop with no money	Nothing should happen, upgrade number for that specific upgrade should not increase	Nothing happens, the upgrade number for the upgrade doesn't increase Passed ☒ SEE STAGE 4 TEST 1.mp4 in evidence folder
2	Is the player able to purchase an upgrade	Click an upgrade in the shop with enough money to buy it to see if the upgrade number increases	The upgrade number for the upgrade clicked should increase by 1.	The upgrade number of the upgrade increased by 1 Passed ☒ SEE STAGE 3 TEST 2.mp4 in evidence folder
3	Does buying upgrades correctly reduce the player's money	Click an upgrade in the shop with enough money to buy it and observe the money display.	The money should decrease by the price of the upgrade	The money decreased by the price of the upgrade purchased Passed ☒

				SEE STAGE 4 TEST 3.mp4 in evidence folder
--	--	--	--	---

4	Does the colour of the shop upgrades become green when the player can afford them	Get enough money to buy an upgrade and see if the colour of the text changes	The colour of the upgrade text should turn green	The colour of the text in the upgrade button turned green Passed ☒ SEE STAGE 4 TEST 4.mp4 in evidence folder
5	Does the multi-purchase feature correctly calculate the price of purchasing the upgrade 10 times	Click the multi-purchase button to x10 and see if the price of the upgrades increases to the price of purchasing them 10 times	The price of the upgrade should increase to the price of purchasing the upgrade 10 times (with price scaling)	The price of the upgrade increased to the price of purchasing the upgrade 10 times, taking into account the price scaling. Passed ☒ SEE STAGE 4 TEST 5.mp4 in evidence folder
6	Does the multi-purchase feature correctly calculate the max number of times the upgrade can be purchased, and the price of doing so	Click the multi-purchase button to MAX and see if the price of the upgrades increases	The price of the upgrade should increase to the price of purchasing it the max number of times	The price of the upgrade increased to the price of purchasing the upgrade the max number of times possible with 400 money. The number on the right of the upgrade became the max number of times it can be purchased

		to the max amount that can be purchased, and to see if the number on the right of the upgrade is the max number of times it can be purchased	possible with the current money. The number on the right of the upgrade should be the number of times it can be purchased	Passed  SEE STAGE 4 TEST 6.mp4 in evidence folder
7	Does the click surge upgrade correctly increase click power by 2%	Purchase click-surge and click the planet to see the population increase	Each click, the population should increase by 1.02	The population increased by 1.02, then 1.04 (with the second click surge purchase), instead of just 1 like normal Passed  SEE STAGE 4 TEST 7.mp4 in evidence folder
8	Does the income boost correctly increase money per second per population by 10%	Purchase income boost and see if the money per second per population	The money per second should be the population multiplied by 1.1	The money per second increased by more than normal, with it being population multiplied by 1.1. At the end, with 2 income boosts purchased, the population was 30. 30 multiplied by 1.1 is 33, and 33 multiplied by 1.1 is 36 (rounded down).

		n increases correctly		Passed ☒ SEE STAGE 4 TEST 8.mp4 in evidence folder
--	--	-----------------------------	--	--

9	Does the industry boom correctly increase money per second by 20% (flat increase)	Purchase industry boom and see if money per second increases correctly	The money per second should increase by 20% (multiplied by 1.2)	The money per second increased by 20%. The money per second went from 0.05 to 0.06 after purchasing the upgrade once. This is a 20% increase. Passed ☒ SEE STAGE 4 TEST 9.mp4 in evidence folder
10	Does worker training correctly increase worker generation by 20%	Purchase a worker, then worker training and see if the total worker generation increases correctly	The total worker gen should increase by 20% (multiplied by 1.2)	The total worker gen increased by 20%. The total worker gen went from 0.4 to 0.48 after purchasing the upgrade once. This is a 20% increase Passed ☒ SEE STAGE 4 TEST 10.mp4 in evidence folder
11	Does worker efficiency correctly reduce worker generation	Purchase a worker, then worker	The workers should generate	The workers began generating population every 1.5s after 10 purchases. After

	time by 0.1s until they are generating every 0.5s	efficiency 10 times and time how quickly they are generating. Purchase 10 more and see if any more purchases are allowed after that	(increase population(every 1.5s after 10 purchases of the upgrade. After another 10, the workers should be generating every 0.5s so no more purchases of the upgrade should be allowed	another 10 purchases of the upgrade, no more were allowed Passed ☒ SEE STAGE 4 TEST 11.mp4 in evidence folder
--	---	---	---	--

Stage review:

In terms of the implementation of the shop, I am happy since it functions correctly with the upgrades and multi-purchase feature correctly implemented. However, the development process could have gone better. There were many errors throughout the process, which made it take much

longer than expected. I also think I overcomplicated the development, and that there were more efficient ways to implement it, especially with the `upgrade()` function which is quite messy. This shows that I should plan my pseudocode more in future, and take care when developing to ensure I make less mistakes, which may be harder with limited time in the later stages.

Stage 5 – Progress Save Feature:

Features:

- All data in the game_data dictionary saved to local storage every 5 seconds
- Clicking save button instantly saves data
- Data loaded each time the page opened

Data Dictionary:

- No additional variables needed

Pseudocode:

Data saving:

```
FUNCTION save_data()  
    localStorage.setItem(JSON.stringify(game_data))  
ENDFUNCTION
```

- I will have a function that saves the data
- The function stores the data in the game_data dictionary into local storage, which is stored in the browser, so will save the game to the browser the player is using
- Since local storage can only store strings, I will need to convert the data in the dictionary to a string, using JSON.stringify()
- When I come to loading the data, I will then convert this back, using JSON.parse(), and set it to the game_data dictionary

```
WHILE true
    save_data()
    WAIT 5
ENDWHILE
```

- I will have an infinite loop that runs every 5 seconds, which calls the save_data function. This will mean the data is saved every 5 seconds, as per my design.

```
save-button EventListener(click) {
    save_data()
}
```

- I will also add a click event listener to the save button, and I will call the save_data() function in the event handler, so that the data will also save anytime the player clicks the save button

Data loading:

- To load the data, I will need to get the data from local storage, then use JSON.parse() to convert it to a string, and then set it to the value of game_data
- This code will need to run when the page loads, however I am not sure what the page load event is in javascript, so I looked it up. I went found the w3schools web page “onload Event”, (w3schools, n.d.), which showed the event that is triggered by the page loading. It showed the syntax for detecting the event:

Syntax

In HTML:

```
<element onload="myScript">
```

Try it Yourself »

In JavaScript:

```
object.onload = function(){myScript};
```

Try it Yourself »

In JavaScript, using the addEventListener() method:

```
object.addEventListener("load", myScript);
```

Try it Yourself »

Therefore, I can use `object.onload` to call the code that loads the local storage data. Since I know that the JavaScript object that contains the web page is `window`, I will use `window.onload` to detect the load event.

I also don't know how to copy the value of an object (the saved local storage data) and set it to the value of another object (the `game_data` dictionary). When I say objects, I am referring to dictionaries, which are objects. So, I also looked this up and found a web page about cloning JavaScript objects, (Tutorials, n.d.), which showed 3 methods of how to clone a JavaScript object. This is useful because, once I have converted the local storage data back from a string to an object, I need to copy the object into the `game_data` object. These are the methods that the web page showed:

1. Use the spread (`...`) syntax
2. Use the `Object.assign()` method
3. Use the `JSON.stringify()` and `JSON.parse()` methods

The following illustrates how to copy an object using three methods above:

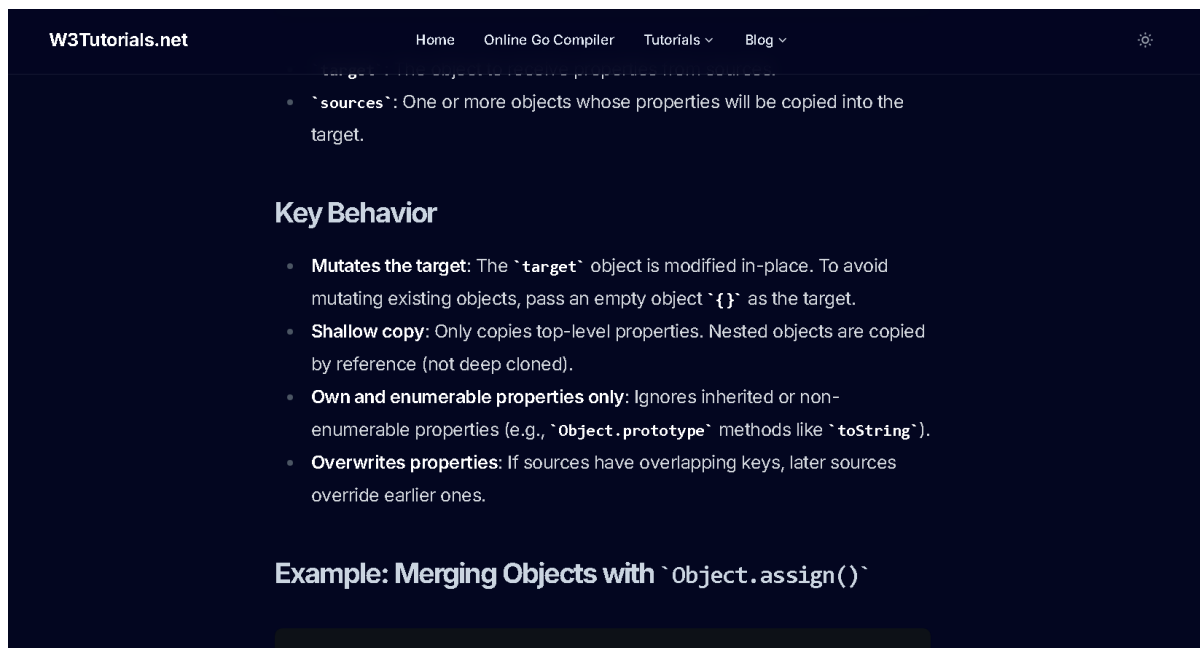
```
const person = {
  firstName: 'John',
  lastName: 'Doe'
};

// using spread ...
let p1 = {
  ...person
};

// using Object.assign() method
let p2 = Object.assign({}, person);

// using JSON
let p3 = JSON.parse(JSON.stringify(person));
```

I decided to use the 2nd method, `Object.assign()`. I could just simply assign the object that I parse from local storage to the `game_data` dictionary, however the `game_data` dictionary is a `const`, so I won't be able to assign a value to it and if I change it to `var`, it would make it less secure and more vulnerable to unintended modifications. I also researched `Object.assign` to check if it was appropriate and found a webpage, [W3Tutorials.net](https://www.w3schools.com/jsref/jsref_obj_assign.asp) (W3Tutorials.net, n.d.), which compares the use of `Object Spread` and `Object.assign`, giving key behaviours of each. I am not interested in `Object spread`, but I just used the webpage to get more information about how `Object.assign` works. I found this information:



Here, it says that `Object.assign` overwrites sources that have “overlapping keys”, meaning that if any keys in the object being copied are the same as the object being assigned, then the keys in the object being assigned will be overwritten. This is appropriate for my scenario as I want everything that has been retrieved from local storage to overwrite everything in `game_data`. It also says that, since it performs a shallow copy, nested objects are not copied by value but reference instead. This applies to me as I have a lot of nested objects in `game_data`, however it is not a problem. This is because I only retrieve the saved data once from local storage and merge it into `game_data`, when the game loads. The nested objects from the saved data will share the same references as those in `game_data`, but I never directly modify the local storage data in my code, so there won’t be a risk of reference issues.

This will be pseudocode for this:

```
window.onload {  
    Object.assign(game_data,  
        JSON.parse(localStorage.getItem("game_data"))  
    )  
}
```

Development:

I started my development of this section by creating the function to save the game data to local storage:

```
function save_data() {  
  localStorage.setItem("game_data", JSON.stringify(game_data))  
}
```

The function saves the `game_data` dictionary to local storage, by converting it to a string using `JSON`. I have set the key of the data to “`game_data`”, as it matches the name of the dictionary. This key will be used to retrieve the data when I need to load it.

After this, I created the infinite loop that runs every 5 seconds, which calls the `save_data()` function:

```
setInterval(() => {  
  save_data()  
}, 5000)
```

The reason I have a function for saving the data which I call in the loop, instead of just putting the code that saves the data straight into the loop, is because I also need this code to be run in the event handler for the `save-button` event listener. Therefore, by having a function that saves the data, I can call it in both the infinite loop and the `save-button` event handler, which makes the code more clean.

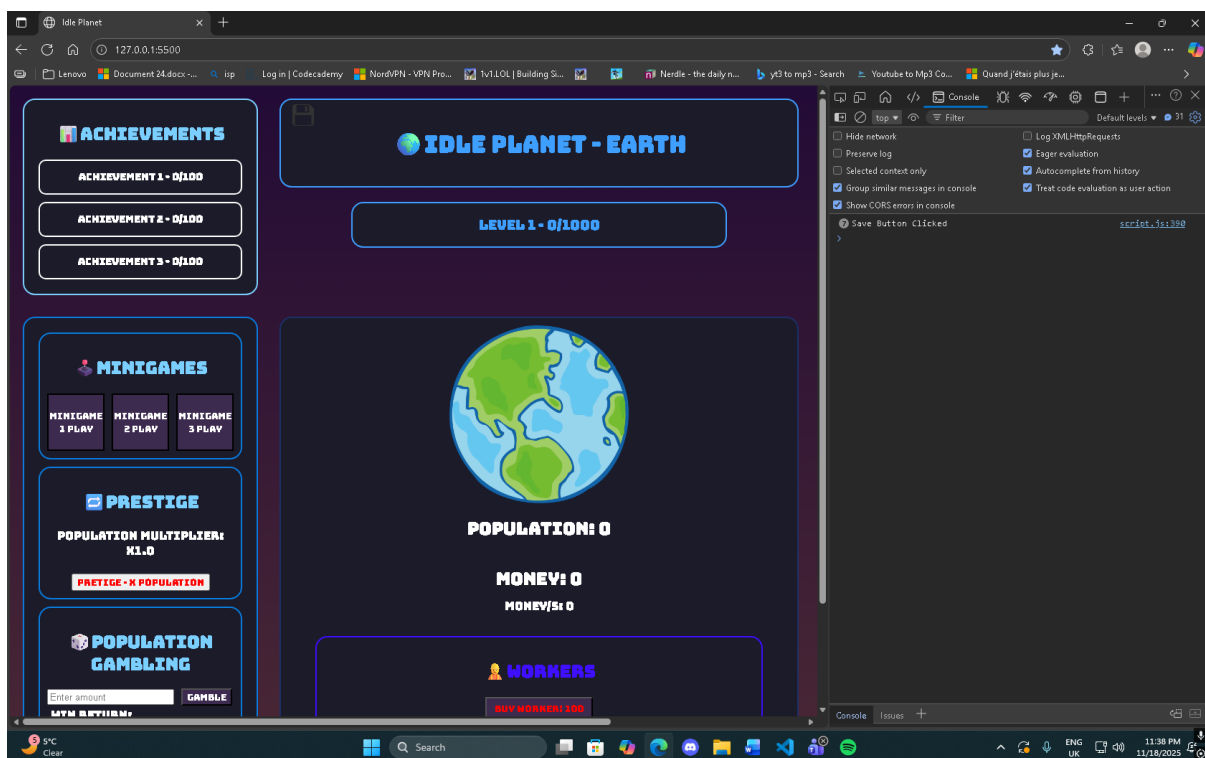
I then went to add the click event listener to the save button, but first I needed to give it an id in the HTML so I could access it:

```
<button class="save-button" id="save-button"> ←  
  </img>  
</button>
```

I then added the click event listener in the JavaScript, and called the `save_data()` function in the event handler:

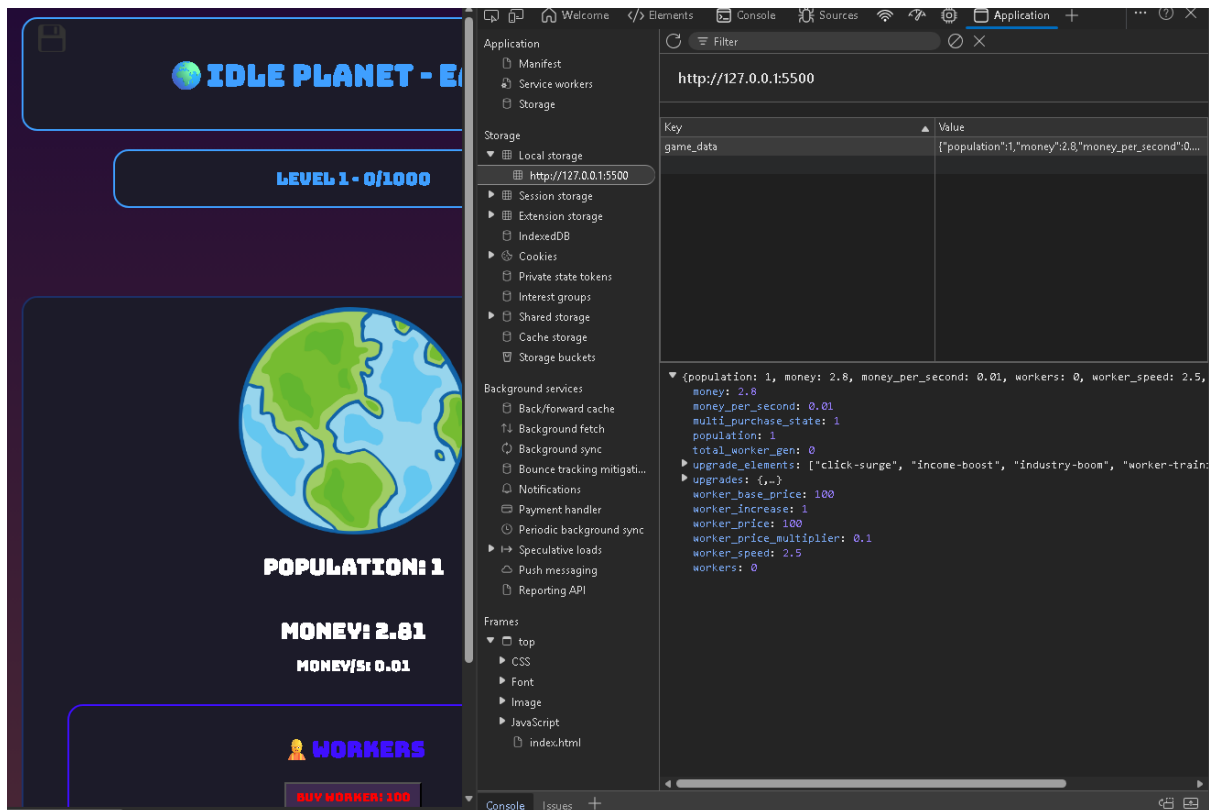
```
document.getElementById("save-button").addEventListener("click", function (event) {  
  save_data()  
  console.log("Save Button Clicked")  
})
```

I also added a line that prints "Save Button Clicked" to the console, so I could check that the click event was being detected. I then ran the code:



Each time I clicked the button, the line was output to the console, so I knew the event listener was working.

Then, to confirm that the data was being correctly stored in local storage, I went to the application tab in the Microsoft Edge developer tools (which I have been using this the whole time to access the console), and selected local storage:



I saw the game_data key in the local storage for the session, with all the correct values, which showed that everything was being correctly saved to local storage.

Finally, I wrote the code to load the saved data in local storage when the game loads:

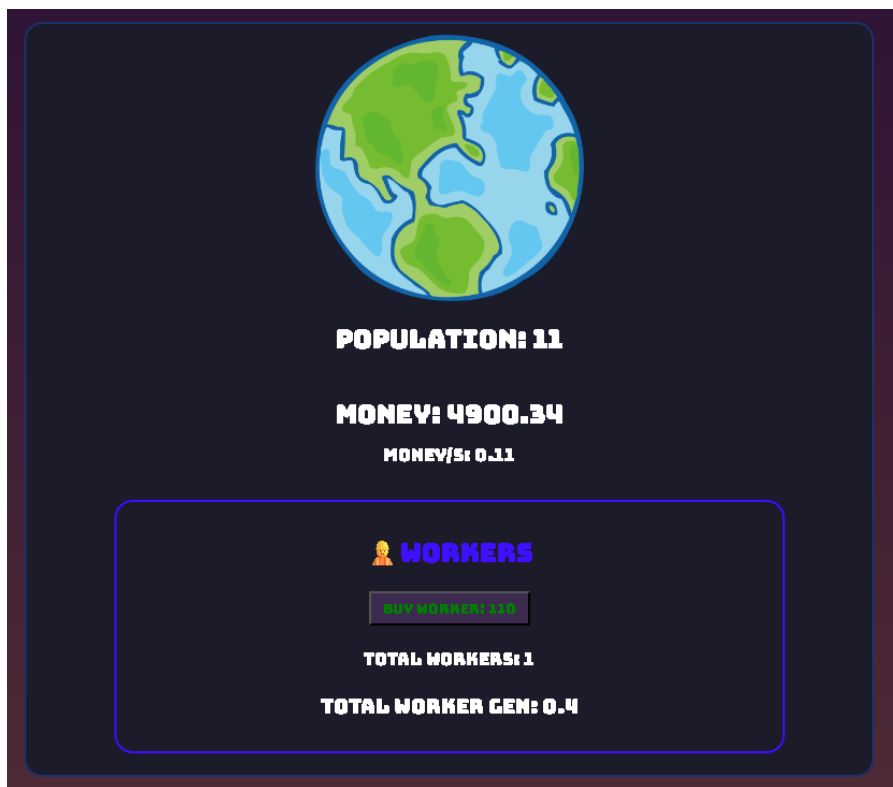
```

window.onload = function() {
  Object.assign(game_data, (JSON.parse(localStorage.getItem("game_data"))))
}

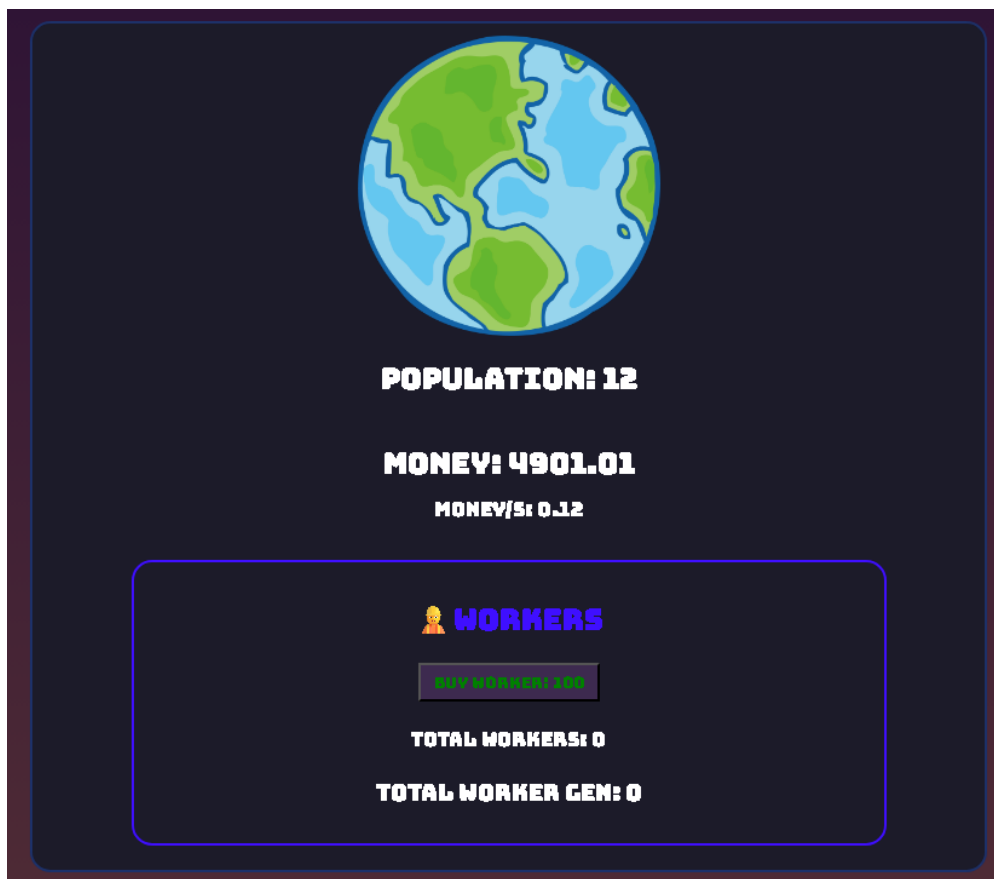
```

Here, I use window.onload, which is used to run the code when the game loads. The window object contains the game page, and the onload event occurs when the page loads. Therefore, when the game is opened/refreshed, the code will run. The code I wrote to save the data retrieves the "game_data" key from local storage which stores the data from the last save. I then used JSON.parse to convert the data back to an Object, because I saved it as a string, and I use Object.assign to assign it to game_data, which merges the saved data with the game_data dictionary.

I tested this to see if it worked:



After closing and re-opening:



After I refreshed the page, everything remained the same (the reason population is 1 higher after refreshing is because the worker increased it), apart from the total workers, total worker gen and worker price. After clicking the buy worker button again, the number increased to 2, meaning the total workers are being saved, but just not updated when the page loads. I realised that this was because the only time I update the worker displays is in the event handler for the worker-button click event listener, and the upgrade() function. Therefore, I added the line to update the worker price, total workers and total worker gen displays in the code that loads the local storage data. I decided to also update the population, money, money per second and shop displays in there as well, so they would update the second the page loads, rather than waiting a small delay to be updated in the loops:

```
✓ window.onload = function() {  
  Object.assign(game_data, (JSON.parse(localStorage.getItem("game_data"))))  
  update_price_and_level()  
  update_prices()  
  
  document.getElementById("population").textContent = game_data.population  
  document.getElementById("money").textContent = game_data.money  
  document.getElementById("money-per-second").textContent = game_data.money_per_second  
  document.getElementById("workers").textContent = game_data.workers  
  document.getElementById("worker-price").textContent = game_data.worker_price  
  document.getElementById("worker-gen").textContent = Math.round((game_data.total_worker_gen / game_data.worker_speed) * 100) / 100  
}
```

Here, I call the update_prices() and update_price_and_level() functions, which calculates all the upgrade prices and then updates their display in the HTML. I then update the text content for population, money, money per second, workers, worker price and worker gen. This will make the game load feel more smooth, as everything gets updated straight away.

Testing:

This is my test plan for this stage:

Test no.	Description	Test type	Test data	Expected Result	Actual result
1	Does the player data like money, population and workers save to local storage correctly		After getting some money and at least 1 worker, close the browser, re-open it and go onto the game.	The money, population, and worker count should be the exact same as before the game was closed	When the game was re-opened, the money, population and worker count were exactly the same as before. Passed ☒ SEE STAGE 5 TEST 1.mp4 in evidence folder
2	Does the save button correctly save game data to local storage		After getting some money and at least 1 worker, press the save	The money, population, and worker count should be the exactly	When the game was re-opened, the

			button and instantly close the browser	same as when the game was closed	money, population and worker count was exactly the same as before Passed ☒ SEE STAGE 5 TEST 2.mp4 in evidence folder
--	--	--	--	----------------------------------	---

Stage review:

Overall, I am very happy with the development of this stage. I correctly implemented the automatic and manual progress saving, according to the stage plan. There were no major errors, only a small issue of not updating some of the HTML text displays after page was reloaded, which was very simple to fix. I also completed this stage in a very short amount of time, and I believe the code I wrote was efficient and clean.

Stage 6 – Prestige:

Features:

- Majority of game data is reset when player prestiges
- Gems, achievements, level, and prestige upgrades are not reset
- Population multiplier applied to player's population after prestige
- Population multiplier increases by 0.1 every 100000 population
- Prestiging will require 1 million population at first, then 10 million, then 100 million, and so on (number multiplied by 10 each time)
- Text colour of prestige button red when player doesn't have enough to prestige, and green when they do
- Population multiplier boost upgrade, which increases the population multiplier 5% per upgrade. This will be reset every prestige
- Quicker prestige upgrade, which reduces the population required to prestige by 5% each time. This will not be reset after a prestiging
- I have decided not to develop the Permanent click boost upgrade, as I believe it is too similar to the population multiplier boost upgrade, so I will remove this from the shop

Data Dictionary:

Variable	Data type	Purpose
default_game_data	Dictionary	To store the default game data, so that when the player prestiges, the game_data dictionary can be reset to this dictionary to restore the default game values
game_data. population_multiplier	Float	This will store the population multiplier, which will be increased with

		population and will increase the population the player earns after prestiging
game_data. prestige_level	Int	This will store the number of times the player has prestiged, to keep track of the amount it should cost. This will not be reset after prestiging, of course, I will keep it the same
game_data. prestige_cost	Float	This will store the population required to prestige

Psuedocode:

Default game data initialisation:

```
default_game_data = JSON.parse(JSON.stringify(game_data))
```

- I will copy the game_data object and assign it to the default_game_data object
- I used the same copying method as I did for merging the saved game_data object from local storage with the current game_data object, which involves converting it to a string and then back to an object (using JSON).

Prestige button text colour:

```
WHILE TRUE
```

```

IF population > population_cost THEN
    HTML.prestige-button.textColour = green
ELSE
    HTML.prestige-button.textColour = red
ENDIF
WAIT 1
ENDWHILE

```

- I will have an infinite loop (that runs every second) that checks if the player's population is greater than the population required to prestige.
- If so, I will set the text colour of the prestige upgrade button to green, and if not I will set it to red
- Since I already have an infinite loop, that also runs every second, which upgrades the text colour of the worker and upgrade buttons, I will combine it with this.

Population multiplier update:

```

WHILE true
    prestige_multiplier = 1 + 0.1 * (population /
    100000)
    prestige-population-multiplier.HTML.text =
    prestige_multiplier
    WAIT 1
ENDWHILE

```

- I will have an infinite loop that runs every second, which sets the population multiplier, incrementing it by 0.1 for every 100000 population.
- The reason I add the calculated value to 1, is because the population multiplier starts at 1.0, so when I calculate the value based on the population, I need to add it to 1.
- I then update the population multiplier display in the HTML, setting it to the population multiplier variable. I will need to make the population multiplier value a separate span in the HTML, with the id population-multiplier so I can update it in the JavaScript.

Prestige button event listener:

```
prestige-button EventListener(click) {
    IF population > prestige_cost THEN
        prestige()
    ENDIF
}
```

- I will have a click event listener for the prestige button, the event handler of which will check if the player has the sufficient population to prestige, and if so will call the prestige() function which will prestige the player.

Prestige function:

```
FUNCTION prestige()
```

```

int prestige_level = game_data.prestige_level
int prestige_cost = game_data.prestige_cost
prestige_level += 1
prestige_cost *= 10

Object.assign(game_data,
JSON.parse(JSON.stringify(default_game_data)))

game_data.prestige_level = prestige_level
game_data.prestige_cost = prestige_cost

```

ENDFUNCTION

- This function initialises 2 temporary variables, `prestige_level` and `prestige_cost`, which are assigned the value of the prestige level prestige cost (respectively) in `game_data`. This allows me to save these variables before `game_data` is reset, so I can restore them. The reason I don't want them to be reset is that they are needed throughout the game, to track the player's all time prestige count and the cost to purchase the next prestige.
- Once I have saved the prestige level and cost, I increment the level by 1 (because the player is prestiging 1 time), and I multiply the prestige cost by 10 (since the population required to prestige increases by a factor of 10 each prestige)
- After this, I copy the `default_game_data` object (using the same copy method I have used before), and then use `Object.assign()` to assign it to the current `game_data` object, which resets it.
- Finally, I set the prestige level and prestige cost of the current `game_data` to the `prestige_level` and `prestige_cost` variables, thus restoring their values

Population multiplier applied to population increase:

```
population *= population_multiplier
```

- Wherever population is updated in the game, I will add this line to apply the population multiplier

Population multiplier boost upgrade

```
WHILE true
```

```
    prestige_multiplier = 1 + 0.1 * (population /  
    100000) + (0.5 *  
    population_multiplier_boost.level)
```



```
    prestige-population-multiplier.HTML.text =  
    prestige_multiplier
```

```
    WAIT 1
```

```
ENDWHILE
```

- This is my infinite loop that calculates the prestige multiplier, but I will add + (0.5 * population_multiplier_boost.level) on the end, as this will increase the prestige multiplier by 0.5 for each population_multiplier_boost level, which is what the upgrade does
- I will not restore this upgrade when I reset the game_data for a prestige, since I want it to increase the prestige multiplier before the player prestiges, and then to reset. Otherwise, the player could get this upgrade to a very high level as they would keep it after every prestige

Quicker Prestige Upgrade:

```

FUNCTION upgrade(upgrade_name)
    // Existing IF checks
    ELSE IF upgrade_name = "quicker_prestige" THEN
        prestige_cost *= 0.95
    ENDIF
    // Rest of function
ENDFUNCTION

```

- This is the existing function that handles upgrades. I already have a few ELSE IF checks to check the name of the upgrade, and apply an effect. Therefore, I will just add another ELSE IF check to check if the name of the upgrade is "quicker_prestige", and if so I will apply the effect, which is multiplying the prestige_cost by 0.95, which reduces it by 5%
- Because I restore the prestige_cost variable after the game_data is reset, this upgrade will remain active after prestige. However, the upgrade dictionary for prestige_cost itself will be reset, therefore I will need to restore this as well by creating a temporary variable for the dictionary which will be a copy of the current one. and then setting it to the quicker_prestige dictionary in game_data after it is reset:

```

FUNCTION prestige()

    int prestige_level = game_data.prestige_level
    int prestige_cost = game_data.prestige_cost
    int quicker_prestige =
    JSON.parse(JSON.stringify(game_data.upgrades.q
    uicker_prestige)) ←
    prestige_level += 1
    prestige_cost *= 10

    Object.assign(game_data,
    JSON.parse(JSON.stringify(default_game_data)))

```



```
game_data.prestige_level = prestige_level  
game_data.prestige_cost = prestige_cost  
Object.assign(game_data..upgrades.quicker_prestige, quicker_prestige
```



ENDFUNCTION

- Here, I use the same method I have used before to copy an object, JSON.parse(JSON.stringify), and then Object.assign to re-assign it
- I am working with a nested object, since quicker_prestige is a nested dictionary inside the upgrades dictionary, however it still works the same

Development:

I will start my development by adding the id “prestige-button” to the prestige button in the HTML, so I can access it in the JavaScript:

```
</div>  
<div class="prestige">  
  <h2>📦 Prestige</h2>  
  <h4>Population multiplier: x1.0</h4>  
  <button id="prestige-button">Pretige - x population</button>  
</div>
```



I then added the `prestige_level`, `prestige_cost`, and `prestige_multiplier` variables to the game data dictionary:

```
// PRESTIGE

prestige_level: 0,
prestige_cost: 1000000,
prestige_multiplier: 1.0,
```

I then developed the loop to update the text colour of the prestige button if it's necessary:

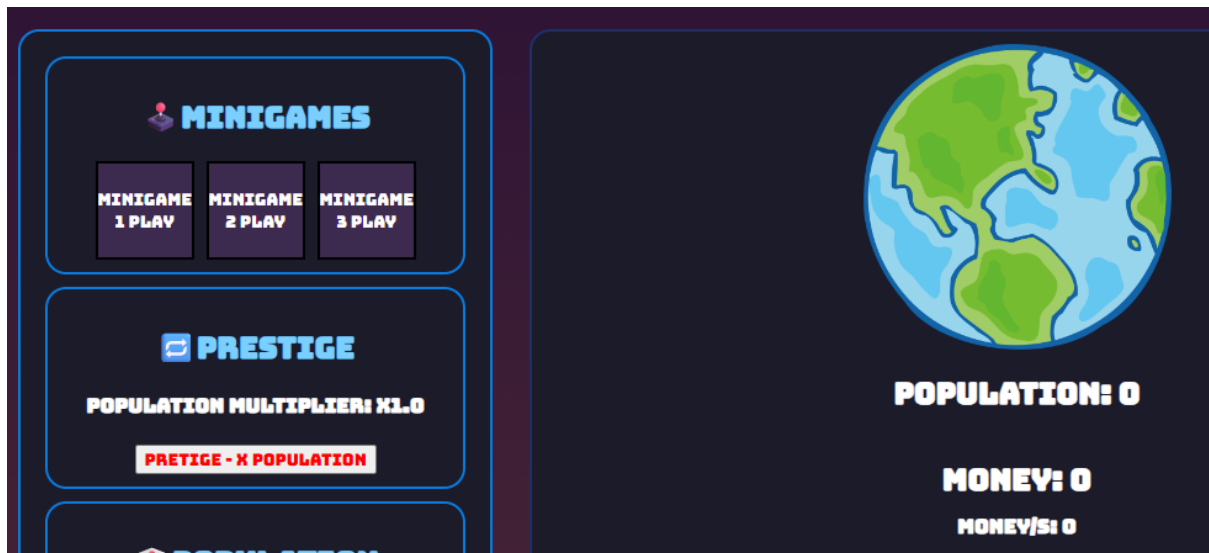
```
setInterval(() => {
  for(let i = 0; i < game_data.upgrade_elements.length; i++) {
    let upgrade_name = game_data.upgrade_elements[i].replaceAll("-", "_")
    if(game_data.money >= game_data.upgrades[upgrade_name].price) {
      document.getElementById(game_data.upgrade_elements[i]).style.color = "green"
    } else {
      document.getElementById(game_data.upgrade_elements[i]).style.color = "red"
    }
  }

  if(game_data.money >= game_data.worker_price) {
    document.getElementById("worker-button").style.color = "green"
  } else {
    document.getElementById("worker-button").style.color = "red"
  }

  if(game_data.population >= game_data.prestige_cost) {
    document.getElementById("prestige-button").style.color = "green"
  } else {
    document.getElementById("prestige-button").style.color = "red"
  }
}, 1000)
```

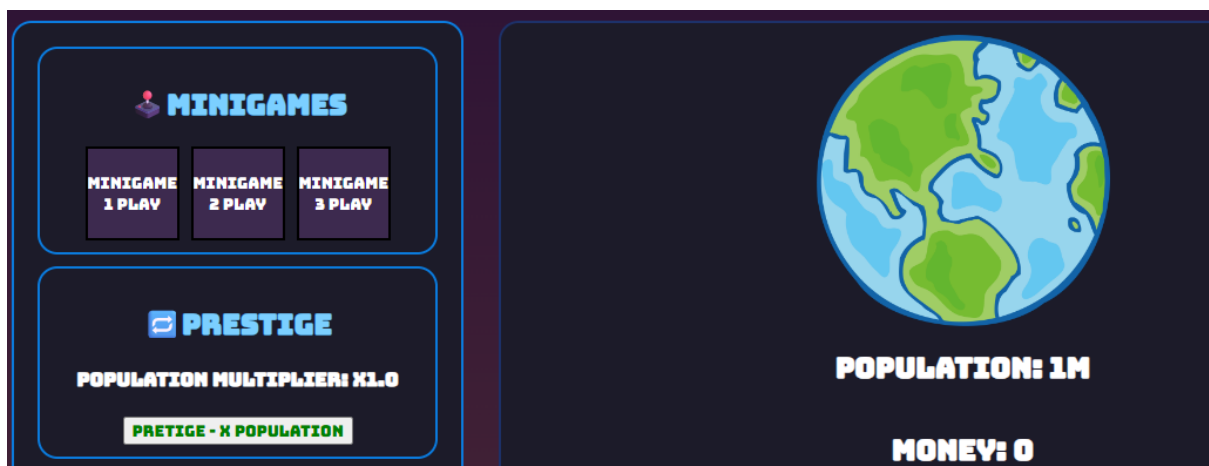
Here, I check if the population is greater than or equal to the `prestige_cost` (the population required to prestige), and if so, I change the colour of the prestige button text to green, and if not I change it to red. Since I already perform this operation for the upgrade and worker buttons, I put the code inside the same loop as them, so that all the checks will be applied at the same time, every second. I ran this to check if it worked:

With 0 population:



Button is red as the population is less than the required amount (1 million)

With 1M population (the required amount to prestige):



The text colour of the prestige button is green, so the code is working

I then added a click event listener to the prestige button in the JavaScript, with an event handler that checks if the player has the required population to prestige (`game_data.prestige_cost`), and then calls the prestige function if so:

```
// Calls prestige function when prestige button clicked, if player has enough population
document.getElementById("prestige-button").addEventListener("click", function (event) {
  if(game_data.population >= game_data.prestige_cost) {
    prestige()
  }
})
```

The reason I don't carry out the `prestige_cost` check in the `prestige` function, is because then the function would be called every time the prestige button was clicked, meaning that regardless of whether the player is able to prestige, the function would be called. This would lead to unnecessary function calls, which would waste memory,

After this, I added the line to copy the `game_data` dictionary and store it in a `default_game_data` dictionary:

```
// Creates default copy of data, used to reset game data after prestige
const default_game_data = JSON.parse(JSON.stringify(game_data))
console.log(default_game_data)
```

Here, I use `JSON.stringify` then `JSON.parse` to create a copy of the `game_data` object. The reason I do this is so that I have a copy of all the default data, so that I can reset the game when the player prestiges. I also output the `default_game_data` to the console afterwards to check that the `game_data` dictionary was being correctly copied and assigned to it:

Here, a complete copy of game_data is output, meaning it is being copied correctly.

```
▼ Object 1
  money: 0
  money_per_second: 0
  multi_purchase_state: 1
  population: 1000000
  prestige_cost: 1000000
  prestige_level: 0
  prestige_multiplier: 1
  total_worker_gen: 0
  ▼ upgrade_elements: Array(14)
    0: "click-surge"
    1: "income-boost"
    2: "industry-boom"
    3: "worker-training"
    4: "worker-efficiency"
    5: "population-multiplier-boost"
    6: "quicker-prestige"
    7: "permanent-click-boost"
    8: "double-minigame-reward"
    9: "temporary-population-surge"
    10: "gamble-luck"
    11: "win-streak"
    12: "achievement-gem-boost"
    13: "achievement-xp-boost"
    length: 14
  ▶ [[Prototype]]: Array(0)
  ▼ upgrades:
    ▶ achievement_gem_boost: {price: 2000, level: 0, price_multiplier: 0.1, base_multiplier: 0.1, multiplier_increase: 0.03, ...}
    ▶ achievement_xp_boost: {price: 5000, level: 0, price_multiplier: 0.1, base_multiplier: 0.1, multiplier_increase: 0.05, ...}
    ▶ click_surge: {price: 100, level: 0, price_multiplier: 0.1, base_multiplier: 0.1, multiplier_increase: 0.01, ...}
    ▶ double_minigame_reward: {price: 30000, level: 0, price_multiplier: 0.1, base_multiplier: 0.1, multiplier_increase: 0.125, ...}
    ▶ gamble_luck: {price: 50000, level: 0, price_multiplier: 0.1, base_multiplier: 0.1, multiplier_increase: 0.125, ...}
    ▶ income_boost: {price: 500, level: 0, price_multiplier: 0.1, base_multiplier: 0.1, multiplier_increase: 0.02, ...}
    ▶ industry_boom: {price: 10000, level: 0, price_multiplier: 0.1, base_multiplier: 0.1, multiplier_increase: 0.05, ...}
    ▶ permanent_click_boost: {price: 20000, level: 0, price_multiplier: 0.1, base_multiplier: 0.1, multiplier_increase: 0.08, ...}
    ▶ population_multiplier_boost: {price: 10000, level: 0, price_multiplier: 0.1, base_multiplier: 0.1, multiplier_increase: 0.08, ...}
    ▶ quicker_prestige: {price: 5000, level: 0, price_multiplier: 0.1, base_multiplier: 0.1, multiplier_increase: 0.07, ...}
    ▶ temporary_population_surge: {price: 20000, level: 0, price_multiplier: 0.1, base_multiplier: 0.1, multiplier_increase: 0.125, ...}
    ▶ win_streak: {price: 5000, level: 0, price_multiplier: 0.1, base_multiplier: 0.1, multiplier_increase: 0.07, ...}
    ▶ worker_efficiency: {price: 7500, level: 0, price_multiplier: 0.1, base_multiplier: 0.1, multiplier_increase: 0.08, ...}
    ▶ worker_training: {price: 1000, level: 0, price_multiplier: 0.1, base_multiplier: 0.1, multiplier_increase: 0.03, ...}
    ▶ [[Prototype]]: Object
  worker_base_price: 100
  worker_increase: 1
  worker_price: 100
  worker_price_multiplier: 0.1
  worker_speed: 2.5
  workers: 0
  ▶ [[Prototype]]: Object
```

After this, I developed my prestige function, starting by initialising 2 temporary variables for the prestige_level and prestige_cost:

```
function prestige() {
  let prestige_level = game_data.prestige_level
  let prestige_cost = game_data.prestige_cost
}
```

I assign both of these variables the value of their associated value in game_data. The reason I do this is because after I reset game_data to the default, I want to keep these values as they are needed throughout the game. prestige_level tracks the number of times the player has prestiged, so needs to increase every time they prestige and not get reset, and prestige_cost stores the population required to prestige so also needs to increase each time the player prestiges and not get reset. When I add gems,

achievements, the level system, and prestige upgrades, I will do the same for these to ensure they don't get reset (as I designed the prestige system to keep these and not reset them). After this, increment the 2 temporary variables I initialised and then reset the game_data dictionary to the default:

```
// Function that handles prestige
function prestige() {

    // Creates temporary variables, to restore specific game data after prestige
    let prestige_level = game_data.prestige_level
    let prestige_cost = game_data.prestige_cost

    // Increments prestige level and scales cost
    prestige_level ++
    prestige_cost *= 10

    Object.assign(game_data, JSON.parse(JSON.stringify(default_game_data))) // Resets game_data for prestige
}
```

I increase prestige level by 1, as the player is prestiging one time, and I multiply the prestige cost by 10 because it needs to increase by a factor of 10 each time the player prestiges. I then reset the game_data dictionary, by using JSON.stringify then JSON.parse to make a copy of default_game_data and then use Object.assign to assign it to the game_data object, which resets it to the default values.

Finally, I set prestige_level and prestige_cost in game_data to the temporary variables I assigned before resetting game_data, which restores them:

```
Object.assign(game_data, JSON.parse(JSON.stringify(default_game_data))) // Resets game_data for prestige

// Restores the game data variables that was assigned to the temporary variables
game_data.prestige_level = prestige_level
game_data.prestige_cost = prestige_cost
```

I also realised that I will need to update all the text displays, as they will all be reset. I want to do the exact same as when I load the game from local storage:

```
window.onload = function() {
    Object.assign(game_data, (JSON.parse(localStorage.getItem("game_data"))))
    update_prices()
    update_price_and_level()

    document.getElementById("population").textContent = formatNum(game_data.population)
    document.getElementById("money").textContent = formatNum(game_data.money)
    document.getElementById("money-per-second").textContent = formatNum(game_data.money_per_second)
    document.getElementById("workers").textContent = formatNum(game_data.workers)
    document.getElementById("worker-price").textContent = formatNum(game_data.worker_price)
    document.getElementById("worker-gen").textContent = formatNum(Math.round((game_data.total_worker_gen / game_data.worker_speed) * 100) / 100)
}
}
```

However, I don't want to just copy all this code into the prestige function, as that would be unnecessary duplication of code, therefore I made it into a function, then called it in the window.onload block, and in the prestige button click event handler:

```
function update_game_displays() {
    update_prices()
    update_price_and_level()

    document.getElementById("population").textContent = formatNum(game_data.population)
    document.getElementById("money").textContent = formatNum(game_data.money)
    document.getElementById("money-per-second").textContent = formatNum(game_data.money_per_second)
    document.getElementById("workers").textContent = formatNum(game_data.workers)
    document.getElementById("worker-price").textContent = formatNum(game_data.worker_price)
    document.getElementById("worker-gen").textContent = formatNum(Math.round((game_data.total_worker_gen / game_data.worker_speed) * 100) / 100)
}
```

```
window.onload = function() {
    Object.assign(game_data, (JSON.parse(localStorage.getItem("game_data"))))
    update_game_displays()
}
```

```
document.getElementById("prestige-button").addEventListener("click", function (event) {
    if(game_data.population >= game_data.prestige_cost) {
        prestige()
        update_game_displays()
    }
})
```

I then ran this and pressed the prestige button to see if it worked:



POPULATION: 0

MONEY: 0

MONEY/S: 0



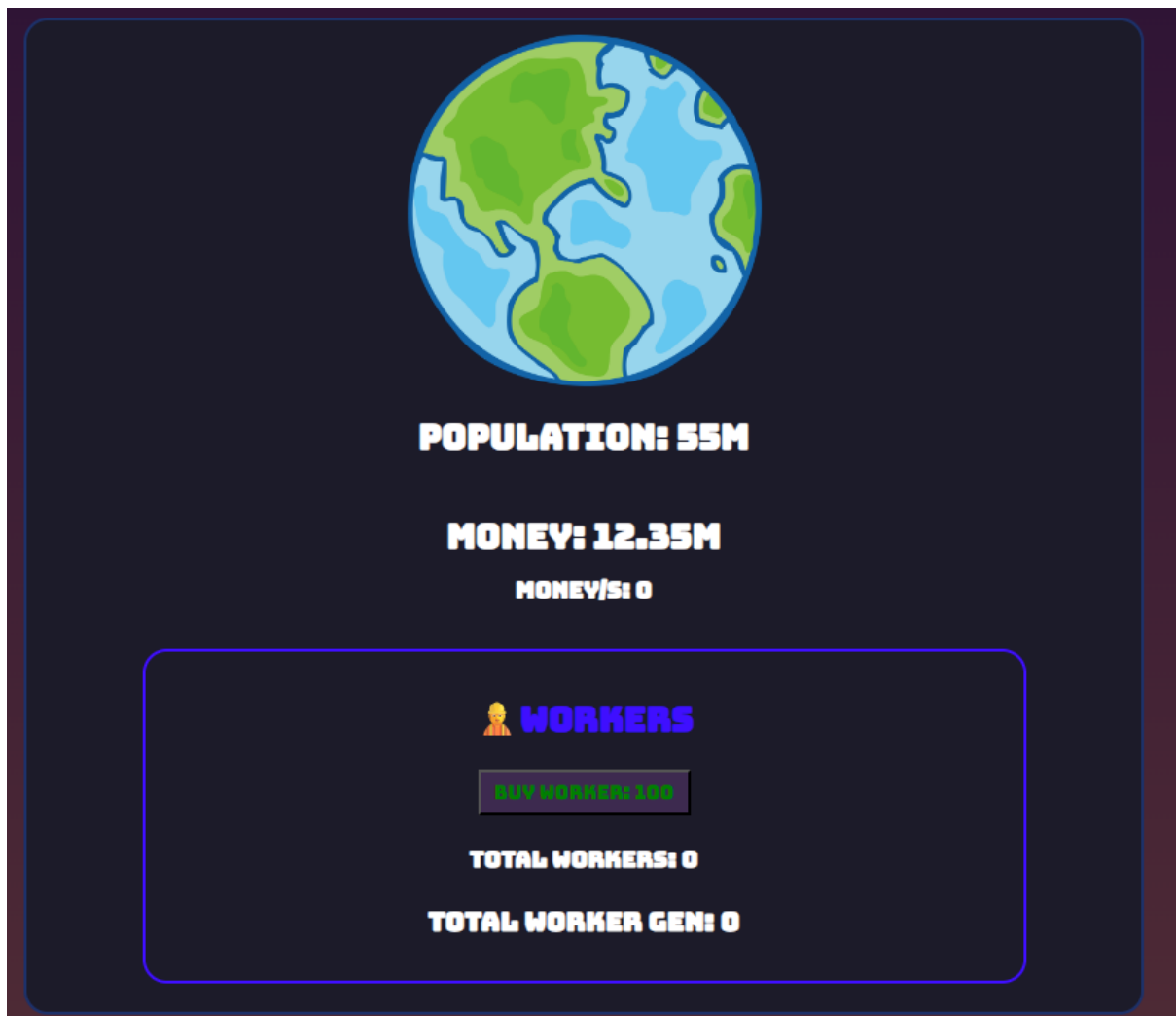
WORKERS

BUY WORKER: 100

TOTAL WORKERS: 0

TOTAL WORKER GEN: 0

Everything reset correctly, however when I refreshed the page everything was restored:



I realised that this was because I did not reset the local storage entry for game_data, so that was being restored. Therefore, I need to remove this in the prestige function:

```
// Removes previous save from local storage
localStorage.removeItem("game_data")
}
```

After this, I realised that I needed to update the population cost display in the HTML, as I had not done this. I will need to put the value in a separate span in the HTML, and give it an id so I can access it in the JavaScript:

```
<button id="prestige-button">Pretige - <span id = "prestige-cost" ></span> population</button>
```

I now need to update it in the JavaScript, and I decided to do it in the function I just created, as this runs every time the game loads and when the player prestiges:

```
function update_game_displays() {  
  update_prices()  
  update_price_and_level()  
  
  document.getElementById("population").textContent = formatNum(game_data.population)  
  document.getElementById("money").textContent = formatNum(game_data.money)  
  document.getElementById("money-per-second").textContent = formatNum(game_data.money_per_second)  
  document.getElementById("workers").textContent = formatNum(game_data.workers)  
  document.getElementById("worker-price").textContent = formatNum(game_data.worker_price)  
  document.getElementById("worker-gen").textContent = formatNum(Math.round((game_data.total_worker_gen / game_data.worker_speed) * 100) / 100)  
  document.getElementById("prestige-cost").textContent = formatNum(game_data.prestige_cost) ←  
}
```

I ran this to see if it worked:



Now, the prestige cost display updates correctly to the `prestige_cost` variable in game data, and is formatted correctly.

After this, I coded the population multiplier update. Before I code the loop that does this, I need to population multiplier value in a separate span in the HTML, with an id so I can access it in the JavaScript:

```
<h4>Population multiplier: x<span id="population-multiplier">1.0</span></h4>
```

I have only put the actual number in the span, so that the rest of the text will always be there, and only the value will change. I gave it the id "population-multiplier", so I could access it in the JavaScript. I then coded the population multiplier update:

```
// Infinite loop that updates prestige multiplier
setInterval(() => {
  // Increases prestige multiplier by 0.1 per 100k population
  game_data.prestige_multiplier = 1 + 0.1 * (game_data.population / 100000)
  //Updates population multiplier display in HTML
  document.getElementById("population-multiplier").textContent = game_data.prestige_multiplier
}, 1000)
```

Here, I have an infinite loop that runs every second, and calculates the prestige multiplier by multiplying 0.1 by the population divided by 100000, then adding the result to 1. This is because the population multiplier needs to increase by 0.1 per 100000 population. The reason I add the result to 1 is because the population multiplier starts at 1, so then the result needs to be added to 1, e.g. $0.1 * 200000 / 100000 = 0.2$, but the multiplier needs to be 1.2, so I add 1 to it. After this, I simply update the HTML display for population multiplier, by setting it's text content to the prestige multiplier. I ran this to check if it worked (by giving myself 200000 population):



Here, the multiplier showed 1.2 which is correct. I also wanted to check what would happen if I had just 1 population, and whether it would stay at 1.0 like it should:



However, the multiplier showed 1.000001, which is correct based on the calculation, however I would not like it to show this many digits. Therefore, I used `.toFixed(1)` to ensure the multiplier only showed 1 decimal place:

```
// Infinite loop that updates prestige multiplier
setInterval(() => {
  // Increases prestige multiplier by 0.1 per 100k population
  game_data.prestige_multiplier = (1 + 0.1 * (game_data.population / 100000)).toFixed(1)
  //Updates population multiplier display in HTML
  document.getElementById("population-multiplier").textContent = game_data.prestige_multiplier
}, 1000)
```

The reason I didn't use the method where I multiply the value by 100 and round the result, then divide it by 100, is because this solves the problem of JavaScript adding extra digits on the end of the value after multiplication, however this is not happening here and I just need to change how many decimal places are shown without rounding. I ran the code to check if this worked:



Now, when I have 1 population, the multiplier remains at 1.0, and when I have 200k:



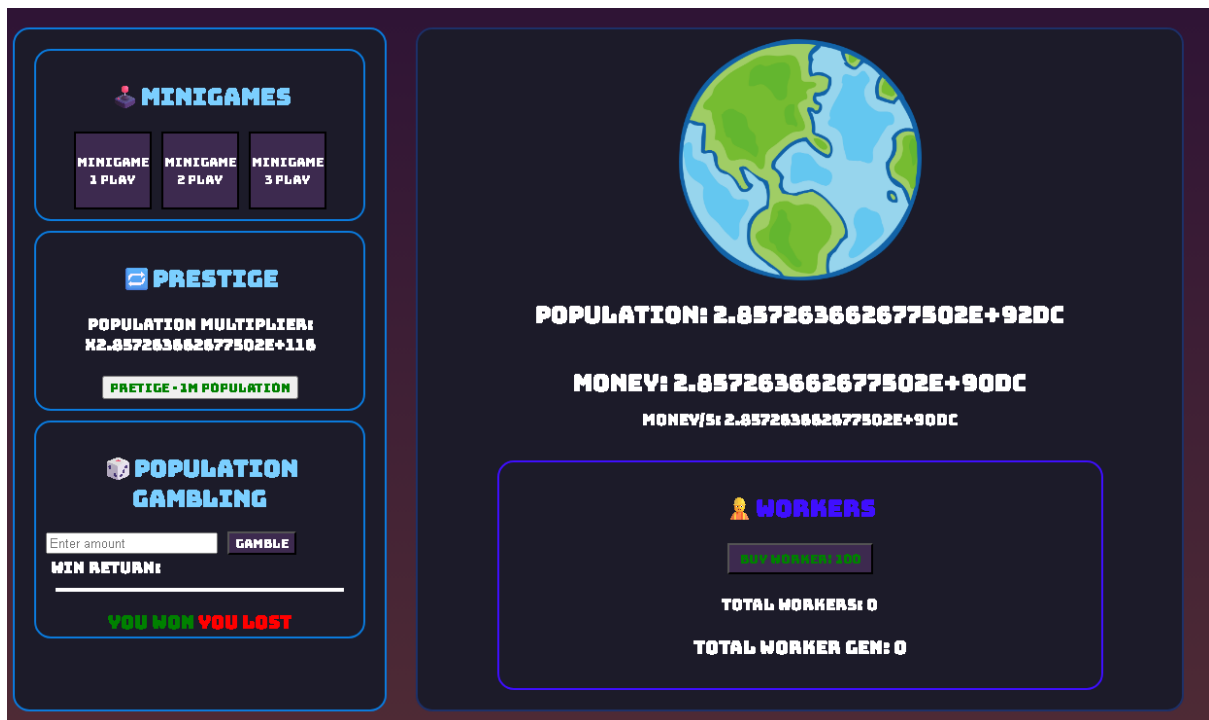
The multiplier becomes 1.2, so everything is working.

Finally, I wrote the code to actually apply the population multiplier, which required adding a line everywhere that I set the population:

```
function worker_click() {  
  clearInterval(interval)  
  game_data.population += game_data.total_worker_gen * (1 + (game_data.upgrades.worker_training.level * 20) / 100)  
  game_data.population += game_data.workers * game_data.prestige_multiplier  
  game_data.population = Math.round(game_data.population * 100) / 100 //Removes extra, unwanted decimal places  
}  
  
document.getElementById("planet-button").addEventListener("click", function (event) {  
  game_data.population += 1 * (1 + (game_data.upgrades.click_surge.level * 2)/100)  
  game_data.population *= game_data.prestige_multiplier  
  game_data.population = Math.round(game_data.population * 100) / 100  
  game_data.money_per_second = game_data.population * (0.01 * (1 + (game_data.upgrades.income_boost.level * 20) / 100))  
})
```

The reason I multiply the workers by the prestige_multiplier is because I need the population to increase for every worker, so I multiply the multiplier by the total number of workers.

I ran this to see if it worked:



I immediately ran into a problem, where the population increased the population multiplier, which then increased the population, which then increased the population multiplier even more, causing the population to increase by way too much. I realised that this was because the population multiplier needs to only active when the player prestiges, not be active all the time. Therefore, I decided to have 2 variables for the population multiplier, one for the current multiplier, and another for the next multiplier. The current multiplier will be the one that is multiplied by the population, and the next multiplier will determine the multiplier that will be applied after the prestige. Therefore, I need to remove the current multiplier, and add these 2 multiplier variables to game_data:

```
prestige_level: 0,
prestige_cost: 1000000,
current_prestige_multiplier: 0,
next_prestige_multiplier: 1.0,
```

The reason I set current_prestige_multiplier to 0, is because this will be added onto the population, so it should start at 0 because the player starts with no active population multiplier, they only get it once they prestige

I then changed the loop that calculates the multiplier, to set the next_prestige_multiplier to $1 + 0.1 * (\text{population} / 100000)$, and to set the multiplier text display to the next_prestige_multiplier. This is because this is the multiplier that will be

applied after the player prestiges, which is what needs to increase with the population and be displayed to them:

```
// Infinite loop that updates prestige multiplier
setInterval(() => {
  // Increases prestige multiplier by 0.1 per 100k population
  game_data.next_prestige_multiplier = (1 + 0.1 * (game_data.population / 100000)).toFixed(1)
  // Updates population multiplier display in HTML
  document.getElementById("population-multiplier").textContent = game_data.next_prestige_multiplier
}, 1000)
```

I then added a line in the prestige() function that creates another temporary variable current_prestige_multiplier, and then sets it to the next_prestige_multiplier:

```
// Function that handles prestige
function prestige() {

  // Creates temporary variables, to restore specific game data after prestige
  let prestige_level = game_data.prestige_level
  let prestige_cost = game_data.prestige_cost
  let current_prestige_multiplier = game_data.next_prestige_multiplier ←
  prestige_level ++
```

I did this so that I could restore the current prestige multiplier, before the game_data was reset, and then apply it after. I then set the current prestige multiplier in game_data to the temporary variable, after game_data is reset:

```
Object.assign(game_data, JSON.parse(JSON.stringify(default_game_data))) // Resets game_data for prestige

// Restores the game data variables that was assigned to the temporary variables
game_data.prestige_level = prestige_level
game_data.prestige_cost = prestige_cost
game_data.current_prestige_multiplier = current_prestige_multiplier ←
```

This restores it and applies it to the current prestige multiplier after the player prestiges, so that they will get their population multiplier. The next_prestige_multiplier will then be reset, and will increase again as normal as the player gets more population. I also changed the lines where I multiply the population, to multiply it by the current_prestige_multiplier value, as this is the active multiplier. I also realised that I should be adding to the population with +=, rather than multiplying the population with *=, because I am just adding the increased population with the multiplier. If I multiplied the population each time with the multiplier, the number would get large very quickly:

```
function worker_click() {
    clearInterval(interval)
    // Increases population by the worker gen value, with boost from worker training upgrade
    game_data.population += game_data.total_worker_gen * (1 + (game_data.upgrades.worker_training.level * 20) / 100)
    game_data.population += game_data.workers * Number(game_data.current_prestige_multiplier) // Applies prestige multiplier to population increase
    game_data.population = Math.round(game_data.population * 100) / 100 // Removes extra unwanted decimal places
}
```



POPULATION: 111

MONEY: 1.11

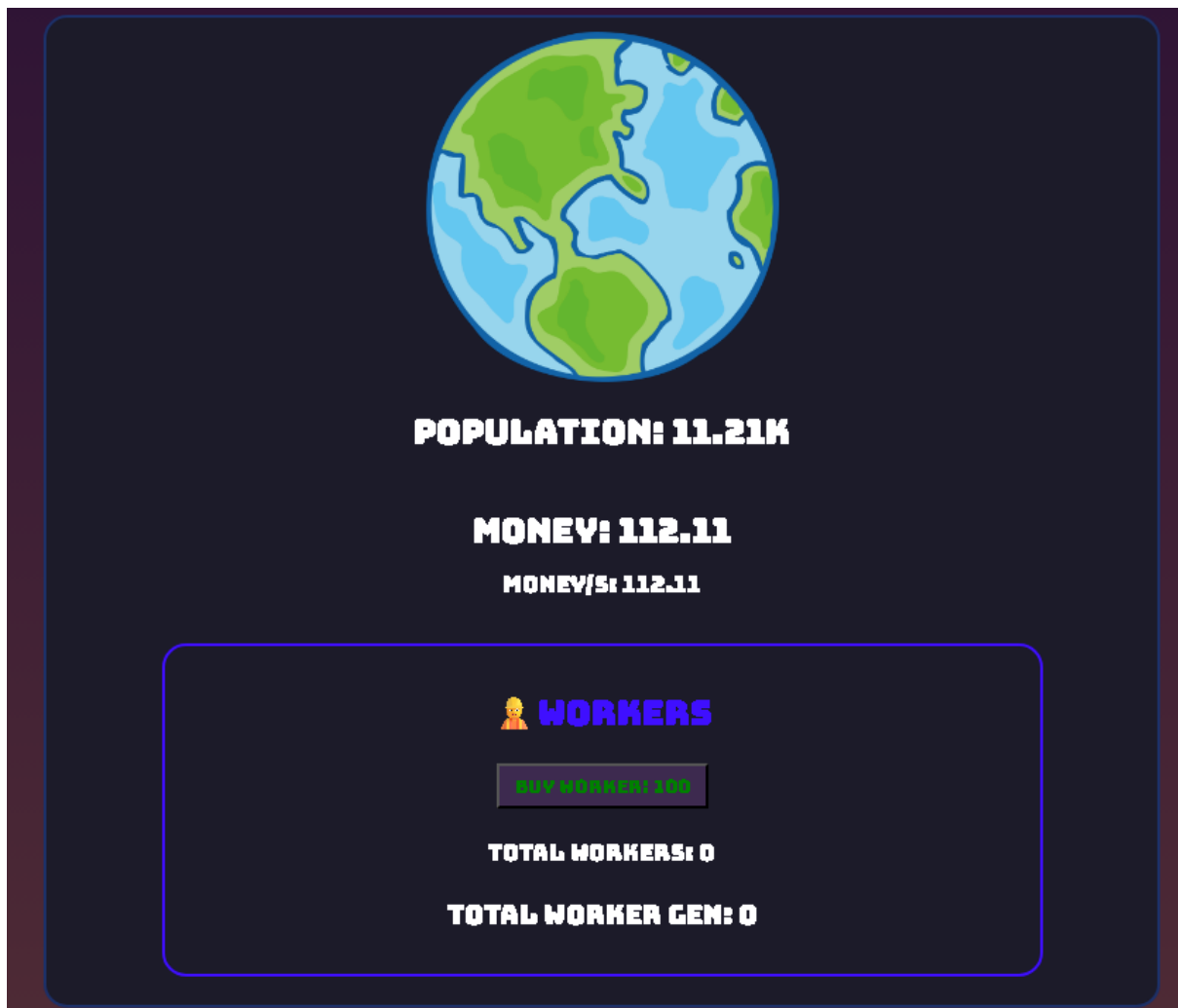
MONEY/S: 1.11

 **WORKERS**

BUY WORKER: 100

TOTAL WORKERS: 0

TOTAL WORKER GEN: 0

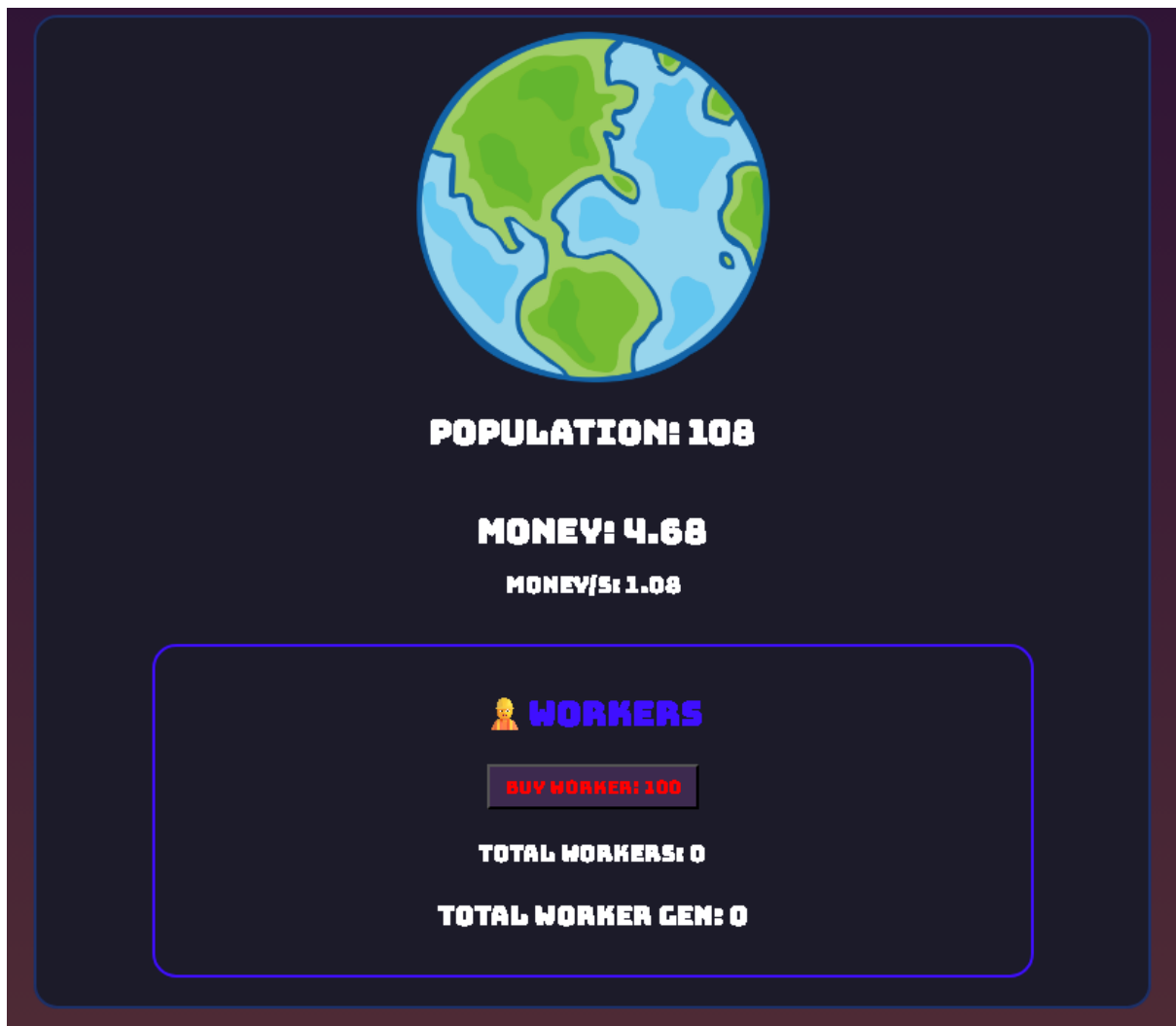


Now, there was another problem. After prestiging with a population multiplier of x11, the population increased to 111 after the first click, and then 11.21k after the second. It seemed like the population multiplier was being literally added onto the population, like it was doing $1 + 11 = 111$. I had a feeling this was a problem with the data type of the multiplier, and that it was not being calculated correctly. To check this, I used the `Number()` function to convert it to an integer, to see if that fixed the problem:

```
game_data.population += Number(game_data.current_prestige_multiplier)
game_data.population = Math.round(game_data.population * 100) / 100

game_data.population += game_data.workers * Number(game_data.current_prestige_multiplier)
game_data.population = Math.round(game_data.population * 100) / 100
```

I ran this to see if it worked:

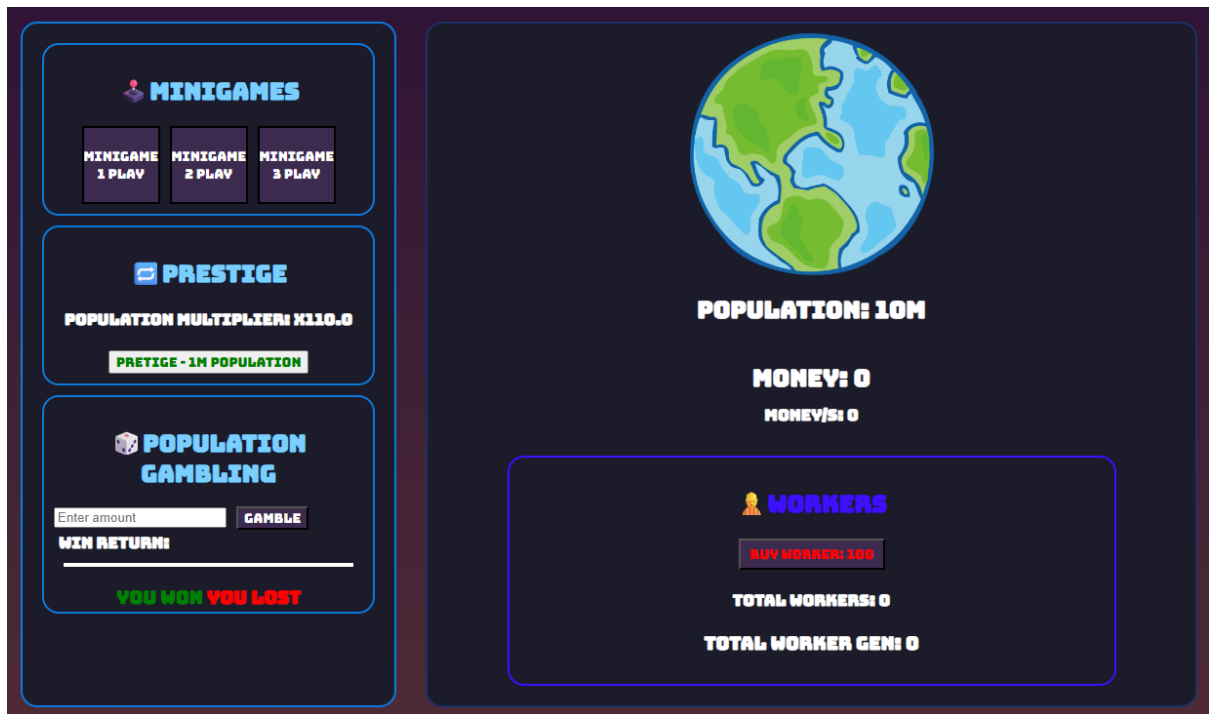


Now, after prestiging with a population multiplier of 11, each time I clicked the planet the population increased by 11, meaning the multiplier was being correctly applied.

After this, I developed the `population_multiplier_boost` upgrade, for which I just needed to add some code onto the end of the line that calculates the population multiplier:

```
setInterval(() => {  
  // Increases prestige multiplier by 0.1 per 100k population, plus the boost from the population multiplier upgrade (+0.5 per upgrade)  
  game_data.next_prestige_multiplier = (1 + 0.1 * (game_data.population / 100000) + (0.5 * game_data.upgrades.population_multiplier_boost.level)).toFixed(1)  
  //Updates population multiplier display in HTML  
}, 1000);
```

Here, I add 0.5 multiplied by the population multiplier boost upgrade level to the prestige multiplier calculation, which increases it by 0.5 per upgrade. I tested this to see if it worked:



Unfortunately, I had a similar problem as before, where the numbers I were adding were literally being concatenated together, as if they were strings. To try to fix this, I converted both the results I was adding to integers, using `Number()`:

```
game_data.next_prestige_multiplier = Number(((1 + 0.1 * (game_data.population / 100000))) + Number((0.5 * game_data.upgrades.population_multiplier_boost.level)).toFixed(1))
document.getElementById("population-multiplier").textContent = game_data.next_prestige_multiplier
```

I then ran this to check if it worked:



However, the problem was still occurring, and I realised that it was because `.toFixed` converts the result to a string. The reason it was working in the `formatNum` function where I return `num.toFixed(2)`, is because I then use `parseFloat()` to convert it back to a float. Therefore, I need to do this here, after `.toFixed()`:

```
game_data.next_prestige_multiplier = parseFloat(((1 + 0.1 * (game_data.population / 100000))) + (0.5 * game_data.upgrades.population_multiplier_boost.level)).toFixed(1))
document.getElementById("population-multiplier").textContent = game_data.next_prestige_multiplier
```

I put the whole result inside `parseFloat`, so that it is reduced to 1 decimal place, and then converted back to a float. I tested this to see if it worked:



Now, the population multiplier is correct, so I then purchased a population multiplier upgrade to check if it updates correctly:



After purchasing, the multiplier increased by 0.5, which is what the upgrade is supposed to do, so it is working.

After this, I coded the quicker prestige upgrade, which just requires adding another else if check in the upgrade function, to then reduce the `prestige_cost` by 5%

```
} else if(upgrade_name == "quicker_prestige") {  
    game_data.prestige_cost *= 0.95 // Reduces pretige cost by 5%  
    document.getElementById("prestige-cost").textContent = formatNum(game_data.prestige_cost)  
}
```

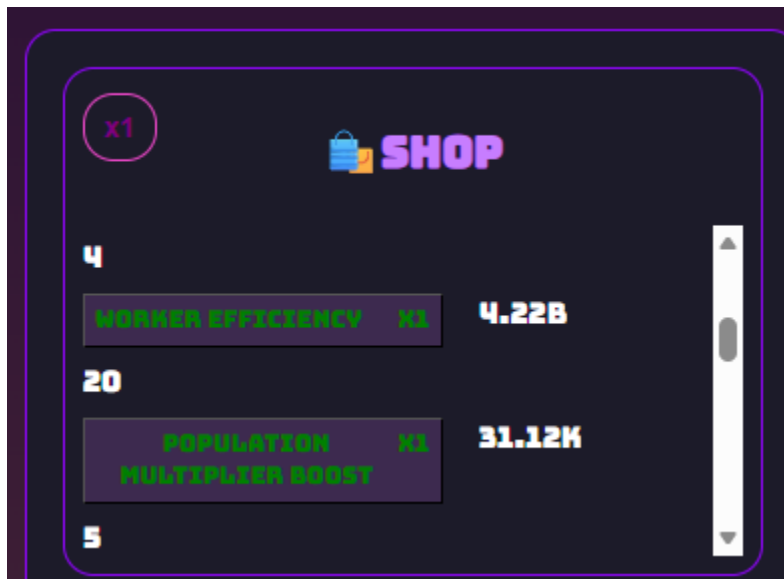
By multiplying the `prestige_cost` by 0.95, I reduce it by 5%, and then I update the prestige cost display in the HTML, setting it to the formatted `prestige_cost`, using the `formatNum` function I developed. Looking at the upgrade function again, I noticed a quite a serious problem:

```
function upgrade(upgrade_element) {
  // Converts upgrade name from the case used in HTML to snake case so the upgrade can be accessed in t
  let upgrade_name = upgrade_element.replaceAll("-", "_")
  // Stores the number of times to upgrade
  let upgrade_number = 0
  // Checks if player can afford upgrade and if worker efficiency upgrade is allowed (cannot be purchas
  if(game_data.money >= game_data.upgrades[upgrade_name].price && worker_efficiency_allowed == true) {
```

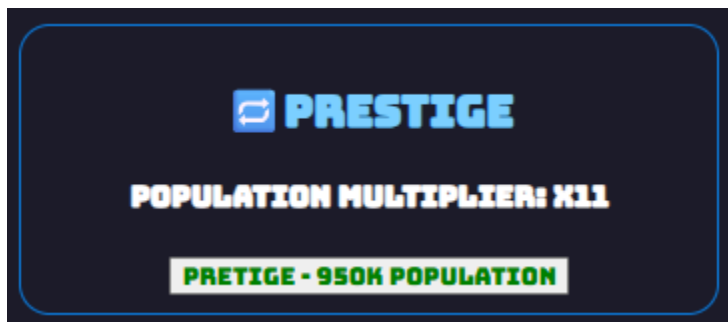
The first if check in the function, which checks if money is greater than the upgrade price and that the `worker_efficiency_allowed` variable is set to true, however I realised that when that variable is set to false (when no more worker efficiency upgrades can be purchased), this check won't pass so no other upgrades can be purchased. This is a problem because it prevents the player from buying anything from the shop after they have purchased 20 worker efficiency upgrades. Therefore, I changed it by splitting the if statement into 2, the second one checking if the `upgrade_name` is "worker_efficiency" and that the `worker_efficiency_allowed` variable is set to false. If so, I will break out of the function, by returning null. This will ensure that the only time the code in the function will not run, is if the upgrade is worker efficiency and the Boolean variable is set to false:

```
function upgrade(upgrade_element) {
  // Converts upgrade name from the case used in HTML to snake case so the upgrade can be
  let upgrade_name = upgrade_element.replaceAll("-", "_")
  // Stores the number of times to upgrade
  let upgrade_number = 0
  // Checks if player can afford upgrade
  if(game_data.money >= game_data.upgrades[upgrade_name].price) {
    // Prevents purchasing worker efficiency if worker speed is at 0.5s (cap)
    if(upgrade_name == "worker_efficiency" && worker_efficiency_allowed == false) {
      return null
    }
  }
```

I ran this to see if it worked:



Now, after reaching the worker efficiency limit, the game still allowed me to purchase other upgrades. I then ran the code to check if the quicker prestige upgrade was being correctly applied:



After the first click, the prestige cost reduced to 950k, which is a 5% reduction from 1 million, and then 902.5k after the second click, which is a 5% reduction from 950k, therefore the upgrade is working correctly. I then clicked the prestige button to check if the prestige cost reduction remained afterwards:



After the prestige, the reduction was still applied, reducing the prestige cost from 10M to 9.03M, meaning it is being restored correctly.

I also realised that there was a spelling mistake in Prestige, as I missed out the s, so I corrected that in the HTML:

```
<n4>population multiplier: x<span id= "population-multiplier" >1.0</span></n4>  
<button id="prestige-button">Prestige - <span id = "prestige-cost" ></span> population</button>  
div>
```

Now I have developed the quicker prestige upgrade, I need to make sure it is restored after the prestige. According to my pseudocode, I need to create a copy of the quicker_prestige dictionary before the game_data is reset, and then restore it after:

```
function prestige() {  
  let prestige_level = game_data.prestige_level  
  let prestige_cost = game_data.prestige_cost  
  let current_prestige_multiplier = game_data.next_prestige_multiplier  
  let quicker_prestige = JSON.parse(JSON.stringify(game_data.upgrades.quicker_prestige))  
  
  prestige_level ++  
  prestige_cost *= 10  
  
  Object.assign(game_data, JSON.parse(JSON.stringify(default_game_data)))  
  
  game_data.prestige_level = prestige_level  
  game_data.prestige_cost = prestige_cost  
  game_data.current_prestige_multiplier = current_prestige_multiplier  
  Object.assign(game_data.upgrades.quicker_prestige, quicker_prestige)  
  
  localStorage.removeItem("game_data")  
}
```

Here, I use `JSON.parse(JSON.stringify)` to create a copy of the quicker prestige upgrade, to ensure all values are copied. I then use `Object.assign` after `game_data` is reset, to assign `quicker_prestige` back to its object in `game_data`. The reason I copy the whole

object is because I want to keep all of the values, including the price, price multiplier, and level, so that the upgrade is completely restored.

Finally, as per my design for this stage, I will remove the permanent click boost upgrade, which I previously planned would increase click power by 5% after a prestige. I believe this is too similar to the population multiplier boost upgrade, which increases the population multiplier, which then increases population increase by 5%. Therefore, I will remove this from the shop in the HTML, from the upgrades dictionary in game_data, and in the upgrade_elements array in game_data

Testing:

This is my test plan for stage 6, with some modifications:

- I have added the specific amount that the population multiplier should increase by for the test. In my design, I just wrote “The population multiplier should gradually increase as the population increases”, as I had not planned the exact amount at that point
- I have changed test 4 so that it just checks whether the player keeps their quicker prestige purchases. I designed the test plan to check whether the player keeps their achievement completions, gems, and level, however I have not developed these features yet so I will test them when I do
- I also added tests for the functionality of the 2 prestige upgrades I developed; population multiplier boost and quicker prestige. I did not plan these tests in the design section, since I hadn’t designed the specific upgrades yet

Test no.	Description	Test type	Test data	Expected Result	Actual result
1	Does the prestige multiplier increase as the population increases		Increase population by 100k and check if the population multiplier increases by 0.1 every 100000 population	The population multiplier increases by 0.1 every 100000 population	After changing money to 100k in the code, the population multiplier increased from x1 to x1.1, a 0.1 increase Passed ☒ SEE STAGE 6 TEST 1.mp4 in evidence folder
2	Is the player prevented from prestiging until they reach a certain population		Attempt to prestige with population to see if it is allowed	Nothing should happen; the player should not be allowed to prestige, all values (e.g. money should remain the same)	Nothing happened, all values stayed the same Passed ☒ SEE STAGE 6 TEST 2.mp4 in evidence folder

3	Does the game reset the player's progress after prestiging		Prestige with enough money to see if the game resets the player's money, population, workers and upgrades (apart from quicker prestige upgrade)	The game should reset the player's money, population, worker count and upgrade count to 0, with all upgrade (apart from quicker prestige upgrade) and worker effects removed.	The game reset; population, money, worker count and upgrade count reset to 0 Passed  SEE STAGE 6 TEST 3.mp4 in evidence folder
4	Does the player keep their quicker prestige upgrade purchases after they prestige, and do they still apply their effect		Prestige with enough money to see if the game keeps their quicker prestige upgrades and whether they still apply their affect to the population cost	The player should keep their quicker prestige purchases, meaning the upgrade number should remain the same, the cost should remain the same. The price reduction form the quicker	The quicker prestige upgrade count remained, and the price reduction form the upgrade still applied to the new prestige cost after prestiging (Price was 9.03M, original

				prestige upgrades should still apply to the new cost of the prestige.	after prestige is 10M) Passed ☒ SEE STAGE 6 TEST 4.mp4 in evidence folder
5	Does the population multiplier boost upgrade increase the population multiplier by 0.5		Purchase the population multiplier boost upgrade, and see if it correctly increases the population multiplier	The population multiplier should increase by 0.5	The population multiplier boost increased from x2 to x2.5, after purchasing the upgrade, and then kept increasing by 0.5 after each purchase. Passed ☒ SEE STAGE 6 TEST 5.mp4 in evidence folder

6	Does the quicker prestige upgrade reduce the prestige cost by 5%		Purchase the quicker prestige upgrade and see if the population required to prestige (The number on the prestige button) correctly increases	The population required to prestige should reduce by 5% each time the upgrade is clicked	The prestige cost reduced from 1M to 950K after the first upgrade purchase, and then kept increasing by 5% each time (950 → 902.5K → 857.38K) Passed ☒ SEE STAGE 6 TEST 1.mp4 in evidence folder
---	--	--	--	--	--

Stage review:

I am generally quite happy with the development of this stage, as I managed to implement the prestige feature correctly according to the stage plan, with game_data being correctly reset and the prestige multiplier being correctly calculated and applied. I also ensured the prestige is not affected by the game_data save in local storage. However, there was quite a big error with the prestige multiplier, at first, where I applied it to the population before the prestige. This meant I had to slightly redesign the way the prestige multiplier worked, by having 2 values for the multiplier, a next multiplier and a current, active multiplier. This shows I could have designed

my pseudocode better, to prevent this error. This also extended the development time of this stage, however I still managed to complete it in a relatively short amount of time, so it was still a success.

Stage 7 – Population Gambling:

Features:

- Win return displays the population the player would earn if they win, based on how much they entered in the form
- The player can gamble their population, only if they have enough, and earn the amount displayed by the win return, if they win
- The win chance will be 50% for all gambles below 1000, and then will reduce by 5% each time the gamble amount increases by a factor of 10
- The win chance will stop reducing once it reaches 5%
- The win population multiplier will start at 1.5 and will also increase by 0.25 each time the gamble amount increases by a factor of 10 and will stop once it reaches 3.75
- This is how this will all scale:

Bet	Win Change	Win multiplier
5	50%	1.5
10	50%	1.5
1000	45%	1.75
10000	40%	2.0
100000	35%	2.25
1000000	30%	2.5
10000000	25%	2.75
100000000	20%	3
1000000000	15%	3.25
10000000000	10%	3.5
100000000000	5%	3.75
1000000000000	5%	3.75

- This makes it so that higher gambles are riskier, but give a bigger reward
- A “YOU WON” message will appear briefly and ping sound effect will play if the player wins the gamble
- A “YOU LOST” message will appear briefly and an error sound effect will play if the player loses the gamble
- Gamble button will be green if player has more population than the amount they enter into the form, but red if they don’t
- Gamble luck upgrade which increases gamble win chance by 5%, capped at 75%
- Win streak upgrade that increases win chance by 5% for each consecutive win

Data Dictionary:

Variable	Data type	Purpose
game_data.win_chance	Float	This will store the probability of winning a gamble, which will be based on the amount the player has entered. Will start at 0.5 (50%)
game_data.win_multiplier	Float	This will store the population multiplier that will

		multiply the population the player gambled, to return to them if they win. Will start at 1.5
game_data.win_return	Float	This will store the population that will be returned to the player if they win the gamble.
game_data.bet	Int	This will store the current population bet the player has entered
game_data.win_streak	Int	Stores the number of consecutive gamble wins, used for the win streak upgrade
game_data.sounds {}	Dictionary	A dictionary inside game_data that will

		store the win and loss sounds for the gambles
game_data.sounds.win	Audio	An audio object that will store the ping sound that will play when the player wins a gamble
game_data.sounds.lose	Audio	An audio object that will store the error sound that will play when the player loses a gamble

Pseudocode:

Add to game_data:

Float win_chance = 0.5

Float win_multiplier = 1.5

Float win_return = 0

Int bet = 0


```
Dict sounds {  
    Audio win = "ping.mp3"  
    Audio lose = "error.mp3"  
}
```

Gamble button text colour:

```
FUNCTION check_gamble  
    IF population >= bet THEN  
        gamble-button.HTML.textColour =  
            "green"  
    ELSE  
        gamble-button.HTML.textColour = "red"  
    ENDIF  
ENDFUNCTION
```

- I will have a function, that checks if the player has enough population to gamble the amount they entered (bet)
- If they do, the button text colour will be set to green, and if not it will be set to red
- I will call this function every time the player makes a change to the gamble input, in the event listener that I will have for the input event

Calculating win chance and population multiplier:

```
FUNCTION calculate_gamble_stats()
    int betLevel = 1000

    WHILE (bet >= betLevel)
        win_chance -= 0.05
        win_multiplier += 0.25
        betLevel *= 10

        IF win_chance <= 0.05 THEN
            win_chance = 0.05
            BREAK
        ENDIF
    ENDWHILE

    win_return = bet * win_multiplier
ENDFUNCTION
```

- I will have a function, into which the bet amount is passed, which will calculate the win chance and multiplier of the bet
- It starts by resetting the win chance and multiplier to their default values, as they will be re-calculated in the function
- Then a while loop is used, so that while the bet is larger than the betLevel (the threshold which, when passed will reduce the win chance and increase the win

multiplier), the win chance will be reduced by 5% (0.05), the multiplier will increase by 0.25 and the betLevel will be increased by a power of 10.

- The while loop will stop when either the bet is less than the next bet level, or when the win chance hits the minimum value of 5% (in which case I break out of the loop)

Displaying win return:

```
gamble-amount EventListener(input) {  
    IF input_value != Number THEN  
        win-return.HTML.text = "Enter valid  
        number"  
    ELSE IF input_value == 0  
        win-return.HTML.text = 0  
    ELSE  
        bet = input_value  
        check_gamble()  
        calculate_gamble_stats(bet)  
        win-return.HTML.text = win_return  
    ENDIF  
}
```

- I will add an input event listener to the gamble-amount form, which will be triggered anytime the player enters anything
- I will have a type check, so that if the player doesn't enter just a number, it will display "Enter valid number"
- I will also have a validation check that checks if the user enters 0, in which case I will simply set the win return to 0
- If both these checks are passed, I will call the check_gamble function to change the colour of the gamble button text depending on whether the amount the player entered is greater than their population.
- I will then set the bet value in game_data to the input value of the form which will be the bet amount. I will then call the calculate_gamble_stats function, and pass in bet. This will calculate the win chance, multiplier and return, so I can then display the win return

Performing the gamble:

```

FUNCTION gamble
    IF population >= bet THEN
        let random_roll = randomNum(0,1)
        IF random_roll < win_chance THEN
            population -= bet

```

```

        population += bet * multiplier
        win-message.HTML.show
        PLAY(sounds.win)
    ELSE
        population -= bet
        loss-message.HTML.show
        PLAY(sounds.lose)
    ENDIF
ENDIF
ENDIF

```

```

gamble-amount.HTML.form.clear
game_data.bet = 0
calculate_gamble_stats()
WAIT 2
win-message.HTML.hide
loss-message.HTML.hide
update_game_displays()
ENDFUNCTION

```

```

gamble-button EventListener(click) {
    gamble()
}

```

- I will have a gamble function, which is called whenever a click event listener for the gamble button detects a click
- The function will first check if the population is greater than the bet, to check if the player actually has enough population to perform the bet

- If they do, then it will generate a random decimal number between 0 and 1, and check if it is less than the win chance. This would indicate that the player has one the gamble
- In this case, the population will be reduced by the bet amount, and then increased by the amount they bet multiplier by the win multiplier
- The win result message will then be shown, and the win sound will play
- If the random roll is greater than the win chance however, this would indicate a loss, so the population will be reduced by the bet, the loss message will be shown, and the loss sound will play
- After this, the input form will be cleared, the bet value will be set to 0, and the calculate_gamble_stats() function will be called to reset all the gamble variables, as the gamble is finished
- The code will then wait 2 seconds, and then hide the win messages (it will hide both, so whichever one is showing will be hidden) and call the update_game_displays() function to update the population display after the gamble, as it would have been changed.

Gamble luck upgrade:

```
FUNCTION upgrade(upgrade_name)
```

```
// Existing IF checks
ELSE IF upgrade_name == "gamble_luck" THEN
    win_chance += 0.5
ENDIF

// Rest of function
ENDFUNCTION
```

Win streak upgrade:

```
// Start of gamble function
FUNCTION gamble
    IF population >= bet THEN
        let random_roll = randomNum(0,1)
        win_chance += win_streak *
            (upgrades.win_streak.level * 0.05)
        IF random_roll < win_chance THEN
            population -= bet
            population += bet * multiplier
            win-message.HTML.show
            PLAY(sounds.win)
            win_streak ++
        ELSE
            population -= bet
            loss-message.HTML.show
            PLAY(sounds.lose)
            win_streak = 0
        ENDIF
    ENDIF
```

```
// Rest of gamble function  
ENDIF
```

Development

I started development of this section by adding the necessary variables to game_data:

```
// POPULATION GAMBLING  
  
win_chance: 0.5,  
win_multiplier: 1.5,  
win_return: 0,  
bet: 0,  
win_streak: 0,
```

I added the win_chance, which will be the probability of winning a gamble, starting at a default value of 0.5 (50%)

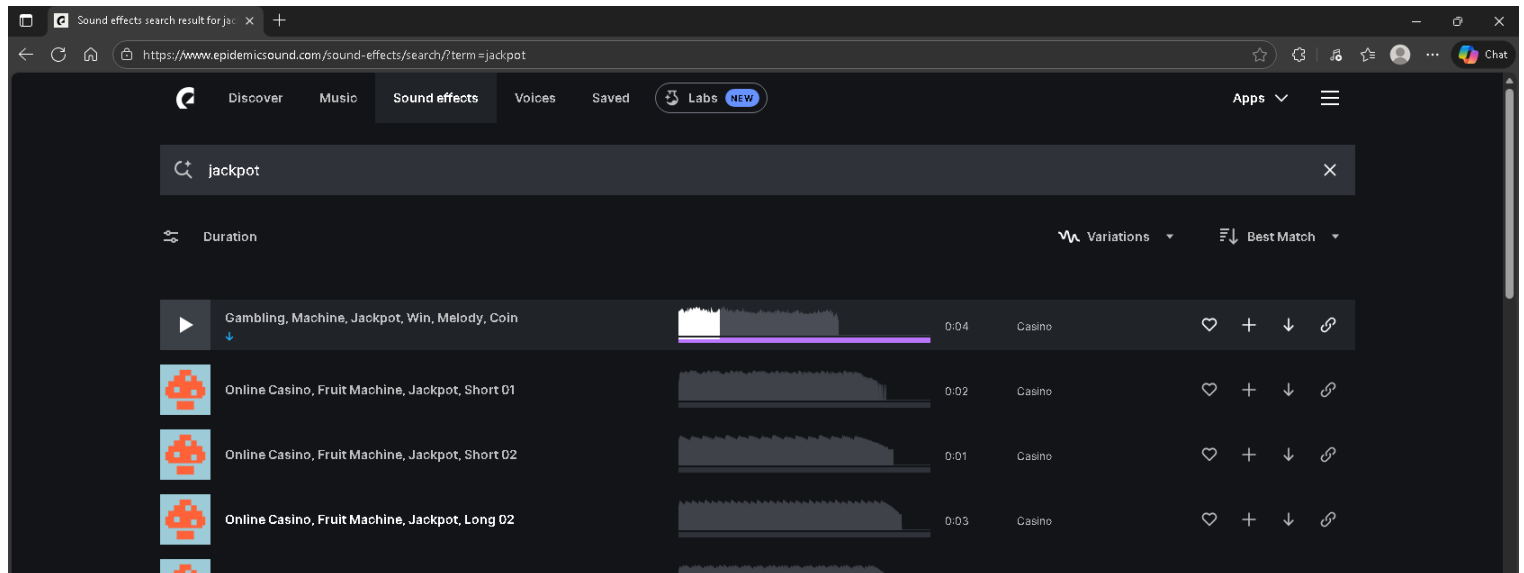
I also added the win_multiplier, which will be the amount that the population returned to the player after a win is multiplied by

I also added a win_return, which will store the population that will be returned to the player after a win

I also added a bet variable which will store the population that the player has gambled

I also added a win streak variable, to count the player's consecutive gamble wins, which will be used for the win streak upgrade.

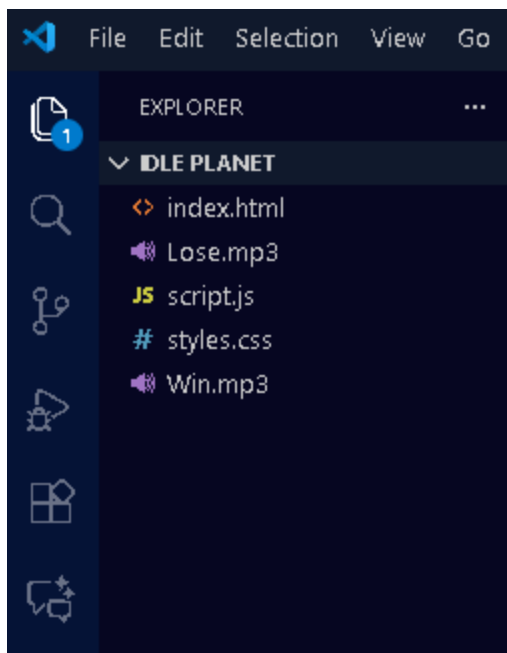
Before I added the sounds dictionary, I decided to get some more interesting sounds, rather than just a ping and error sound. So, I made a free account with Epidemic Sound (Sound, n.d.), and downloaded a slot



machine jackpot win for the gamble win sound ([SFX - Gambling, Machine, Jackpot, Win, Melody, Coin | Epidemic Sound](#)) :

and a retro lose sound for the gamble loss sound ([SFX - Retro, Lose, Negative 01 | Epidemic Sound](#))

I named the files “win.mp3” and “loss.mp3”, put them in my project folder and then added them to a sounds dictionary which I created inside game_data:



```
sounds: {  
  win : new Audio("win.mp3"),  
  lose: new Audio("lose.mp3")  
}
```

I think this will make the game feel more alive and engaging, rather than just a plain, boring ping and error sound.

I then developed the `check_gamble` function, which will be used to change the colour of the gamble button text colour depending on whether the player has enough population to gamble the amount they entered. Since I will be accessing the gamble button in the HTML, I added an id to it first:

```
<button id="gamble-button" class="gamble-button">Gamble</button>
```

I then developed the function:

```
function check_gamble() {
  if(game_data.population >= game_data.bet) {
    document.getElementById("gamble-button").style.color = "green"
  } else {
    document.getElementById("gamble-button").style.color = "red"
  }
}
```

Here, I simply check whether the population is greater than the player's bet, and if so then I change the text colour of the button to green, but if not I change it to red. I will then put this function call in the infinite loop that sets the colour of other buttons, since it does the same thing and saves me writing another setInterval loop:

```
setInterval(() => {
  for(let i= 0; i<game_data.upgrade_elements.length; i++) {
    let upgrade_name = game_data.upgrade_elements[i].replaceAll("-", "_")
    if(game_data.money >= game_data.upgrades[upgrade_name].price) {
      document.getElementById(game_data.upgrade_elements[i]).style.color = "green"
    } else {
      document.getElementById(game_data.upgrade_elements[i]).style.color = "red"
    }
  }

  if(game_data.money >= game_data.worker_price) {
    document.getElementById("worker-button").style.color = "green"
  } else {
    document.getElementById("worker-button").style.color = "red"
  }

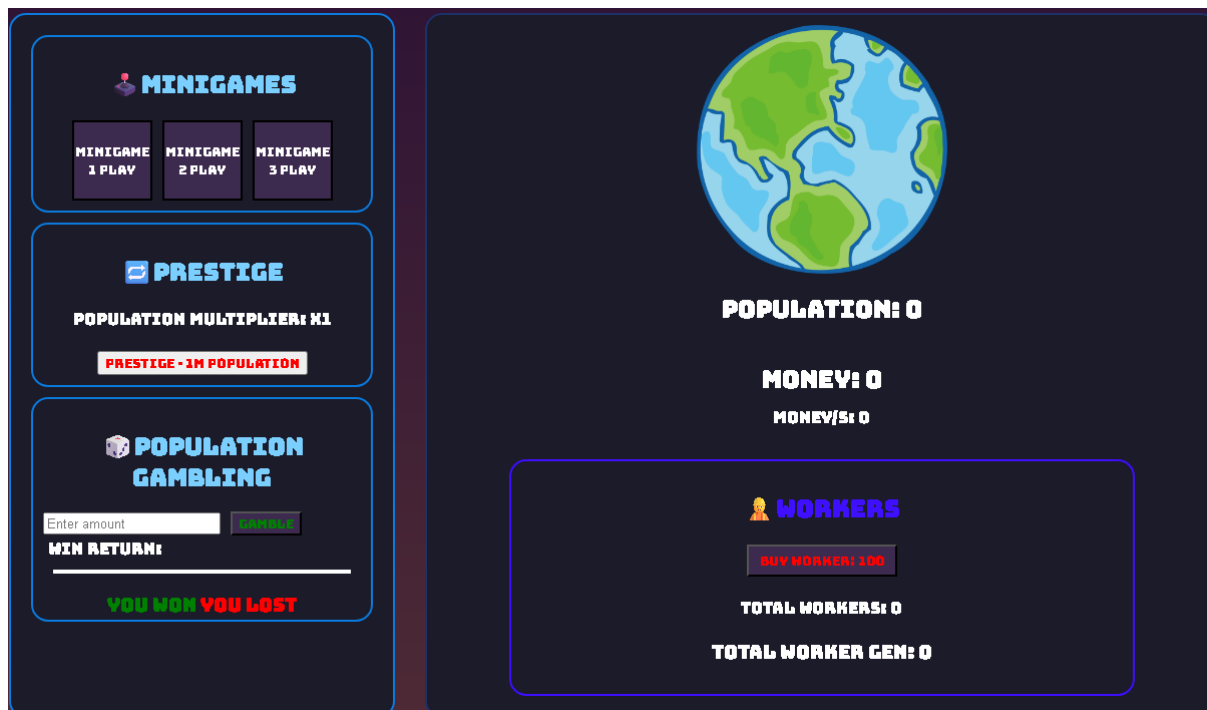
  if(game_data.population >= game_data.prestige_cost) {
    document.getElementById("prestige-button").style.color = "green"
  } else {
    document.getElementById("prestige-button").style.color = "red"
  }

  check_gamble()
}, 1000)
```

The reason I have a function for the code, in which I call in the loop, rather than having the code directly in the loop like for the rest of the button colour changes, is because I want to call this code elsewhere, not just every second. The code needs to be called in the input event listener for the

gamble input form, so that it updates to let the player know whether they can gamble as they are typing.

I tested this to see if it worked:



Now, the gamble button text colour is changed to green so it is working. Currently, the bet variable is just set to 0 by default, which is why the button is green, however once I develop the code to set bet to whatever the player enters in the form, it will change depending on whether the player has enough population to gamble the amount they entered.

After this, I developed the `calculate_gamble_stats()` function, to calculate the `win_chance`, `win_multiplier` and `win_return` based on the player's input:

```
function calculate_gamble_stats() {
  let betLevel = 1000

  while(game_data.bet >= betLevel) {
    game_data.win_chance -= 0.05
    game_data.win_multiplier += 0.25
    betLevel *= 10
  }
}
```

I started by creating the function and initialising a betLevel variable, which will store the threshold for the bet amount, meaning that each time the population surpasses it, the win chance is decreased by 0.05 (5%) and the win multiplier is increased by 0.25 in the while loop. The bet level is then increased by a power of 10, and the while loop runs again to check if the bet is still higher than the threshold.

After this, I added an if statement inside the loop to check if the win_chance was below or equal to 0.05, and if so I set the win_chance to 0.05 and break out of the loop:

```
while(game_data.bet >= betLevel) {
  game_data.win_chance -= 0.05
  game_data.win_multiplier += 0.25
  betLevel *= 10

  if(game_data.win_chance <= 0.05) {
    game_data.win_chance = 0.05
    break
  }
}
```

The reason I wrote this check is because I designed the win chance to stop decreasing at 0.05, so if it gets to 0.05 I need to exit the loop. The reason I set the win_chance to 0.05 in the if statement is because it might have dropped below 0.05, before breaking out of the loop, so I reset it just in case to ensure it is exactly 0.05.

Finally, at the end of the function (outside of the while loop), I set the win return to the bet multiplier by the win multiplier:

```

    }

    game_data.win_return = game_data.bet * game_data.win_multiplier
}

```

This sets the win return to the amount the player gambled, multiplied by the multiplier they get for winning the gamble.

After this, I developed the code to update the win return display, starting by creating an input event listener for the gamble amount form, for which I realised I needed to add an id to so I could access it. I also realised that I had the type of the form set to “text” when it should be “number”, since the player will be entering a number:

```

<div class="gamble-input">
  <input id="gamble-amount" class="gamble-amount" type="number" placeholder="Enter amount"></input>
</div>

```



After this, I created the input event listener for it:

```

// Input event listener, triggers anytime the player types or removes a digit from the input form
document.getElementById("gamble-amount").addEventListener("input", function(event) {

```

This will trigger each time the player enters anything in the input form. They don't have to press the button, as anything typed into the form will trigger the event. The reason I did this is because I want the win return display to update in real time, so the player gets a live display of how much population they will get from winning gamble. This makes the game more responsive and gives the user a chance to decide whether they want to gamble based on the reward.

Inside the event handler, I performed the 2 validation checks according to my pseudocode:

```
document.getElementById("gamble-amount").addEventListener("input", function(event) {  
    if(isNaN(this.value)) {  
  
    } else if(this.value == 0) {  
  
    } else {  
  
    }  
})
```

First, I check if the amount entered is NaN, meaning not a number. I use `this.value` as this stores the value currently in the input form. Although I have set the input type of the form to “number”, meaning it only accepts numbers, I still carry out this check just in case the player finds a way of entering something that isn’t a number. If this check passes, I check if the value is 0, in which case I will just set the win return to 0. This is because it would be unnecessary to call the `update_gamble_stats` function for this and would waste memory, when I can simply set the win return display to 0 here. If both these checks pass, I will calculate the win return and set the display to the result.

Before I do anything with the win return, however, I need to make sure I can access it from the JavaScript, so I added a span inside the header tag in the HTML and gave it an id so I could give it a value in the JavaScript:

```
<h4 style="margin: 5px; text-align: left;">Win return: <span id="win-return">0</span></h4>  
<div class="separator"></div>
```

I also gave it a default value of 0, as this will be the return if nothing is entered in the input form.

After this, I set the win-return display in each of the checks:

```
// Input event listener, triggers anytime the player types or removes a digit form the input form
document.getElementById("gamble-amount").addEventListener("input", function(event) {

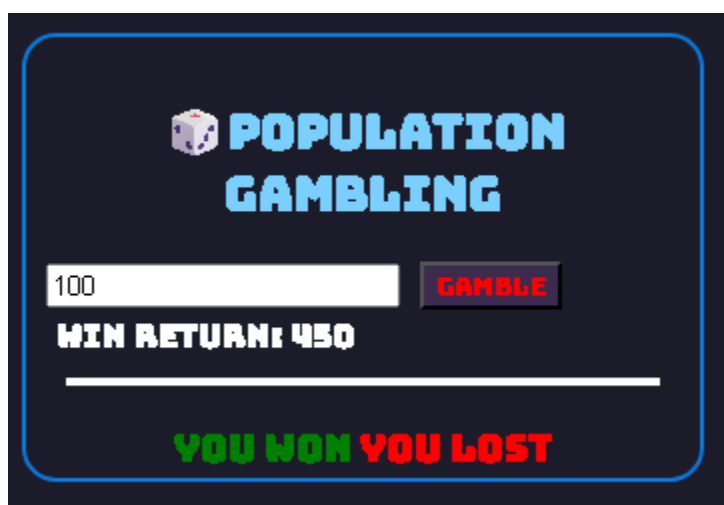
    // Checks if player has entered a non-numeric character
    if(isNaN(this.value)) {
        document.getElementById("win-return").textContent = "Enter valid number"
    } // Checks if player has entered 0
    } else if(this.value == 0) {
        // Sets win return to 0 if player enters 0. Can't win if bet is 0.
        document.getElementById("win-return").textContent = "0"
    } else {
        // Sets player bet to the value in the input form
        game_data.bet = this.value
        check_gamble() // Updates gamble button text colour depending on whether the player can afford to bet the amount entered
        calculate_gamble_stats() // Calculates the win return based on the input#
        // Updates win return and win chance display in HTML
        document.getElementById("win-return").textContent = game_data.win_return
    }
});
```

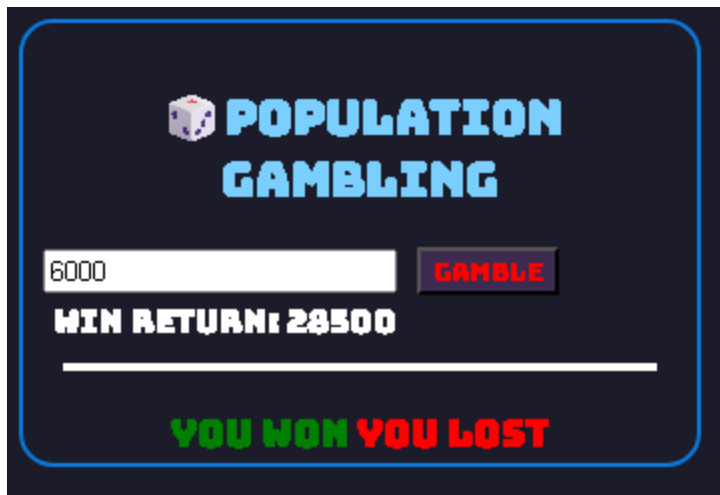
In the first check, if the value is not a number, I set the win-return display to “Enter valid number” so that the player is warned that they have entered incorrect data

In the second check, if the value is 0, I simply set the win-return display to 0

Finally, if the user enters correct data above 0, I set the bet value to the value in the form. I then call the `check_gamble()` and `calculate_gamble_stats()` functions, so that the gamble button text colour is updated, along with the win chance, win multiplier, and win return values, so they are ready for the gamble. I then update the win return display.

I tested this to see if it worked:





And the win return updated to the correct win return in real time, as I entered values. One thing I forgot to do was format the win return value, using the `formatNum()` function I developed, so that the number would be abbreviated, preventing long numbers from going over the line:

```
// Updates win return and win chance display in HTML
document.getElementById("win-return").textContent = formatNum(game_data.win_return)
```

After this, I developed the actual gamble function:

```
// Function that handles the gambles
function gamble() {
  // Checks if player has enough population to bet the amount they entered
  if(game_data.population >= game_data.bet) {
    // Generates random roll value
    let random_roll = Math.random()
    // Checks if player has won the gamble
    if (random_roll < game_data.win_chance) {

    } else {

    }
  }
}
```

I started by creating the function and running a check, to make sure the player has enough population to gamble the amount they entered. I then initialise a `random_roll` variable and set it to `Math.random()`, which (on its

own) generates a random number between 0 and 1, e.g 0.42. I then check if the value of random_roll is less than the win_chance, which indicates a gamble win. This is because as the win chance gets less, it will be less likely that the random_roll is less than it, meaning the win chance will decrease the win chance variable decreases. After this, I developed the code for the gamble win and loss:

```
// Checks if player has one gamble
if (random_roll < game_data.win_chance) {
    // Removes bet from population, and then adds it back with the win multiplier applied
    game_data.population -= game_data.bet
    game_data.population += game_data.bet * game_data.win_multiplier
}
```

I started developing the win code, by removing the bet from the population and then adding back the bet with the multiplier applied, however I realised I am now going to need to set the win message, which I don't have an id for in the HTML, so I added one. I have a span for both the win and loss, message, so I added one for each:

```
<div>
  <h3><span id="win" class="gamble-win">YOU WON</span> <span id="loss" class="gamble-loss">YOU LOST</span></h3>
</div>
```



Since I only want these to show when the player wins or loses a gamble, I will set them both to hidden in the update_game_displays() function, which runs each time the page is loaded:

```
document.getElementById("win").hidden = true
document.getElementById("loss").hidden = true
}
```

Now, I can carry on developing the gamble win and loss code, as I can access the win and loss messages:

```
// Checks if player has one gamble
if (random_roll < game_data.win_chance) {
  // Removes bet from population, and then adds it back with the win multiplier applied
  game_data.population -= game_data.bet
  game_data.population += game_data.bet * game_data.win_multiplier
  // Shows win message
  document.getElementById("win").hidden = false
  // Plays win sound
  game_data.sounds.win.play()
} else {
  game_data.population -= game_data.bet
  // Shows loss message
  document.getElementById("loss").hidden = false
  // Plays loss sound
  game_data.sounds.lose.play()
}
```

For both a win and loss, I show the associated message, by setting their hidden property to false, and I then play the associated sound, either win or lose. For the loss however, I just remove the bet amount from the population, and don't add anything on, since losing a gamble means the player loses all the population they gambled.

After this if statement, will reset the gamble displays and hide the messages:

```
// Recalculates gamble stats to reset win return and chance value, then updates them in HTML
calculate_gamble_stats()
document.getElementById("win-return").textContent = formatNum(game_data.win_return)
document.getElementById("win-chance").textContent = `${Math.round((game_data.win_chance * 100) * 100) / 100}%`
// Hides win/loss message after 2 seconds
setTimeout(() => {
  document.getElementById("win").hidden = true
  document.getElementById("loss").hidden = true
}, 2000)
```

I set the value of the gamble amount form to "", an empty string, which will clear it. I then reset the bet value and call calculate_gamble_stats, which will reset the win_chance, win_multiplier and win_return, since they will become their default values if the bet is 0.

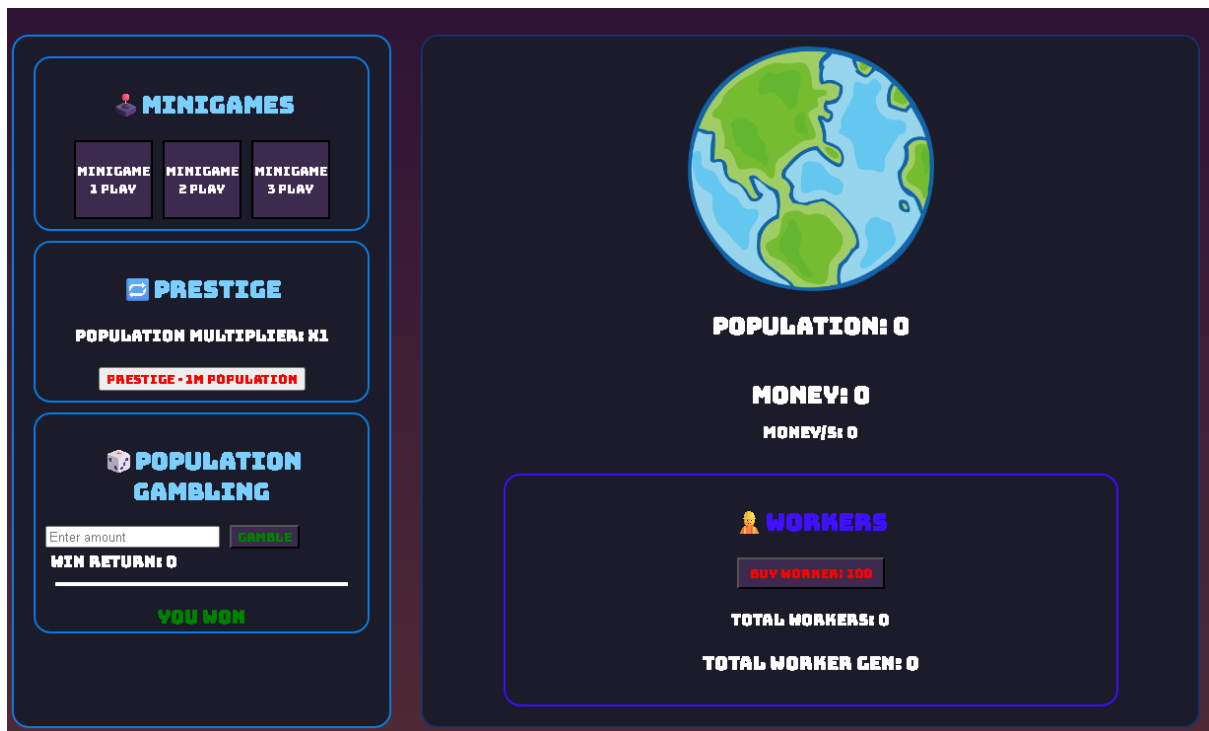
I then make the program wait 2 seconds, after which I hide both the win messages and call update_game_displays() so that the population display will be reset to the current population, as it would have changed after the gamble. The reason I hide both the win and loss messages, is because I am

outside the if statement that checks for a win or loss, so I don't know if the player has won or lost, so I just hide both the messages. Also, the reason I didn't use setInterval(), but setTimeout() instead, is because setInterval() will start an infinite loop that runs every 2 seconds, whereas I just want to create a 2 second wait, and then run the code once.

I tested this to see if it worked:



The gambles were working successfully, I was gaining the population that was displayed by the win return, when I won, and the win message appeared for 2 seconds. The only problem was that it was letting me gamble even when I didn't have enough to:



I realised that this was because I was not performing a validation check in the event handler for the click event listener for the gamble button, I was only performing it in the event handler for the input event listener for the gamble amount input form. Therefore, the input was being validated, but it didn't stop the gamble() function from being called. Therefore, I implemented the same validation check before calling the gamble() function:

```
document.getElementById("gamble-button").addEventListener("click", function(event) {  
    if(!isNaN(game_data.bet) && game_data.bet > 0) {  
        gamble()  
    }  
})
```

Here, I check if the bet is a number (not not a number) and if it is bigger than 0, and only in that case do I call the gamble() function. Now, the only time a gamble should be allowed is if there is something entered in the form, which is above 0 and less than or equal to the population:



No gambles were allowed when 0 was entered



No gambles were allowed when nothing was entered



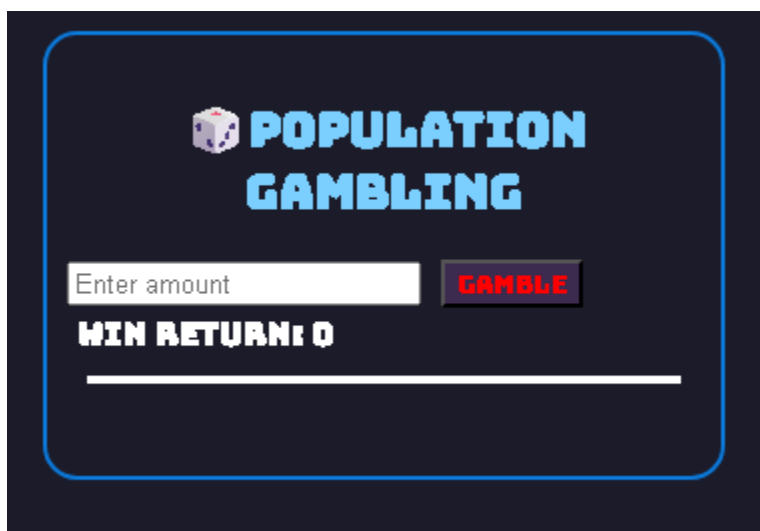
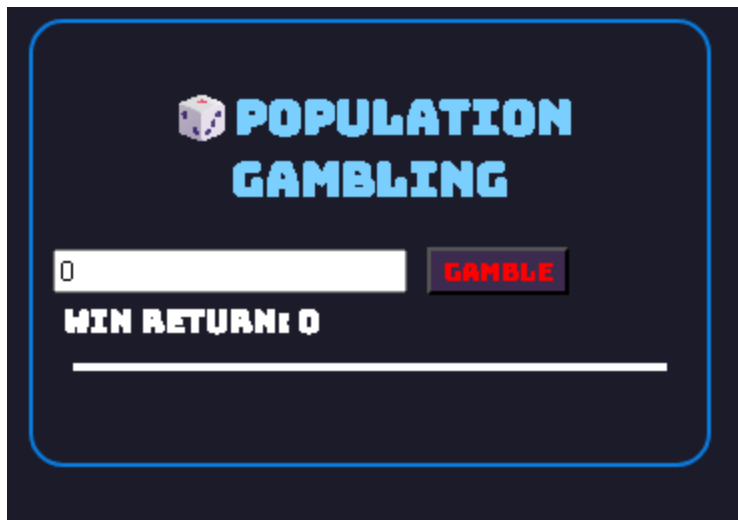
And no gambles were allowed then the amount entered was more than the current population

The only thing I noticed was that the gamble button was only red when the gamble amount entered was more than the current population, not when there was nothing entered or the amount entered was 0. I need to make it so the button is always red, unless a gamble is allowed. Therefore, I will modify my `check_gamble()` function so that it also checks to ensure the bet is a number and bigger than 0 for it to set the gamble button text colour to green:

```
if(game_data.population >= game_data.bet && !isNaN(game_data.bet) && game_data.bet > 0) {  
    document.getElementById("gamble-button").style.color = "green"  
} else {  
    document.getElementById("gamble-button").style.color = "red"  
}
```

Now, the text colour will only be set to green if the population is greater than the bet and if the bet is a number and if the bet is bigger than 0.

I tested this to see if it worked:



Now, with nothing or 0 entered in the form, the gamble button text colour stays red, so it is working.

Another small problem I had was that the population doesn't immediately update after a gamble, since I call the `update_game_displays()` function in the `setTimeout` loop, so it takes 2 seconds. I can't call it outside the loop, however, or the win/loss message will be hidden, as this happens in the `update_game_displays()` function. It will ignore the loop wait time. Therefore, I decided to remove this call, since the only reason I have it is to update population, and then to simply set the population display in the `gamble()` function, after the loop:


```

    setTimeout(() => {
        document.getElementById("win").hidden = true
        document.getElementById("loss").hidden = true
    }, 2000)

    document.getElementById("population").textContent = formatNum(game_data.population)
}

```

Finally, I noticed that after a gamble, the money per second did not update, until the planet was clicked where it was re-calculated, therefore I updated it at the bottom of the gamble() function, and then displayed it:

```

// Recalculates money per second, then updates money per second and population displays in HTML
game_data.money_per_second = game_data.population * (0.01 * (1 + (game_data.upgrades.income_boost.level * 10) / 100))
game_data.money_per_second = Math.round(game_data.money_per_second * 100) / 100 //Removes extra, unwanted decimal places
document.getElementById("money-per-second").textContent = game_data.money_per_second
document.getElementById("population").textContent = formatNum(game_data.population)

```

Now, after testing it, the money per second updated correctly, however I was noticing that after entering a number above 1 trillion, and then entering a smaller number like 1, I lost almost every gamble. This was probably because the win chance was not getting reset after being set to 0.05. Therefore, I set them to their default values at the start of the calculate_gamble_stats() function, to ensure they were being reset before getting re-calculated:

```

function calculate_gamble_stats() {
    let betLevel = 1000
    game_data.win_chance = 0.5
    game_data.win_multiplier = 1.5
}

```

I also decided to add another display, under win return, for the win chance. This makes it easier for me to test, as I won't have to use the console to constantly check whether the win chance was being set and reset correctly. It will also make it clear to the player that the win chance reduces with higher bets, since this is not explained at all. I started by duplicating

the win return display, and changing the id to “win-chance” and the text to “Win chance: ”:

```
<h4 style="margin: 5px; text-align: left;">Win return: <span id="win-return">0</span></h4>  
<h4 style="margin: 5px; text-align: left;">Win chance: <span id="win-chance">0</span></h4>
```

I then decided to make the text colour of the win return green, as it shows the reward for a win, so should be a green which is a colour associated with money and rewards. I then made the colour of the win chance start at green when at 50%. I will then change it to a orange from 45% to 35%, then red from 30% to 5%, which will be done in the JavaScript. This makes it more clear that the chance is decreasing, switching from a colour associated with positivity and reward, down to red which is a colour associated with loss. I started by setting the colours of the displays in the HTML:

```
<h4 style="margin: 5px; text-align: left; color: green">Win return: <span id="win-return">0</span></h4>  
<h4 style="margin: 5px; text-align: left; color: green">Win chance: <span id="win-chance">0</span></h4>
```

I used inline css, as it is quick and simple, and I am only making one modification to the elements so would not be worth doing it in the CSS.

I then need to set the value of the win-chance display in the JavaScript. I will do this in the `update_game_displays()` function since this is called when the page loads, so this display will be updated instantly when the player loads the game:

```
document.getElementById("win-chance").textContent = `${game_data.win_chance * 100}%`  
}
```

I used string interpolation so I could add the % symbol on the end, as the chance is a percentage. The reason I multiplied the `win_chance` by 100, is because it's a decimal number, like 0.5 which would represent 50%, so multiplying by 100 gets the correct percentage value.

I tested this to see if it worked:



After loading the page, the win chance display updated to 50%, which is the correct, default value.

After this, I needed to update the win chance display when and input was typed in the form, so that it updates in real time. Therefore, I will update it in the event handler for the input event listener for the gamble amount input form:

```
document.getElementById("gamble-amount").addEventListener("input", function(event) {  
  // Checks if player has entered a non-numeric character  
  if(isNaN(this.value)) {  
    document.getElementById("win-return").textContent = "Enter valid number"  
  }  
  // Checks if player has entered 0  
} else if(this.value == 0) {  
  // Sets win return to 0 if player enters 0. Can't win if bet is 0.  
  document.getElementById("win-return").textContent = "0"  
} else {  
  // Sets player bet to the value in the input form  
  game_data.bet = this.value  
  check_gamble() // Updates gamble button text colour depending on whether the player can afford  
  calculate_gamble_stats() // Calculates the win return based on the input#  
  // Updates win return and win chance display in HTML  
  document.getElementById("win-return").textContent = formatNum(game_data.win_return)  
  document.getElementById("win-chance").textContent = `${(game_data.win_chance * 100)}%` ←
```

In the else block, I set the win chance to the win chance in game data, in the same way as before. I tested this to see if it worked:



Now, after entering numbers greater than 1000, the win chance decreased correctly, so it was all working. I then updated it to change the colour depending on the win chance. To do this, I realised that I will need to update the whole h4 tag, not just the element with the “win-chance” id, since this is just the span that holds the win chance value. Therefore, I added an id to the h4 tag, “win-chance-display”, so I could access all the text:

```

<h4 style="margin: 5px; text-align: left; color: green" id="win-chance-display">Win chance: <span id="win-chance">0</span></h4>
</div>

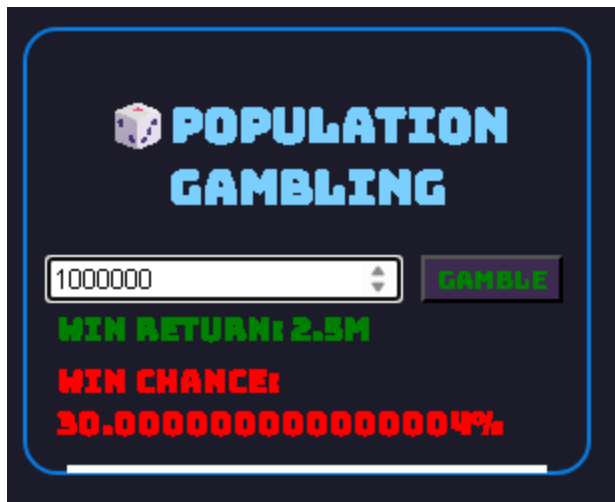
```

I then set the colour of this according to the value of win chance in the JavaScript, in the event handler for the input event listener for the gamble amount input form:

```
// Updates win return display text colour
// 50%+ Green
// 35-45% Orange
// <35% Red
if(game_data.win_chance >= 0.5) {
    document.getElementById("win-chance-display").style.color = "green"
} else if(game_data.win_chance >= 0.35 && game_data.win_chance <= 0.45) {
    document.getElementById("win-chance-display").style.color = "orange"
} else if(game_data.win_chance < 0.35) {
    document.getElementById("win-chance-display").style.color = "red"
}
```

I check if the win_chance is bigger equal to 0.5, if not then if it is between 0.35 and 0.45, and if not then if it is less than 0.35, and I set it to either green, orange or red respectively. I tested this to see if it worked:





The colour was now being updated correctly, however once the win chance hit 30%, there were lots of extra 0s added on the end, so I used my method to remove these (by rounding the result of the value multiplied by 100, and then dividing the result) at the bottom of the `calculate_gamble_stats()` function:

```
// Sets win return to the amount the player bet, multiplied by the win multiplier calculated
game_data.win_return = game_data.bet * game_data.win_multiplier
game_data.win_chance = Math.round(game_data.win_chance * 100) / 100
}
```

This then fixed the problem:



After this, I developed the gamble luck upgrade, which increases the win chance by 5%, until it is 75% at which the upgrade cannot be purchased. I started by adding a nother check in the upgrade function, to check if the name of the upgrade being purchased was “gamble luck”:

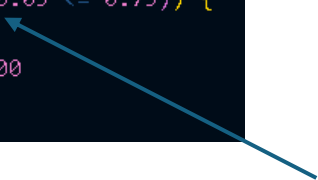
I then added 0.05 to the win chance in the else if block, which represents a 5% increase, and used my method to remove the extra digits:

```
document.getElementById( "prestige-cost" ).textContent = formatNum( game_data.prestige_cost )
} else if (upgrade_name == "gamble_luck") {
}
```



```
} else if (upgrade_name == "gamble_luck") {
    game_data.win_chance += 0.05 // Increases win chance by 5%
    game_data.win_chance = Math.round(game_data.win_chance * 100) / 100
}
```

```
} else if (upgrade_name == "gamble_luck" && (game_data.win_chance + 0.05 <= 0.75)) {
    game_data.win_chance += 0.05 // Increases win chance by 5%
    game_data.win_chance = Math.round(game_data.win_chance * 100) / 100
}
```



I then added the check to ensure the win chance won't go above 0.75:

The reason I check if the win_chance + 0.05 is more than 0.75, instead of just checking if it is equal to 0.75, is because the win chance somehow got just below 0.75, e.g. 0.73 due to a rounding error or something else like that, then a purchase would still not be allowed, since 0.73 + 0.05 is greater than 0.75. This just makes it more error-proof.

Finally, I updated the win-chance display in the HTML, so it would display the updated win chance:

```

} else if (upgrade_name == "gamble_luck" && (game_data.win_chance + 0.05 <= 0.75)) {
  game_data.win_chance += 0.05 // Increases win chance by 5%
  game_data.win_chance = Math.round(game_data.win_chance * 100) / 100
  document.getElementById("win-chance").textContent = `${(game_data.win_chance * 100) * 100 / 100}%`
}

```

I then tested this to see if it worked:



After purchasing the update, the win chance correctly increased to 55%, however there were still extra digits added on the end, and I realised that this was because I set the value of win_chance multiplied by 100, which is likely where they are coming from. Therefore, I used the rounding method again, where I set the value of the win chance display:

```

} else if (upgrade_name == "gamble_luck" && (game_data.win_chance + 0.05 <= 0.75)) {
  game_data.win_chance += 0.05 // Increases win chance by 5%
  game_data.win_chance = Math.round(game_data.win_chance * 100) / 100
  document.getElementById("win-chance").textContent = `${Math.round((game_data.win_chance * 100) * 100) / 100}%`
}

```

I tested this to see if it worked:



Now, all extra digits were removed after purchasing the upgrade. I then also did this everywhere I set the win chance display, to ensure the problem doesn't happen at all:

```
document.getElementById("win-chance").textContent = `${Math.round((game_data.win_chance * 100) * 100) / 100}%`
```

```
document.getElementById("win-chance").textContent = `${Math.round((game_data.win_chance * 100) * 100) / 100}%`
```

The only problem now was that after purchasing the upgrade, and then entering a number in the input form, the win chance reset back to its default:



I realised that this was because I set the win chance (and win multiplier) to its default value at the start of the `calculate_gamble_stats()` function, which ignores the upgrade. Therefore, I add on the value of gamble luck upgrade level multiplied by 0.5, instead of just setting it 0.5:

```
// function that calculates gamble win chance, win multiplier and win return
function calculate_gamble_stats() {
  let betLevel = 1000
  game_data.win_chance = 0.5
  // Adds on win chance boost from gamble luck upgrade (+5% per upgrade)
  game_data.win_chance += game_data.upgrades.gamble_luck.level * 0.05
  game_data.win_multiplier = 1.5
}
```

I tested this to see if it worked:



Now, the win chance increase remained, after entering a new number.

After this, I checked to see if the game prevented any more purchases of the gamble luck upgrade after the win chance hit 75%:



After the win chance became 75%, it did not increase, however I still allowed me to purchase the upgrade, with the upgrade number and price

increasing. This was a similar problem that I had with worker efficiency, so I solved it in the same way. I added a boolean variable before the upgrade function, called `gamble_luck_allowed` and set it to `true`:

```
var worker_efficiency_allowed = true
var gamble_luck_allowed = true
// Function that handles purchasing upgrades
function upgrade(upgrade_element) {
```

I then added a check at the start of the function, to check if the upgrade name was equal to "gamble_luck" and the boolean value was equal to `false`. If so I returned `null`:

```
if(game_data.money >= game_data.upgrades[upgrade_name].price) {
  // Prevents purchasing worker efficiency if worker speed is at 0.5s (cap)
  if(upgrade_name == "worker_efficiency" && worker_efficiency_allowed == false) {
    return null
  }
  // Prevents purchasing gamble luck if win chance is at 75% (cap)
  if(upgrade_name == "gamble_luck" && gamble_luck_allowed == false) {
    return null
  }
}
```

This is the exact same as for worker efficiency, and it means the program will exit out of the function if the upgrade being purchased is gamble luck, and the upgrade is capped, thus preventing the upgrade from being purchased. Finally, I added an if statement to check if the win chance + 0.05 is greater than 0.75, and if so I set `gamble_luck_allowed` to `false`. This is, again, the same as for worker efficiency, so I put the if statement under it:

```
// Caps win chance at 75%, prevents further upgrades once reached
if(game_data.win_chance + 0.05 > 0.75) {
  gamble_luck_allowed = false
}

// Takes upgrade price of money
```

Now, the player won't be able to purchase the upgrade anymore, once the win_chance hits 0.75.

Finally, I developed the win streak upgrade, where the the win chance increases by 5% per upgrade, per consecutive gamble win. I started by incrementing the win streak variable when the player wins a gamble, and setting it to 0 when they lose, in the gamble() function:

```
if (random_roll < game_data.win_chance) {  
    // Removes bet from population, and then adds it back with the wi  
    game_data.population -= game_data.bet  
    game_data.population += game_data.bet * game_data.win_multiplier  
    // Shows win message  
    document.getElementById("win").hidden = false  
    // Plays win sound  
    game_data.sounds.win.play()  
  
    game_data.win_streak ++ ←  
  
} else {  
    game_data.population -= game_data.bet  
    // Shows loss message  
    document.getElementById("loss").hidden = false  
    // Plays loss sound  
    game_data.sounds.lose.play()  
  
    game_data.win_streak = 0 ←  
  
}
```

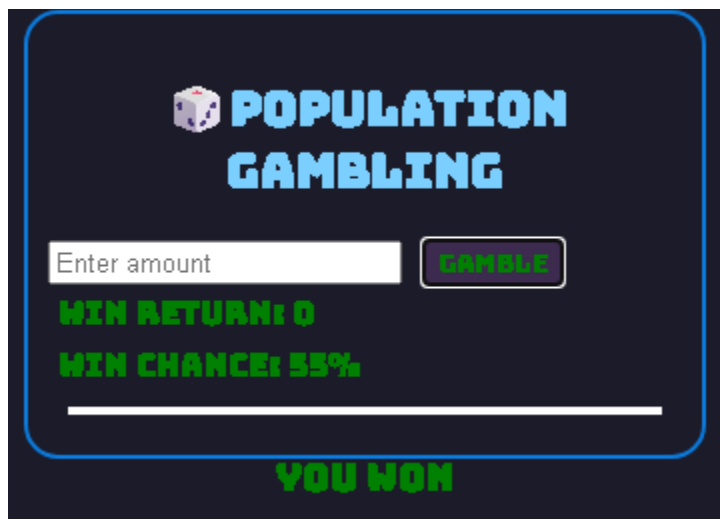
The reason I did this, was because the win streak variable will now increase each the player wins a gamble, but as soon as they lose one gamble, they will instantly lose their streak because the variable is set to 0.

I then need to add to the win chance based on the win_streak. I will do this in the calculate_gamble_stats () function first, as I already set win chance here:

```
function calculate_gamble_stats() {  
  let betLevel = 1000  
  game_data.win_chance = 0.5  
  // Adds on win chance boost from gamble luck upgrade (+5% per upgrade)  
  game_data.win_chance += game_data.upgrades.gamble_luck.level * 0.05  
  // Increases win chance by 5% per win streak upgrade, per consecutive gamble win  
  game_data.win_chance += (game_data.upgrades.win_streak.level * 0.05) * game_data.win_streak
```

This multiplies the win streak upgrade level by 0.05, then multiplies it by the win streak, since the upgrade increases win chance by 0.05 (5%), per upgrade, per consecutive win. I also

I tested this to see if it worked:



Now, after purchasing the win streak upgrade, a successful gamble increased the win chance 5%, and then a second consecutive gamble increased it by 5% again. However, I realised that just winning one gamble increased the win chance, whereas it should start increasing at 2, because a single gamble win is not a streak of consecutive wins. Therefore, I added an if statement to check if the win streak was above 1, and only then will the win chance be updated:

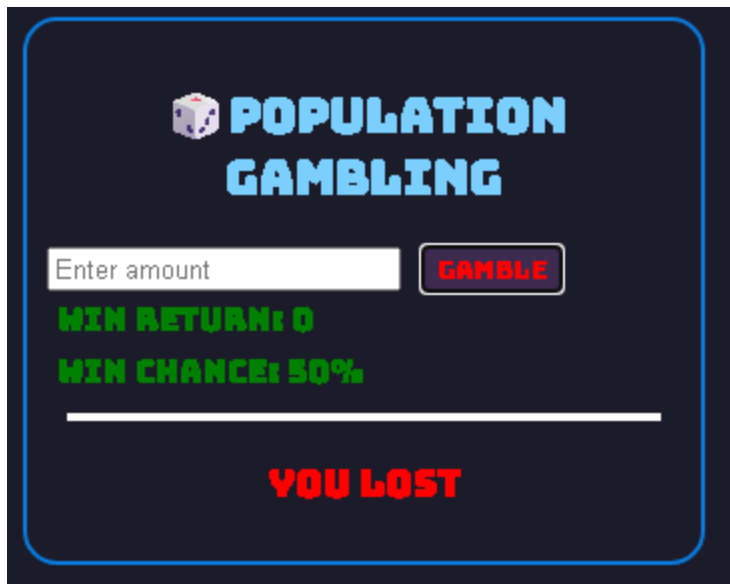
```
// Function that calculates gamble win chance, win multiplier and win return
function calculate_gamble_stats() {
  let betLevel = 1000
  game_data.win_chance = 0.5
  // Adds on win chance boost from gamble luck upgrade (+5% per upgrade)
  game_data.win_chance += game_data.upgrades.gamble_luck.level * 0.05

  // Increases win chance by 5% per win streak upgrade, per consecutive gamble win
  if(game_data.win_streak > 1) {
    game_data.win_chance += (game_data.upgrades.win_streak.level * 0.05) * (game_data.win_streak-1)
  }
}
```

Also, the reason I subtract 1 from the win streak, is because I want to ignore the first win, since it is not part of the win streak until there is a second win. If I did not do this, by the second win the win chance would increase to 60%, as it will have increased by 5% for the first and second win. I only want it to start increasing by the second win, and for future consecutive wins.

The last problem was that the win message was outside of the population gambling div, because I added the win chance display. Therefore, I increased the height of the div in the CSS, from 200px to 250px:

```
.population-gambling {
  height: 250px;
}
```



Now, it correctly fits the win/loss messages

Finally, I decided to increase the price of both the gamble luck and the win streak upgrades to 500k, as I want them to be much harder to obtain due to how powerful they are:

```
gamble_luck: {  
  price: 500000, ←  
  level: 0,  
  price_multiplier: 0.1,  
  base_multiplier: 0.1,  
  multiplier_increase: 0.125,  
  base_price: 500000, ←  
  max_purchase: 0,  
},  
win_streak: {  
  price: 500000, ←  
  level: 0,  
  price_multiplier: 0.1,  
  base_multiplier: 0.1,  
  multiplier_increase: 0.07,  
  base_price: 500000, ←  
  max_purchase: 0,  
},
```

Throughout testing this stage, I gave myself 1 trillion population to make sure I could easily afford upgrades and test large gambles, and while I was testing I noticed the prestige population multiplier was very large. Since it increases by 0.1 per 100000 population, it was 1 million with a population of 1 trillion. This would be way too overpowered, so I decided to cap the population multiplier at x50:

```
// Infinite loop that updates prestige multiplier
setInterval(() => {
  // Increases prestige multiplier by 0.1 per 100k population, plus the boost from the population multiplier upgrade (+0.5 per upgrade)
  game_data.next_prestige_multiplier = (1 + 0.1 * (game_data.population / 100000) + (0.5 * game_data.upgrades.population_multiplier_boost.level)).toFixed(1)
  // Caps multiplier at 50
  if(game_data.next_prestige_multiplier > 50) {
    game_data.next_prestige_multiplier = 50
  }
  //Updates population multiplier display in HTML
  document.getElementById("population-multiplier").textContent = game_data.next_prestige_multiplier
}, 1000)
```

I did this by adding an if statement in the infinite loop that calculates the next prestige multiplier, that checks if it is greater than 50, and if so just sets it to 50. This ensures it never goes above 50. Now, I need to prevent the population multiplier boost upgrade from being purchases once the population multiplier reaches 50, so I added an if statement at the start of the upgrade() function which returns null if so:

```
if(game_data.money >= game_data.upgrades[upgrade_name].price) {
  // Prevents purchasing worker efficiency if worker speed is at 0.5s (cap)
  if(upgrade_name == "worker_efficiency" && worker_efficiency_allowed == false) {
    return null
  }
  // Prevents purchasing gamble luck if win chance is at 75% (cap)
  if(upgrade_name == "gamble_luck" && gamble_luck_allowed == false) {
    return null
  }
  // Prevents purchasing population multiplier boost if prestige multiplier has reached x50 (cap)
  if(upgrade_name == "population_multiplier_boost" && game_data.next_prestige_multiplier >= 50) {
    return null
  }
}
```

I also decided to add a warning message, using alert(), to let the player know why they can't purchase any more of the upgrade. I did this for worker efficiency and gamble luck as well:

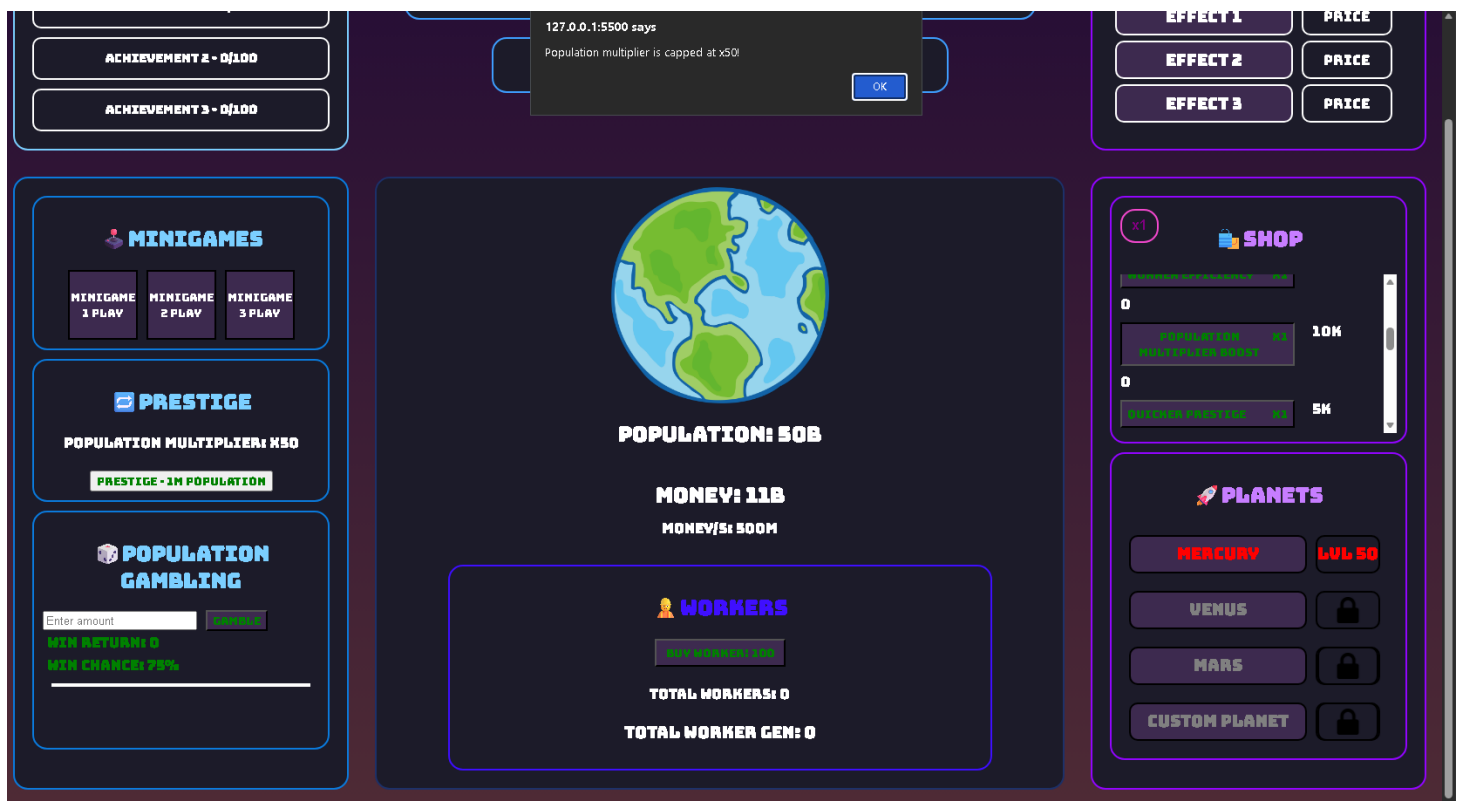

```

if(game_data.money >= game_data.upgrades[upgrade_name].price) {
    // Prevents purchasing worker efficiency if worker speed is at 0.5s (cap)
    if(upgrade_name == "worker_efficiency" && worker_efficiency_allowed == false) {
        alert("Worker speed is capped at 0.5s!")
        return null
    }
    // Prevents purchasing gamble luck if win chance is at 75% (cap)
    if(upgrade_name == "gambleLuck" && gambleLuck_allowed == false) {
        alert("Win chance is capped at 75%")
        return null
    }
    // Prevents purchasing population multiplier boost if prestige multiplier has reached x50 (cap)
    if(upgrade_name == "population_multiplier_boost" && game_data.next_prestige_multiplier >= 50) {
        alert("Population multiplier is capped at x50!")
        return null
    }
}

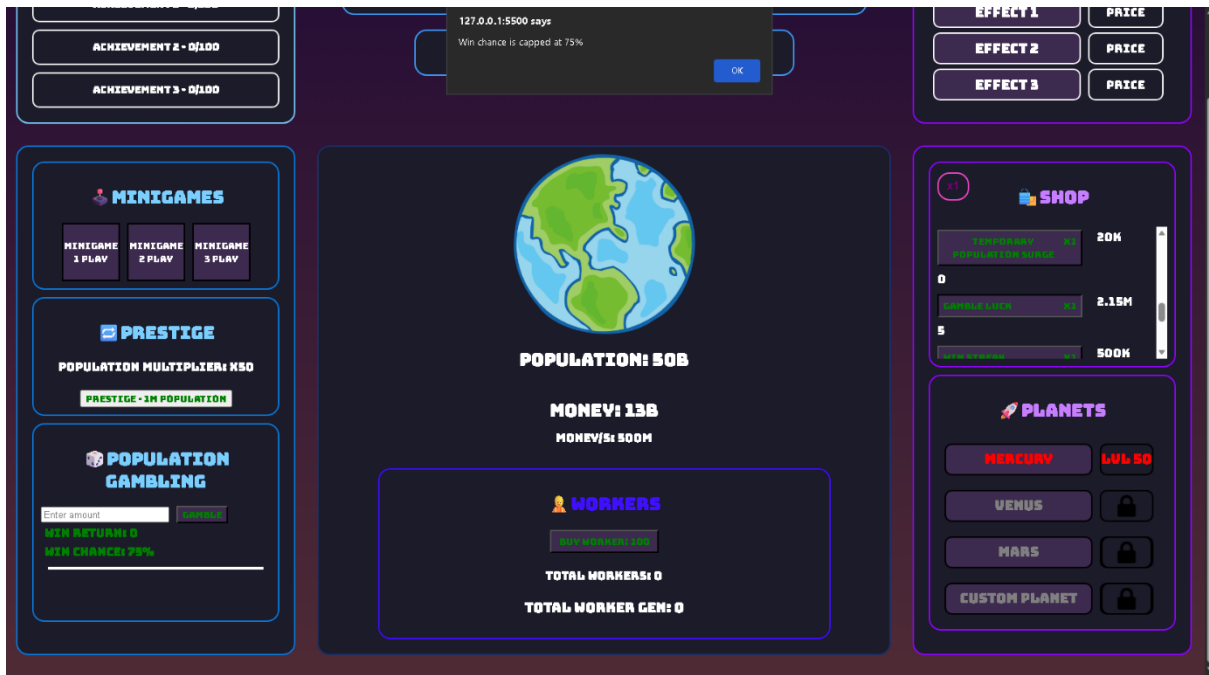
```

This makes it obvious, since it is not explained, why the player cannot purchase these upgrades when they are capped.

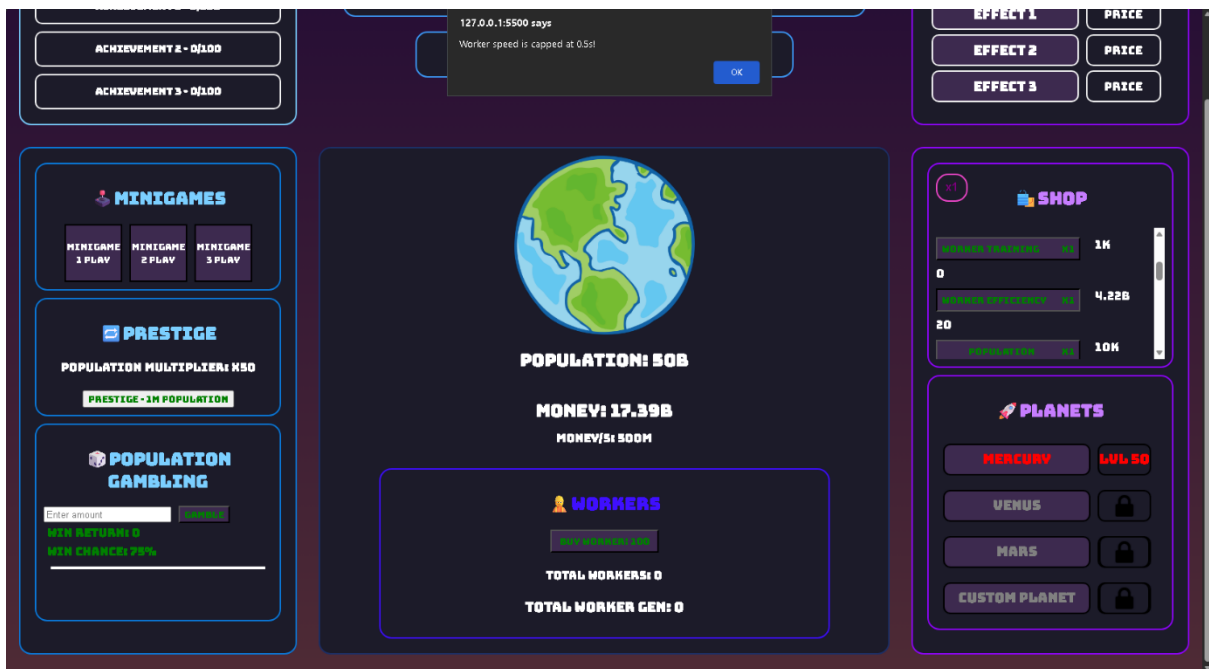
I tested this to see if it worked:



Attempting to purchase population multiplier boost when population multiplier = 50 shows the correct warning



Attempting to purchase gamble luck when win chance = 75% showed the correct warning



Attempting to purchase worker efficiency when workers are already generating at 0.5s showed the correct warning

Testing:

This is my test plan for this stage, with some modifications:

- I have added a test for the gamble button text colour, to test whether it is green when the player has enough population to gamble the amount they entered, and red when they don't. I have put this in now because it was a feature I added in the design for this stage, since it made sense and I had done it for the worker button, upgrade buttons and prestige button. I didn't design it in the stages of development however, which is why I have added the test for it to the test plan here
- I have also added a test for the win chance display, to test whether its colours update correctly. Again, I have added this test here because it was a feature I decided to implement in the development of this stage, not in the stages of development.
- I have also added a test for the actual win chance value, to ensure it decreases correctly as the bet amount increases, I had not decided to decrease the win chance as the bet amount increased, in my stages of development, but instead in the design of this stage.
- I have also added 2 additional tests for this stage, for the gamble luck and win streak upgrades. In the test plan I made in the stages of development section, I did not have these tests since I had not planned any specific upgrades at that point.
- **NOTE:** I don't have a test for the win and loss sounds, since these can be heard in the first test, so it is clear that they are working

Test no.	Description	Test type	Test data	Expected Result	Actual result
1	Does the game correctly show how		Enter an amount of population to gamble in	The win return display should	The win return display showed 150

	much population the player will win once they enter their gamble		the gambling section and look at the return display	display a number that is the population entered multiplied by the win multiplier, which is x1.5	with a bet of 100 (x1.5) and then 1500 with a bet of 1000 (x1.5) Passed  SEE STAGE 7 TEST 1.mp4 in evidence folder
2	Does the player receive the correct amount of population when they win a gamble		Gamble a certain amount of population , remembering the population before the gamble, until a win. Observe the increase	The population, subtracted by the amount gambled, should increase by the amount entered multiplied by the win multiplier (1.5)	With a bet of 10 and a population of 10, the population increased to 15 after a successful gamble. This means that the player earned their population back times 1.5 (15) Passed  SEE STAGE 7 TEST 2.mp4

					in evidence folder
3	Does the player lose the correct amount of population when they lose a gamble		Gamble a certain amount of population, remembering the population before the gamble, until loss.	The population should decrease by the amount that was gambled	With a population of 10 and a bet of 10, the population decreased to 0 after an unsuccessful gamble. This means the player lost everything they bet Passed ☒ SEE STAGE 7 TEST 3.mp4 in evidence folder
4	Does the player win a random number of gambles		Gamble 10 times and count the number of wins and losses	Around 50% of gambles should be wins, and 50% should be losses. There should be no pattern	After 10 gambles, 6 were lost and 4 were won, so around 50% with no common pattern. Passed ☒

					SEE STAGE 7 TEST 4.mp4 in evidence folder
5	Is the player prevented from gambling when they have entered more population than they currently have		With 0 population, enter 1 in the gamble amount form and try to gamble	Nothing should happen. There should be no win or loss message, and the population should not increase or decrease	Nothing happened, no win or loss message appeared and the population stayed the same, so the gamble was not allowed Passed ☒ SEE STAGE 7 TEST 5.mp4 in evidence folder
6	Does the gamble button text colour turn green when the player has enough population to gamble the amount they		With 0 population, enter 1 into the gamble amount form and observe the colour of the gamble button text. Then click the planet	The text colour of the button should start at red, and then turn green after clicking the button	With a population of 0 and a gamble of 1, the gamble button text colour was red. After clicking the planet and increasing

	entered, and red when they don't		button once and observe again.		the population, it became green. Passed  SEE STAGE 7 TEST 6.mp4 in evidence folder
7	Is the win chance display green when the win chance is above 50%, and orange when it is between 35 and 45%, and red when it is less than 35%		Enter 1 into the gamble amount form, and observer the colour of the win chance display. Then enter 1000 and observer again. Then enter 1000000 and observer again.	The colour of the win chance display should be green when 1 is entered, orange when 1000 is entered, and red when 1000000 is entered	After entering a bet of 1, the win chance display text colour was green, then orange with a bet of 1000, then red once the bet reached 1000000. Passed  SEE STAGE 7 TEST 7.mp4 in evidence folder

8	Does the win chance decrease correctly as a higher bet is entered, stopping at 5%		Enter 1000 into the gamble amount form and observe the win chance. Keep adding a 0 onto the end of the number and observe the win chance value. Keep adding 0s when it hits 5% and see if it decreases anymore	The value of the win chance should decrease from 50% to 45% when 1000 is entered, and then by 5% each time a 0 is added onto the end of the number in the form. Once at 5%, the win chance should stay the same even when more 0s are added.	Each 0 added onto the bet after 1000 reduced the win chance by 5%, and once it stopped at 5% Passed ☒ SEE STAGE 7 TEST 8.mp4 in evidence folder
9	Does the gamble luck upgrade increase the win chance correctly and stop at the cap		Purchase the gamble luck upgrade and observe the win chance display. Keep attempting to purchase once the win chance hits 75%	The win chance should increase by 5% each upgrade, until 75%, after which no more purchases should be	Purchasing the gamble luck upgrade increased the win chance by 5%, until 75% where purchasing showed the warning

				allowed and a warning should display	message and did not allow any more purchases Passed ☒ SEE STAGE 7 TEST 9.mp4 in evidence folder
10	Does the win streak upgrade increase win chance correctly with each consecutive gamble win		Purchase the gamble luck upgrade and gamble until 2 consecutive gamble wins. Observe the win chance display. Gamble again (and win) and observe the win chance to confirm	The win chance display should increase by 5% from its previous value. It should then increase by 5% after the 3 rd consecutive gamble win	Once the win streak upgrade was purchased, after 2 consecutive gamble wins, the win chance increased by 5%, and then 5% again after the 3rd consecutive win. Passed ☒ SEE STAGE 7 TEST 10.mp4

					in evidence folder
--	--	--	--	--	-----------------------

Stage review:

Overall, I am quite happy with the development of this stage. I correctly implemented the population gambling feature, with all the gamble stats being correctly calculated, correct population rewards granted, with the win/loss messages and sounds. There were several errors throughout the process, such as formatting errors and forgetting validation, however these were minor and did not take long to fix. The overall process took longer than expected, however this was because it is a relatively complicated feature, and I also had to develop the gamble upgrades on top of it all. Therefore, I am generally happy with the development.

Stage 8 – Achievements:

Features:

- Achievements with a progress bar that increases as the player increases as they complete more of the achievement
- Progress bar will be an east, linear gradient from #3CFF6B to #6AFFB8 (Chosen from bing colour picker).
- There is also a text display of the achievement completion and achievement goal as a number, which will be updated as achievements are completed as well.
- Achievements will grant gems and xp upon completion. The rewards will scale each time the achievement is completed
- The 3 achievements I will have are:
 - Click the planet – Goal starts at 100. Scales
 - Earn money – Goal starts at 100
 - Win gambles in a row – Goal starts at 5
- Each achievement goal will scale by x5 each time the achievement is completed
- The gem reward for click the planet and earn money will start at 5, but for win gambles in a row it will be 10.
- The xp reward for click the planet and earn money will start at 25, but for win gambles in a row it will be 50.
- The rewards for win gambles in a row will be higher since it is a much harder achievement.

- All rewards for achievement completion will scale by x1.5 each time the achievement is completed
- The goal of each achievement is the number that the player needs to reach to complete it. For example, the first click achievement will be “Click the planet 100 times”
- The achievement goal will increase each time the achievement is completed, e.g. Click the planet 100 times → Click the planet 500 times
- Achievement gem upgrade, that will increase gem rewards from all achievements by 2% (rounded up), per upgrade
- Achievement xp upgrade that will increase xp rewards from all achievements by 10%, per upgrade
- All achievements and their progress will be kept after prestige
- I decided to not to implement the win popup, which shows the gems and xp the player has won after completing the achievement, as it would take too long and is not worth it. The player will see their level increase from the xp, and their gems increase from the gem display, so I don't need to tell them how much they've earned.

Data dictionary:

Variable	Data type	Purpose
game_data.achievements {}	Dictionary	This will store all 3 achievements as nested dictionaries, and their associated attributes, similar to how

		upgrades{} stores each upgrade and its variables
achievements.clicks{}	Dictionary	Stores the clicks achievement as a dictionary
achievements.money{}	Dictionary	Stores the money achievement as a dictionary
achievements.gambles{}	Dictionary	Stores the gambles achievement as a dictionary
achievements.clicks.progress achievements.money.progress achievements.progress	Int	This will store the progress the player has made for each achievement. E.g. if they have clicked the planet 20 times, the first achievement progress will be 20
achievements.clicks.goal achievements.money.goal	Int	This will store the

achievements.goal		achievement goal for each achievement, which is the amount it takes to complete it
achievements.clicks.gems achievements.money.gems achievements.gems	Int	This will store the gems that each achievement will reward for completion
achievements.clicks.xp achievements.money.xp achievements.xp	Int	This will store the xp that each achievement will reward for completion
achievements.clicks.element achievements.money.element achievements.element	Document	This will store the HTML element for the div of each achievement, so the different elements inside them can be queried to update them
game_data.gems	Int	This will store the gems of the player,

		which will be used to purchase effects.
game_data.xp	Int	This will store the xp of the player, which will be used to increase their level later on.

HTML and CSS modifications:

- I will have to make some modifications to the HTML or CSS, so I can correctly update the achievements display
- I have not previously planned my modifications to my HTML for the design of a stage, however I will have to make quite a few for this stage

1. Add ID to each achievement div:

- Click the button achievement: "achievement-clicks"
- Earn money achievement: "achievement-money"
- Win gambles in a row achievement: "achievement-gambles"

2. Add a div for the progress bar:

- Class = "progress-fill"
- ID = "achievement-[achievement-name]-progress-fill"

3. Add 2 spans in the <h3> tag for each achievement:

- 0
- 0

- Spans will be split by a “/”, so the format will be x/y, e.g. 0/100

4. Style progress-fill div:

- 10px border radius (same as each achievement div)
- East (90deg in CSS) gradient, from #3CFF6B to #6AFFB8
- 0 left margin so it starts at the left of the achievement, rather than the centre
- 0 z-index so it is behind the text
- 0% width, since it starts empty
- 100% height so it fills the whole achievement div height
- Position absolute, so it stays positioned inside the achievement div (relative to it)

5. Add necessary styles to achievement divs:

- Overflow hidden so its contents, the progress fill bar, will not overflow it
- Position relative so it stays in its normal position

6. Set z-index of h3 elements inside each achievement div so it is above the progress bar

Pseudocode:

Updating achievement completion:

```
FUNCTION update_achievement(achievement_type, amount)

    Const ACHIEVEMENT =
        game_data.achievements.achievement_type

    ACHIEVEMENT.progress += amount

    IF ACHIEVEMENT.progress >= ACHIEVEMENT.goal THEN
```



```

        ACHIEVEMENT.progress = ACHIEVEMENT.goal
        complete_achievement(achievement_type)
    ELSE
        ACHIEVEMENT.element.progress-bar.HTML.width
        = (ACHIEVEMENTS.progress /
        ACHIEVEMENT.goal) * 100

        ACHIEVEMENT.element+“-progress”.HTML.text =
        ACHIEVEMENT.progress
    ENDIF
ENDFUNCTION

```

- I will have a function that is called everytime an achievement needs to be updated (e.g. when the player clicks the planet or earns any amount of money)
- It will take 2 arguments, achievement_type and amount. The achievement_type will be the name of the achievement, e.g. “clicks”, and will be used to access the achievements dictionary in game_data, to get the current achievement being updated.
- It starts by creating a constant variable, which gets the specific achievement dictionary that is needs to be updated. This prevents having to write out game_data.achievements.achievement_type every time I need to access a variable in the achievement dictionary.
- The function will then increase the achievement progress by the second argument value, the amount. This will be how much of the achievement the player has progressed by. For example, if they click the button once, this will be 1.
- It then checks if the achievement is completed, by checking if it's goal is bigger than or equal to its goal, and if so resets the progress to the goal, in case it has gone over (e.g. 101/100 clicks). The complete_achievement function is then called,

passing in the `achievement_type`, which will carry out the code to complete the achievement, reward the player, and scale it.

- If not however, the function will update the width of the progress-fill bar in the HTML to the value of the achievement progress divided by goal, multiplied by 100. This gets a percentage value of the achievement's completion, which will set the width to a percentage of how much the player has completed. It will then update the achievement progress display value, to the value of the achievement progress.

Completing achievements:

```
FUNCTION complete_achievement(achievement_type)

    Const ACHIEVEMENT =
        game_data.achievements.achievement_type

    gems += ACHIEVEMENT.gems
    xp += ACHIEVEMENT.xp

    ACHIEVEMENT.goal *= 5
    ACHIEVEMENT.gems *= 1.5
    ACHIEVEMENT.xp *= 1.5
    ACHIEVEMENT.progress = 0

    ACHIEVEMENT.element+"-completion".HTML.text = 0
    ACHIEVEMENT.element+"-goal".HTML.text =
        ACHIEVEMENT.goal
    ACHIEVEMENT.element+"-progress-fill".HTML.width
= 0
```

ENDFUNCTION

- I will have a function that completes achievements, which starts by creating a constant variable for the current achievement, in the same way as with the previous function
- It will then increase the gems and xp by the reward amount specified in the achievement dictionary
- It will then scale the achievement goal by multiplying it by 5, as per the design, and it will then also scale the gems and xp reward amount by multiplying them by 1.5
- It will then reset the progress of the achievement to 0, as it is being reset
- It will then update the completion and goal spans in the HTML, setting them to their recently updated and scaled amounts in the achievement dictionary. It concatenates -completion and -goal onto the end of the achievement's element name, to access the specific spans for the goal and completion. I will use string interpolation for this in the Javascript as it's cleaner
- Finally, it sets the width of the progress-fill bar in the HTML to 0, as the achievement has been reset so the progress will be 0

Restoring achievements after prestige:

```
FUNCTION prestige()  
    // Start of function  
  
    Object achievements =  
        JSON.parse(JSON.stringify(game_data.achievements  
        ))  
    Int gems = game_data.gems  
    Int xp = game_data.xp  
  
    // game_data reset
```

```
Object.assign(game_data,  
JSON.parse(JSON.stringify(default_game_data)))
```

```
Object.assign(game_data.achievements,  
achievements)
```

```
game_data.gems = gems
```

```
game_data.xp = xp
```

```
// Rest of function
```

- I will add temporary variables in the prestige() function, for all the values used for achievements, including the achievements dictionary, which I will use JSON.parse(JSON.stringify) to make a copy.
- Then, after game_data is reset, I will assign these variables back to their associated keys in game_data, which will restore them.

Achievement gem and xp upgrades:

```
FUNCTION upgrade(upgrade_name)
```

```
// Existing IF checks
```

```
ELSE IF (upgrade_name) == "achievement-gem-  
boost" THEN
```

```
    achievements.clicks.gems =  
    ceil(achievement.clicks * 1.02)
```

```
    achievements.money.gems =  
    ceil(achievement.clicks * 1.02)
```

```
    achievements.gambles.gems =  
    ceil(achievement.clicks * 1.02)
```

```
ELSE IF (upgrade_name) == "achievement-xp-boost"
    THEN
        achievements.clicks.xp *= 1.1
        achievements.money.xp *= 1.1
        achievements.gambles.xp *= 1.1
    ENDIF
// Rest of function
```

- I will add to else if checks in the upgrade function, to check for the achievement gem and achievement xp boost upgrades.
- For the achievement gem boost upgrade, I multiply the gems of each achievement by 1.02, which increases the values by 2%. I also ceil the result, which means rounding upwards, since I don't want the gem value to ever be a decimal because the price of effects (which are purchased with gems) will always be whole numbers.
- For the achievement xp boost upgrade, I multiply the xp of each achievement by 1.1, which increases the values by 10%.

Development:

I started the development of this stage by adding the achievement dictionary, containing the 3 achievement dictionaries with all their values:

```
// ACHIEVEMENTS

achievements : {
  clicks: {
    progress: 0,
    goal: 100,
    gems: 5,
    xp: 25,
    element: document.getElementById("achievement-clicks"),
  },
  money: {
    progress: 0,
    goal: 100,
    gems: 5,
    xp: 25,
    element: document.getElementById("achievement-money"),
  },
  gamble: {
    progress: 0,
    goal: 5,
    gems: 10,
    xp: 50,
    element: document.getElementById("achievement-gambles"),
  },
},
},
```

For each achievement, I added the progress and set it to a default value of 0, as the completion starts at 0. I also added a goal to each one, which is 100 for the first 2, since the achievements are to click the planet 100 times and earn 100 money. I set the goal of the gamble achievement to 5, since it is to win 5 gambles in a row. I also added the gems and xp reward for the achievements, setting them to 5 and 25 respectively, for the first 2, and 10 and 50 for the gamble one since it is a lot harder than the first 2. I also added the element value for each achievement, which stores the document object for the div element for each one, allowing me to update their displays in the functions. I would have done this as well for the upgrades, instead of having an array with all their element id names which I iterate over and it would have made it much simpler, however I only thought about doing this during the design of this stage.

I then added the gems and xp variables, so the achievements could increase them, even though I have not developed the effects or level system yet:

```
    gems: 0,  
    xp: 0,  
  }
```

I then modified the achievements section in the HTML, according to the design (HTML and CSS modifications):

```
<div class="achievements">  
  <h2> 🎮 Achievements </h2>  
  <div class="achievement" id="achievement-clicks">  
    <div class="progress-fill" id="achievement-clicks-progress-fill"></div>  
    <h3>Click planet 100 times - <span id="achievement-clicks-completion">0</span>/<span id="achievement-clicks-goal">100</span></h3>  
  </div>  
  
  <div class="achievement" id="achievement-money">  
    <div class="progress-fill" id="achievement-money-progress-fill"></div>  
    <h3>Earn 100 money - <span id="achievement-money-completion">0</span>/<span id="achievement-money-goal">100</span></h3>  
  </div>  
  
  <div class="achievement" id="achievement-gambles">  
    <div class="progress-fill" id="achievement-gambles-progress-fill"></div>  
    <h3>Win 5 gambles in a row - <span id="achievement-gambles-completion">0</span>/<span id="achievement-gambles-goal">100</span></h3>  
  </div>  
</div>
```

I then styled the progress fill bar and achievement divs according to the design:


```

.progress-fill {
  border-radius: 10px;
  background: linear-gradient(90deg, ■ #3CFF6B, ■ #6AFFB8);
  width: 0%;
  height: 100%;
  position: absolute;
  z-index: 0;
  left: 0;
  overflow: hidden;
}

.achievement {
  border: 2px solid ■ white;
  border-radius: 10px;
  margin: 10px;
  text-align: center;
  height: 45px;
  font-size: 12px;
  position: relative;
}

.achievement h3 {
  z-index: 1;
}

```

I then added a line in the javascript, to increase the width of the bar each time the planet button was clicked, to check if it worked:

```

document.getElementById("planet-button").addEventListener("click", function (event) {
  game_data.achievements.clicks.element.querySelector("#achievement-clicks-progress-fill").style.width = `${game_data.population}`
  game_data.population += 1 * (1 + (game_data.upgrades.click_gauge_level * 2) / 100)
})

```

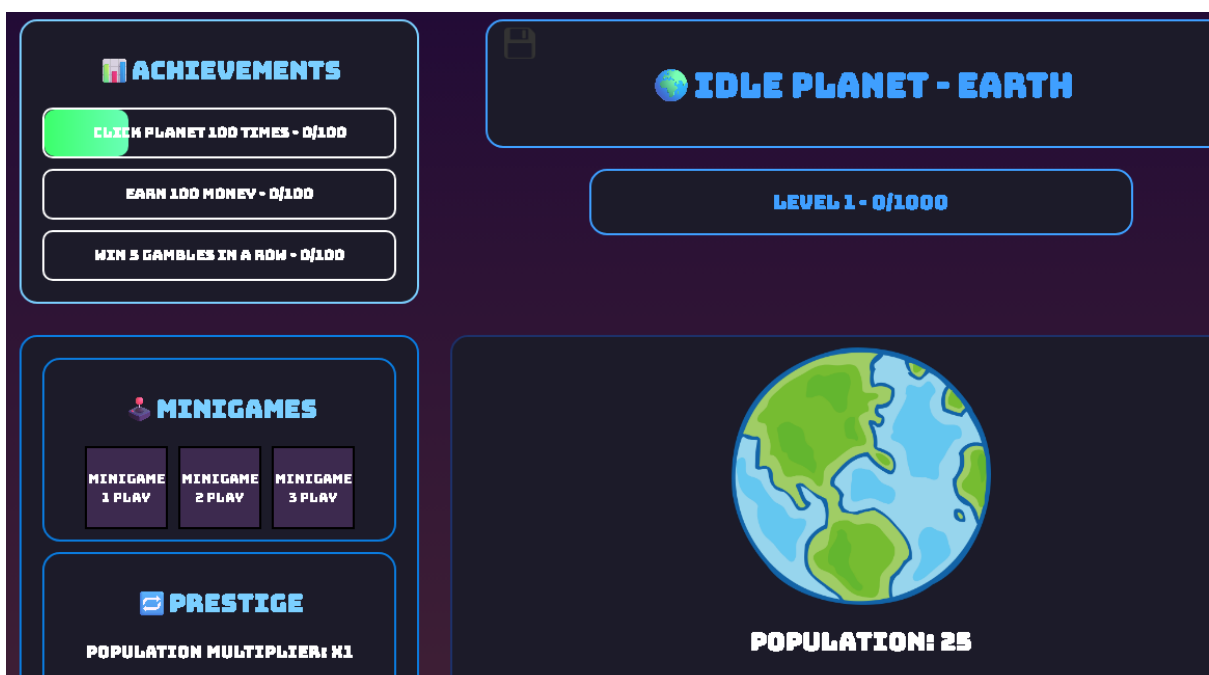


Unfortunately, nothing happened after clicking the planet button. I realised that this was because I am just setting the width to a number, when it needs to be a number followed by the % symbol, so I added this at the end of the string:

```
...").style.width = `${game_data.population}%`
```



I then tested it again:



Now, the progress bar was increasing correctly. I also decided to add a transition for the width increase, to make it slightly more smooth, using the transition property and setting it to width, 0.5 seconds and ease to make it smoothly increase:

```
.progress-fill {  
  border-radius: 8px;   
  background: linear-gradient(90deg, #3CFF6B, #6AFFB8);  
  width: 0%;  
  height: 100%;  
  position: absolute;  
  z-index: 0;  
  left: 0;  
  overflow: hidden;  
  transition: width 0.5s ease;  
}
```

I also reduced the border-radius from 10px to 8px since it was not quite fitting exactly with the outline of the achievement. I then removed the temporary line in the JavaScript as I had tested that the progress bar worked correctly.

I then developed the `update_achievement` function, which updates the progress and progress bar of each achievement when the player completes an action that goes towards it:

```
// Function that handles updating achievements  
function update_achievement(type, amount) {  
  const achievement = game_data.achievements[type]  
  // Increases achievement by amount passed in  
  achievement.progress += amount  
}
```

I started by defining the function with the 2 parameters, type and amount, which will be the type (name) of the achievement and the amount to

increase the progress by. I changed the first parameter to type, as it was achievement_type in my pseudocode but type is just shorter and quicker to write.

I then defined a constant variable which stores the current achievement being updated, using the type variable to index the achievements dictionary to get the achievement. After this, I just increase the achievement's progress by the amount variable.

After this, I added an if statement to check if the achievement progress was more than the achievement goal, in case it had somehow gone over, and if so I just set the progress to the goal and call the complete_achievement() function (which I will develop after this function is complete) and pass in the type variable, because if the achievement progress is bigger than or equal to the goal, then the achievement is complete:

```
// Function that handles updating achievements
function update_achievement(type, amount) {
  const achievement = game_data.achievements[type]
  // Increases achievement by amount passed in
  achievement.progress += amount

  if(achievement.progress >= achievement.goal) {
    achievement.progress = achievement.goal // Sets progress back to goal in case it's gone over
    complete_achievement(type) // Completes achievement
  }
}
```

I then added an else statement, which sets the progress bar width to the achievement progress divided by the achievement goal multiplied by 100:

```
...
} else {
  document.getElementById(`${achievement.element}-progress-fill`).style.width = `${(achievement.progress / achievement.goal) * 100}%`
}
```

This gets the percentage value of the achievements completion, and sets it to a percentage of the full width. For example, if the player had clicked 50/100 times, then $50 / 100 * 100 = 50$, so the width will be set to 50%,

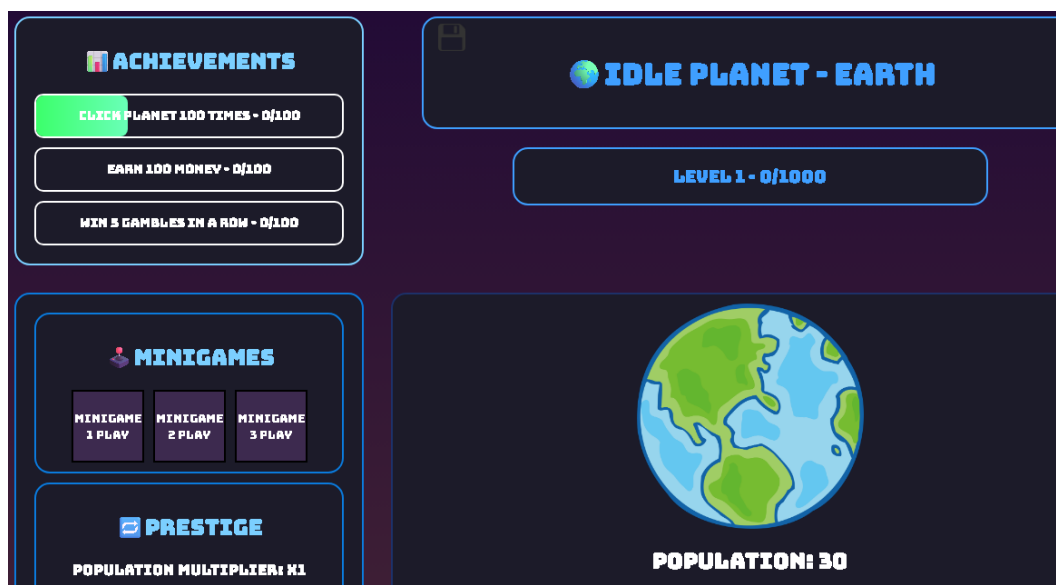
which is half of its full width, since the player has completed half of the achievement. I also use string interpolation to get the progress fill element for the specific achievement, by adding -progress-fill onto the achievement element object, to get the progress bar element.

However, I realised that my pseudocode was wrong and this won't work because the element key in each achievement dictionary stores the document object of the achievement div, not the name of the element. Therefore, I could change each one to element id as a string, however I realised that I could just use `.querySelector` to select all divs inside the achievement element div, and since there is only one div in there which is the progress bar div, it would select the progress bar for me to update the width:

```
} else {  
  achievement.element.querySelector("div").style.width = `${(achievement.progress / achievement.goal) * 100}%`  
}
```

I then called the `update_achievement()` function in the event handler for the planet button click, and passed in "clicks" for the achievement type as this is the name of the achievement dictionary, and 1 for the amount since the event handler runs once per click. I will use this to test the progress bar increase, but I will keep the line there as it will be how the click achievement is updated:

```
document.getElementById("planet-button").addEventListener("click", function (event) {  
  update_achievement("clicks", 1) // Updates click achievement each time player clicks planet  
})
```



Clicking the planet successfully increased the progress bar for the click achievement, so the function is working.

After this, I developed the function for completing the achievement, where I apply the rewards, reset the values and update the achievement display:

```
// Function that handles completing achievements
function complete_achievement(type) {
  const achievement = game_data.achievements[type]

  game_data.gems += achievement.gems
  game_data.xp += achievement.xp

  // Scales achievement goal and rewards
  achievement.goal *= 5
  achievement.gems *= 1.5
  achievement.xp *= 1.5

  achievement.progress = 0
}
```

I started by creating a constant variable for the achievement, like with the `update_achievement` function, and then increased the gems and xp by their associated reward values in the achievement dictionary. This rewards the player for completing the achievement.

I then multiply the achievement by 5, making the next one harder to achieve as it will give more rewards, since I then multiply the gems and xp reward for the achievement by 1.5. I then also set the achievement progress to 0, so the player will then start at 0 completion for the next achievement.

After this, I set the progress completion span to 0 and the progress goal span to goal value of the achievement:

```
achievement.element.querySelector("span").textContent = 0
}
```

However, as I started doing this, I realised that I would need to querySelector both the spans in the achievement div, however I will only be able to get the first one since querySelector only returns the first instance of the element. Therefore, I decided to change the HTML so that inside the h3 for the achievement, there is just one whole span, and then I can just update that with both the achievement progress and completion in one line:

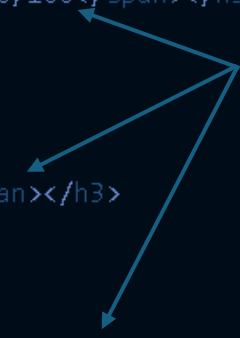
```
<div class="progress-fill" id="achievement-clicks-progress-fill"></div>
<h3>Click planet 100 times - <span id="achievement-clicks-completion">0/100</span></h3>
</div>
```

I did this for all the achievements:

```
<div class="achievement" id="achievement-clicks">
  <div class="progress-fill" id="achievement-clicks-progress-fill"></div>
  <h3>Click planet 100 times - <span id="achievement-clicks-completion">0/100</span></h3>
</div>

<div class="achievement" id="achievement-money">
  <div class="progress-fill" id="achievement-money-progress-fill"></div>
  <h3>Earn 100 money - <span id="achievement-money-completion">0/100</span></h3>
</div>

<div class="achievement" id="achievement-gambles">
  <div class="progress-fill" id="achievement-money-progress-fill"></div>
  <h3>Win 5 gambles in a row - <span id="achievement-gambles-completion">0/5</span></h3>
</div>
```

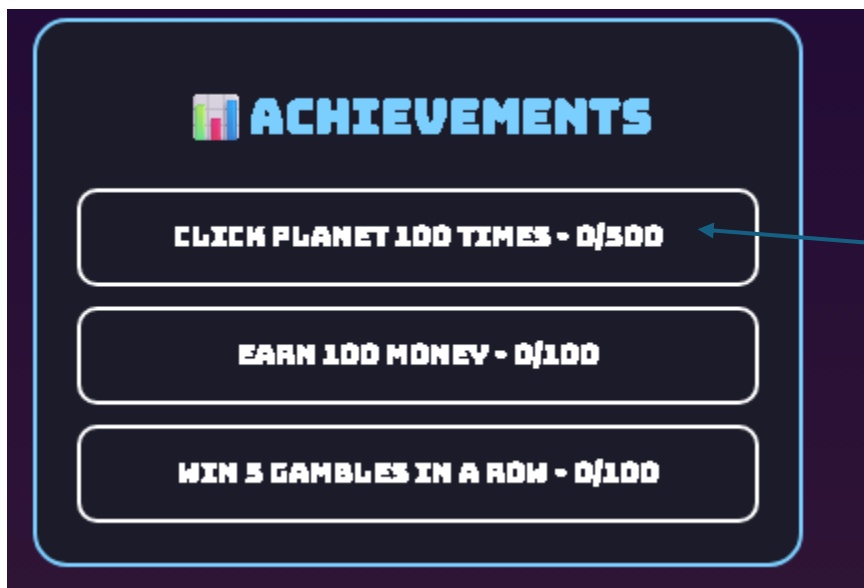


Now, I can update the whole span in the JavaScript, in one line:

```
achievement.element.querySelector("div").textContent = 0
```

This sets the text content to achievement_progress / achievement_goal. I tested this by calling the function at the end of the program:

```
}  
  
complete_achievement("clicks")
```



This increased the achievement goal to 500, which is correct as this is 5 times 100, so the function is working.

Finally, I updated the width of the progress bar of the achievement to 0%, as the completion resets to 0 when the previous achievement is completed:

```
// Reset achievement goal and progress bar  
achievement.element.querySelector("span").textContent = `0/${achievement.goal}`  
achievement.element.querySelector("div").style.width = "0%"
```

I also realised I forgot to update the achievement progress value in the update_achievement() function, so I did this:


```

} else {
  // If achievement is not complete, update progress bar to percentage value of completion
  achievement.element.querySelector("div").style.width = `${(achievement.progress / achievement.goal) * 100}%`
  // And update achievement progress display
  achievement.element.querySelector("span").textContent = `${achievement.progress} / ${achievement.goal}`
}

```

I then tested the code to see if the `update_achievement()` function was working:



After getting past 100 clicks, the progress bar reset and the achievement goal scaled properly, so the function is working.

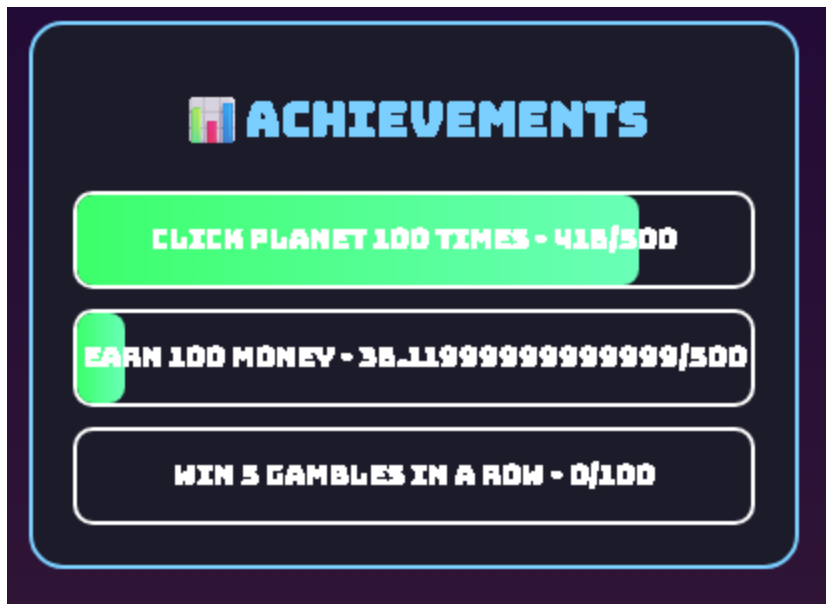
Finally, I added the code to update the achievements the money and gambles achievements (since I already have the code to update the clicks achievement):

Money achievement update:

```
setInterval(() => {  
    let money = game_data.money  
    game_data.money += game_data.money_per_second  
    game_data.money = Math.round(game_data.money * 100) / 100 //Removes extra, unwanted decimal places  
    document.getElementById("money").textContent = formatNum(game_data.money)  
    update_achievement("money", game_data.money - money)  
}, 1000)
```

I create a temporary value called money, which is set to game_data.money before it is updated. I then call the update_achievement and for the amount argument, I pass in the current game_data.money subtract the temporary variable. This gets the amount the money has just increased by. The reason I don't just pass in money_per_second is because this is an accurate way of indicating how much money the player has earned, until they purchase the industry boom upgrade, which increases money per second by 20%. This means that the achievement would be updated by money per second, but money per second would be more than the current money because of the 20% increase. Therefore, the achievement would increase by more than the player has actually completed. Additionally, my method is unaffected by purchasing upgrades which reduces money. This is because I simply calculate the amount the money has increased by in the loop, so even if the players money goes down due to purchasing an upgrade (or a worker), the achievement will always be updated by how much it increases each second.

I tested this to see if it worked:



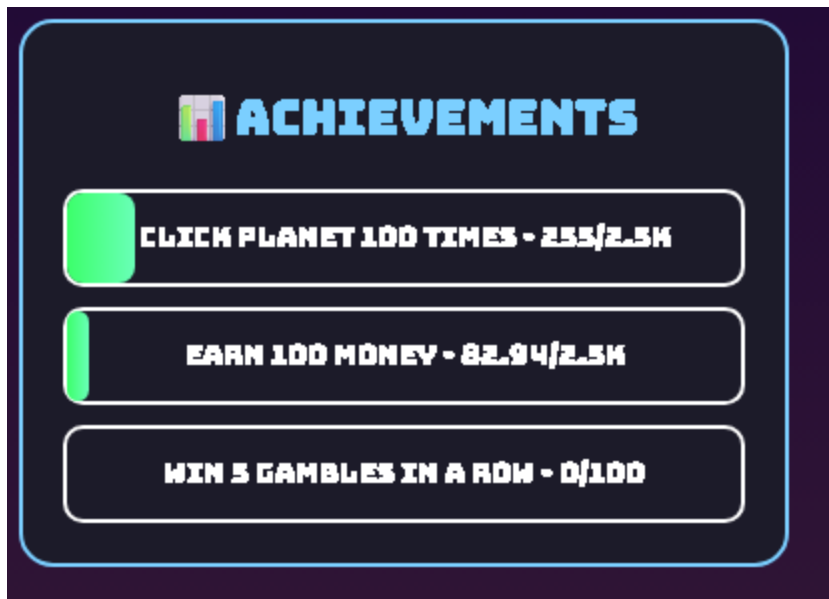
The achievement was correctly increasing, and scaling after completion, however I noticed there were extra digits added on the end of the progress value, a problem which I have faced a lot during development of this project. To solve this, I firstly called the `formatNum` function in the line of code where I update the achievement progress and goal values, to prevent the extra digits and to ensure that they will be abbreviated later when the numbers get big:

```
} else {  
  // If achievement is not complete, update progress bar to percentage value of completion  
  achievement.element.querySelector("div").style.width = `${(achievement.progress / achievement.goal) * 100}%`  
  // And update achievement progress display  
  achievement.element.querySelector("span").textContent = `${formatNum(achievement.progress)} / ${formatNum(achievement.goal)}`  
}
```

However, this still did not fix the issue, so I used my rounding method, to get rid of extra digits, on the achievement progress and achievement goal:

```
achievement.element.querySelector("span").textContent =  
  `${formatNum(Math.round(achievement.progress * 100) / 100)} / ${formatNum(Math.round(achievement.goal * 100) / 100)}`
```

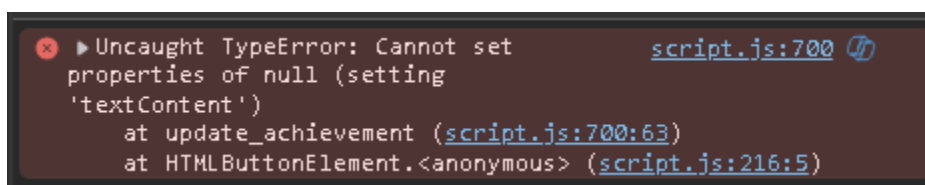
I tested this to see if it fixed the problem:



Now, there were no extra digits added on the end. However, I realised that although I was updating the achievement progress displays, I was not updating the name of the achievement. For example, “Click the planet 100 times” should change to “Click the planet 500 times” after the first completion. Therefore, at the bottom of the `complete_achievement()` function, I set the `h3` of each achievement to its title, using string interpolation to embed the goal value of it:

```
// Updates each achievement title with updated achievement goal
game_data.achievements.clicks.element.querySelector("h3").textContent = `Click planet ${formatNum(game_data.achievements.clicks.goal)} times`
game_data.achievements.money.element.querySelector("h3").textContent = `Earn ${formatNum(game_data.achievements.money.goal)} money`
game_data.achievements.gamble.element.querySelector("h3").textContent = `Win ${formatNum(game_data.achievements.gamble.goal)} gambles in a row`
}
```

I tested this to see if it worked, however I would get this error after completing the first achievement:



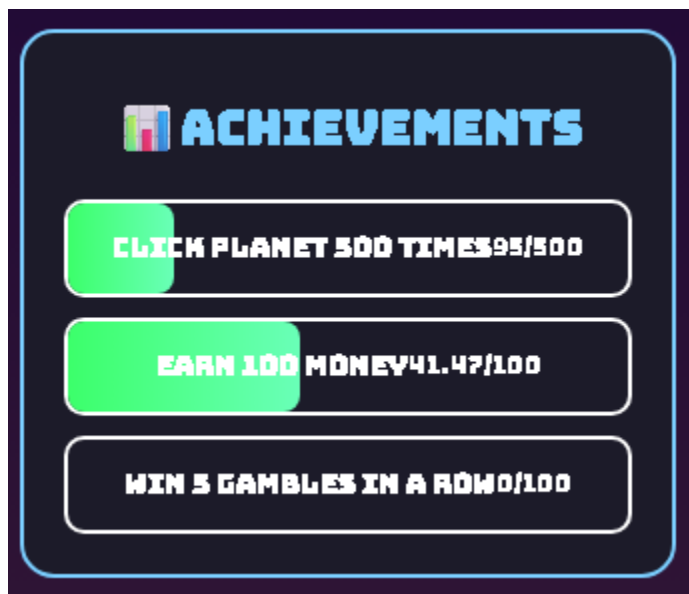
I realised that this was because I was setting the text content of the the h3 tag, which contained a span. Therefore, the span was getting deleted so couldn't be read from anymore. Therefore, I decided to take the spans out of all the h3 tags, and have them adjacent to them:

```
<div class="achievement" id="achievement-clicks">
  <div class="progress-fill" id="achievement-clicks-progress-fill"></div>
  <h3>Click planet 100 times - </h3><span id="achievement-clicks-completion">0/100</span>
</div>

<div class="achievement" id="achievement-money">
  <div class="progress-fill" id="achievement-money-progress-fill"></div>
  <h3>Earn 100 money - </h3><span id="achievement-money-completion">0/100</span>
</div>

<div class="achievement" id="achievement-gambles">
  <div class="progress-fill" id="achievement-money-progress-fill"></div>
  <h3>Win 5 gambles in a row - </h3><span id="achievement-gambles-completion">0/5</span>
</div>
```

Now, the content of the h3 will be overwritten, but not the spans. I tested this to see if it worked:



Now, the achievement titles were being correctly updated, however the text from the span was pushed right up against the title, so I added a non-breaking white space () followed by a dash in the span, before the progress value. I then removed the dash in the h3 tags:

```

<div class="achievement" id="achievement-clicks">
  <div class="progress-fill" id="achievement-clicks-progress-fill"></div>
  <h3>Click planet 100 times </h3><span id="achievement-clicks-completion">&nbsp; - 0/100</span>
</div>

<div class="achievement" id="achievement-money">
  <div class="progress-fill" id="achievement-money-progress-fill"></div>
  <h3>Earn 100 money </h3><span id="achievement-money-completion">&nbsp; - 0/100</span>
</div>

<div class="achievement" id="achievement-gambles">
  <div class="progress-fill" id="achievement-money-progress-fill"></div>
  <h3>Win 5 gambles in a row </h3><span id="achievement-gambles-completion">&nbsp; - 0/5</span>
</div>

```

This creates a space after the title, so it is not too squashed. I also gave them a z-index of 1, so they would be above the progress bar, in the CSS by selecting spans in the achievement class then applying the z-index:

```

.achievement span {
  z-index: 1;
}

```

I then needed to add this space in the JavaScript, when I set the value of the span, both the `update_achievement()` and `complete_achievement()` functions:

```

achievement.element.querySelector("span").textContent =
  '&nbsp; - ${formatNum(Math.round(achievement.progress * 100) / 100)}/${formatNum(Math.round(achievement.goal * 100) / 100)}'
}

achievement.element.querySelector("span").textContent = `&nbsp; - 0/${achievement.goal}`
achievement.element.querySelector("div").style.width = "0%"

```

I tested this to see if it worked:



This did not work, and the text “ ” was being added instead of the actual whitespace. I looked up how to use a non-breaking space in JavaScript when setting text content:

Bing

how to use in javascript .textContent

ALL SEARCH VIDEOS IMAGES MAPS NEWS COPILOT MORE

Copilot Search

When you use .textContent in JavaScript, HTML entities like are **not parsed** — they are treated as plain text. If you want a **non-breaking space** in .textContent, you must use the actual Unicode character \u00A0 instead.

Example:

```
JavaScript
// Select an element
const el = document.getElementById("mySpan");

// Set textContent with a non-breaking space
el.textContent = "Hello" + "\u00A0" + "World";
```

The copilot result said that I need to use the unicode character for it when setting text content, so I did this:

```
// And update achievement progress display, ensures there is whitespace and dash between achievement title and progress display
achievement.element.querySelector("span").textContent =
'\u00A0 - ${formatNum(Math.round(achievement.progress * 100) / 100)}/${formatNum(Math.round(achievement.goal * 100) / 100)}
'
```

```
// And update achievement progress display, ensures there is whitespace and dash between achievement title and progress display
achievement.element.querySelector("span").textContent = `${achievement.title} - ${achievement.progress}/${achievement.goal}`
// Resets progress bar
achievement.element.querySelector("div").style.width = "0%"
```

And this fixed the issue:



Gambles achievement update:

```
if (random_roll < game_data.win_chance) {
  // Removes bet from population, and then adds it back with the win multiplier applied
  game_data.population -= game_data.bet
  game_data.population += game_data.bet * game_data.win_multiplier
  // Shows win message
  document.getElementById("win").hidden = false
  // Plays win sound
  game_data.sounds.win.play()
  game_data.win_streak ++
  update_achievement("gamble", 1) // Increases gamble achievement completion each consecutive win
```

Here, I call the `update_achievement()` function when the player wins, passing in "gamble" for the achievement type and 1 for the amount

because the win streak increases. The only problem is that even when the win streak goes back to 0, the gambles achievement progress won't be reset. Therefore, I added a unique check in the `update_achievement()` function, that will reset the progress for the gambles achievement including the progress bar and progress display:

```
// Function that handles updating achievements
function update_achievement(type, amount) {
  if(type == "reset_gamble_achievement") {
    console.log("test")
    // Resets gamble achievement progress completely
    // Only runs if gamble win streak is lost
    game_data.achievements.gamble.progress = 0
    game_data.achievements.gamble.element.querySelector("span").textContent = '\u00A0 - 0/${game_data.achievements.gamble.goal}'
    game_data.achievements.gamble.element.querySelector("div").style.width = "0%"
  } else {
    const achievement = game_data.achievements[type]
    // Increases achievement by amount passed in
    achievement.progress += amount
    // Checks if achievement is completed
    if(achievement.progress >= achievement.goal) {
      achievement.progress = achievement.goal // Sets progress back to goal in case it's gone over
      complete_achievement(type) // Completes achievement
    } else {
      // If achievement is not complete, update progress bar to percentage value of completion
      achievement.element.querySelector("div").style.width = `${(achievement.progress / achievement.goal) * 100}%`
      // And update achievement progress display, ensures there is whitespace and dash between achievement title and progress display
      achievement.element.querySelector("span").textContent =
        '\u00A0 - ${formatNum(Math.round(achievement.progress * 100) / 100)}/${formatNum(Math.round(achievement.goal * 100) / 100)}'
    }
  }
}
```

(I also moved the lines that creates the achievement const variable and increases the progress amount by the amount argument passed in, so it only runs if the type is not equal to “reset_gamble_achievement”. This is because I can't reference the achievements dictionary with type “reset_gamble_achievement” as this is a special argument for 1 purpose only, and I don't need to increase the progress amount if the win streak is reset, since I set it to 0.)

I will then call the function when the player loses a gamble, and pass in “reset_gamble_achievement” for the parameter, so that when the win streak is set back to 0, the progress for the gamble achievement will also be reset:

```
} else {  
  game_data.population -= game_data.bet  
  // Shows loss message  
  document.getElementById("loss").hidden = false  
  // Plays loss sound  
  game_data.sounds.lose.play()  
  game_data.win_streak = 0  
  update_achievement("reset_gamble_achievement") // Resets gamble achievement after win streak loss  
}
```



I tested this to see if it worked

WIN 5 GAMBLER IN A ROW - 0/5



MINIGAMES

MINIGAME
1 PLAY

MINIGAME
2 PLAY

MINIGAME
3 PLAY



PRESTIGE

POPULATION MULTIPLIER:
X1

PRESTIGE - 1M POPULATION



POPULATION GAMBLING

Enter amount

GAMBLE

WIN RETURN: 0

WIN CHANCE: 50%

YOU LOST

After losing a gamble, the progress bar and progress display reset back to default, so it was working.

After this, I added the code in the prestige() function to restore the achievements after a prestige:

```
function prestige() {
  let prestige_level = game_data.prestige_level
  let prestige_cost = game_data.prestige_cost
  let current_prestige_multiplier = game_data.next_prestige_multiplier
  let quicker_prestige = JSON.parse(JSON.stringify(game_data.upgrades.quicker_prestige))

  let achievements = JSON.parse(JSON.stringify(game_data.achievements))
  let gems = game_data.gems
  let xp = game_data.xp

  prestige_level ++
  prestige_cost *= 10

  Object.assign(game_data, JSON.parse(JSON.stringify(default_game_data)))

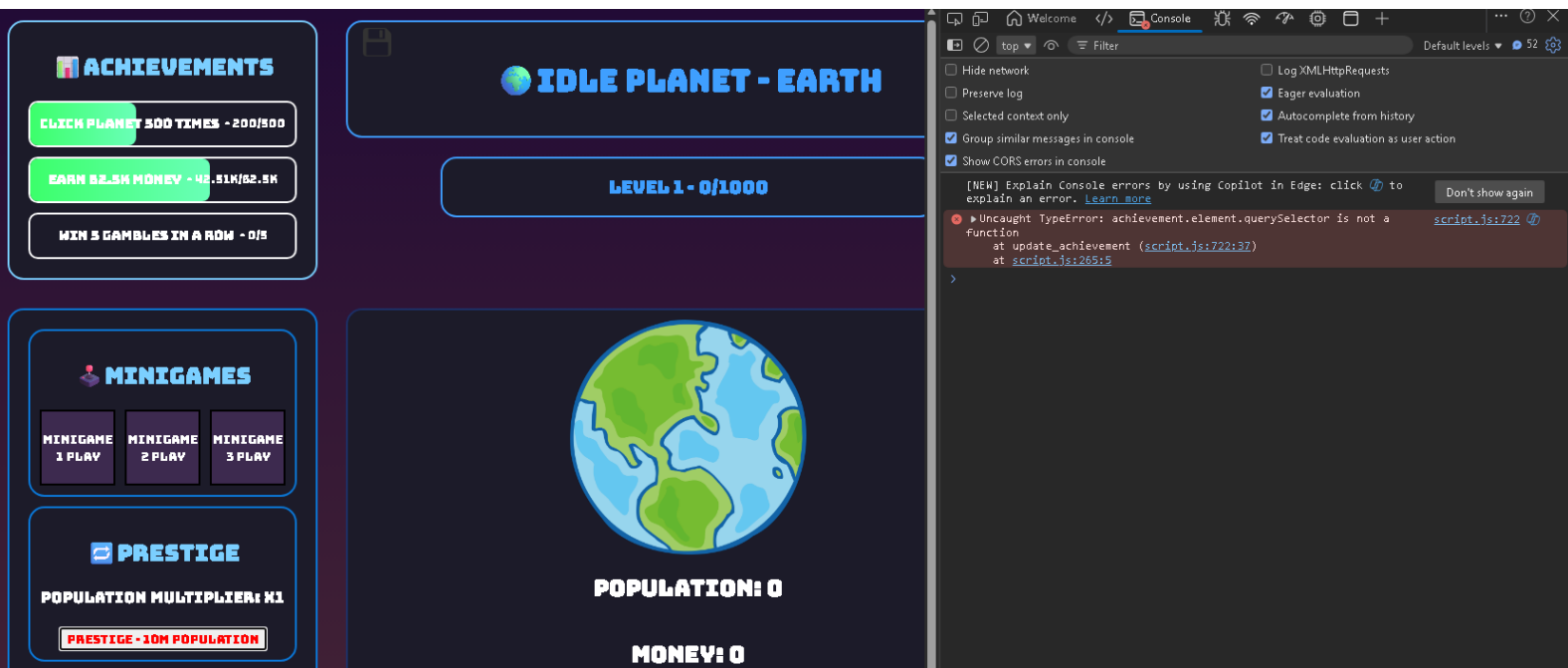
  game_data.prestige_level = prestige_level
  game_data.prestige_cost = prestige_cost
  game_data.current_prestige_multiplier = current_prestige_multiplier
  Object.assign(game_data.upgrades.quicker_prestige, quicker_prestige)

  Object.assign(game_data.achievements, achievements)
  game_data.gems = gems
  game_data.xp = xp

  localStorage.removeItem("game_data")
}
```

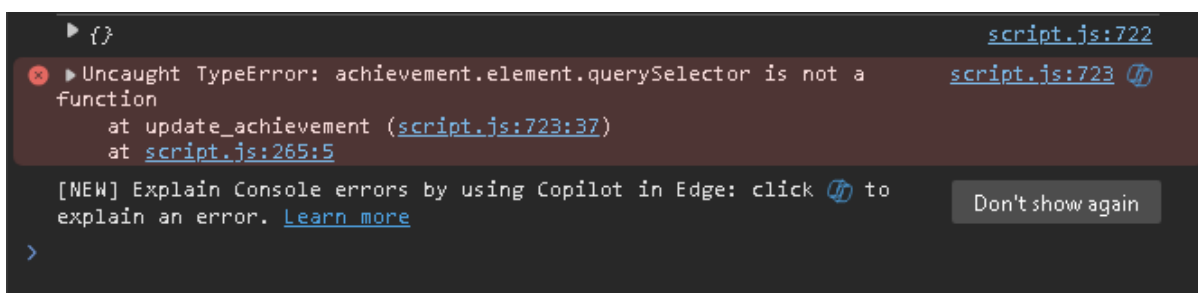
Here, I create temporary variables for all the achievement data, just like I do for the prestige data that I restore, and then assign them back to game_data after it is reset. For the achievements dictionary, I use JSON.parse(JSON.stringify) to create a copy of the object, so then I can use Object.assign() to assign it back to game_data, which restores everything inside. The reason I restore achievements after prestige, is because I don't want the player to feel like they have completely lost all their progress. Achievements are a way to track how much progress the player has made, so keeping the achievements will make them feel like they are making constant progress throughout the game, rather than it being completely reset each time.

I tested this to see if it worked:



However, I was getting the error “script.js:722 Uncaught TypeError: achievement.element.querySelector is not a function”, meaning there was a problem setting the querying the achievement elements. I added a console.log statement in the update_achievement() function, to output the element of the achievement, to see if it was being correctly reset after a prestige:

```
} else {  
  console.log(achievement.element) ←  
  // If achievement is not complete, update progress bar to percentage value of completion  
  achievement.element.querySelector("div").style.width = `${(achievement.progress / achievement.goal) * 100}%`  
  // And update achievement progress display, ensures there is whitespace and dash between achievement title and progress display  
  achievement.element.querySelector("span").textContent =  
    `⌚ - ${formatNum(Math.round(achievement.progress * 100) / 100)} / ${formatNum(Math.round(achievement.goal * 100) / 100)}`  
}
```



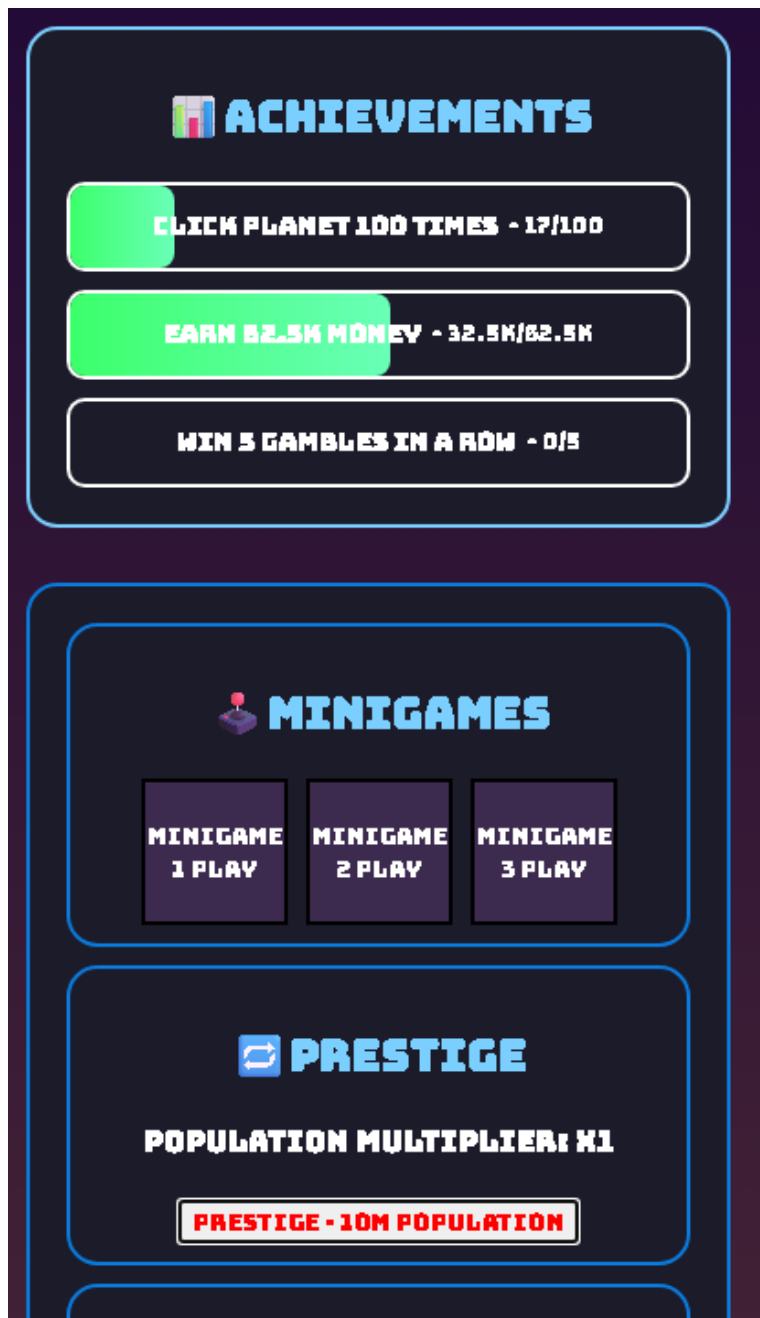
After prestiging, the achievement element was not being restored properly, as an empty value was output. Therefore, I decided to explicitly assign the elements of all 3 achievements in the prestige function, after game_data is reset, to ensure they are all correctly restored:

```
Object.assign(game_data.achievements, achievements)
game_data.achievements.clicks.element = document.getElementById("achievement-clicks")
game_data.achievements.money.element = document.getElementById("achievement-money")
game_data.achievements.gamble.element = document.getElementById("achievement-gambles")
game_data.gems = gems
game_data.xp = xp
```

I then put these in the window.onload function, so that they would also be reassigned when the page loads after retrieving game_data from local storage:

```
window.onload = function() {
  Object.assign(game_data, (JSON.parse(localStorage.getItem("game_data"))))
  game_data.achievements.clicks.element = document.getElementById("achievement-clicks")
  game_data.achievements.money.element = document.getElementById("achievement-money")
  game_data.achievements.gamble.element = document.getElementById("achievement-gambles")
  update_game_displays()
}
```

I then tested this again, to see if it worked:



Now, after prestiging the achievements were restored, including their element property, and there were no errors.

I also realised that I will need to restore the gamble win streak, so that the gamble achievement would be correctly restored:

```
let win_streak = game_data.win_streak
```

(Before game_data is reset)

```
game_data.win_streak = win_streak
```

(After game_data is reset)

I tested this to see if it worked:

ACHIEVEMENTS

CLICK PLANET 100 TIMES - 53/100

EARN 62.5K MONEY - 22.63K/62.5K

WIN 7 GAMBLER IN A ROW - 6/7

MINIGAMES

MINIGAME
1 PLAY

MINIGAME
2 PLAY

MINIGAME
3 PLAY

PRESTIGE

POPULATION MULTIPLIER: X1

PRESTIGE - 10M POPULATION

POPULATION GAMBLING

1

GAMBLE

WIN RETURN: 1.5

WIN CHANCE: 50%

YOU WON

However, after prestiging and then winning another gamble, nothing happened to the achievement. Also, the win message appeared, but no sound played. I check the console to see if there was an error, and I got this:

```
2 Uncaught TypeError: game_data.sounds.win.play is not a function    script.js:682
    at gamble (script.js:682:34)
    at HTMLButtonElement.<anonymous> (script.js:714:9)
```

Therefore, I realised that the sounds dictionary was not being correctly restored after a prestige. This was a similar problem with the element key for the achievements, so I fixed it in the same way, by explicitly redefining the win and lose sounds after game_data was reset:

```
Object.assign(game_data, JSON.parse(JSON.stringify(default_game_data))) // Resets game_data for prestige

// Restores the game data variables that was assigned to the temporary variables
game_data.prestige_level = prestige_level
game_data.prestige_cost = prestige_cost
game_data.current_prestige_multiplier = current_prestige_multiplier
Object.assign(game_data.upgrades.quicker_prestige, quicker_prestige)

Object.assign(game_data.achievements, achievements)
game_data.achievements.clicks.element = document.getElementById("achievement-clicks")
game_data.achievements.money.element = document.getElementById("achievement-money")
game_data.achievements.gamble.element = document.getElementById("achievement-gambles")
game_data.gems = gems
game_data.xp = xp
game_data.win_streak = win_streak

game_data.level = level
game_data.level_goal = level_goal
game_data.level_gem_reward = level_gem_reward
game_data.level_population_reward = level_population_reward

game_data.sounds.win = new Audio("win.mp3"), ←
game_data.sounds.lose = new Audio("lose.mp3") ←
```

I then also put these in the window.onload function so they would also be redefined when the page loads:

```
window.onload = function() {
    Object.assign(game_data, (JSON.parse(localStorage.getItem("game_data"))))
    game_data.achievements.clicks.element = document.getElementById("achievement-clicks")
    game_data.achievements.money.element = document.getElementById("achievement-money")
    game_data.achievements.gamble.element = document.getElementById("achievement-gambles")
    game_data.sounds.win = new Audio("win.mp3"), ←
    game_data.sounds.lose = new Audio("lose.mp3") ←
    update_game_displays()
}
```

I tested this to see if it worked:

ACHIEVEMENTS

CLICK PLANET 100 TIMES - 75/100

EARN 62.5K MONEY - 42.5K/62.5K

WIN 5 GAMBLER IN A ROW - 3/5

MINIGAMES

MINIGAME
1 PLAY

MINIGAME
2 PLAY

MINIGAME
3 PLAY

PRESTIGE

POPULATION MULTIPLIER: X1

PRESTIGE - 10M POPULATION

POPULATION GAMBLING

Enter amount

GAMBLE

WIN RETURN: 0

WIN CHANCE: 50%

YOU WON

Now, after prestiging, gambling worked again. The sound played like normal, and the win streak was restored so the gamble achievement didn't reset.

Finally, I developed the achievement gem and xp boost upgrades:

```
} else if (upgrade_name == "achievement_gem_boost") {  
    // 2% boost to achievement gem rewards  
    game_data.achievements.clicks.gems = Math.ceil(game_data.achievements.clicks.gems * 1.02)  
    game_data.achievements.money.gems = Math.ceil(game_data.achievements.money.gems * 1.02)  
    game_data.achievements.gamble.gems = Math.ceil(game_data.achievements.gamble.gems * 1.02)  
} else if (upgrade_name == "achievement_xp_boost") {  
    // 10% boost to achievement xp rewards  
    game_data.achievements.clicks.xp *= 1.1  
    game_data.achievements.money.xp *= 1.1  
    game_data.achievements.gamble.xp *= 1.1  
}
```

As per my pseudocode, I created an else if check for both of the upgrades. For the achievement gem boost, I multiply the gem reward for each by 1.02, which is a 2% increase. The reason I ceil the result is so that it will round up and there will be no decimal gem values. This is because I don't want the user having decimal gems, because all effect prices will be whole numbers. Then, for the achievement xp boost upgrade I simply multiply the xp reward for each achievement by 1.1, which is a 10% increase according to the upgrade design.

To make these easier to test, I added the gem and xp displays early, so I could see if they were increasing when achievements were completed:

```
<div class="effects">  
    <div class="gems"><h3>Gems: <span id="gems">0</span></h3></div>  
    <div class="xp"><h3>XP: <span id="xp">0</span></h3></div>  
</div>
```

```
<div class="level">  
    <div class="progress-bar"></div>  
    <h3>Level 1 - <span id="xp">0</span>/1000</h3>  
</div>
```

For the gems, I added a span to display the gems, with the id "gems" so I could update it in the JavaScript

For the level, I put the value for the current xp inside a span, with the id “xp”.

I am not developing these features here, I am just adding the displays so I can test the achievements more easily.

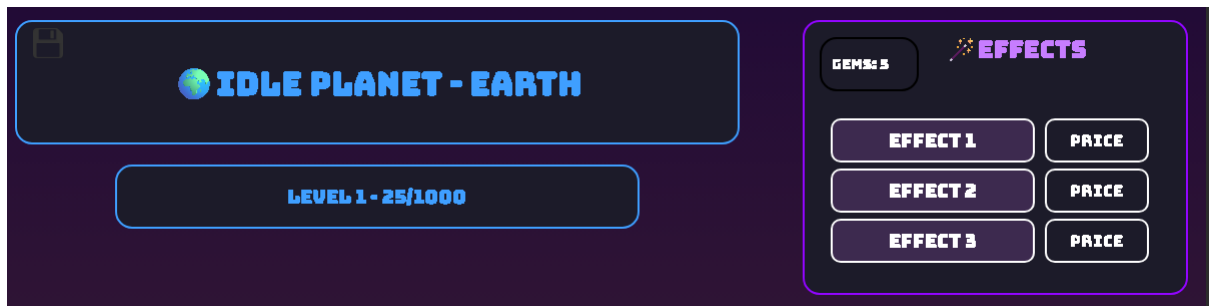
I also decreased the font size of the gems display, since it is too large to also fit the gem number inside the container:

```
.gems {  
  border: 2px solid black;  
  border-radius: 17px;  
  padding: 10px;  
  margin: 10px;  
  text-align: center;  
  width: 75px;  
  height: 30px;  
  display: flex;  
  justify-content: left;  
  align-items: center;  
  position: absolute;  
  font-size: 12px;  
}
```

I then updated the gem and xp displays in the complete_achievement() function after I increase their values:

```
// Scales achievement goal and rewards  
achievement.goal *= 5  
achievement.gems *= 1.5  
achievement.xp *= 1.5  
  
achievement.progress = 0  
  
// Updates gems and xp display in HTML  
document.getElementById("gems").textContent = formatNum(Math.round(game_data.gems * 100) / 100)  
document.getElementById("xp").textContent = formatNum(Math.round(game_data.xp * 100) / 100)
```

I also passed them into the formatNum function so they would be abbreviated correctly when they get larger, and used the rounding method on them to remove extra digits. I tested this by completing the clicks achievement once, to see if the displays updated:

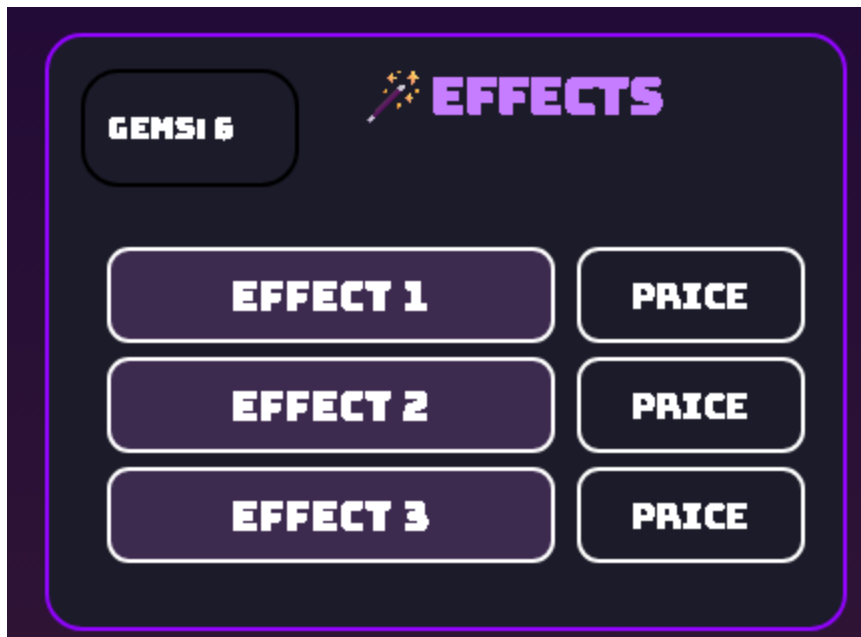


The xp display updated to 25, and the gems display updated to 5, which is correct so it is working.

The only problem was that after purchasing the achievement gem boost upgrade and completing an upgrade, the reward didn't seem to increase, however this is because it was only being increased by 2%, and also rounded so it wasn't increasing at all. I decided that 2% was a bit too low for the upgrade, so I increased it to 20%:

```
} else if (upgrade_name == "achievement_gem_boost") {  
    // 20% boost to achievement gem rewards  
    game_data.achievements.clicks.gems = Math.ceil(game_data.achievements.clicks.gems * 1.2)  
    game_data.achievements.money.gems = Math.ceil(game_data.achievements.money.gems * 1.2)  
    game_data.achievements.gamble.gems = Math.ceil(game_data.achievements.gamble.gems * 1.2)  
}
```

I tested this to see if the gem display increased now:



Now, the gem display increased to 6 after completing the upgrade with an achievement gem boost upgrade.

I also decided I want to change the way the achievements reset. Instead of resetting the progress to 0, I will keep it the same. For example, when the player completes the first click achievement, they will still have 100/500 clicks, therefore the achievement will track their total clicks, rather than their clicks just for that achievement. Therefore, I remove the line in `complete_achievement` that resets the progress to 0:

```
achievement.progress = 0

// Updates gem and xp display in HTML
document.getElementById("gems").textContent = formatNum(Math.round(game_data.gems * 100) / 100)
document.getElementById("xp").textContent = formatNum(Math.round(game_data.xp * 100) / 100)
```

Then, in the line that sets the progress bar width to 0%, I will change it to set it to the percentage value of its completion:

```
achievement.element.querySelector("span").textContent = `\u00A0 - ${achievement.progress}/${achievement.goal}`
achievement.element.querySelector("div").style.width = `${(achievement.progress / achievement.goal) * 100}%`
```

I tested this to see if it worked:



Now, after completion, the progress bar goes back down but doesn't reset to 0, it just becomes the percentage value of the current completion, with the updated achievement goal value. The only problem was that for the gamble achievement, the progress bar did not go up to full and then back down at all, because the goal only increases by 1. Therefore, I added some code to make it clear than the achievement was completed:

```
// Updates progress bar to percentage value of completion
if(type == "gamble") {
  // For gamble achievement, increases progress bar to full for 0.5s then resets it to current value
  // This makes it clear that the achievement is completed, since the progress bar doesn't move otherwise
  achievement.element.querySelector("div").style.width = "100%"
  setTimeout(() => {
    achievement.element.querySelector("div").style.width = `${(achievement.progress / achievement.goal) * 100}%`
  }, 500)
} else {
  achievement.element.querySelector("div").style.width = `${(achievement.progress / achievement.goal) * 100}%`
}
```

Here, I check if the achievement type is "gamble", and if so I set the progress bar to full. I then use setTimeout to wait 0.5s, before putting it back to its current value. This makes it clear that the achievement has been completed. I also set the progress bar as normal, if the type is not gamble.

Also, looking at this function again, I realised that when I set the value of the title of the achievement, I do not pass it through the formatNum function, so I did this:

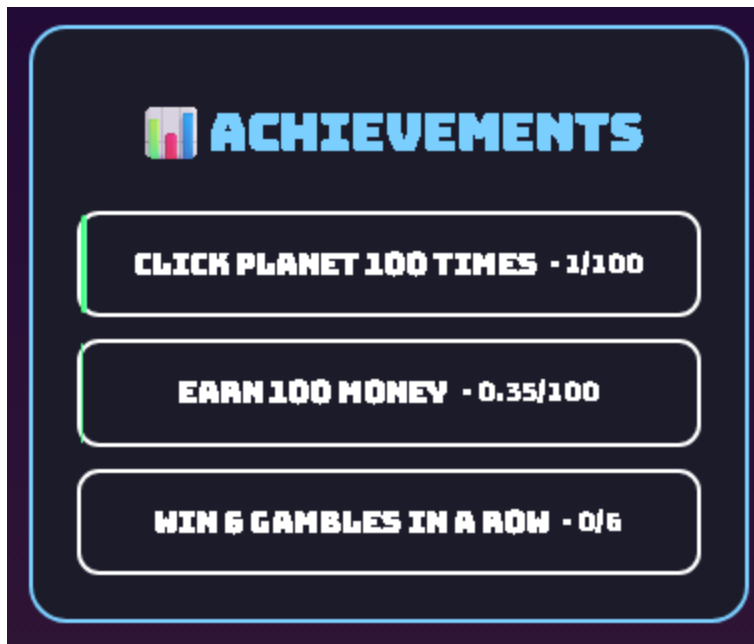
```
// Updates achievement title with updated achievement goal
game_data.achievements.clicks.element.querySelector("h3").textContent = `Click planet ${formatNum(game_data.achievements.clicks.goal)} times`
game_data.achievements.money.element.querySelector("h3").textContent = `Earn ${formatNum(game_data.achievements.money.goal)} money`
game_data.achievements.gamble.element.querySelector("h3").textContent = `Win ${formatNum(game_data.achievements.gamble.goal)} gambles in a row`
```

This ensures that when the achievement goals get bigger, they will be abbreviated correctly.

Finally, after testing completion all the achievements, I noticed something quite obvious, that after just the first completion of the gambles achievement, the goal increases to 25 because it is multiplied by 5. Winning 25 gambles in a row would be extremely hard, even with a 75% win rate, and especially with bigger gambles where the win rate drops. Therefore, I decided to increase the goal for this achievement by only 1 each time, to make it more obtainable:

```
if(type == "gamble") {
  // Increases gamble achievement goal by 1
  achievement.goal ++
} else {
  // Scales other achievements by x5
  achievement.goal *= 5
}
```

I added an if statement in the complete_achievement() function, that checks if the achievement type is “gamble”. If so, it increases the goal by 1, but if not it increases it normally, by multiplying it by 5. I tested this to see if it worked:



Now, after completing the achievement once, the goal increased to 6, which is much more obtainable than 25, and then 125 which would have been only the 3rd achievement goal.

Testing

This is my test plan for this stage, with some modifications:

- For the second test, I have changed it to test whether, after completion, the progress bar decreases to the new percentage value of completion with the updated progress goal. In the

original test plan, I planned to test whether it just reset back to 0, since that was the design, however I changed the way it worked in the development so I have slightly modified the test.

- I have added the exact amount that the achievement goal should scale by in the 3rd test, since this was not planned yet in the stages of development
- I have also added the exact gem and xp value that the achievements should reward the player, in the 4th test, since this was also not yet planned in the stages of development
- I also added 2 tests for the 2 achievement upgrades which I developed in this stage, since I had not planned any specific upgrades in the stages of development

Test no.	Description	Test type	Test data	Expected Result	Actual result
1	Do the achievement progress bars and progress counts increase correctly		Click the planet and observe the click the planet achievement progress bar	The progress bar should increase as the planet is clicked, and the progress count should increase	The progress bar increased with each click, with the progress count increasing by one each click. Passed  SEE STAGE 8 TEST 1.mp4 in evidence folder
2	Does the achievement progress bar update		Complete an achievement and look at the progress bar	The progress bar should get smaller and decrease to the new	Completing the click achievement reset the planet bar, so it was now

	correctly after completion			percentage value of completion with the updated progress goal.	at 20% completion (100/500). Passed  SEE STAGE 8 TEST 2.mp4 in evidence folder
3	Does the achievement goal scale when the achievement is completed, and is this update reflected in the achievement title		Complete each achievement once and look at the achievement goal value, and the achievement title	The achievement goal should scale by x5 unless it's the gamble achievement in which case it should increase by 1. The achievement title should update with the achievement goal.	After completing the click achievement, the achievement goal value increased to 500 (x5 increase from 100) Passed  SEE STAGE 8 TEST 3.mp4 in evidence folder
4	Do achievements correctly reward the player with gems and xp		Complete the click achievement and look at the level and gem display	The the xp in the level display should increase by 25 and the gems in the gem display should increase by 5	After completing the click achievement, the xp increased to 25, and the gems increased to 5 Passed 

					SEE STAGE 8 TEST 4.mp4 in evidence folder
5	Are achievements and their completion restored after prestige		Prestige with completion of all achievements and observe them	Nothing should happen to the achievements, their progress bars and progress count should remain exactly the same	Prestiging did not reset the achievements or their completion, they remained exactly the same Passed ☒ SEE STAGE 8 TEST 5.mp4 in evidence folder
6	Does the achievement gem boost correctly increase gems earned from achievements by 20%		Purchase the achievement gem boost upgrade and complete the click achievement once. Look at the gem display	The gems should increase by 6	After purchasing the upgrade, completing the click achievement increased gems by 6 (instead of 5, so 20% increase) Passed ☒ SEE STAGE 8 TEST 6.mp4 in evidence folder
7	Does the achievement xp boost upgrade		Purchase the achievement xp boost upgrade and complete	The xp should increase by 27.5	After purchasing the upgrade, completing the click achievement

	correctly increase xp earned from achievements by 10%		the click achievement once. Look at the xp in the level display.		increased gems by 27.5 (Instead of 25, so 10%) Passed  SEE STAGE 8 TEST 7.mp4 in evidence folder
--	---	--	--	--	--

Stage review:

I am happy with the final implementation of the achievements feature, as I correctly implemented 3 separate achievements that scale with completion, with progress bars that increase, and with rewards granted for completion that scale each time. However, there were many errors throughout the development which lenthened the process. I made an error with my pseudocode with the achievement element object, and I had to make a lot of changes to my HTML to correctly access all the achievement elements. This shows that I did not plan this stage carefully enough, and did not consider the HTML enough when designing it. The time this took to develop was definitely too long, especially since I am running out of time.

Stage 9 – Level System:

Features:

- Player has a level that increases when they get a certain amount of xp
- The xp required (xp goal) to level up scales by x5 each level up, just like achievement goals
- A progress bar that increases as the player gets xp, showing how much of the level the player has completed
- Player is rewarded population and gems for leveling up, starting at 100k population and 50 gems.
- The population and gem rewards scale by x1.5 each level up, just like achievement rewards
- The player's level will be restored after they prestige
- I have decided not to implement the level up pop-up screen, which displays the level the player has levelled up to, as well as their gem and population rewards. I had planned this in my GUI design and stages of development, however I don't have enough time to implement it, and it is also not absolutely necessary as it will be clear from looking at the gems and population displays that they have increased after levelling up.

Data dictionary:

Variable	Data type	Purpose
game_data.level	Key (Integer)	Stores the current level of the player, to be displayed to them in the level display
game_data.level_goal	Key (Float)	Stores the xp required to get to the next level
game_data.level_gem_reward	Key (Float)	Stores the gem reward for the next level up
game_data.level_population_reward	Key (Float)	Stores the population reward for the next level up

Pseudocode:

Add to game_data:

```

level = 1
level_goal = 1000
level_gem_reward = 50
level_population_reward = 100000

```

Increase level:

```
FUNCTION increase_level()  
  
    IF xp >= level_goal THEN  
        xp = level  
        level_up()  
    ELSE  
        HTML.xp.text = xp  
        HTML.level-progress-bar.width = (xp /  
            level_goal) * 100  
    ENDIF
```

- I will have an increase level function, which will be called in the update_achievement function, to update the xp and progress bar of the level
- I first check if the current xp is greater than or equal to the level goal, the same check I do in the update_achievement() function to with the achievement progress. This just checks if the xp has accidentally gone over the level goal, and if so I set it back to the level goal. I then call the level_up() function (which I will plan after this), because if the xp is greater than or equal to the level goal, then the player has reached the end of the level so needs to level up.
- If the player does not need to level up, however, I just set the value of the xp display in the HTML to the xp in game_data, and then set the width of the progress bar to the xp divided by the level goal multiplied by 100. This calculates the width in the same way as for the

achievement progress bars, as it gets the percentage value of completion and assigns it to the width of the bar.

Level up:

```
FUNCTION level_up()  
    level += 1  
    gems += level_gem_reward  
    population += level_population_reward  
    level_goal *= 5  
    level_gem_reward *= 1.5  
    level_population_reward *= 1.5  
  
    HTML.xp.text = xp  
    HTML.level.text = level  
    HTML.level-goal = level_goal  
    HTML.level-progress-bar.width = (xp /  
goal) * 100  
ENDFUNCTION
```

- I will have a level up function, that will be called when the xp reaches the current level goal

- It will increment the level value by 1
- It will then increase the gems and population by their reward values for the current level
- It will then scale the level goal by x5, scale the gem and population rewards by x1.5
- It will also update the xp, level and level goal displays with their associated values.
- Finally, it will set the width of the level progress bar to the xp divided by the level goal, multiplied by 100, to get the new percentage value of completion.

Restore level after prestige:

```

FUNCTION prestige()
    // Start of function
    int level = game_data.level
    int level_goal = game_data.level_goal
    int level_gem_reward =
    game_data.level_gem_reward
    int level_population_reward =
    game_data.level_population_reward

    // game_data reset
    Object.assign(game_data,
    JSON.parse(JSON.stringify(default_game_data)))

    game_data.level = level
    game_data.level_goal = level_goal

```

```
        game_data.level_gem_reward =  
level_gem_reward  
  
        game_data.level_population_reward =  
level_population_reward  
  
        // Rest of function
```

- I will add temporary variables in the prestige() function, for all the values used for the level system
- Then, after game_data is reset, I will assign these variables back to their associated keys in game_data, which will restore them.

Development:

I started the development of this section by adding the required level variables to game data, and setting them to their default values:

```
level: 1,  
level_goal: 1000,  
level_gem_reward: 10,  
level_population_reward: 100000,
```

After this, I needed to update the level div in the HTML. I started by giving the progress bar an id of “level-progress-bar” so I could access it in the JavaScript:

```
<div class="level">  
|   <div class="progress-bar" id="level-progress-bar"></div>
```

I then styled the level div and the progress bar div in the CSS, in the same way as the achievement div and progress bar:

```

.level {
  grid-row: 2;
  grid-column: 2;
  height: 40px;
  width: 500px;
  text-align: center;
  display: flex;
  justify-content: center;
  align-items: center;
  position: relative;
  overflow: hidden;
}

.progress-bar{
  border-radius: 10px;
  background: linear-gradient(90deg, ■ #3CFF6B, ■ #6AFFB8);
  width: 0%;
  height: 100%;
  z-index: 0;
  left: 0;
  overflow: hidden;
  transition: width 0.5s ease;
  position: absolute;
}

```

I gave the level container the same position and overflow as the achievement div.

I gave the progress bar the same gradient as the achievement progress bar, as I wanted them to look the same. I also gave it the same width, height, z-index, left margin, overflow, transition, and position.

This ensures it works in the same way as the achievement progress bars, which I know are already working.

I also set the z-index of the h3 inside the level div to 1, like with the h3 in the achievement div, so it would be above the progress bar:

```
.level h3 {  
  z-index: 1;  
}
```

I tested this by adding a line in the planet button event handler, which will set the width of the progress bar to the current population, so that it would increase each time I clicked the planet:

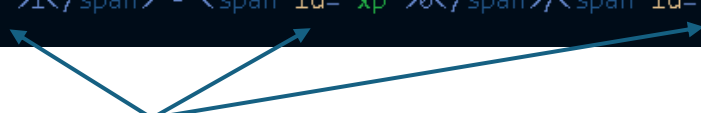
```
document.getElementById("planet-button").addEventListener("click", function (event) {  
  document.getElementById("level-progress-bar").style.width = `${game_data.population}%`  
})
```



Each time I clicked the planet, the level progress bar increased, so I knew it was working.

Finally, I added 3 spans in the h3 for the level text. The first span will store the level, with the id “level”, so I can increase the value from the JavaScript when the player levels up. The second span will store the current xp, with the id “xp” so I can increase this when the player earns xp. The 3rd span will store the level goal, with the id “level-goal”, so I can increase this when the player levels up:

```
<div class="progress-bar" id="level-progress-bar"></div>  
<h3>Level <span id="level">1</span> - <span id="xp">0</span>/<span id="level-goal">1000</span></h3>  
</div>
```



I then developed the `increase_level()` function:

```
// Function that increases the player's level  
function increase_level() {  
  // Checks if player has reached and of level  
  if(game_data.xp >= game_data.level_goal) {  
    game_data.xp = game_data.level_goal // Sets xp to level goal incase it has gone over  
    level_up() // Levels up player  
  } else {  
    // Updates xp display and progress bar width  
    document.getElementById("xp").textContent = formatNum(Math.round(game_data.xp * 100) / 100)  
    document.getElementById("level-progress-bar").style.width = `${(game_data.xp / game_data.level_goal) * 100}%`  
  }  
}
```

I started with an if statement to check if the xp is bigger than or equal to the level goal, in case it went over by accident, in which case I just set it back to the level goal. I then call the `level_up()` function, which I will develop after, because if the xp is bigger than or equal to the level goal, it means that the player has reached the end of the level and needs to be levelled up.

If the xp is not bigger than or equal to the level goal, the code just updates the xp display to the current xp and updates the progress bar width to its percentage value of completion, by dividing xp by the level goal and multiplying it by 100 to get the percentage.

I then developed the `level_up()` function, which is called when the player reaches the end of the current level:

```
// Function that levels up the player
function level_up() {

    game_data.level ++

    game_data.gems += game_data.level_gem_reward
    game_data.population += game_data.level_population_reward

    // Scales level goal and level reward
    game_data.level_goal *= 5
    game_data.level_gem_reward *= 1.5
    game_data.level_population_reward *= 1.5
}
```

I started by incrementing the level by 1, as the player is going up a level. I then increase the gems and population by the gem and population level reward. After this, I scale the level goal, by multiplying it by 5, making the next level 5x harder to complete.

I then scale the gem and population rewards by x1.5, meaning the next level is 1.5x more rewarding, since it is harder to complete.

Finally, I updated the xp, level and level-goal displays to their associated values in `game_data`, and set the progress bar again to its percentage value of completion according to xp and the level goal:

```
// Updates level displays in HTML
document.getElementById("xp").textContent = formatNum(Math.round(game_data.xp * 100) / 100)
document.getElementById("level").textContent = game_data.level
document.getElementById("level-goal").textContent = formatNum(game_data.level_goal)
document.getElementById("level-progress-bar").style.width = `${(game_data.xp / game_data.level_goal) * 100}%`
}
```

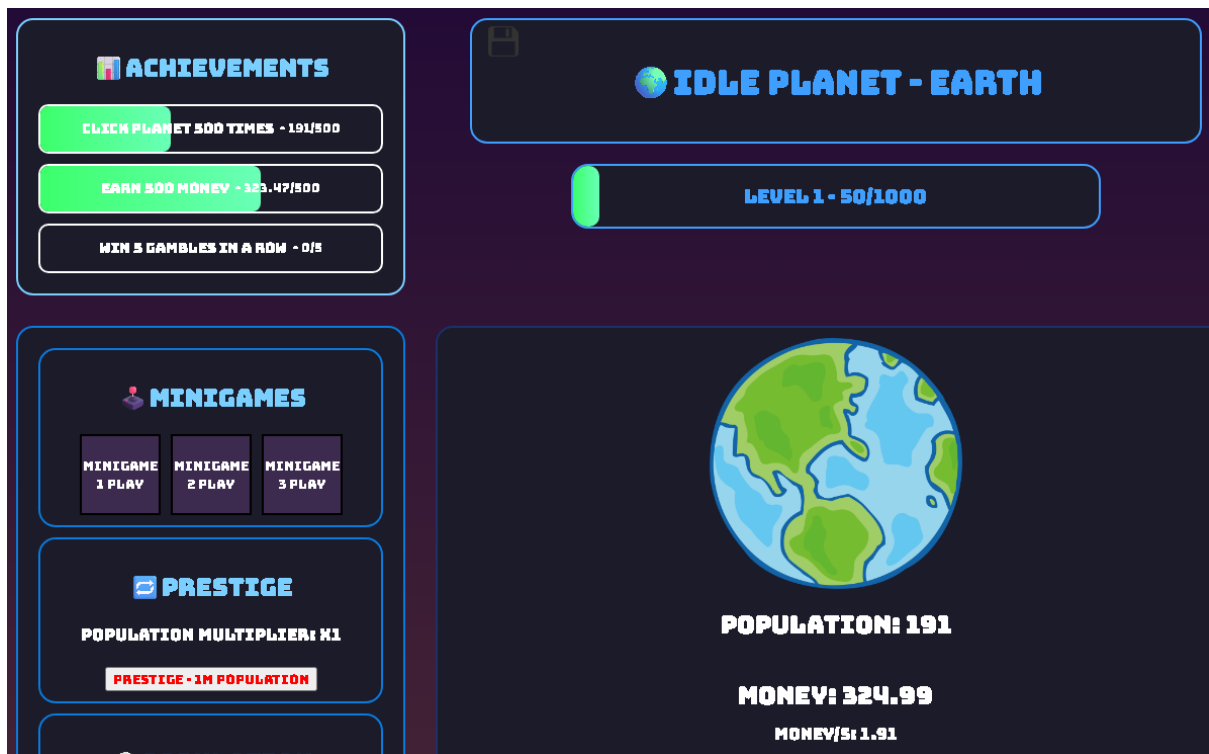
The reason I used `formatNum` and the rounding method for xp and level-goal, but not for level, is because I don't plan on the player's level ever

going over 1000 (in which case it would be formatted to “1k”), so it does not need to be formatted and will only increase by 1 each time so will never have extra digits to remove (which happens when multiplying the value)

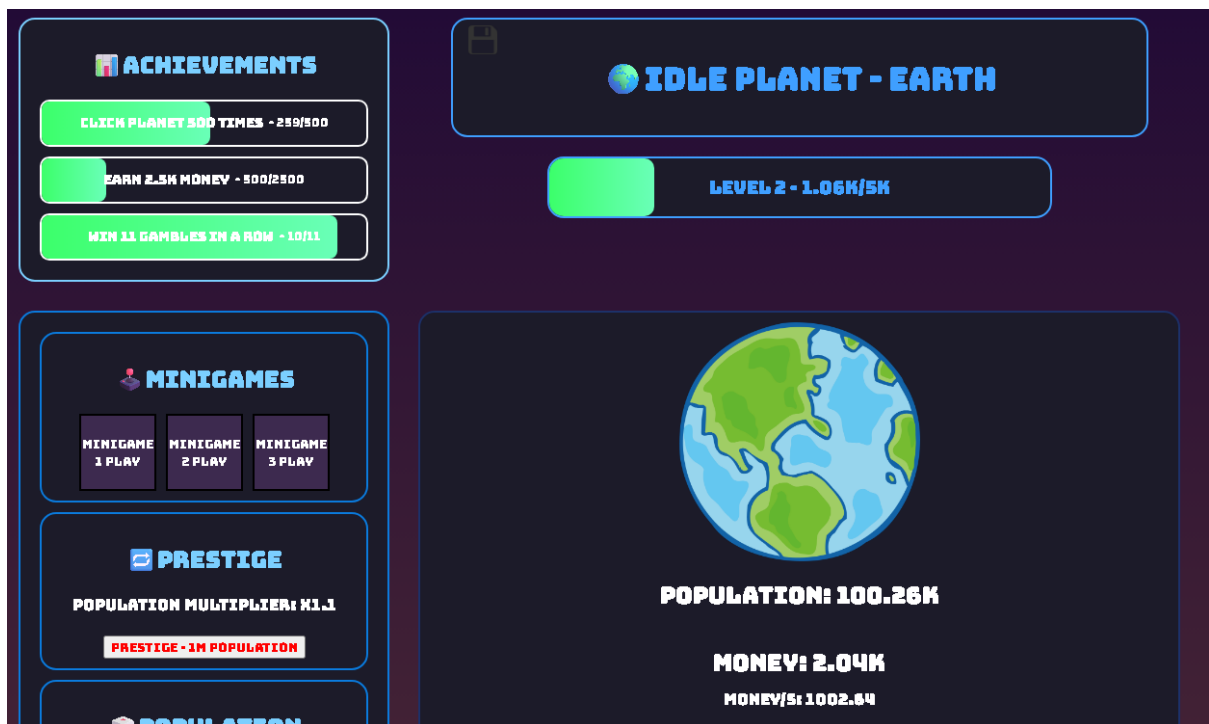
After this, I called the `increase_level()` function in the `complete_achievement()` function, after the gems and xp are increased, so that completing an achievement will update the players level because gems and xp have been rewarded:

```
function complete_achievement(type) {  
  const achievement = game_data.achievements[type]  
  
  game_data.gems += achievement.gems  
  game_data.xp += achievement.xp  
  
  increase_level() ←
```

I tested this all to see if it worked:



The level progress bar correctly increased after completing achievements, so the `increase_level()` function is working



The level increased and the level goal scaled, after levelling up, with the progress bar resetting to its new percentage value of completion, so the level_up() function is working

Finally, I wrote the code to restore the players level after they prestige, in the prestige() function:

```
let level = game_data.level
let level_goal = game_data.level_goal
let level_gem_reward = game_data.level_gem_reward
let level_population_reward = game_data.level_population_reward

prestige_level ++
prestige_cost *= 10

Object.assign(game_data, JSON.parse(JSON.stringify(default_game_data)))

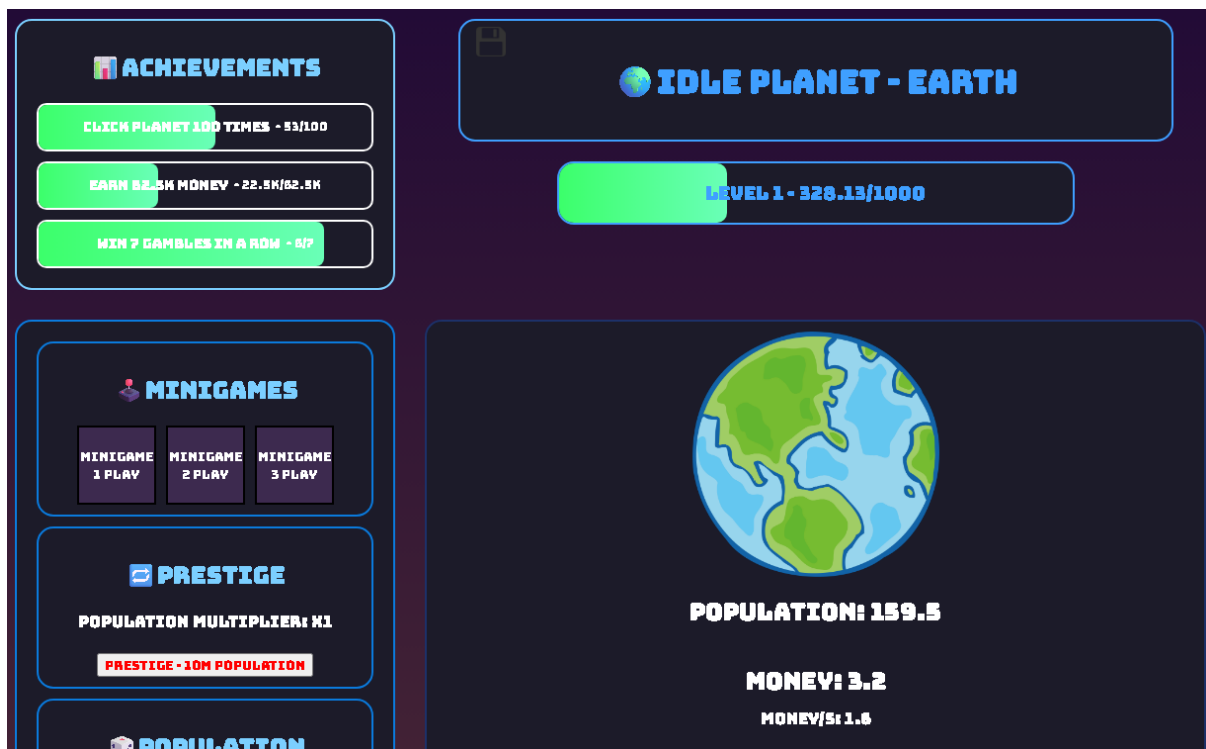
game_data.prestige_level = prestige_level
game_data.prestige_cost = prestige_cost
game_data.current_prestige_multiplier = current_prestige_multiplier
Object.assign(game_data.upgrades.quicker_prestige, quicker_prestige)

Object.assign(game_data.achievements, achievements)
game_data.achievements.clicks.element = document.getElementById("achievement-clicks")
game_data.achievements.money.element = document.getElementById("achievement-money")
game_data.achievements.gamble.element = document.getElementById("achievement-gambles")
game_data.gems = gems
game_data.xp = xp
game_data.win_streak = win_streak

game_data.level = level
game_data.level_goal = level_goal
game_data.level_gem_reward = level_gem_reward
game_data.level_population_reward = level_population_reward
```

This creates temporary variables for the level, level goal, and gem and population rewards. It then restores these values by setting them back in game_data after it is reset. The reason I want to restore the players level after prestiging, is for the same reason as achievements. The players level is a way to track their progress in the game, so I don't ever want to reset it, so they feel like they are making constant progress, even after they have prestiged.

I tested this to see if it worked:



Now, after prestiging, the level was restored and kept increasing like nothing had happened, so everything is working.

Testing:

This is my test plan for this stage, with some modifications:

- For the first test, I have changed the expected result to the actual amount the xp count should increase by (25), since I had not yet planned the exact reward number in the stages of development
- I originally planned the second test to test if the progress bar and count reset to 0 after levelling up, since this was the plan for the level system. However, I changed the way they work in the development. Now, when achievements are completed,

the xp count does not reset, but instead just the progress goal. The progress bar is then recalculated to its new completion. For example, after clicking the planet 100 times, the completion will be 100/500, so the achievement bar will reset to 20% completion, rather than 0. Therefore, I changed the expected result for this test to accommodate for this.

- For test 3, I changed the expected result to include the exact value that levelling up should scale to (5000 because 1000×5), since I had not planned the amount it would scale by in the stages of development
- For test 4, I included the exact amount that the population and gems should increase by after levelling up (100k and 50), since I had also not yet planned this in the stages of development
- Finally, I have removed test 5, for the level up pop-up screen, since I decided to remove this in the design of this stage

Test no.	Description	Test type	Test data	Expected Result	Actual result
1	Do the level progress bar and xp count increase when xp is granted		Complete the click the planet achievement and observe the level progress bar and xp count	The level bar should increase slightly, and the xp count should increase to 25	After completing the click achievement, the xp count increased to 25 (reward for click achievement), and the progress bar increased slightly. Passed 

					SEE STAGE 9 TEST 1.mp4 in evidence folder
2	Does the level progress bar and xp count reset correctly after level up		Level up to level 2 by completing achievements and observe the level progress bar and xp count	The level bar should reset to its new percentage value of completion, and the xp count should reset to 1000	After levelling up to level 2, the level progress bar reset to 20% completion (1k/5k) and the xp count reset to 1000 Passed ☒ SEE STAGE 9 TEST 2.mp4 in evidence folder
3	Does the level goal scale correctly after level up		Level up to level 2 by completing achievements and observe the level goal value	The level goal value should scale by x5, so levelling up to level 2 should increase it to 5000	After levelling up to level to, the level goal scaled to 5000 (x5) Passed ☒ SEE STAGE 9 TEST 3.mp4 in evidence folder

4	Does levelling up correctly reward population and gems		Level up to level 2 by completing achievements and observe the population and gem increase	Population should increase by 100000 and gems should increase by 50	After levelling up to level 2, the population increased by 100000, however the gems increased by only 10 (They increased by 86.94, however completing 10 successful gambles in a row gives the gamble achievement reward, which is 75.94 after 10 consecutive wins, so the reward from the level up was just 10) Failed ✖ SEE STAGE 9 TEST 4.mp4 in evidence folder
---	--	--	--	---	--

Remedial action for failed test 4;

- The reason for this test failing was because I set the level_rem_reward in game_data to 10, instead of 50, since the reward for levelling up should be 50
- Therefore, I changed it to 50:

```
// LEVEL SYSTEM

gems: 0,
xp: 0,

level: 1,
level_goal: 1000,
level_gem_reward: 50, ←
level_population_reward: 100000,
```

Stage review:

Overall, I am very happy with the development of this stage, as I managed to implement the level system with the progress bar correctly updating with xp gain, and rewards being correctly applied. This was because I reused the progress bar from the achievements, allowing me to use the same styling to make it look the same, and the same JavaScript code to calculate the percentage value of completion and update the width of the bar. Therefore, I was able to complete this stage in a short amount of time, which was great considering I have very little time left. There was one small error that I discovered in the final testing for the stage, which was just a mistake with the level gem reward, which I had set to 10 instead of 50, however this was a very simple fix which I did after the testing.

Stage 10 – Effects:

Features:

- 3 effects that apply a temporary multiplier when *purchased*, and require gems to purchase
 1. Double click power for 10 seconds – Clicking the planet generates 2x population for 10 seconds. Cost: 5 gems.
 2. Double money for 20 seconds – Money per second is doubled for 20 seconds. Cost: 10 gems.
 3. Double worker generation for 30 seconds – Workers generate 2x population for 30 seconds. Cost 50 gems.
- Effects cannot be stacked. Once an effect is purchased, it cannot be purchased until it's effect has run out.
- Effect buy button text colour will be red when the player doesn't have enough gems to afford it, and green when they do

Data dictionary:

Variable	Data type	Purpose
game_data.effects {}	Dictionary	Stores the dictionary for each

		effect and their values
game_data.effects. double_click_power {}	Dictionary	Stores the double click power effect dictionary
game_data.effects. double_money {}	Dictionary	Stores the double money effect dictionary
game_data.effects. double_worker_gen {}	Dictionary	Stores the double worker gen effect dictionary
game_data.effects.[effect_name]. cost	Key (Integer)	Stores the cost in gems of the effect
game_data.effects.[effect_name]. time	Key (Integer)	Stores the length the effect it active for
game_data.effects.[effect_name]. button	Key (Document Object)	Stores the button of the object in the HTML, so event listeners can be added

game_data.effects.[effect_name].active	Key (Boolean)	Stores the boolean value for whether the effect is currently active or not
game_data.effect_names[]	Key (Array)	Stores the names (strings) of each effect, so I can iterate over it and add an event listener to each button

Pseudocode:

Add to game_data:

```
dict effects {
  dict double_click_power {
    int cost = 5
    int time = 10
```

```

        Button button = HTML.Element.ID("click-
effect-button")

        Boolean active = false
    }

    dict double_money {

        Int cost = 10

        Int time = 20

        Button button = HTML.Element.ID("money-
effect- button")

        Boolean active = false
    }

    double_worker_gen {

        Int cost = 50

        Int time = 30

        Button button = HTML.Element.ID("worker-
effect-button")

        Boolean active = false
    }

}

```

```

Array effect_names = ["double_click_power",
"double_money", "double_worker_gen"]

```

Effect buy button text colour:

```

WHILE true
    FOR N = 0 TO effect_names.length
        IF gems >= effects.effect_names[N].cost
            THEN

```

```

        HTML.effects.effect_names.button.text
        = green
    ELSE
        HTML.effects.effect_names.button.text
        = red
    ENDIF
ENDFOR
WAIT 1
ENDWHILE

```

- I will have an infinite loop that runs every second, that will update the text colour of each effect's buy button
- It iterate over each effect, using the effect_names array to access the effect in game_data.effects, to check if the gems is greater than the cost of the effect. If so , the text colour will be set to green, if not then red
- Since I already have an infinite loop that does this for other buttons, I will put it in there

Effect price update:

```

WHILE true
    FOR N = 0 TO element_names.length
        HTML.effects.element_names[N].button.
        text = effects.element_name.cost
    ENDFOR
    WAIT 1
ENDWHILE

```

- I will have an infinite loop, that runs every second, to update the price display of the effect to its cost

Effect button event listeners:

```

FOR N = 0 TO element_names.length

    effects.element_names[N].button
    EventListener(click) {

        IF(gems >= effects.element_names[N].cost)
        THEN

            purchase_effect(effect_names[N])

        ENDIF

    }

ENDFOR

```

- I will have a for loop that will iterate over each effect, adding an event listener to each button
- In the event handler, I will check if the player's gems is greater than or equal to the cost of the effect, and if so I will call the purchase_effect() function (which I will develop after this) and pass in the effect name

Effect purchase:

```

FUNCTION purchase_effect(effect_name)

    gems -= effects.effect_name.cost

    HTML.gems.text = gems

```

```
        effects.effect_name.active = true
        WAIT effects.effect_name.time
        effects.effect_name.active = false
    ENDFUNCTION
```

- I will have a function that will handle purchasing the effect
- It will start by reducing the player's gems by the cost of the effect, and updating the gem display
- It will then set the active variable of the effect to true, so it can take effect
- It will then wait for a certain amount of time, that time being the time variable of the effect, and then set the active variable of the effect to false.

Development

I started development of this section by adding the names of all the effects in each h3 tag inside each effect-row div:


```

<div class="effects-list">
  <div class="effect-row">
    <div class="effect"><h3>Double click power <br> (10 secs)</h3></div>
    <div class="effect-price"><span>Price</span></div>
  </div>
  <div class="effect-row">
    <div class="effect"><h3>Double money <br> (20 secs)</h3></div>
    <div class="effect-price"><span>Price</span></div>
  </div>
  <div class="effect-row">
    <div class="effect"><h3>Double worker gen <br> (30 secs)</h3></div>
    <div class="effect-price"><span>Price</span></div>
  </div>
</div>

```

I added a line break before the time for each effect, so it would display underneath the effect title and wouldn't overflow the container. I also made the text size smaller so it would fit:

```

.effect {
  border: 2px solid black;
  border-radius: 10px;
  width: 200px;
  height: 40px;
  display: flex;
  align-items: center;
  justify-content: center;
  font-size: 12px;
}

```

I then realised I needed to put the span for the price of each effect inside a button, so that it could be clicked to purchase:

```

<div class="effect-row">
  <div class="effect"><h3>Double click power <br> (10 secs)</h3></div>
  <div class="effect-price"><button id="click-effect-button"><span>Price</span></button></div>
</div>
<div class="effect-row">
  <div class="effect"><h3>Double money <br> (20 secs)</h3></div>
  <div class="effect-price"><button id="money-effect-button"><span>Price</span></button></div>
</div>
<div class="effect-row">
  <div class="effect"><h3>Double worker gen <br> (30 secs)</h3></div>
  <div class="effect-price"><button id="worker-effect-button"><span>Price</span></button></div>
</div>

```

I gave each button an id so I could access it in the JavaScript to add an eventListener.

I then removed the background for each button and set the border radius to the same as its container. I also set the height to 40 and width to 100 so it would fit inside but be slightly smaller than the container, so its clear it is a button:

```
.effect-row button {  
  background: none;  
  border-radius: 10px;  
  height: 35px;  
  width: 95px;  
}
```

This is what it looked like at this point:



I then realised I needed to set the font of the price button text to the same as the rest of the game, and I increased the font size slightly:

```
.effect-row button {
  background: none;
  border-radius: 10px;
  height: 40px;
  width: 100px;
  font-family: "Bungee", sans-serif;
  font-size: 15px;
}
```

After this, I developed the code to set the price display of each effect to it's gem cost:

```
// Iterates over effect objects
for(let i = 0; i < game_data.effect_names.length; i++) {
  // If player can afford effect, sets effect button text colour to green
  if(game_data.gems >= game_data.effects[game_data.effect_names[i]].cost) {
    game_data.effects[game_data.effect_names[i]].button.style.color = "green"
  // If not, sets it to red
  } else {
    game_data.effects[game_data.effect_names[i]].button.style.color = "red"
  }
}

check_gamble()
}, 1000)
```

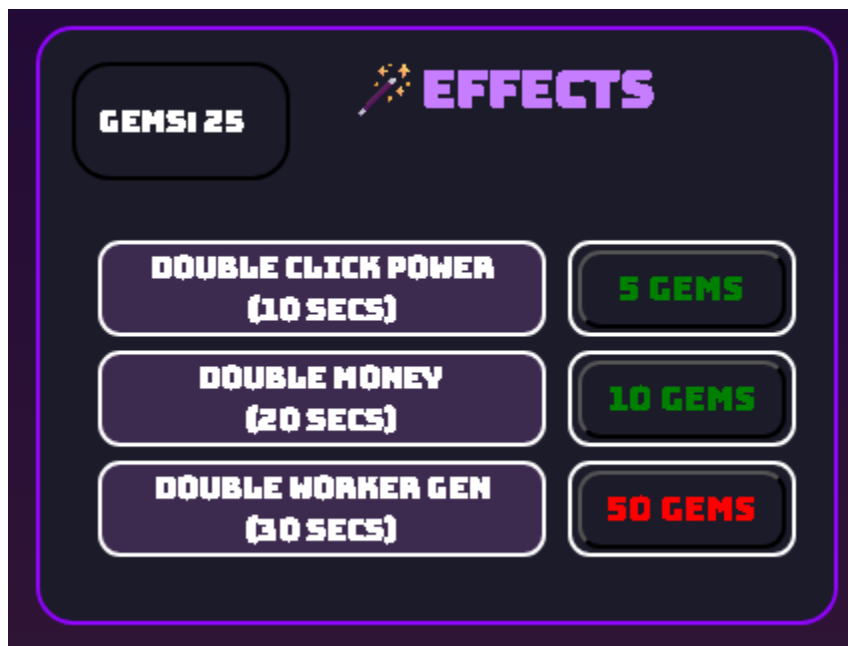
I iterate over each effect dictionary, using the effect_names array, and set the colour of the text to green if the player can afford the effect, and red if not. I put this in the same loop that does this for the other buttons, preventing me having to write a separate loop.

I then created an infinite loop to update the price display of each effect to its gem cost:

```
// Loop that runs every second to update the price of each effect
setInterval(() => {
  // Iterates over effect objects
  for(let i = 0; i < game_data.effect_names.length; i++) {
    // Sets text content of each button to the price of the effect
    game_data.effects[game_data.effect_names[i]].button.textContent = `${game_data.effects[game_data.effect_names[i]].cost} gems`
  }
}, 1000)
```

Here, like with the text colour update, I iterate over the effect dictionaries, accessing them using the effect_names array, and set the price of their HTML button to the cost in gems of the effect, with “gems” concatenated on the end.

I tested this to see if it worked:



Now, the effect buttons show the correct value of gems, and the text colour reflects whether the player can afford the effect

After this, I developed the code to add event listeners to all the effect buttons, and call the `purchase_effect()` function in the event handler if the player can afford the effect:

```
// Adds event listener to each effect button
for(let i = 0; i < game_data.effect_names.length; i++) {
  game_data.effects[game_data.effect_names[i]].button.addEventListener("click", function(event) {
    let effect_active = game_data.effects[game_data.effect_names[i]].active
    // Checks if player can afford effect and if it is not already active (Cannot be purchased again while active)
    if(game_data.gems >= game_data.effects[game_data.effect_names[i]].cost && !effect_active) {
      // Purchases effect
      purchase_effect(game_data.effect_names[i])
    }
  })
}
```

I use a for loop to iterate over each item in the `effect_names` array, using it to access the effect dictionary in `effects`, to add a click event listener to it's button. I then use an if statement to check if the player's gems are greater than the effect cost and that the effect is not currently active. This ensures the player can afford the effect and that it is not currently active (because the player cannot purchase an effect whilst it is already active). If both the checks pass, I call the `purchase_effect()` function pass in the current effect. The reason I use a for loop to add the event listeners, is because it prevents me having to type out the event listener for each button. Using just 1 for loop and an array, I can add all 3 event listeners, which saves time and prevents unnecessary duplication of code.

After this, I coded the `purchase_effect()` function:

```
// Function to handle purchasing of an effect
function purchase_effect(effect_name) {

  // Subtracts effect cost from players gems and updates gem display in HTML
  game_data.gems -= game_data.effects[effect_name].cost
  document.getElementById("gems").textContent = formatNum(Math.round(game_data.gems * 100) / 100)
```

I start by defining the function, with the parameter `effect_name`, which will be passed in from the previous code, and will be the name of the effect

dictionary for the effect that is being purchased. I then reduce the gems by the cost of the effect, and update the gem display. This is because the effect is being purchased, so the player needs to lose gems.

```
// Activates effect
game_data.effects[effect_name].active = true
setTimeout(() => {
  // Deactivates effect after effect time has passed
  game_data.effects[effect_name].active = false
}, game_data.effects[effect_name].time * 1000)
}
```

After this, I set the active variable of the effect to true, as it is becoming active, and then I use `setTimeout` to wait the effect time, and then set it to false. The reason I multiply the effect time by 1000 when setting the loop time, is so that it converts it to milliseconds, which is what `setTimeout` uses to count the time.

I added a 2 `console.log` statements to ensure it was working:

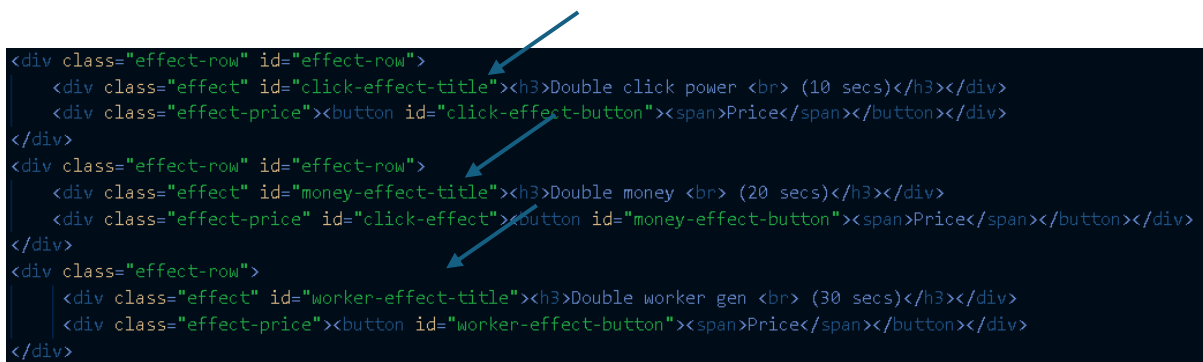
```
// Activates effect
game_data.effects[effect_name].active = true
console.log(game_data.effects[effect_name] + " activated")
setTimeout(() => {
  // Deactivates effect after time has passed
  game_data.effects[effect_name].active = false
  console.log(effect_name + " deactivated")
}, game_data.effects[effect_name].time * 1000)
}
```

double_click_power activated	script.js:872
double_money activated	script.js:872
double_worker_gen activated	script.js:872
double_click_power deactivated	script.js:875
double_money deactivated	script.js:875
double_worker_gen deactivated	script.js:875

After clicking each effect buy button, the first `console.log` statement ran for each effect, and then after 10 seconds, the `double_click_power` output ran, and then after another 10 seconds, the `double_money` output ran, and then

after another 10 seconds, the double_worker_gen output ran. Therefore, the function is working.

I also decided to make it clear that the effect is active by making the background colour of the effect flash green. To do this, I gave each effect title a unique id so I could access it in the JavaScript:



```
<div class="effect-row" id="effect-row">
  <div class="effect" id="click-effect-title"><h3>Double click power <br> (10 secs)</h3></div>
  <div class="effect-price"><button id="click-effect-button"><span>Price</span></button></div>
</div>
<div class="effect-row" id="effect-row">
  <div class="effect" id="money-effect-title"><h3>Double money <br> (20 secs)</h3></div>
  <div class="effect-price" id="click-effect"><button id="money-effect-button"><span>Price</span></button></div>
</div>
<div class="effect-row">
  <div class="effect" id="worker-effect-title"><h3>Double worker gen <br> (30 secs)</h3></div>
  <div class="effect-price"><button id="worker-effect-button"><span>Price</span></button></div>
</div>
```

I then added a variable to each of the effect dictionaries, where I access this element using its id:

```

effects: {
  double_click_power: {
    cost: 5,
    time: 10,
    button: document.getElementById("click-effect-button"),
    active: false,
    title: document.getElementById("click-effect-title")
  },
  double_money: {
    cost: 10,
    time: 20,
    button: document.getElementById("money-effect-button"),
    active: false,
    title: document.getElementById("money-effect-title")
  },
  double_worker_gen: {
    cost: 50,
    time: 30,
    button: document.getElementById("worker-effect-button"),
    active: false,
    title: document.getElementById("worker-effect-title")
  }
},

```

I then added a setInterval loop that runs every 0.5s in the purchase_effect() function, before the setTimeout, to make the element flash:

```

let flash = false;
// Flashes effect background green, on and off, every 0.5 seconds whilst effect is active
const flash_interval = setInterval(() => {
  if(flash) {
    game_data.effects[effect_name].title.style.backgroundColor = "limegreen"
    flash = false
  } else {
    game_data.effects[effect_name].title.style.backgroundColor = "■#3D2A4F"
    flash = true
  }
}, 500);

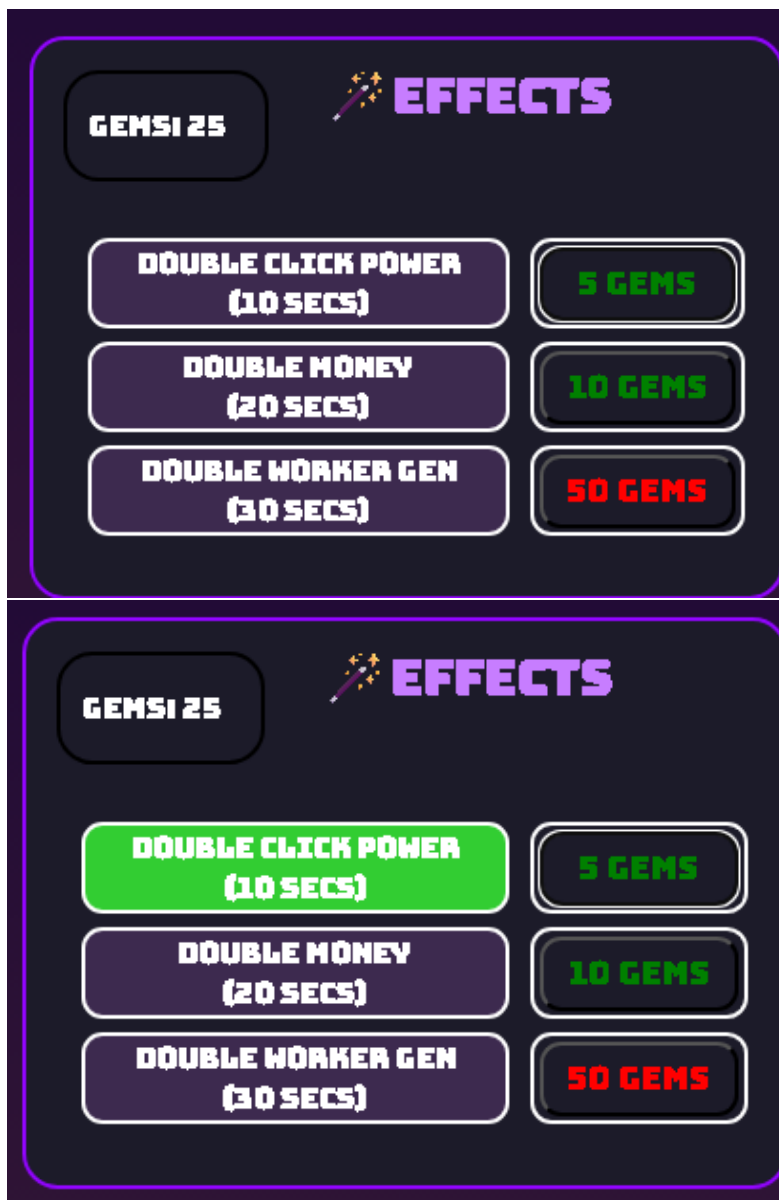
```

I use a flash variable, which I toggle on and off each loop. When it is toggled to true, I set the background colour of the effect title to limegreen then set

toggle flash off, and when it is not I set it to its default colour and toggle flash back on. I then clear the interval in the setTimeout loop and set the background colour of the effect back to its default colour, so the flashing stops when the effect is deactivated:

```
setTimeout(() => {  
  clearInterval(flash_interval) // Stops effect flashing  
  game_data.effects[effect_name].title.style.backgroundColor = "#3D2A4F"  
  // Deactivates effect after time has passed  
  game_data.effects[effect_name].active = false  
  console.log(effect_name + " deactivated")  
}, game_data.effects[effect_name].time * 1000)  
}
```

I tested this to see if it worked:



Now, the background colour of the effect flashed green on and off when purchased

Finally, I developed the code to apply the effects.

For each effect, I will check if the effect is active in the correct part of the code, then calculate the amount that the population or money has increased by, and then double it and apply it.

Double click power effect:

```
document.getElementById("planet-button").addEventListener("click", function (event) {
  update_achievement("clicks", 1)
  let population = game_data.population // Creates a temporary variable which stores the population before increase
  game_data.population += 1 * (1 + (game_data.upgrades.click_surge.level * 2)/100) // Increases population by 1 mult
  game_data.population += Number(game_data.current_prestige_multiplier)

  // Adds the population increase if the double click effect is active, which doubles the total increase.
  if(game_data.effects.double_click_power.active) {
    game_data.population += game_data.population - population
  }
})
```

I create a temporary variable for population, before it is increased, then check if double click is active, and if so I increase population by the amount the population has increased since it was updated. This gets the amount population increases by, taking into account click surge and the prestige multiplier, and adds it onto population which doubles the total amount it increases by

Double money power effect:

```
setInterval(() => {
  let money = Math.round(game_data.money * 100) / 100
  game_data.money += game_data.money_per_second
  if(game_data.effects.double_money.active) {
    game_data.money += game_data.money - money
  }
}, 1000)
```

I already have a temporary variable for money, since I used it to calculate the earn money achievement, so I just use that to double the amount money increases by.

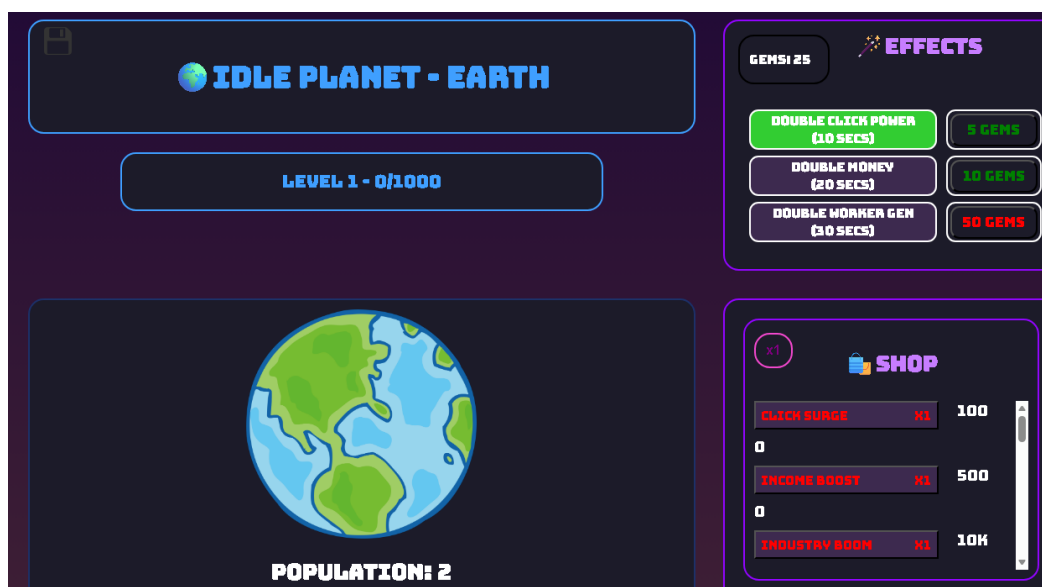
Double worker generation effect:

```
setInterval(() => {  
    let money = Math.round(game_data.money * 100) / 100 // Creates a temporary variable which stores the population before increase  
    game_data.money += game_data.money_per_second  
    // Adds the population increase if the double click effect is active, which doubles the total increase.  
    if(game_data.effects.double_money.active) {  
        game_data.money += game_data.money - money  
    }  
}, 1000)
```

Here, I do the same thing again, create a temporary variable for population and if the double worker gen effect is active, I increase population by the amount it has increased by, which doubles the total increase. I commented this whole loop as well, since I had not fully explained it:

```
// Infinite loop that increases money by money per second  
setInterval(() => {  
    let money = Math.round(game_data.money * 100) / 100 // Creates a temporary variable which stores the population before increase  
    game_data.money += game_data.money_per_second  
    // Adds the population increase if the double click effect is active, which doubles the total increase.  
    if(game_data.effects.double_money.active) {  
        game_data.money += game_data.money - money  
    }  
    game_data.money = Math.round(game_data.money * 100) / 100 //Removes extra, unwanted decimal places  
    document.getElementById("money").textContent = formatNum(game_data.money)  
    update_achievement("money", game_data.money - money) // Updates money achievement  
}, 1000)
```

I tested this to see if it worked:



Now, after purchasing the double click power effect, clicking the planet once increased the population by 2, which is double the normal value of 1.

I also realised that since I have 2 document Object values in each effect dictionary (button and title), I will need to make sure these are correctly restored after prestiging. I had a problem with the achievements, where their “element” value which was a document object would not be correctly copied after prestige, so I had to redefine it after game_data was reset. Therefore, I also did this for the effects:

```
game_data.effects.double_click_power.button = document.getElementById("click-effect-button")
game_data.effects.double_click_power.title = document.getElementById("click-effect-title")
game_data.effects.double_money.button = document.getElementById("money-effect-button")
game_data.effects.double_money.title = document.getElementById("money-effect-title")
game_data.effects.double_worker_gen.button = document.getElementById("worker-effect-button")
game_data.effects.double_worker_gen.title = document.getElementById("worker-effect-title")
// Removes previous save from local storage
localStorage.removeItem("game_data")
}
```

I then also added these in the window.onload function, so that they are also redefined when the page loads:

```
window.onload = function() {
    Object.assign(game_data, (JSON.parse(localStorage.getItem("game_data"))))
    game_data.achievements.clicks.element = document.getElementById("achievement-clicks")
    game_data.achievements.money.element = document.getElementById("achievement-money")
    game_data.achievements.gamble.element = document.getElementById("achievement-gambles")
    game_data.sounds.win = new Audio("win.mp3"),
    game_data.sounds.lose = new Audio("lose.mp3")
    game_data.effects.double_click_power.button = document.getElementById("click-effect-button")
    game_data.effects.double_click_power.title = document.getElementById("click-effect-title")
    game_data.effects.double_money.button = document.getElementById("money-effect-button")
    game_data.effects.double_money.title = document.getElementById("money-effect-title")
    game_data.effects.double_worker_gen.button = document.getElementById("worker-effect-button")
    game_data.effects.double_worker_gen.title = document.getElementById("worker-effect-title")
    update_game_displays()
}
```

I then also updated the gem, level and achievement displays in the `update_game_displays()` function, as I realised that the display will need to be updated after loading the `game_data` from local storage:

```
function update_game_displays() {
  update_prices()
  update_price_and_level()

  document.getElementById("population").textContent = formatNum(game_data.population)
  document.getElementById("money").textContent = formatNum(game_data.money)
  document.getElementById("money-per-second").textContent = formatNum(game_data.money_per_second)
  document.getElementById("workers").textContent = formatNum(game_data.workers)
  document.getElementById("worker-price").textContent = formatNum(game_data.worker_price)
  document.getElementById("worker-gen").textContent = formatNum(Math.round((game_data.total_worker_gen / game_data.worker_speed) * 100) / 100)
  document.getElementById("prestige-cost").textContent = formatNum(game_data.prestige_cost)
  document.getElementById("win").hidden = true
  document.getElementById("loss").hidden = true
  document.getElementById("win-chance").textContent = `${Math.round((game_data.win_chance * 100) * 100) / 100}%`
  document.getElementById("gems").textContent = game_data.gems
  game_data.achievements.clicks.element.querySelector("div").style.width = `${(game_data.achievements.clicks.progress / game_data.achievements.clicks.goal) * 100}%`
  game_data.achievements.clicks.element.querySelector("span").textContent =
    '\u00A0 - ${formatNum(Math.round(game_data.achievements.clicks.progress * 100) / 100)} / ${formatNum(Math.round(game_data.achievements.clicks.goal * 100) / 100)}'
  game_data.achievements.money.element.querySelector("div").style.width = `${(game_data.achievements.money.progress / game_data.achievements.money.goal) * 100}%`
  game_data.achievements.money.element.querySelector("span").textContent =
    '\u00A0 - ${formatNum(Math.round(game_data.achievements.money.progress * 100) / 100)} / ${formatNum(Math.round(game_data.achievements.money.goal * 100) / 100)}'
  game_data.achievements.gamble.element.querySelector("div").style.width = `${(game_data.achievements.gamble.progress / game_data.achievements.gamble.goal) * 100}%`
  game_data.achievements.gamble.element.querySelector("span").textContent =
    '\u00A0 - ${formatNum(Math.round(game_data.achievements.gamble.progress * 100) / 100)} / ${formatNum(Math.round(game_data.achievements.gamble.goal * 100) / 100)}'
  document.getElementById("xp").textContent = formatNum(Math.round(game_data.xp * 100) / 100)
  document.getElementById("level").textContent = game_data.level
  document.getElementById("level-goal").textContent = formatNum(game_data.level_goal)
  document.getElementById("level-progress-bar").style.width = `${(game_data.xp / game_data.level_goal) * 100}%`
}
```

Finally, since this is the last stage I will have time for, meaning I would not be able to implement all the animations I wanted to, I added a simple button bounce animation to the planet button, which I got from [TheLinuxCode](#), (TheLinuxCode, n.d.):

Bounce Click Effect

A subtle bounce is one of the most common click animation effects. It provides positive feedback that the button was activated. Here is one way to achieve a gentle bounce with CSS:

```
button {  
  transition: transform 0.1s ease-out;  
}  
  
button:active {  
  transform: translateY(2px);  
}
```

Breaking this down:

- `transition: transform` – Allows smooth change of the transform property
- `transform: translateY(2px)` – Moves the button down 2px on click
- `active state` – Applies while button is being clicked

This creates a quick bounce down effect on click. We're only moving the button 2px to keep it subtle. Adjust the `translateY` to increase the bounce strength if desired.

```
.planet-button {  
  transition: transform 0.3s ease-out;  
}  
  
.planet-button:active {  
  transform: translateY(20px);  
}
```

I changed the selector to `.planet-button` so it would only apply to the planet button, increased the bounce size to 20px due to the size of the button, and increased the transform time to 0.3s to make it more smooth

I also wanted to add it to all the buttons, but a smaller one, so I applied the same effect to all buttons, but with a size of only 5px:

```

button {
  transition: transform 0.1s ease-out;
}

button:active {
  transform: translateY(5px);
}

.planet-button {
  transition: transform 0.3s ease-out;
}

.planet-button:active {
  transform: translateY(20px);
}

```

I then added a hover glow effect to the planet button, from SDR (superdevresources), (SDR, n.d.):

```

.button {
  text-decoration: none;
  color: rgba(255, 255, 255, 0.8);
  background: rgb(145, 92, 182);
  padding: 15px 40px;
  border-radius: 4px;
  font-weight: normal;
  text-transform: uppercase;
  transition: all 0.2s ease-in-out;
}

```

Now we add the definition of `glow-button` in our CSS. In order to have this glow animation only when user hovers over the button, we are using the [CSS selector](#) `:hover`. If you need to have a permanent glow effect on the button, all you need to do is to remove this `:hover` selection from the `glow-button` definition.

```

.glow-button:hover {
  color: rgba(255, 255, 255, 1);
  box-shadow: 0 5px 15px rgba(145, 92, 182, .4);
}

```

```

.planet-button {
  transition: all 0.2s ease-in-out;
}

.planet-button:hover {
  background: none;
  border-radius: 120px;
  color: ■ rgba(255, 255, 255, 1);
  box-shadow: 0 10px 20px ■ rgba(145, 92, 182, .4);
  transition: 0.3s ease-in-out;
}

```

I increased the size of the box shadow by 5px as I wanted it larger, and set the border radius to 120px as this was the approximate border radius of the planet image

Testing:

This is my test plan for this stage, with some modifications:

- I have added a test at the strat to check if the effect button text colour changes to green when the player can afford the effect, and red when they can't, since this was a feature I planend in the design of this development cycle
- I also added tests for the functionality of the specific tests I design in this stage. In the test plan, I simply planned to test whether the effects correctly applied their effect, since I had not planned the specific effects at that point.

Test no.	Description	Test type	Test data	Expected Result	Actual result
----------	-------------	-----------	-----------	-----------------	---------------

1	Does the effect button text colour update correctly depending on whether the player can afford the effect		Complete the click the planet achievement once to get 5 gems and observe the effect buy buttons	The button text colour for the first effect should be green, and the second 2 should be red	After getting 5 gems, the text colour for the first effect was green, since the player can afford it, while the rest remained red since they cost more gems than the player has. Passed  SEE STAGE 10 TEST 1.mp4 in evidence folder
2	Does the double click power apply its effect correctly		Purchase the double click power effect and click the planet button	The population should increase by 2 each click, instead of the normal 1	Passed  SEE STAGE 10 TEST 2.mp4 in evidence folder
3	Does the double money apply its effect correctly		Purchase double money and click the planet to get 5 population	Money should increase by 0.1, instead of the normal 0.05	After purchasing the effect, with a population of 5, money was increasing by 0.1 per second, instead of the normal 0.05 (5 x 0.1) without the effect Passed  SEE STAGE 10 TEST 3.mp4 in evidence folder
4	Does the double worker		Purchase a worker and then the double	The population should increase by 2	After purchasing the

	gen apply its effect correctly		worker gen effect and observe the population increase	every 2.5 seconds, instead of the normal 1	effect, the population increased by 2 each worker action, instead of 1. Passed  SEE STAGE 10 TEST 4.mp4 in evidence folder
--	--------------------------------	--	---	--	--

Stage review:

Overall, I am very happy with the development of this stage, as I was able to correctly implement the effects, with an additional feature of the effect background flashing green whilst the effect is active. I even implemented a simple bounce on click animation for all the buttons, which was not related to the effects, however I am almost out of time for the development so I know I won't be able to reach the animations stage. I managed to do all of this in a relatively short amount of time, without any errors.

Evaluation

Post-development testing:

1. Functionality Testing:

- According to my post-dev test plan, I recorded a video run-through of my game to test its overall functionality, to ensure all the components correctly integrate together to create a working game.
- **See** “*Idle Planet Video Runthrough.mp4*” in evidence folder
- **Results:**
 - Consistent evidence of **main game loop** (planet clicking and money generation) functioning throughout gameplay:
E.g. 0:00 – 0:10, 2:57 – 3:11, 5:30 – 5:42, 6:35 – 6:45, 7:44 – 8:15, 11:51 – 11:58
 - Clicking was smooth, population increased as normal throughout, and money generated automatically the whole time.
 - Consistent evidence of the **population gambling feature** functioning correctly throughout gameplay:
E.g. 0:11 - 0:14, 0:40 - 1:15, 1:39 - 1:41, 2:45 - 2:52, 3:35 - 3:38, 4:56 - 5:05, 6:14 - 6:20, 6:27 - 6:34, 7:07 - 7:11, 8:24 - 8:28, 11:05-11:15, 13:52-14:10
 - Gambling is allowed when appropriate, sounds playing and result messages showing correctly, win return and win chance correctly calculated, rewards correctly granted, consecutive gambles correctly updating gamble achievement

- Consistent evidence of **temporary effects** being correctly applied when purchased, throughout gameplay:

0:26 - 0:37 - Double click power, 2:33 - 2:44, 3:23, 10:34

1:59 - 2:09 - Double money, 4:37, 5:09 - 5:19, 10:20, 12:36,

13:28 - Double worker gen

- Effect purchase is consistently handled correctly, temporary effect is consistently applied.
- Evidence of effects overlapping and functioning as normal without affecting each other at 10:34 (Double money and double click power)

- Consistent evidence of **upgrades** correctly applying their effects when purchased, throughout gameplay:

1:46 - 1:58 - Click surge, 2:25-2:30, 3:18, 3:53,

4:48 - 4:53 - Worker Training, 5:46 - 5:49,

6:24 - Income boost + Click surge,

8:21 - Worker training, 10:29, 10:44,

12:10 - Worker efficiency,

12:20 - Worker training, 13:10 - 13:19,

13:33 - 13:37 - Achievement Gem and XP Boost,

14:18 - Population Multiplier Boost,

14:30 - Quicker Prestige,

18:07 - 18:45 - Gamble luck,

- Upgrade purchase is consistently handled correctly, upgrade effects correctly applied, upgrades stack correctly throughout gameplay, prices correctly scale.
- Upgrade caps correctly applied whenever a purchase attempt is made that would hit the cap:

18:07 - 18:45 – Gamble luck cap

21:31 - Population Multiplier Boost and Worker Efficiency Caps

- Consistent evidence of **multi-purchase feature** functioning and correctly integrating with upgrade purchases:

9:50 - 10:10, 12:43 - 13:09, 17:31 - 17:59

- Multi-purchase feature correctly and dynamically scales prices and upgrade purchase amounts whenever button pressed
- Handles upgrade purchases by increasing upgrade level and reducing price based of the setting selected

- Consistent evidence of **workers** functioning and stacking when purchased:

2:11 - 2:22, 3:17, 4:06, 7:38, 8:36, 10:30, 10:55, 13:39

- Worker purchase handled correctly, with price scaling and displays updating dynamically
- Automatic worker generation consistent throughout the gameplay, with population increasing with each worker action and the population display updating dynamically
- Stacking workers correctly increases worker generation throughout the game
- Worker generation and speed correctly upgraded by Worker Training and Worker Efficiency upgrades throughout the gameplay

- Consistent evidence of **achievements** functioning correctly:

E.g. 0:15 - 0:25 - Click achievement completion, 1:17 - 1:35, 4:32

1:43 - Money achievement completion, 2:30, 12:27 – 12:34

16:42 – 16:53 - Gamble achievement completion

- Achievement progress bars correctly updating throughout the game, automatically, as the player makes progress towards them, with progress completion values updating correctly
- Achievement completion correctly granting rewards, with the achievement goal scaling correctly, achievement progress bars

resetting and progress count value resetting, throughout the gameplay.

- Achievement xp rewards correctly and automatically updating level progress bar and progress count, throughout the gameplay

- Consistent evidence of **level ups** functioning correctly:

13:25 – Level 2, 20:35 - Level 3

- Level completion correctly grants rewards, with the level goal scaling correctly, level progress bar and progress count resetting, each level up.

- Consistent evidence of **progress saving** functioning correctly:

7:17 - 7:32, 12:00 - 12:05, 14:58, 21:54

- Progress saved automatically throughout gameplay, with the game being restored to last save whenever it is refreshed.
- Progress saved whenever save button pressed, throughout the game, with the game being restored to the point at which the button was pressed
- All displays and progress bars update to their correct values as soon as the page loads

- Consistent evidence of **prestige** functioning correctly:

14:36, 22:10

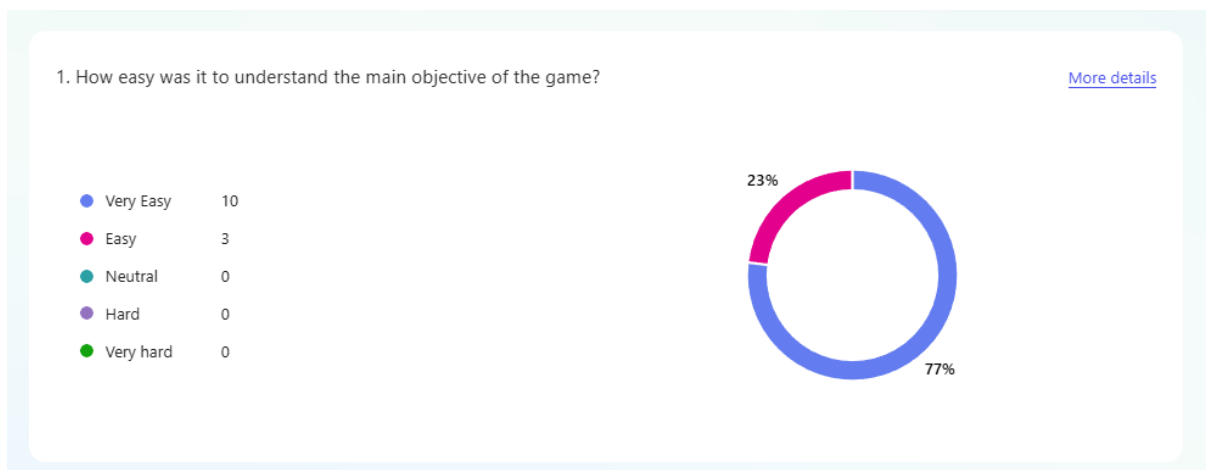
- Prestige handled correctly, being allowed when the player has the correct population, and correctly resetting the game
- Achievements, level and gem restored correctly whenever there is a prestige, and correctly functioning afterwards
- Population multiplier dynamically and automatically updating throughout the game, as the population increases
- Main game loop and all features functioning correctly and consistently after prestige:

14:49-16:40 – Game functioning as normal with all features correctly integrating, after prestige.

2. Usability Testing:

- As per my post-dev test plan, I created a survey with the questions I planned to get feedback on the usability and overall user experience, and distributed it to a range of stakeholders, including my class peers and friends.
- **Survey results:**

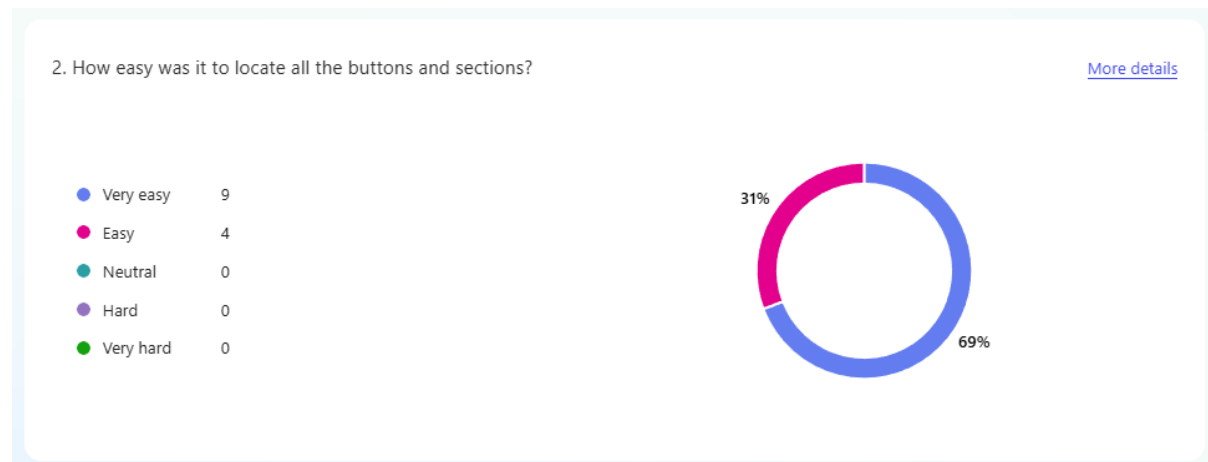
Question 1:



This question was designed to find out whether it was clear to the player how to play the game and what they needed to do to make progress,

The results were very positive for this question, showing that 77% of participants found the main objective very easy to understand, with all participants saying it was at least easy. This shows that I have designed my GUI in a way that makes the game objective obvious, and it is clear to the players, without explanation, how they need to play. This shows that the game is accessible and easy to understand, which I am happy with.

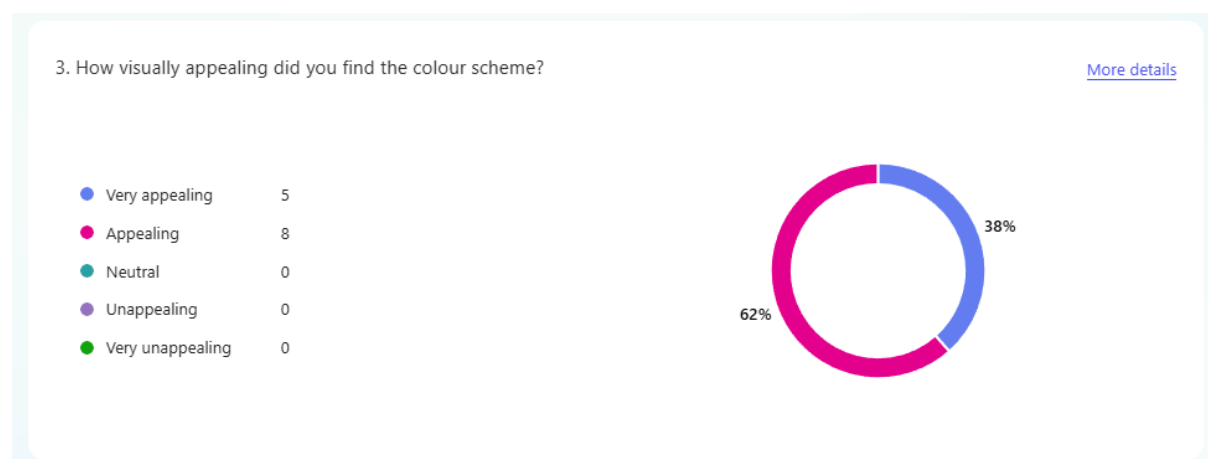
Question 2:



This question was designed to find out whether the GUI was easy to navigate, with the player easily being able to locate and identify all the buttons and sections.

Again, the results for this question were very positive. 69% of participants said that it was very easy, with all participants saying it was at least easy. This shows that the GUI is well designed and easy to navigate, and that I have laid out all the sections clearly, making all buttons easy to find and identify. I am happy with this result as it further proves the accessibility of the game, since players will easily be able to find everything they need to play without getting stuck.

Question 3:

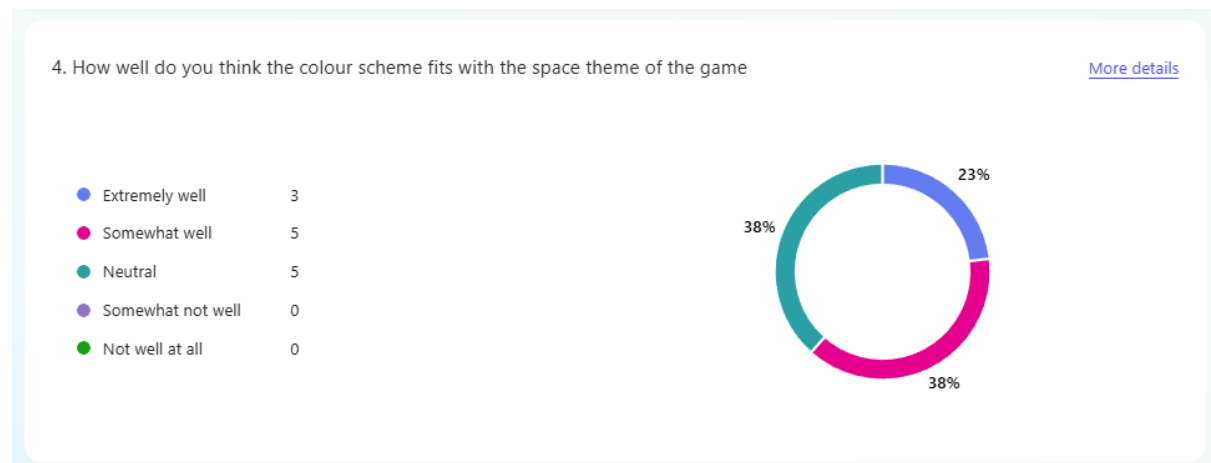


This question as designed to find out how aesthetic and well designed the colour scheme is, and whether it was appealing to the players.

The results for this question were, again, positive. All participants said that the colour scheme is at least appealing, with 38% saying it is very appealing. I am very happy with this result as it shows I have designed an aesthetic, and visually pleasing colour

scheme, which the players have recognised. The results show there still could be some improvement, since the majority of participants said that the colour scheme was just appealing, rather than very appealing. However, these improvements would just be small tweaks like making the colours more vibrant or adding glowing text, since the results still show that the colour scheme is visually appealing to all users.

Question 4:



This question was also about the colour scheme, to find out whether it fits the space theme of the game.

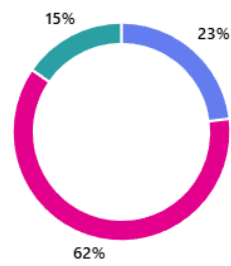
The results from this question were generally positive. 62% (rounded up) of participants said the GUI fit at least somewhat well with the space theme, with 23% of them saying it fits extremely well. This shows that the colour scheme is correctly designed, with correctly chosen colours and gradients that evoke an image of space. However, 38% of the participants responded “Neutral” to this question, suggesting that they didn’t associate the colour scheme with the theme of space. This tells me that the colour scheme could do with some improvements to ensure it correctly fits the space theme.

Question 5:

5. How easy was the population gambling feature to user and understand

[More details](#)

Very easy	3
Easy	8
Neutral	2
Hard	0
Very hard	0



This question

Question 6:

6. How would you rate the overall usability of this game out of 10

[More details](#)



Question 7:



7. Do you have any suggestions for improving the layout, colours or accessibility of the game?

8 Responses

ID ↑	Name	Responses
1	anonymous	Explain a few of the features a bit more to the user
2	anonymous	add a help feature that explains buttons that do not have descriptions
3	anonymous	add sounds and animations
4	anonymous	Give a tutorial
5	anonymous	Explain what the shop upgrades do
6	anonymous	Make the text easier to read
7	anonymous	Animations
8	anonymous	make a tutorial that explains how everything works

3. Robustness Testing:

- Finally, I carried out my robustness testing according to the post-dev test plan, by completing a series of tests to push the game to it's extremes and to test compatibility.
- **Test plan + Results:**

Test No.	Description	Test data	Expected result	Actual result
----------	-------------	-----------	-----------------	---------------

1	Can the game handle all invalid data into input forms (The population gambling form)	Attempt to enter/paste characters, symbols and negative numbers into the gambling input form	The game should not allow these inputs as only numbers are accepted. It should not crash or cause any errors	The game did not allow inputs of letters or symbols, including when pasted (The reason it allowed the letter e is because it is a mathematical constant). Therefore, the game correctly validates user inputs to prevent errors and crashing. Passed  SEE Robustness Test 1.mp4 in evidence folder
2	Can the game handle rapid clicking of all buttons	Use an autoclicker to click all buttons at 1000 clicks per millisecond	The game should continue to run smoothly without any lag, and should not crash	The game continued to run smoothly with no lag, with the game loop running as normal, and did not crash. The animation of the buttons was quite glitchy, but this is just because of the speed of the autoclicker and is not a problem since the player will not be clicking this fast. Passed  SEE Robustness Test 2.mp4 in evidence folder
3	Is the game compatible on multiple browsers	Run the game on Chrome, Firefox and Edge. Perform basic actions like clicking, gambling, switching between multi-purchase setting, saving.	The GUI should be consistent across all browsers with no display errors, and the game should run as normal. All game features should work the same.	The game opened and ran as normal on all 3 browsers. Planet clicking, population gambling, multi-purchase and progress saving worked the same on all browsers. The layout was mostly the same on all browser, apart from a slightly different font for the multi-purchase button text. showing that it is compatible on multiplier browsers and the functionality and layout is generally consistent across all. Passed 

				SEE Robustness Test 3.mp4 in evidence folder
4	Does the layout adapt to the screen size of the browser	Resize the browser window by zooming in and out out and test on a smaller screen like a laptop	The GUI should adapt to all sizes, keeping all the elements visible	The GUI did not correctly adapt to zooming out, as it was squashed at the top of the page, with the rest of the page empty. This made the text unreadable and the game virtually unplayable at 50% size and below. The game GUI did adapt correctly to zooming in, as all elements scaled correctly and the spacing stayed the same. The game remained completely playable, and the text stayed visible. Failed ✗ Despite correctly adapting to zooming in, the GUI doesn't adapt to zooming out so it is not consistent, and doesn't fully adapt to all resizing. SEE Robustness Test 3.mp4 in evidence folder
5	Can the game handle extremely large numbers for population and money	Set the population to an extremely large number and click the planet to start the money update loop	The game should continue to run smoothly with no lag, crashes or layout issues.	The actual game ran smoothly and did not crash. However as more money was generated and the money achievement was completed more and more, there were layout issues. The values in the money achievement became so large and were not being formatted correctly, causing the text in the progress bar to be squashed to the left, and the achievement container to increase in size, which then increased the width of the page. This caused the money achievement values to be unreadable, and created constant resizing of the page. This shows that the game was unable to

				handle such a large number for money without layout issues.
6	Can the game handle extremely large upgrade and worker numbers	Purchase each upgrade 100 times, and purchase 100 workers	The game should run smoothly with no lag, crashes or layout issues.	The game was not able handle the extremely large number of upgrades and workers, as it caused many layout issues. The game still ran smoothly and didn't crash, but the large numbers causes constant resizing of containers, which was very messy. The upgrade prices became very large, causing them to increase the shop container width, making it go off the screen. Also, the worker price overflowed the button due to how large it was, and the same problem with the money achievement occurred from test 5. This shows that the GUI is unable to handle extreme values, and cannot adapt properly to ensure it stays consistent.

Crash attempt:

- To improve my robustness testing, I also decided to test to see if I could cause the game to crash, by pushing it to it's absolute extremes
- I did this because I wanted to see if it would be possible for the user to crash the game, to inform future development and to uncover flaws in the way the game handles extreme values.
- I did this by purchasing a large number of the Worker Training upgrade, and then purchasing a large number of workers. This is because, during development, I noticed that this upgrade was quite overpowered and significantly increased the worker generation value when purchased in high quantities.
- **SEE** Robustness Test – CRASH ATTEMPT.mp4 in test evidence folder
- **Result:** After purchasing 265 of the worker training upgrade, with over 100 workers, the values for population, money, money per second and total worker gen displayed "Infinity", and the game became unresponsive and crashed soon after. This shows further proves that the game cannot handle extreme values and also will eventually crash if these values get too high.

Evaluation:

To conclude this project, I will evaluate the solution I have produced, by evaluating how far each success criteria has been met according to test evidence.

Success Criteria Evaluation:

1. The player should be able to click a planet, to increase it's population, which automatically generates money

MET ☒

There is a planet button in the centre of the game, which can be clicked by the player. Each time the planet is clicked, the population increases. The amount that the population increases by each time the planet is clicked starts at 1, but increases with multipliers (click surge upgrade, prestige multiplier, double click effect). As the population increases, money is automatically generated every second, the amount of which depends on the current population (starting at 0.01 per 1 population, increases with multipliers). Evidence for this can be found in **Stage 2 Development**, where I code the population increase via the planet button and the automatic money generation, and in **Stage 2 Testing**, where I test the functionality of the planet clicking, population growth and automatic money generation. Further evidence for the functionality of the planet clickgin and money generation can be found in the functionality testing (See **Post-development Testing – 1. Functionality Testing**), where I show consistent evidence of the main game loop functioning correctly, with planet clicking updating population, which lead to idle income.

2. GUI should be clean and space/sci-fi themed with separate sections for different features, and space-themed animations.

Partially Met:

The GUI has clearly separated sections for all the key features. The left section includes the achievement section in the top left, with a container underneath containing the minigames, prestige and population gambling containers. There is a title section in the top centre, with a level container

underneath. There is a central section for the main game loop (planet + population, money and money per second stats), with a worker section at the bottom of it. The right section contains the effects section in the top right, with a container underneath containing the shop and planets section. All sections are separated with borders, creating a clear and structured layout. Evidence for this can be seen in **Stage 1 Development**, where I develop the GUI for the game, and in Stage 1 Testing, where I test the layout created by identifying and explaining all the sections.

However, there is not enough evidence from the usability testing to show that the GUI is space themed. I have chosen space-related colours (pink, purple, blue), with a cosmic-style gradient for the background. However, since 38% of participants of the usability responded “Neutral” to the question “How well do you think the colour scheme fits with the space theme of the game”, there is not enough evidence for me say for certain that the GUI is “space-themed”, as there were participants that did not associate the colour scheme of the GUI with a space theme (This evidence can be seen in the Usability Testing in the Evaluation, page 457).

Furthermore, I was unable to implement any space-themed animations, even though I did implement a simple click animation where the buttons bounce on click (which I decided to implement at the end of the effects section since I realised I did not have enough time to reach the animations stage), however it hasn’t got a space theme. Therefore, this is another part of the criterion that was unmet. This does not harm the outcome too much, however, since the game still functions, has an appealing colour scheme and still has a simple click animation. I would still have liked to implement more animations, to add to the space theme of the game and make it more aesthetic.

Further development:

The lack of a space theme withing the GUI could be addressed in further development, by adding improvements to the GUI to make it fit better with the space theme of the game. These improvements could include making the colours more vibrant, adding space-themed gradients (like the gradient for the background) to all the sections, or adding a glow to the section borders, to ensure the space theme is well conveyed. Furthermore, space-themed objects could be added into the game to give it more of a space feel, e.g. stars and asteroids in the background, or UFOs that occasionally fly across the screen.

If I had more time, I would also implement some space themed animations, like space particles or unique button animations for each button rather than the same bounce animation for all buttons, however I would need to do a lot

of research on how to implement animations in CSS. As mentioned in the **Limitations** section in the **Analysis**, these would not be complex animations, but I would still need to research how to implement them since I have very little knowledge of CSS animations (I even had to research how to implement the simple button bounce animation that I implemented).

3. The game should include a shop system where the player can purchase upgrades using their money, like worker, population and money upgrades upgrades.

Met ☒

The game has a shop, located on the right of the page, containing shop upgrades for all the main features, including population upgrades, money (income) upgrades, worker upgrades, prestige upgrades, gamble upgrades, minigame upgrades and achievement upgrades. The player can purchase these upgrades with their money, after which they apply their effect. Upgrades can be stacked, applying their effect multiple times when purchased more than once. The upgrade prices also scale when purchased, making them harder to purchase each time. Evidence for this is shown in **Stage 4 Development**, where I develop the shop system and the population and income upgrades, and **Stage 4 Testing** where I test the shop system and the functionality of the upgrades. Further evidence for this can be found in the functionality testing (See **Post-development Testing – 1. Functionality Testing**), where I show consistent evidence of upgrades functioning correctly, with purchases being handled correctly and upgrades applying their various effects, Also see:

Stage 6 Development for evidence of prestige upgrades development and **Stage 6 Testing** for the testing of these upgrades (Tests 5 and 6).

Stage 7 Development for evidence of gamble upgrades development and **Stage 7 Testing** for the testing of these upgrades (Tests 9 and 10).

Stage 8 Development for evidence of achievement upgrades development and **Stage 8 Testing** for the testing of these upgrades (Tests 6 and 7).

4. The shop system should include a multi-purchase feature, where the player can buy multiplier upgrades at once (either 1, 10, 100 or the MAX they can buy with their money)

Met ☒

There is a multi-purchase button, in the top left of the shop, which the player can press. Pressing the button switches between x1, x10, x25 and MAX, allowing the player to switch between purchasing 1, 10, 25 or MAX of the upgrade. MAX means the maximum number of the upgrade that the player can purchase with the money they have. Of course, I designed this feature to allow the player to purchase 1, 10, 100 or MAX of the upgrade. However, once I got to designing the exact workings of the feature, I changed the 3rd option to 25 instead of 100, because I calculated that purchasing 100 of any upgrade at once would cost a very large number. This is still a success though, because I planned to have 4 options for the multi-purchase setting, getting increasingly larger, which the solution still has but just with a slight modification to make it more accessible. Evidence for this can be found in **Stage 4 Development**, where I develop the multi-purchase feature, and **Stage 4 Testing**, where I test it. Further evidence for this can be found in the functionality testing (See **Post-development Testing – 1. Functionality Testing**), where I show consistent evidence of the multi-purchase feature functioning correctly, with upgrade prices and purchase counts being upgraded correctly whenever the multi-purchase button is pressed.

5. The player should be able to hire workers, that automatically generate population, with upgrades available that can increase their speed and generation

Met ☒

There is a worker button, which allows players to purchase workers using their money, the price of which scales like with shop upgrades. Workers begin automatically increasing the population (starting at 1 every 2.5 secs). A total workers display increases by 1 for each worker purchased, and a total worker gen display updates to display the total population being generated every second. Workers can be stacked, e.g. 2 workers with no upgrades will generate 2 population every 2.5 seconds. There are 2 worker upgrades available in the shop, Worker Training which increases the worker generation by 20%, and Worker Efficiency which reduces the time inbetween each worker action by 0.1seconds (Until they are generating every 0.5s). Evidence for this can be seen in **Stage 3 Development**, where I develop the worker purchase and automatic population generation, and **Stage 3 Testing**, where I

test them. Evidence for the Worker Training and Worker Efficiency can be found in **Stage 4 Development** and **Stage 4 Testing**, which is where I develop and test them. Further evidence for this can be found in the functionality testing (See **Post-development Testing – 1. Functionality Testing**), where I show consistent evidence of workers functioning correctly, with purchases being handled correctly, price scaling, and workers correctly generating population automatically.

6. The player can prestige, where they reset their game progress, to earn permanent multipliers that increase their progress

Met ☒

There is a prestige button, in the prestige container on the left of the screen. On the button, there is a display that shows the population required to prestige. Once the player reaches this amount, they can press the prestige button, which will reset their game including their population, money, shop upgrades and workers. However, their achievements, level, gems and quicker prestige upgrades will be restored. The quicker prestige upgrade is a shop upgrade that reduces the population required to prestige by 5%, all purchases of which are restored after prestige. There is also a population prestige multiplier, which increases during gameplay, by 0.1 per 100k population. Once the player prestiges, this multiplier is permanently applied to their population growth (until they prestige again), and the population prestige multiplier for the next prestige resets and starts growing again. There is also a population multiplier boost, which increases the prestige population multiplier by 0.5 per upgrade. The population prestige multiplier is capped at x50, therefore it will stop increasing once it hits that, and player's will also be prevented from purchasing the population multiplier boost once it is at x50. Evidence for this can be found in **Stage 6 Development**, where I develop the game reset, prestige multiplier growth, the applying of the prestige multiplier, and the Quicker Prestige and Population Multiplier Boost upgrade, and **Stage 6 Testing** where I test this. Further evidence for this can be found in the functionality testing (See **Post-development Testing – 1. Functionality Testing**), where I show consistent evidence of the prestige feature functioning correctly, with the game being correctly reset after prestiging, and the prestige multiplier being correctly applied.

7. The player should be able to unlock and develop multiple planets, which have different stats

Not Met ✖

Unfortunately, this criterion has not been met as I did not have time to implement the additional planets. This was because I overestimated how much time I had to develop all the stages I planned. I would like to have implemented this feature, as it was requested by 93% of participants of the stakeholder survey (See **Stakeholder survey** in the **Analysis** section, **question 3**), and it would be a good way to make the game feel more fresh, as the player can start over on different planets. The planning for this stage can be found in **Stages of development – 12. Multiplie Planets**, but there is no Stage 12 in the Development, since I did not have time to implement the feature. Additionally, at 21:58 – 22:01 in the video runthrough for the functionality testing (See **Post-development Testing – 1. Functionality Testing**), it shows me clicking the minigame buttons, which don't do anything because the minigames were not implemented.

Further Development:

If I had more time, I would implement this feature fully, as it would not be that complicated and would be relatively easy to implement. The feature would be similar to the prestige feature, where the player's progress is reset, except they would be starting over on a different planet. I would simply need to change the colour scheme for each additional planet, to fit it's theme (e.g. orange/red colour scheme for Mars), change the planet image and planet name in the title, and modify the stats to make upgrade prices higher and minigames grant more rewards. This was all planned in **Stages of development – 12. Multiplie Planets**. This would prevent the player getting bored of playing through the game on the same planet, which would make the game more engaging therefore attract a larger audience.

8. The game should include simple, space-themed minigames that the player can play to earn an additional currency

Not Met ✖

This criterion was also unment, due to lack of time. I was unable to implement any of the minigames I planned in **Stages of development – 11. Minigames**. This was unfortunate as minigames were another highly requested feature by participants of the stakeholder survey (See **Stakeholder survey** in the **Analysis** section, **question 3**), with 93% selecting it as a desired feature, and it would have added another active and engaging feature to the game, like population gambling, that keeps the game fresh. There is no

Stage 11 for minigames in the **Development** section, since I did not implement the feature. Additionally, at 22:02 – 22:05 in the video runthrough for the functionality testing (See **Post-development Testing – 1. Functionality Testing**), it shows me clicking the planet unlock buttons, which are just containers with text or a lock icon, instead of buttons as I did not implement the feature.

Further Development:

If I had more time for this project, I would implement all 3 of the minigames I planned, and potentially even more if there was demand in the future. I would need to research how to use modals in HTML and CSS, which are overlays which I would use to overlay the minigames on top of the main game. I would also need to implement all the minigame GUI screens I planned (see **GUI Design** in the **Design section**), including the minigame player, minigame win screen, loss screen, and score screen, which would also all be modals.

9. There should be an additional currency (like gems), that the player earns through winning minigames, completing achievements and levelling up, which can be spent on temporary effects/multipliers

Met ☒

There is an additional currency, gems, which is displayed in the top left of the effects section. Each time the player completes an achievement or levels up they earn gems. The player does not earn gems from winning minigames, since I did not implement this feature (See previous success criteria).

Evidence for this can be found in **Stage 8 Development**, where I develop the gem increase from achievements and **Stage 9 Development** develop the xp rewards for levelling up, and **Stage 8 and 9 Testing** where I test these rewards. Achievements can be spent on effects, which apply temporary multipliers. There are 3 effects, Double Click Power (Doubles the amount clicking increases population by) for 10 seconds, Double Money (Double's income per second) for 20 seconds, and Double Worker Gen (Double's the total worker generation per worker action) for 30 seconds. The gem price for effects does not scale, it stays the same the whole game. Evidence for these effects can be found in **Stage 10 Development**, where I develop the temporary multipliers for each effect and their purchase handling, and **Stage 10 Testing** where I test the effects.

10. The player can gamble some or all of their population, with a chance of winning and multiplying their bet, or losing it all

Met ☒

There is a population gambling section in the bottom left of the game, with an input field to allow the player to enter an amount of population to gamble (which must be no more than their current population). There is a win chance (percentage chance of winning the gamble) and win return (population reward for winning the gamble) display which both update dynamically each time the player enters or removes a digit in the input field. There is also a gamble button, which the player can press (after entering the amount) to gamble, after which they will either win or lose (with the chance depending on the win chance). There is also a win message that appears and a win sound that plays after a gamble win, and a loss message that appears with a loss sound that plays after a gamble loss. Evidence for this can be found in **Stage 7 Development**, where I develop the gambling feature, including the win chance and return calculation, population rewarding and the win and loss message and sounds, and **Stage 7 Testing** where I test the feature. Additionally, I test the gambling feature multiple times in the video runthrough for the functionality testing (See gamble evidence in **Post-development Testing – 1. Functionality Testing**), showing consistent evidence of the win chance and return being correctly calculated, the win/loss message and sounds being correctly shown and played, and the population reward being correctly granted.

11. The player unlocks achievements for completing a certain amount of an action (e.g. clicking the planet), which reward them with xp and the secondary currency

Met ☒

There are 3 achievements, located in the top left of the page. There is a planet click achievement, where the player clicks the planet to complete the achievement, an earn money achievement, where the player earns money to complete the achievement, and a consecutive gamble achievement, where the player wins consecutive gambles to complete the achievement. The achievement goal (The amount of the action the player must complete to complete the achievement) scales with each achievement completion. The

goal for the click achievement starts at 100 (Click the planet 100 times), for the money achievement starts at 100 (Earn 100 money), and for the gamble achievement at 5 (Win 5 gambles in a row), and each time the achievement is completed these get harder. Each time the player completes an achievement, they earn a gem and xp reward. The reward they get from each achievement also scales with completion, making the rewards better as the achievement gets harder. Evidence for this can be found in **Stage 8 Development**, where I develop the achievement completion, scaling and progress bar updates, and **Stage 8 Testing** where I test this. Further evidence for this can be found in the functionality testing (See **Post-development Testing – 1. Functionality Testing**), where I show consistent evidence of achievements functioning correctly, with progress correctly updating, the progress bars updating with progress, and achievements correctly scaling and granting rewards upon completion.

12. After finishing the main game (unlocking all the default planets), the player should be able to create and customise their own planets, with unique stats that they can choose

Not Met ❌

This criterion was unmet, again due to a lack of time, meaning I was unable to implement the custom planet creator. I would have really like to implement this feature, since one of the main goals of my project was to create an idle game with more customisability than other similar games, making the player feel like they really own their tycoon and can customise it how they want. So, it's unfortunate that I was unable to implement this feature, as it limits the customisability of the game and potentially reduce the time the player can play it before getting bored. Evidence for the planning of this feature can be found in **Stages of development – 13. Custom Planets**, however there is so **Stage 13** in the **Development** section since I did not implement the feature. Additionally, at 22:02 – 22:05 in the video runthrough for the functionality testing (See **Post-development Testing – 1. Functionality Testing**), it shows me clicking the planet unlock buttons, including the custom planet unlock button at the end, which is just a container with a lock icon since I did not implement the feature.

Further Development:

In the future, if I had more time, I would implement the custom planet creator. Like with the minigames, this would require me to research how to implement modals in HTML and CSS, since I planned to overlay the planet

creator on top of the main game page. I have a GUI design for the custom planet creator, which I would implement. This would require implementing drop down menus for each category of planet customisation, with input fields and tick boxes for the player to customise their planet. I would then be able to simply run the main game loop with the modified stats.

13. The player's progress should automatically save to local storage and whenever a save button is pressed

Met ☒

Every 5 seconds, the player's progress automatically saves to local storage, meaning that when they close the game and re-open it, their game is restored to the point when the game was last saved. Additionally, there is a save button which allows the player to save their progress at any point, meaning when they close the game and re-open it, their game is restored to the point when they last pressed the button. Evidence for this can be found in **Stage 7 Development**, where I develop the automatic and manual progress saving, and **Stage 7 Testing** when I test it. Further evidence for this can be found in the functionality testing (See **Post-development Testing – 1. Functionality Testing**), where I show consistent evidence of progress automatically saving, and saving whenever the save button is pressed

14. The player has a level which increases when they gain xp (through minigames or completing achievements), with a level up congratulations screen and reward granted when they level up

Partially Met

There is a level display, located in the top middle of the page, underneath the title section. When the player earns xp from completing achievement, the level progress bar goes up, and the progress count value increases. When the player levels up, the level goal (Xp required to reach next level) scales which makes the next level harder to reach, causing their progress bar and progress count resets to their new value of completion. Levelling up grants an xp and population reward, however I did not implement the level up congratulations screen (See **Stage 9 Features**), because of a lack of time and necessity for

the feature. The reason I have decided that this is partially met, is because I did quite a lot of planning for the level congratulations screen, with a separate GUI design for it (See **GUI Design - 2. Level Up Screen** in the **Stages of development** section) . Therefore, it is quite a significant feature missed, however it does not remove any functionality from the level system, which is why it is not completely unmet. Evidence for level system can be found in **Stage 9 Development**, where I develop the level increase, progress bar update, and rewards, and **Stage 9 Testing**, where I test the feature.

Further Development:

To address the unmet part of this criterion, I would implement the level congratulations screen that I planned. Like with the minigames and custom planet creator, I would implement the level up screen as a modal overlay on top of the main game, which would also require research on modals in HTML and CSS. Other than that, the implementation would be very simple, as I would just have to display the reward values for the level up. This would add to the sense of achievement from levelling up, as it would clearly show that the player has reached the xp milestone to level up, and clearly display the rewards they have earned for their progress.

Usability Evaluation:

Overall, I am quite happy with the usability of my game, due to generally positive feedback from stakeholders. The average rating of the game's usability out of 10 by participants of the usability survey was 8.38, with 7 being the minimum rating and 62% rating it a 9. This shows that users generally found the game easy to use. However, the evidence shows that there are still some improvements that could be made.

In response to the question in the usability survey asking how easy the population gambling feature was to use and understand (See **Post-development testing – 2. Usability Testing, Question 5**), 15% of participants responded “Neutral”. This is a small number, however it shows that to some users, the population gambling feature may not be very intuitive. This could be addressed in further development by explaining the feature and how to use it, perhaps by adding a tutorial for it or a help box explaining how to use it. This would ensure that all users know exactly how the feature works, preventing confusion or frustration. This would be very simple to implement, as I would just need to add a button that displays some text or shows a video that explains the feature.

Furthermore, in response to the question in the usability survey asking for suggestions for improving the layout, colours or accessibility of the game (See **Post-development**

testing – 2. Usability Testing, Question 7), there were 5 responses about explaining features more to the user. 4 responses talked about this specifically: “Explain a few of the features a bit more to the user”, “add a help feature that explains buttons that do not have descriptions”, “Explain what the shop upgrades do”, “make a tutorial that explains how everything works”, and 1 response was “Give a tutorial” which suggests the need for better explanation of features. This evidence shows that the game is not intuitive enough and needs further explanation. For example, the shop upgrades have no descriptions and it is definitely not clear what some of them do just based off their name. For example, the win streak upgrade which increases gamble win chance by 5% for each consecutive gamble win is just called “Win streak”, with no reference to the population gambling feature, so is definitely not clear enough. This could be addressed in further development by adding explanations to all the features, especially the shop upgrades. I could also add a tutorial page with information on how to use all features in the game in one place, making the information easy to access at all times. This would not be complex at all, I could just make a tutorial button on the page, which takes the player to a separate page with information about how to play the game and how to use all its features. This could also include videos to explain active features like population gambling, which would make sure the player knows exactly how to use it.

Finally, one of the responses to the final question of the usability survey was “Make the text easier to read”, which makes sense because the font I use is quite thick and the spacing between the letters of some text in the game is quite small due to the small font size (e.g. the effect titles in the top right). This may make it harder for some players to read, which reduces the accessibility of the game. This could be addressed in further development by using a thinner font, or increasing the spacing between letters, which would be very easy to implement. A tutorial page may also be a good fix for this issue, as the information explaining all the features would be in a different font to that of the main game, one that is much clearer and easier to read. This would allow players to read and understand all the features, clearing up any confusion about what the text says on the main game page.

Limitations:

One limitation of my game is the lack of a tutorial, which is something that came up in the usability testing, where many users stated that the game should have explanations for features that aren't clear. Therefore, it may not be clear how to use all the features in my game, which may stop people from playing it. As I mentioned in the usability evaluation, this could be addressed in future development by adding a tutorial page to explain how to use the game.

Another limitation is the length of play. Since I was unable to implement the additional planets or the custom planet creator, the game will likely get boring and repetitive quite quickly, meaning less motivation to play the game long-term. This will make it hard to keep a long-term playerbase, meaning in further development I would definitely implement this feature to keep the game fresh in the long-term.

Additionally, the player's progress only saves to local storage, with no way of transferring it across browsers or devices. This means that the player can only play the game on one device and on browser, and would have to start over if they wanted to switch. Therefore, in future development I could implement the ability to export a save code, which can be imported on other devices and browsers, allowing the player to access their save whatever browser or device they're on.

Finally, a lack of sounds is a limitation of my game, since the only sounds are those that play after a gamble. This makes the game feel less alive and engaging, which may be another factor that contributes to the player getting bored more easily. In the usability survey, one of the responses to question 7 asking for suggestions for improving the layout, colours or accessibility of the game (See **Post-development testing – 2. Usability Testing, Question 7**) was “add sounds and animations”, showing that the lack of sounds was recognised by a user. This could be addressed in further development by adding appropriate sounds to player actions within the game. For example, I could add a ping sound that plays when a player purchases an upgrade, or a level up sound that plays each time the player levels up. This would make the game more engaging, making the player feel more excited when they complete actions. This would also not be hard to implement at all, as I would just need to download the different sounds, add them into the code and play them in the code that handles the actions (e.g. the click event listener for shop upgrade buttons for the `level_up()` function that is called when the player levels up).

Maintenance:

Overall, my project would not require much maintenance as it is run in a browser which handles all software updates, so the code should always work. Additionally, I have clear comments throughout the code, making it easy for other developers to understand and maintain it. However, some of the functions are quite complicated and slightly inefficient, so may require even more explanation. In future development, I would fix these inefficiency by re-designing the way they work, to reduce messy code and make it easier to understand. I could also write some in-depth explanations on how the code works, in a separate file, for other developers to read.

Conclusion:

To conclude this project, I am very proud of the finished product, despite the lack of all planned features that were not implemented due to a lack of time.

I have produced a functioning idle game, with a main game loop where the player clicks a planet to increase their population, which leads to idle income. I have purchase features, including a shop for upgrade and an effects section for temporary boosts, allowing the player to spend money and gems that they earned from progression. I have active features, including a population gambling feature and a prestige mechanic, which keeps the game engaging and encourages replaying of the game. I also have achievements and a level system so the player's progress can be tracked and rewarded. I have also designed a visually appealing GUI, with clearly separated sections with an aesthetic colour scheme

There are some features that I were unable to implement, including additional planets, minigames and a custom planet creator, however I believe I have enough active features combined with idle generation and progression to say that the game is a success. In further development, I would implement all these features fully, to make sure that the game stays fresh and engaging in the long term, however I have produced enough to ensure a positive playing experience for a decent amount of time.

I received generally positive feedback from stakeholders regarding the usability of the game, and my functionality testing shows evidence of a working solution. However, there are definitely some improvements that I would make in further development to make it more intuitive, and ensure all users can understand how to play it and use all of the features.

Additionally, if I had more time, I would have designed my stages of development more carefully, to reduce persistent errors and make my code more efficient.